Lecture Notes: Quantitative Reasoning and Mathematical Thinking[1]

Subhashis Banerjee

Department of Computer Science
Ashoka University
Sonipat, Haryana, 131029
email: suban@ashoka.edu.in

October 22, 2026

[1]Copyright © 2025, Subhashis Banerjee. All Rights Reserved. These notes may be used in an academic course with prior consent of the author.

Contents

1 Introduction **7**

2 God gave us numbers, and human thought created algorithms **9**
2.1 Numbers . . . 9
2.1.1 Numbers may be represented in multiple ways . . . 10
2.2 Sets . . . 10
2.3 The set of Natural numbers . . . 11
2.3.1 The Well-Ordering Principle . . . 11
2.3.2 Addition . . . 12
2.3.3 Multiplication . . . 13
2.3.4 Subtraction . . . 13
2.3.5 Division . . . 14
2.4 The Set of Integers . . . 14
2.5 The Set of Rationals . . . 15

3 Ruler and compass algorithms **17**
3.1 Constructing a line perpendicular to a given line passing through a point . . . 18
3.2 Constructing a line parallel to a given line passing through a point . . . 19
3.3 Constructibility and the compass equivalence theorem . . . 19
3.4 Rational numbers are constructible . . . 20
3.5 Euclid's GCD using ruler and compass . . . 23

4 Abstraction turns problems and concepts into principles **25**
4.1 Relations . . . 25
4.2 Function . . . 26
4.2.1 One-One (injective), Onto (surjective), and bijective Functions . . . 27
4.3 Counting, Finite and Infinite Sets . . . 27
4.3.1 Finite sets . . . 28
4.3.2 Infinite sets and bijections to $\mathbb{N}$ . . . 28
4.3.3 Integers and Rationals are countable . . . 28
4.4 Equivalence Relations, Classes, and Partitions . . . 29
4.4.1 Equivalence classes and partitions . . . 30
4.5 Modular Arithmetic, Magic Squares, and One-Time Pads . . . 31
4.5.1 Modular arithmetic . . . 31
4.5.2 Magic Squares . . . 31
4.5.3 Perfect Secrecy and One-Time Pads . . . 32

5 We need precision in thought and action to win arguments **35**
5.1 Propositions, Basic Boolean Logic and Truth Tables . . . 36
5.1.1 The basic operations . . . 36
5.1.2 Truth tables . . . 36
5.1.3 Three-variable examples of truth tables . . . 37
5.2 Vacuous Truth . . . 38

5.3 Mathematical proofs . . . 39
5.3.1 Proof by Explicit Construction . . . 40
5.3.2 Proof by Counter-Example . . . 42
5.3.3 Direct Proof . . . 43
5.3.4 Proof by Contrapositive . . . 44
5.3.5 Proof by Contradiction . . . 45
5.3.6 Proof by the Pigeonhole Principle . . . 47
5.3.7 Proof by Exhaustion of cases . . . 48
5.3.8 Proof by Induction . . . 49
5.3.9 Proofs of Equivalence . . . 54
5.3.10 Equivalence of Several Statements: Circular Implication . . . 56
5.3.11 Conclusion . . . 59

6 Induction gives us algorithms 63
6.1 Why do we need a model of computation? . . . 63
6.2 Functions defined by induction . . . 64
6.2.1 Addition and multiplication . . . 64
6.2.2 Conditions, specifications, and algorithms . . . 65
6.2.3 Quotient and remainder . . . 65
6.2.4 Powers and factorials . . . 66
6.2.5 Unfolding recursion: work that waits . . . 67
6.2.6 Summing the values of a function . . . 67
6.2.7 Perfect numbers . . . 68
6.3 How much work? Complexity and Big O . . . 68
6.3.1 Recurrences count the work . . . 68
6.3.2 Seeing the difference in growth rates . . . 69
6.3.3 A faster way to compute powers . . . 71
6.3.4 Euclid's algorithm: correctness and efficiency . . . 71
6.4 Iteration through tail recursion . . . 72
6.4.1 Invariants are induction over state changes . . . 73
6.4.2 Further examples of iterative processes . . . 74
6.4.3 Summation as iteration . . . 74
6.5 How far does this model reach? . . . 75
6.5.1 Why these rules are a reasonable model . . . 75
6.5.2 Bounded search and unbounded search . . . 75
6.5.3 What completeness means . . . 76

7 Changing state, preserving truth: the Imperative model of computation 79
7.1 Variables become places to store values . . . 79
7.1.1 Order matters: exchanging two values . . . 80
7.2 Commands, decisions, and repetition . . . 81
7.2.1 What a procedure promises . . . 81
7.3 A loop invariant is an induction hypothesis . . . 82
7.3.1 From the postcondition to the invariant and the loop . . . 83
7.4 Familiar computations, now with stored state . . . 84
7.4.1 Summation: the processed part and the remaining part . . . 84
7.4.2 Quotient and remainder together . . . 85
7.4.3 Euclid: preserving the answer while changing the problem . . . 85
7.5 Lists and a non-numerical invariant . . . 86
7.6 Searching for an entry in a list . . . 87
7.6.1 Sequential search . . . 87
7.6.2 Binary search . . . 88
7.7 Sorting by selecting the smallest remaining entry . . . 89
7.7.1 The outer loop: extend a sorted partition . . . 90

7.7.2 The inner task: find a minimum and exchange it . . . 91
7.7.3 Counting the work . . . 92
7.8 Procedures can be recursive too . . . 92
7.8.1 Trominoes: carrying out an inductive proof . . . 93
7.8.2 The Tower of Hanoi: a computation that produces actions . . . 94
7.9 What the two models share . . . 95

8 Measuring the world with geometry 99
8.1 From a place to a geometric model . . . 99
8.2 Measuring what we cannot reach . . . 100
8.2.1 A building and a stick . . . 100
8.2.2 How much should we trust the answer? . . . 100
8.2.3 Can a photograph replace a shadow? . . . 101
8.2.4 Across a river, without crossing it . . . 106
8.3 Measuring the Earth: shadows, angles, and a long journey . . . 107
8.3.1 Latitude and longitude: angles that locate a place . . . 107
8.3.2 From a shadow angle to a central angle . . . 108
8.3.3 Why did the measurement involve such a long journey? . . . 109
8.3.4 What the model tells us, and what it assumes . . . 110
8.4 Locating a point from distances . . . 110
8.4.1 Coordinates turn a diagram into equations . . . 110
8.4.2 One circle, two circles, three measurements . . . 110
8.4.3 Which clinic is nearest? . . . 111
8.4.4 Changing the meaning of distance . . . 113
8.5 Finding a shortest route . . . 113
8.5.1 A route that must touch a straight boundary . . . 113
8.5.2 Unfolding a surface . . . 113
8.6 Shape, size, and efficiency . . . 114
8.6.1 The largest rectangular garden . . . 114
8.6.2 Enlarging a shape . . . 115
8.6.3 Cutting a cube into pieces . . . 116
8.6.4 Packaging: state what is being optimised . . . 116
8.7 An optional bridge back to algorithms: area from vertices . . . 117
8.8 What a geometric answer should contain . . . 117

9 Primes, GCD, and and the intrigue of Cryptography 121
9.1 Primes . . . 122
9.1.1 There are infinitely many primes . . . 122
9.1.2 Unique prime factorisation . . . 122
9.1.3 The Sieve of Eratosthenes . . . 122
9.1.4 The Prime Number Theorem . . . 124
9.1.5 Fermat's Little Theorem . . . 125
9.2 The Greatest Common Divisor . . . 126
9.2.1 Euclid's Algorithm . . . 126
9.2.2 The Extended Euclidean Algorithm . . . 127
9.3 Chinese Remainder Theorem . . . 129
9.4 Public-Key Cryptography . . . 131
9.4.1 How to encrypt? . . . 131
9.4.2 Digital signature . . . 132
9.5 The RSA Cryptosystem . . . 133
9.5.1 The RSA procedure . . . 133
9.5.2 Why does it work? . . . 134
9.5.3 How secure is RSA? . . . 134

10 Counting to Beat the Odds **137**
10.1 Introduction . . . 137
10.2 The rules of sum and product . . . 138
10.2.1 The rule of sum . . . 138
10.2.2 The rule of product . . . 138
10.3 Permutations . . . 138
10.4 Counting functions and power sets . . . 139
10.4.1 Power set . . . 139
10.5 Combinations . . . 140
10.5.1 Combination with repetitions . . . 141
10.6 The principle of Inclusion and Exclusion . . . 141
10.7 Recurrences: Counting by Relations . . . 143
10.7.1 Pascal's triangle . . . 144
10.8 The Binomial Theorem . . . 145
10.9 Discrete Probability . . . 147
10.9.1 Experiments, Sample Spaces, and Events . . . 147
10.9.2 Probability Measure and Axioms . . . 148
10.9.3 Random Variables, probability distribution, expectation and variance . . . 149
10.9.4 The Bernoulli Distribution . . . 151
10.9.5 Independence . . . 152
10.9.6 Conditional Probability . . . 153

11 Even infinity admits a hierarchy we must confront: The Real Numbers **157**
11.1 Why Do We Need Real Numbers? . . . 157
11.1.1 Incompleteness of the Rationals and the irrational persecution of Hippasus . . . 158
11.1.2 Zeno's paradox and infinite processes: Achilles and the Tortoise . . . 159
11.2 The Real Line as a Continuum . . . 161
11.2.1 Cantor: The Reals are uncountable . . . 161
11.3 Algebraic and Transcendental Numbers . . . 162
11.3.1 How fast do our money grow? Simple and compound interests . . . 163
11.3.2 From compound interest to the number e . . . 164
11.3.3 The Number π from Geometry . . . 165

12 The Real Toolkit: Sequences, Functions, and Equations **169**
12.1 Arithmetic and geometric progressions . . . 169
12.1.1 Arithmetic progressions (AP) . . . 169
12.1.2 Geometric progressions (GP) . . . 170
12.2 Polynomials . . . 172
12.2.1 Definition and degree . . . 172
12.2.2 Basic algebraic properties . . . 172
12.2.3 The decimal system as a polynomial representation . . . 173
12.2.4 The unique interpolation theorem . . . 173
12.2.5 Polynomials and curve fitting . . . 175
12.3 Systems of linear equations . . . 177
12.3.1 Why linear equations matter . . . 177
12.3.2 Some real-world examples of linear systems . . . 178
12.4 Some fascinating real-valued functions . . . 179
12.4.1 The quadratic function . . . 179
12.4.2 The exponential function . . . 180
12.4.3 The negative exponential . . . 180
12.4.4 The logarithm . . . 180
12.4.5 The sinusoid . . . 180
12.4.6 The Gaussian (normal curve) . . . 181
12.4.7 The logistic function . . . 181

Chapter 1

Introduction

Courant and Robbins, in What is Mathematics? (1941), present mathematics not as a dry collection of formulas and tools, but as a living, creative discipline rooted in human thought and curiosity. For them, mathematics is both a pathway for understanding the natural world and an autonomous intellectual pursuit that reveals structures of order, beauty, and generality. They stress that its essence lies in the interplay between abstraction and concrete problem-solving: starting from simple, practical problems, mathematics ascends to general concepts and theories that then illuminate new domains.

They emphasize accessibility and unity: mathematics belongs to everyone who is willing to think rigorously, and its spirit combines logic with imagination. Rather than reducing it to calculation or technical skill, Courant and Robbins describe mathematics as "an expression of the human mind" where precision, creativity, and aesthetic appreciation converge. Their central idea is that mathematics is at once useful, philosophical, and artistic — simultaneously a language of science, a training ground for reasoning, and a source of intellectual delight.

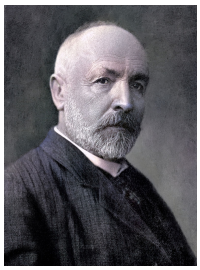
Early mathematics was computational when the emphasis was on finding methods to obtain solutions. However, over the years, the disciplines of mathematics and computer science – the subject of designing algorithms for problem solving – have diverged. In mathematics abstraction is symbolic and logical. It seeks general structures, patterns, and proofs independent of implementation. It often endeavours to seek and capture common structures across different abstractions. The primary aim is truth and understanding – developing rigorous proofs, ensuring logical consistency, and uncovering general laws. Utility often follows from this pursuit but is not always the main driver. In contrast, the role of abstraction in computational thinking is more operational and algorithmic. It emphasizes creating computational process models for natural, social and even abstract phenomena for operational analysis. The primary aim is to construct effective procedures – designing algorithms that solve problems efficiently, often under constraints of time, memory, and real-world complexity. The power lies in execution and exploration — trying to reveal insights about systems too complex to solve analytically. Both have become fundamental strands of epistemology that are essential for critical scientific thinking.

Data-driven inference represents a third way of knowing, distinct from the deductive rigour of mathematics and the constructive procedures of computational thinking. As practiced in modern data science and machine learning, it seeks knowledge not by proving theorems or designing explicit algorithms, but by discovering patterns and regularities directly from empirical data. Its epistemic core is induction at scale: hypotheses, models, or predictors are justified by their ability to capture hidden correlations and to generalize to new observations. Unlike mathematics, correctness is not absolute, and unlike computational thinking, procedures are not always fully transparent. Instead, credibility arises from empirical adequacy — the degree to which models explain, predict, or align with observed phenomena. This mode of inference expands our epistemic toolkit for a world where complexity and abundance of data overwhelm deductive or constructive methods, but it also brings new philosophical challenges: uncertainty about correctness, bias, and the gap between correlation and causation.

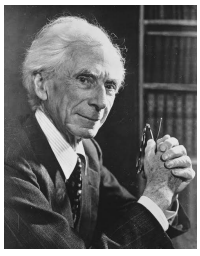
In this course we will try to cover some fundamentals of all of the above.

Chapter 2

God gave us numbers, and human thought created algorithms



"In mathematics the art of proposing a question must be held of higher value than solving it."
"A set is a Many that allows itself to be thought of as a One." – Georg Cantor



"A number will be a set of classes such as that any two are similar to each other, and none outside the set are similar to any inside the set."
"Mathematics rightly viewed possesses not only truth but supreme beauty." – Bertrand Russel

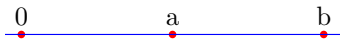
2.1 Numbers

Our discussion on mathematics and computing must start with numbers. What are numbers after all? The same number may be represented with symbols such as 3, III, or even as a line of a fixed length. But what is the underlying concept behind the different representations?

Bertrand Russell defined numbers as sizes (or *cardinality*) of *collections*. Some examples of *equinumerous* collections are $\{Red, Blue, Green\}$, $\{Amir, Salman, Shahrukh\}$, $\{Godavari, Kaveri, Krishna\}$. *Collections* are also called *Sets* or *Classes* in Mathematics. All three Sets above are of cardinality 3.

Russel defined a number as – *the number of a class is the class of all those classes that are similar (equinumerous) to it.* So, according to Russel, a number is a class of classes.

Note that *cardinality* of sets is not the only way to describe the concept of a number. A number may also be a measure of a length. For example, in the straight line below, if we define the segment $\overline{0a}$ to be the *unit length* representing the number 1, then the line segment $\overline{0b}$ which is twice the length of $\overline{0a}$ may represent the number 2.



Without belabouring the point, it will suffice to say for our purpose that all of us intuitively understand what numbers mean.

2.1.1 Numbers may be represented in multiple ways

However, we need to do useful stuff with numbers – we need to add, subtract, multiply and divide them for obvious practical reasons. Indeed, the history of numbers date back to the Mesolithic stone age. The early humans had to figure out – due to a variety of practical considerations – that if they put two similar collections of size two and size three together, the larger collection becomes of size five.

Civilisations have found many ways to represent numbers through the ages. Some examples are as tally marks in the prehistoric to early civilisations – as straight marks on bones, sticks, or stones – as can be observed in the archaeological evidence of the Ishango bones from around 20000 BCE; as Egyptian – a stroke for 1, heel bone for 10, coil of rope for 100, etc. – or Roman – I, V, X, L, C, D, M – numerals; as Base-60 (sexagesimal) numbers written as combinations of "1" and "10" wedges by the Babylonians around 2000 BCE; as used rods arranged on counting boards in base-10 with positional notation in Chinese rod systems; as positional decimal systems in Indian numerals in the Gupta period around 5th century CE; as beads or stones moved on rods or grooves to represent numbers in Abacus systems in China, Rome, Mesopotamia and Jerusalem; with Indo-Arabic numerals in the medieval period; with various mechanical calculators such as Napier's bones, Slide rules, Pascal's calculator, and Leibniz's stepped reckoner in the 17th century; as gears and levers in Charles Babbage's first programmable computer – the Analytic Engine; and as bits and bytes in modern digital computers. Note, also, that the methods of carrying out these operations – the algorithms – will necessarily depend on the representation we choose for numbers.

2.2 Sets

We will use *Sets* quite a bit in this course. We may describe a *Set* or a *Collection* by explicitly listing out its elements without duplicates, such as in the examples above. We may sometimes also describe a Set with a property like "all students enrolled in the QRMT section FC-0306-3". We write this formally using a variable x as $\{x \mid x \text{ is a student in the QRMT section FC-0306-3}\}$. The symbol $\mid$ is read as "such that".

If an element x belongs to a set A, we usually write this as $x \in A$.

Here are some more examples of Sets:

1. $A = \{x \mid x \text{ is a student pursuing a degree in India}\}$
2. $B = \{x \mid x \text{ is a CS Major student at Ashoka University}\}$
3. $C = \{x \mid x \text{ is a CS Major student at Ashoka University } \textbf{and } x \text{ is female}\}$

Clearly, all members of C are also members of B, and all members of B are members of A. We then say that C is a *subset* of B ($C \subseteq B$), and B is a *subset* of A ($B \subseteq A$). Formally, a set B is a *subset* of another set A, denoted as $B \subseteq A$, if $x \in A$ whenever $x \in B$. The empty set is denoted by ϕ, its size is zero (0), and it is a subset of all sets.

Given two sets A and B, the *union* $A \cup B$ is the set of all elements that are in A, or in B, or in both. Formally, $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$. For example, if $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, then $A \cup B = \{1, 2, 3, 4, 5\}$.

Given two sets A and B, the *intersection* $A \cap B$ is the set of all elements that are in both A and B. Formally, $A \cap B = \{x \mid x \in A \text{ and } x \in B\}$. For example, if $A = \{1, 2, 3\}$ and $B = \{3, 4, 5\}$, then $A \cap B = \{3\}$.

Clearly, for any set A, $A \cup \phi = A$ and $A \cap \phi = \phi$,

Exercise 2.1 *Suppose $B \subseteq A$. Argue that*

1. $A \cup B = A$
2. $A \cap B = B$

2.3 The set of Natural numbers

Some sets can also be unbounded or infinite. We define the set of *Natural numbers* as $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$[1].

While we all intuitively understand this set, note that the elements of the set are as yet uninterpreted and undefined. We can overcome this lacunae by assuming a *God-gifted* ability to count. Given a number n as the size of a Set or a length, let us assume that we can interpret and construct the successor of n as $S(n) = n + 1$. Then, we can formally define the set of Natural numbers $\mathbb{N}$ as:

Definition 2.1 (Natural numbers)

1. $0 \in \mathbb{N}$, *where* 0 *is the symbol that denotes the size of the empty set, and*
2. *if* $n \in \mathbb{N}$, *then* $S(n) = n + 1 \in \mathbb{N}$

We can then adopt a suitable representation for successive elements in the set $\mathbb{N}$. Note that the set $\mathbb{N}$ is unbounded, because every number – no matter how large – has a successor.

The natural numbers are the numbers that we use for counting. Their usual order begins

$$0 < 1 < 2 < 3 < \cdots .$$

Thus there is a first natural number, and every natural number has a successor. There is another important feature of this ordering: whenever we select a non-empty collection of natural numbers, that collection has a smallest member. This observation is called the *Well-Ordering Principle*.

2.3.1 The Well-Ordering Principle

Formally, the Well-Ordering Principle states that every non-empty subset of $\mathbb{N}$ has a least element. That is, if $A \subseteq \mathbb{N}$ and $A \neq \phi$, then there is an $m \in A$ such that

$$m \leq a \qquad \text{for every } a \in A.$$

For example, the set $\{5, 8, 11, 14, \ldots\}$ has least element 5. Similarly, if

$$A = \{n \in \mathbb{N} \mid n^2 > 1000\},$$

[1] 0 is usually not included in the Set of Natural numbers in Mathematics. We will however include 0 in the set of Natural numbers in this course. After all, it is quite natural to score a 0 in an examination

then $30^2 = 900$, $31^2 = 961$, and $32^2 = 1024$, so the least element of A is 32.

This property is special to the natural numbers with their usual ordering.

The Well-Ordering Principle makes precise an idea that is already present in counting. If some natural numbers have a particular property, then there is a *first* natural number having that property. This also gives a useful way of reasoning about a claim $P(n)$ that is supposed to hold for every $n \in \mathbb{N}$. If the claim were false, the set

$$C = \{n \in \mathbb{N} \mid P(n) \text{ is false}\}$$

would be non-empty and would therefore have a least element. This element would be the *smallest counterexample*. Since every smaller natural number would satisfy P, one can often use that fact to show that the smallest counterexample must also satisfy P, obtaining a contradiction.

This method foreshadows the Principle of Mathematical Induction. In fact, the Well-Ordering Principle for $\mathbb{N}$ and the Principle of Mathematical Induction are equivalent: each can be used to prove the other. We shall return to this connection when we study methods of proof.[2]

2.3.2 Addition

We observed that the underlying concept of a number is independent of specific representations. Ideally, so should be the concepts of carrying out various operations with numbers. We may think of addition – the sum $a + b$ of two numbers a and b – as just combining two similar sets of sizes a and b. However, the procedure for "combining" is not representation independent. While simple "putting together" may work if we represent the numbers as collections of stones or marbles, it is not well defined for adding two numbers in the place-value representation that we are familiar with from junior school. Hence "combining" is a somewhat unsatisfactory way of defining addition.

A better way of defining $a + b$ is by using the successor operation $S(a) = a + 1$, b times. As long as we have a primitive method for computing $a + 1$ in any representation for an arbitrary a, this definition of $a + b$ becomes representation independent. We may define the basic property of addition using counting as:

For all $m, n \in \mathbb{N}$:

1. $m + 0 = m$
2. $m + S(n) = S(m + n)$

In the above definition we have used the same trick as in definition of the set $\mathbb{N}$ above, of defining a larger concept as a successor of a smaller concept. The process repeats, and the actual additions happen in the return path. For example,

$$\begin{aligned}
&7 + 5 \\
&\quad = (7 + 4) + 1 \\
&\quad = ((7 + 3) + 1) + 1 \\
&\quad = (((7 + 2) + 1) + 1) + 1 \\
&\quad = ((((7 + 1) + 1) + 1) + 1) + 1 \\
&\quad = (((((7 + 0) + 1) + 1) + 1) + 1) + 1 \\
&\quad = ((((7 + 1) + 1) + 1) + 1) + 1 \\
&\quad = (((8 + 1) + 1) + 1) + 1 \\
&\quad = ((9 + 1) + 1) + 1 \\
&\quad = (10 + 1) + 1 \\
&\quad = 11 + 1 \\
&\quad = 12
\end{aligned}$$

[2]The Well-Ordering Principle used here should not be confused with the much more general *Well-Ordering Theorem*, which states that every set can be supplied with some well-ordering. The Well-Ordering Theorem is equivalent to the Axiom of Choice. Neither that theorem nor the Axiom of Choice is required for our present discussion of $\mathbb{N}$.

Note that the repeated substitution of a larger problem with a smaller problem is bounded, because the first condition of the definition works as a sentinel that we are bound to encounter as we keep reducing n.

We can then describe a procedure for computing $a+b$ (Algorithm 1) based on the above principle, but avoiding the deferred computations. The procedure takes a and b as input and returns *sum* as the output. $sum \leftarrow sum + 1$ denotes the operation "*sum* is *assigned* $sum + 1$" indicating that *sum* is incremented by 1.

Algorithm 1 An algorithm for $a + b$ by $+1$ b-times.

```
1: procedure ADD(a, b)
2:     counter ← 0
3:     sum ← a
4:     while counter < b do
5:         sum ← sum + 1
6:         counter ← counter + 1
       return sum
```

Exercise 2.2 *1. Assuming that the operation $a + 1$ is available as a primitive, convince yourself that the above procedure for adding two numbers are correct.*

2. Argue that if the operation $a+1$ is available as a primitive, then the above algorithm for addition is representation independent.

3. Describe how the algorithm may be implemented using pebbles or marbles to represent numbers.

2.3.3 Multiplication

We can now define multiplication as repeated additions:

1. $n \times 0 = 0$, for all $n \in \mathbb{N}$
2. $n \times S(m) = n \times m + n$, for all $n, m \in \mathbb{N}$

Note that here again we have defined $n \times S(m)$, in terms of a smaller problem $m \times n$ of the same type.

Exercise 2.3 *1. Convince yourself that according to the above definition $n \times m = \underbrace{n + n + n + \ldots + n}_{m \text{ times}}$.*

2. Provide a representation independent algorithm, using only the successor function and addition, for multiplication of two numbers.

3. Describe how the algorithm may be implemented using pebbles or marbles to represent numbers.

2.3.4 Subtraction

To define the subtraction operation $m - n$, we may first define a predecessor operation $P(n)$ – analogous to $S(n)$ – as

1. $P(0)$ is undefined
2. $P(n) = n - 1$ for all $n > 0$.

We assume, as before, that we have a primitive counting based procedure for computing $P(n) = n - 1$ in any representation. We can define the subtraction operation $m - n$ similarly to addition:

For all $m, n \in \mathbb{N}, m \geq n$

1. $m - m = 0$

2. $m - n = S(P(m) - n)$

As before, note that $P(m) - n$ is a smaller problem that $m - n$.

The subtraction algorithm may then be given as:

Algorithm 2 An algorithm for $a - b$, $a \geq b$ by -1 b-times.

```
1: procedure SUBTRACT(a, b)
2:     counter ← 0
3:     while counter < b do
4:         a ← a − 1
5:         counter ← counter + 1
       return a
```

Exercise 2.4 *Provide alternative versions of Algorithms 1 and 2 without using the counter. Instead decrement b using $b \leftarrow b - 1$ repeatedly till $b = 0$.*

2.3.5 Division

Division is a natural requirement in civilised societies, mainly for sharing. However, it may not always be possible to divide natural numbers in equal proportions. For example, a collection of size 3 cannot be divided in two proportions of equal sizes without breaking up at least one member element. We have the *division theorem*:

Theorem 2.1 *Given two numbers $a, b \in \mathbb{N}$, there exist unique $q, r \in \mathbb{N}$ (quotient and remainder, respectively) such that $a = bq + r$ and $0 \leq r < b$.*

Proof: Let us first argue that such q and r exist. Repeatedly compute $a-b, a-2b, a-3b, \ldots, a-kb$, $k \geq 0$, till $a - kb < b$ and subtraction is possible no more. Set $q = k$ and $r = a - kb$. Clearly, q is the total number of times b can be subtracted from a, and $0 \leq r < b$. If $r = 0$ then b divides a exactly.

To argue that that q and r obtained by the above procedure are unique, let us suppose they are not. Then, there exist q_1, r_1 and q_2, r_2 such that

$$a = bq_1 + r_1, 0 \leq r_1 < 0$$
$$a = bq_2 + r_2, 0 \leq r_2 < 0$$

Without loss of generality, let us assume that $q_1 \geq q2$. The above implies that $b(q_1 - q_2) = r_2 - r_1$. One of two cases arise:

1. $q_1 = q_2$. This implies that $r_1 = r_2$, and hence uniqueness.
2. $q_1 > q_2$. This implies that $q_1 - q_2 \geq 1 \in \mathbb{N}$. Hence $r_2 - r_1 \geq b$. But this is not possible because $0 \leq r_1, r_2 < b$. Hence q and r must be unique.

□

In the above proof, we used *explicit construction* as a proof technique for establishing existence of such q and r, and *contradiction* for establishing their uniqueness. We will revisit these techniques later in the course when we discuss proofs.

Exercise 2.5 *Describe an algorithm using repeated subtraction that computes q and r given a and b.*

2.4 The Set of Integers

We defined the subtraction operation $m - n, m \geq n, m, n \in \mathbb{N}$ as the number of times the successor operation $S()$ needs to be applied to reach m from n. This definition requires the restriction that $m \geq n$. An obvious generalisation is to remove the restriction and measure the difference in terms of

either the successor $S()$ or the predecessor $P()$ operator. Subtraction then becomes directional, and we require negative numbers to represent the direction. This leads us to the set of integers

$$\mathbb{Z} = \{\ldots, -3, -2, -1, 0, 1, 2, \ldots\}$$

Arithmetic in the set $\mathbb{Z}$ follows the same principles as in $\mathbb{N}$, except that they are now directional.

Exercise 2.6 *Rework the definitions and the algorithms for addition, multiplication, subtraction and division in $\mathbb{Z}$.*

Note, that the set of negative integers

$$\{-1, -2, -3, \ldots\}$$

has no least element: given any member, there is always a smaller one. So, the set of integers is not *well-ordered*.

2.5 The Set of Rationals

The division theorem tells us that given $m, n \in \mathbb{N}$, there exist $q, r \in \mathbb{N}$, such that m can be divided in to q parts of size n, possibly leaving a remainder $0 \leq r < n$. Division is an obvious fundamental need for resource sharing. If each unit is indivisible – like live cattle, for example – then the division theorem is the best we can do. However, items measured in units such as weight, volume or length – such as meat from a hunted animal, or a pile of grains – are often divisible in smaller proportions like $1/3^{rd}$, $2/25^{th}$ etc. So, division inevitably leads us to fractions. We define the set of Rational numbers as

$$\mathbb{Q} = \{x | x = p/q, p \in \mathbb{Z}, q \in \mathbb{N}, q \neq 0\}$$

We often also write this as $\frac{p}{q}$. These are numbers of the type $\pm 1/1, \pm 1/2, \pm 1/3, \pm 1/4, \pm 2/5$ etc. We may also insist that p and q should have no common factors (i.e., $gcd(p, q) = 1$; see Section 3.5 for a formal definition of gcd) to avoid multiple representations for the same Rational number. Clearly $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q}$.

Note, however, that we now have a situation where between any two rational numbers there are infinitely many other rational numbers.

Exercise 2.7 *Convince yourself of the above statement.*

This implies that there is no well defined successor function for a rational number, and we need to revisit our definition of addition for rationals. We start by noting that, for example,

$$\frac{1}{3} + \frac{2}{3} = \frac{1+2}{3} = 1$$

i.e., we can add the numerators as in $\mathbb{Z}$ if the denominators are the same. However, the addition

$$\frac{2}{3} + \frac{3}{4}$$

is not well defined unless the two fractions can be expressed in the same unit. But we can multiply the numerator and denominator of the first fraction by 4, and the second by three to convert to the same unit where the denominator of both is 12

$$\frac{2 \times 4}{3 \times 4} + \frac{3 \times 3}{4 \times 3} = \frac{8}{12} + \frac{9}{12} = \frac{8+9}{12} = \frac{17}{12}$$

Note that multiplying the numerator and the denominator of a fraction with the same number does not change the fraction. So, we can define the general rule for addition of two rational numbers as

$$\frac{p_1}{q_1} + \frac{p_2}{q_2} = \frac{p_1 \times q_2 + p_2 \times q_1}{q_1 \times q_2}$$

where all the additions and multiplications are defined on the set $\mathbb{Z}$.

Exercise 2.8 *Extend the above idea to define subtraction, multiplication and division in the set* $\mathbb{Q}$.

The set of positive rational numbers

$$\left\{1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots\right\}$$

also has no least element, since $1/(n+1) < 1/n$ for every positive natural number n. Thus not every set of numbers is well-ordered by its usual order.

Problems

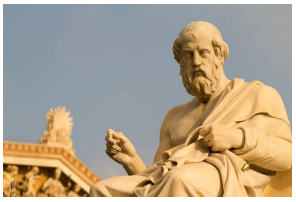
1. Give three different representations for the number 6 (for example: tally marks, Roman numerals, line segment lengths). Explain how the operation "+1" is carried out in each representation.
2. Research and briefly describe how numbers were represented in one historical number system not discussed in class (e.g., Mayan or Incan). Compare it with the decimal positional system.
3. Let

   $$A = \{x \mid x \text{ is an even number less than } 20\}, \quad B = \{x \mid x \text{ is a prime number less than } 20\}.$$

   Compute $A \cap B$, $A \cup B$, and $A \setminus B$.
4. Prove or disprove: If $A \subseteq B$ and $B \subseteq C$, then $A \subseteq C$.
5. Construct a real-world example of three sets A, B, C such that $C \subset B \subset A$.
6. Using only the successor function $S(n) = n + 1$, show step by step how to compute $4 + 3$.
7. Show that addition defined by the recursive rules in the text is associative, i.e. prove $(a+b)+c = a + (b + c)$ using the definition.
8. Starting from the definition $n \times S(m) = (n \times m) + n$, compute 3×4 step by step.
9. Write pseudocode (representation-independent) for multiplication using only the successor function and addition.
10. Explain how you could simulate multiplication using only pebbles to represent numbers.
11. Using the predecessor function $P(n)$, compute $7 - 4$ step by step as in the recursive definition.
12. Implement Algorithm 2 (subtraction by repeated decrementing of b) on the input $a = 10, b = 6$. Show the intermediate steps.
13. Using the Division Theorem, compute the quotient and remainder when $a = 29, b = 5$ using repeated subtraction.
14. Prove that the quotient and remainder obtained from the Division Theorem are unique.
15. Extend the recursive definition of addition from natural numbers to integers, and compute $(-3) + 5$.
16. Explain why we need negative numbers to generalize subtraction. Give a real-world example where negative numbers are essential.
17. Give an example of two distinct rational numbers between $\frac{1}{3}$ and $\frac{1}{2}$.
18. Prove that between any two rational numbers there exists another rational number. (Hint: use their average.)
19. Compute $\frac{2}{3} + \frac{4}{5}$ using the common-denominator method.
20. Extend the definition to show how to compute $\frac{3}{7} \div \frac{2}{5}$.

Chapter 3

Ruler and compass algorithms



"Geometry is knowledge of the eternally existent... it compels the soul to look upwards, and leads us away from the world of appearance to the vision of truth."
"Let no one ignorant of geometry enter here" – Plato

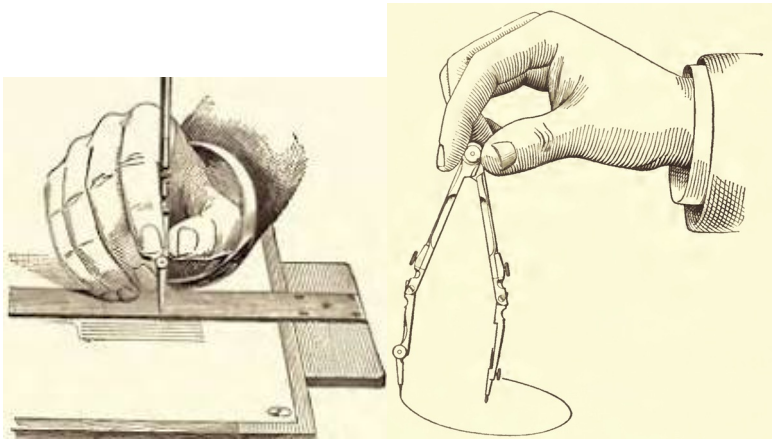


Figure 3.1: Ruler and compass. We will assume that the ruler is unmarked, and lengths can only be measured by adjusting the width of the compass.

Our endeavour so far has been to define numbers, and operations on them, in a representation independent manner. Let us now consider a specific computational model – straightedge and compass constructions introduced by the ancient Greeks – and examine whether the abstract operations we have defined above can be translated in to definite constructible procedures, or *algorithms*. Most of the geometric constructions date back to Euclid's books of Elements from around 300 BCE. We will often – by force of habit – refer to them as *ruler and compass constructions* but with the understanding that the ruler has no markings for length measurements, and can only be used to draw straight edges. See Figure 3.1.

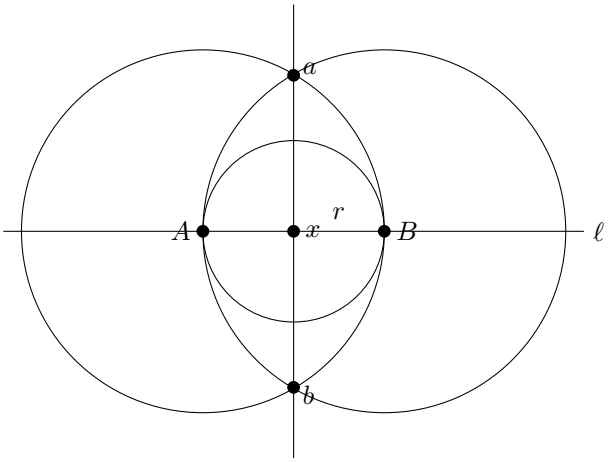


Figure 3.2: Constructing a line perpendicular to a given line passing through a point

Let us first consider some basic constructions. In what follows, the correctness of the constructions will often rely on the basic geometric properties of similar triangles. A reader may revise them from here and here.

3.1 Constructing a line perpendicular to a given line passing through a point

Given a straight line ℓ, and a point x on it, let us consider the problem of constructing a line perpendicular to ℓ and passing through x. We give a construction in Algorithm 3 (See Figure 3.2).

Algorithm 3 Constructing a line perpendicular to a given line passing through a point

1: **procedure** PERPENDICULAR(x, ℓ)
2: $c = CIRCLE(x, r)$, where r is a random length
3: $(A, B) = c \cap \ell$
4: $c_A = CIRCLE(A, 2r)$
5: $c_B = CIRCLE(B, 2r)$
6: $(a, b) = c_A \cap c_B$
7: $result = LINE(a, b)$

We have described the algorithmic procedure using some standard primitives. $c = CIRCLE(x, r)$ denotes the construction of a circle c centred at x of radius r. A and B are the intersection points of c with ℓ, denoted in the algorithm as $(A, B) = c \cap \ell$. Similarly, c_A and c_B are circles of radius $2r$ centred at A and B respectively, and a and b are the intersection points of c_A and c_B. $LINE(a, b)$ joins a and b and is the *result*.

Exercise 3.1 *1. Convince yourself that the above construction is correct. Use properties of similar triangles.*

2. Argue that $LINE(a, b)$ is also the perpendicular bisector of AB.

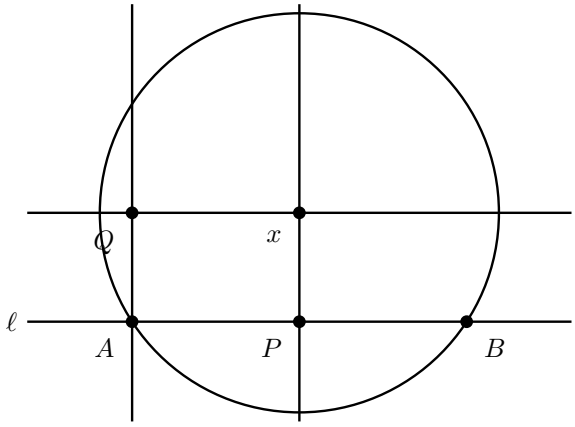


Figure 3.3: Constructing a line parallel to a given line passing through a point

3.2 Constructing a line parallel to a given line passing through a point

Given a line ℓ, and an arbitrary point x, consider the problem of construction of a line parallel to ℓ passing through x. We give a construction in Algorithm 4 (See Figure 3.3).

Algorithm 4 Constructing a line parallel to a given line passing through a point

1: **procedure** PARALLEL(x, ℓ)
2: $c = CIRCLE(x, r)$, where r is a random length
3: $(A, B) = c \cap \ell$
4: Construct p, the perpendicular bisector of AB using Algorithm 3. Argue that p passes through x.
5: $P = p \cap \ell$
6: Construct q, a perpendicular to ℓ passing through A using Algorithm 3.
7: Construct s, a perpendicular to p passing through x using Algorithm 3.
8: $Q = q \cap s$
9: $result = LINE(Q, x)$

Note that in steps 4,6 and 7 of the algorithm, we have used the procedure of Algorithm 3. We will routinely use a previously defined algorithm as a primitive to define a new algorithm.

3.3 Constructibility and the compass equivalence theorem

The above two sections give us several examples of construction of points, lines, line segments and circles. The informal definition of *constructibility* is as follows. Given points are by definition constructible. A line joining two constructible points is constructible. So is the circle centred on one constructible point passing through another constructible point. A point is constructible if it is an intersection of constructible lines and circles.

The compass advocated by the Greek philosopher Plato in these constructions is a *collapsing compass*, i.e., a compass that "collapses" whenever it is lifted from a page, so that it may not be directly used to transfer distances unlike in a modern fixable aperture compass. Note that nowhere in Sections 3.1 and 3.2 have we transferred distances using a fixed size compass lifted from the page.

This is however not a limitation as the following construction shows.

Theorem 3.1 *A collapsing compass can be used to transfer a given length to an arbitrary given*

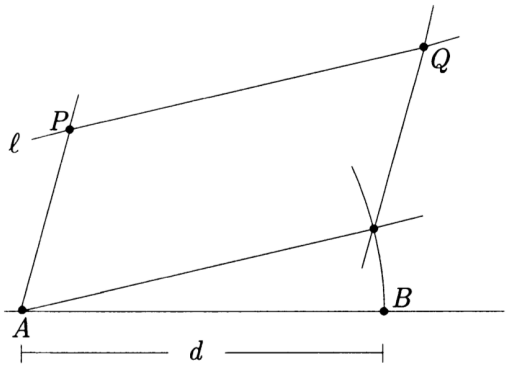


Figure 3.4: A compass-equivalence construction

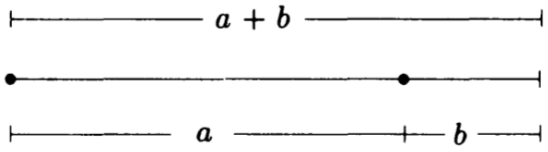


Figure 3.5: Construction of $a + b$.

point on a given line. That is, given an arbitrary length $\overline{AB}$, a line ℓ and a point P on it, it is possible to construct using a collapsing compass a point Q on ℓ such that $\overline{PQ} = \overline{AB}$.

Proof:

Algorithm 5 A compass-equivalence construction

1: **procedure** COMPASS-EQUIVALENCE(A,B,ℓ,P)
2: $c = CIRCLE(A, B)$; note that $\overline{AB} = d$
3: Construct p, the line parallel to ℓ passing through A using Algorithm 4
4: $R = c \cap p$
5: $q = LINE(A, P)$
6: Construct s, the line parallel to q passing through R using Algorithm 4
7: $Q = s \cap \ell$
8: $\overline{PQ}$ is the result on ℓ

Exercise 3.2 *Convince yourself that $\overline{PQ} = \overline{AB}$.*

□

The above is different from the original construction and proof of correctness as given by Euclid in his Book of Elements. Interested readers may study a modern version of the original construction here.

3.4 Rational numbers are constructible

First, we can consider any given length as 1, measure it using a non-collapsing compass – which we just proved is equivalent to a collapsing compass – and add it to any point on a given line by marking it off with the compass. We can similarly subtract. The operations $S(n)$ and $P(n)$ are thus

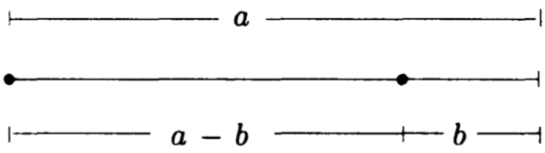


Figure 3.6: Construction of $a - b$.

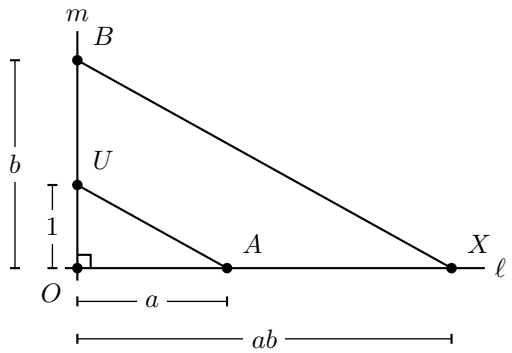


Figure 3.7: Construction of ab given a and b. Join A to U and draw the parallel through B, meeting ℓ at X. Then $\overline{OX} = ab$.

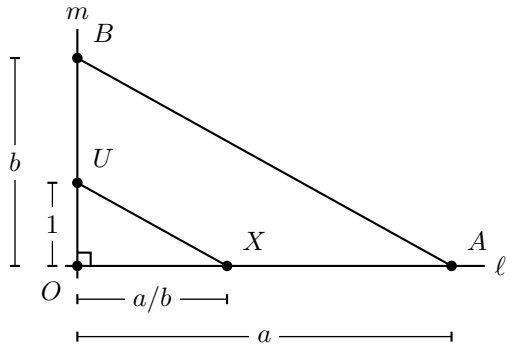


Figure 3.8: Construction of a/b given a and b, $b > 0$. Join A to B and draw the parallel through U, meeting ℓ at X. Then $\overline{OX} = a/b$.

realisable using ruler and compass. Consequently, the elements of the set $\mathbb{Z}$ are constructible. In Figures 3.5 and 3.6 we give the direct constructions for $a+b$ and $a-b$.

Given integers a and b as line segments, we can also construct rational number ab and a/b directly using similar triangles. The construction of Figure 3.7 involves marking off the lengths a and b in two perpendicular segments from O, constructing the unit length in the direction of b, and constructing a line parallel to the line $\overline{a1}$ through a or b. The intercept of the parallel line in the direction of a then marks the length ab by similarity of the triangles. We can similarly construct a/b as depicted in Figure 3.8. Rational numbers are thus constructible. See the following Algorithms:

Algorithm 6 Constructing ab given a and b

procedure MULTIPLY(a, b)
  Choose a horizontal line ℓ and a point O on it
  Construct m, the perpendicular to ℓ passing through O using Algorithm 3
  $c_a = CIRCLE(O, a)$
  $A = c_a \cap \ell$, choosing the intersection to the right of O
  $c_1 = CIRCLE(O, 1)$
  $U = c_1 \cap m$, choosing the intersection above O
  $c_b = CIRCLE(O, b)$
  $B = c_b \cap m$, choosing the intersection above O
  $p = LINE(A, U)$
  Construct q, the line parallel to p passing through B using Algorithm 4; if $B = U$, take $q = p$
  $X = q \cap \ell$
  $\overline{OX}$ is the result on ℓ

Algorithm 7 Constructing a/b given a and b, $b > 0$

procedure DIVIDE(a, b)
  Choose a horizontal line ℓ and a point O on it
  Construct m, the perpendicular to ℓ passing through O using Algorithm 3
  $c_a = CIRCLE(O, a)$
  $A = c_a \cap \ell$, choosing the intersection to the right of O
  $c_b = CIRCLE(O, b)$
  $B = c_b \cap m$, choosing the intersection above O
  $c_1 = CIRCLE(O, 1)$
  $U = c_1 \cap m$, choosing the intersection above O
  $p = LINE(A, B)$
  Construct q, the line parallel to p passing through U using Algorithm 4; if $U = B$, take $q = p$
  $X = q \cap \ell$
  $\overline{OX}$ is the result on ℓ

Exercise 3.3 *1. Describe a ruler and compass procedure for multiplication using repeated additions.*

2. Describe a ruler and compass procedure for division (computing quotient and remainder) using repeated subtractions.

3.5 Euclid's GCD using ruler and compass

GCD of two integers $a > 0$, $b \geq 0$ is defined as the largest integer $d, d > 0$ that divides both a and b. Consider the following algorithm for computing the GCD:

$$gcd(a,b) = \begin{cases} a & \text{if } b = 0 \\ a & \text{if } a = b \\ gcd(a-b,b) & \text{if } a > b \\ gcd(a,b-a) & \text{if } b > a \end{cases}$$

Exercise 3.4 *1. Convince yourself that the above algorithmic specification (rule) is correct for computing GCD. Carry out the pencil and paper computation using the above algorithm for some special cases.*

2. Describe the procedure for executing the algorithm using ruler and compass.

The algorithm described above is from Euclid's Elements. You can find a description of it here. This is also considered to be the oldest non-trivial algorithm in common use.

Now that we have defined our first computational model, several questions arise. What are the full powers of the model? What are the other things that can be constructed? What are the limits of the model, and are there easily defied concepts that are not constructible? Can there be other computational models more powerful than ruler and compass? These are the kind of questions we interrogate every computational model with. We will revisit some of these questions in the latter chapters.

Exercise 3.5 *Try to construct $\sqrt{2}$ using ruler and compass. (Hint: Use the Pythagoras theorem on a right triangle with sides $1, 1$.) Argue why this length is constructible.*

Problems

1. Construct, using ruler and compass, the perpendicular bisector of a line segment AB. Show that any point P on the perpendicular bisector is equidistant from A and B.

2. Given a triangle ABC, construct the three medians using ruler and compass. Argue that they intersect at a single point (the centroid).

3. Using Algorithm 3 (perpendicular construction), describe step by step how to construct the altitude from a vertex of a triangle. Illustrate your steps with a figure.

4. Given a line ℓ and an external point P, use Algorithm 4 (parallel construction) to draw the line parallel to ℓ through P. Prove that the two lines do not intersect.

5. Show that the compass-equivalence construction (Algorithm 5) indeed allows one to transfer a given length AB to an arbitrary point P on a line ℓ. Verify your construction with a worked example.

6. Describe a ruler and compass procedure for multiplication of two integers a, b using repeated additions. Demonstrate the construction for $a = 3, b = 4$.

7. Similarly, describe a ruler and compass procedure for division using repeated subtractions. Apply your method to compute the quotient and remainder when dividing a segment of length 11 into parts of length 3.

8. Using constructions based on similar triangles, show step by step how to obtain the product ab given line segments a and b.

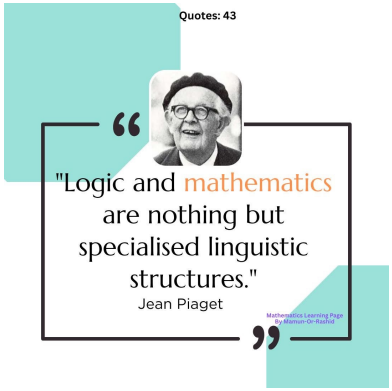
9. Construct the rational number $\frac{3}{5}$ on a line, starting with a unit segment. Explain each step of your construction.

10. Apply Euclid's GCD algorithm (as given in Section 3.5) to the lengths $a = 21$, $b = 15$, using repeated subtraction with compass and ruler. Show all intermediate steps.

11. Prove that Euclid's GCD algorithm terminates in a finite number of steps for all $a, b \in \mathbb{N}$. (Hint: in each step one of the arguments strictly decreases.)

Chapter 4

Abstraction turns problems and concepts into principles



Modern mathematics and computer science are built upon a few simple but very powerful ideas. Among the most important are the notions of *relations* and *functions*, which allow us to describe how objects are connected or transformed. Counting, infinity, and the ways in which sets can be grouped into classes help us measure size, structure, and complexity. These ideas may look abstract at first, but they underlie the methods used in algorithms, data structures, and logical reasoning.

In computer science, functions capture the essence of computation: a program takes inputs and produces outputs, just as a function does. Understanding one-one and onto functions helps us reason about whether information is lost, preserved, or fully covered. Equivalence classes and partitions allow us to organise data into categories. Modular arithmetic, often called "clock arithmetic," is fundamental in cryptography, error detection, and digital systems. Learning these concepts gives us a foundation to explore deeper mathematics and to apply it to practical computational problems.

4.1 Relations

The *Cartesian product* of two sets A and B, denoted by $A \times B$, is the set of all ordered pairs (a, b) such that $a \in A$ and $b \in B$. Thus,

$$A \times B = \{(a, b) \mid (a \in A) \text{ and } (b \in B)\}$$

A^n is the set of all ordered n-tuples $(a_1, a_2, \ldots, a_n)$ such that $a_i \in A$ for all i. i.e.,

$$A^n = \underbrace{A \times A \times \cdots \times A}_{n \text{ times}}$$

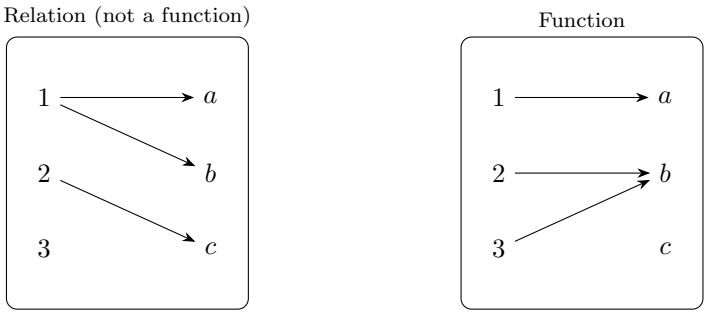


Figure 4.1: Relation vs function: In a relation, an element in A can be mapped to multiple elements in B, but not so in a function. In a function, all elements in A must be covered, but not necessarily so in a relation. The set B need not be fully covered in either.

A *relation* tells us which elements of one set are connected to elements of another.

A *binary relation* R from A to B is a subset of $A \times B$. It is a characterisation of the intuitive notion that some of the elements of A are related to some of the elements of B.

If $A = \{1, 2, 3\}$ and $B = \{a, b, c\}$, then $R = \{(1, a), (2, b), (3, a)\}$ is a relation from A to B. Familiar binary relations from $\mathbb{N}$ to $\mathbb{N}$ are $=$, $\neq$, $<$, $\leq$, $>$, $\geq$. Thus the elements of the set $\{(0, 0), (0, 1), (0, 2), \ldots, (1, 1), (1, 2), \ldots\}$ are all members of the relation $\leq$ which is a subset of $\mathbb{N} \times \mathbb{N}$.

4.2 Function

A *function* from A to B – written as $f : A \to B$ – is a special relation in which:

1. every element of A is related to some element of B, and
2. no element of A is related to more than one element of B.

Equivalently, each input has *exactly one* output.

Some familiar examples of functions are

1. $+$ and $*$ (addition and multiplication) are functions of the type $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$
2. $-$ (subtraction) is a function of the type $f : \mathbb{N} \times \mathbb{N} \to \mathbb{Z}$.
3. *div* and *mod* are functions of the type $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$. If $a = q * b + r$ such that $0 \leq r < b$ and $a, b, q, r \in \mathbb{N}$ then the functions *div* and *mod* are defined as $div(a, b) = q$ and $mod(a, b) = r$. We will often write these binary functions as $a * b$, a *div* b, a *mod* b etc.
4. The binary relations $=$, $\neq$, $<$, $\leq$, $>$, $\geq$ are also functions of the type $f : \mathbb{N} \times \mathbb{N} \to \mathbb{B}$ where $\mathbb{B} = \{false, true\}$.
5. $f : \mathbb{N} \to \mathbb{N},\ f(x) = x^2$.

We write the definition of a function formally as follows.

A *function* from A to B is a binary relation f from A to B such that for *every* element $a \in A$ there is a *unique* element $b \in B$ so the $(a, b) \in f$ (or $f(a) = b$)[1]. We will use the notation $f : A \to B$ to denote a function f from A to B. The set A is called the *domain* of the function f and the set B is called the *co-domain* of the function f. The *range* of a function $f : A \to B$ is the set $\{b \in B \mid \text{for some } a \in A,\ f(a) = b\}$ denoting the subset of elements in B that are actually covered by f.

[1] This is sometimes written using mathematical notation as $\forall a \in A$, $\exists$ unique $b \in B$. $\forall$ is the usual symbol for *for all*, and $\exists$ is the usual symbol for *there exists*

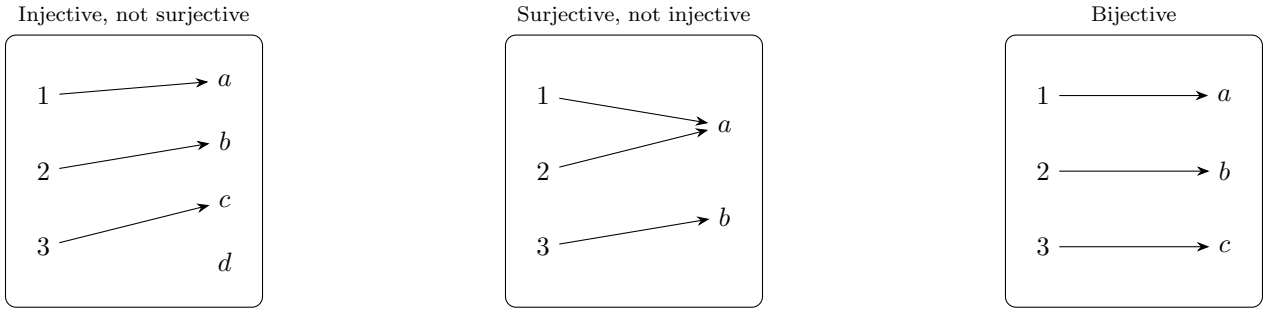


Figure 4.2: *One-one (injective)*: different inputs give different outputs. (like roll numbers: no two students share one roll); *Onto (surjective)*: every element of the codomain gets hit by *some* input. (no empty seats.); *Bijective*: both one-one and onto. (perfect pairing; has an inverse.)

4.2.1 One-One (injective), Onto (surjective), and bijective Functions

When we study functions, it is often not enough to know that "every input has exactly one output." We also want to understand how well the function uses its codomain and whether different inputs remain distinct after applying the function.

Think of a classroom with students and seats:

- If no two students sit in the same seat, the "assignment" of students to seats is one-one (injective).
- If every seat is occupied by at least one student, the assignment is onto (surjective).
- If both conditions happen together — each student has exactly one seat, and every seat is filled — then the assignment is bijective. In such a case we may also define an inverse function from seats to students.

The formal definitions below make the concepts precise.

Let $f : A \to B$ be a function. f is

Injective if whenever $f(a_1) = f(a_2)$ for $a_1, a_2 \in A$, we can conclude that $a_1 = a_2$

Surjective if for all $b \in B$, there exists $a \in A$ such that $f(a) = b$.

Bijective if it is both injective and surjective.

Example 4.1 *1.* $f : \mathbb{N} \to \mathbb{R},\ f(n) = 2n$ *is injective but not surjective (odd numbers are not covered).*

2. $g : \mathbb{N} \to \mathbb{N},\ g(0) = 0;\ g(n) = n - 1$ *is surjective but not injective (why?).*

4.3 Counting, Finite and Infinite Sets

Counting is one of the most fundamental activities in mathematics: it is how we measure the size of a set. At first glance this seems very straightforward — counting laddoos in a box or students in a class is familiar to everyone. However, mathematics asks us to extend this simple idea to more abstract settings: What does it mean for a set to be "finite"? How can we formally compare the sizes of different sets, especially when some sets are infinite? For finite sets, the answer is clear: we can match the elements of the set to the numbers $\{1, 2, \ldots, n\}$. For infinite sets, the situation is more subtle, but surprisingly we can still talk about "countable" sets — those whose elements can be placed in a one-to-one correspondence with the natural numbers $\mathbb{N}$. This point of view leads to some striking and important results: for instance, that the set of all even numbers, the set of all

integers, and even the set of all rational numbers are all countably infinite. At the same time, we will see later in this course that there are sets of numbers so large that they cannot even be listed in sequence: these are called uncountable sets. These distinctions between finite, countably infinite, and uncountable sets form the foundation for much of modern mathematics, and are crucial in computer science as well, where questions of size, encoding, and enumeration play a central role.

4.3.1 Finite sets

A set is *finite* if it has a finite number of elements. Formally, a set A is finite if there exists a natural number n and a bijection

$$f : A \to \{1, 2, \ldots, n\}.$$

This means that the elements of A can be paired exactly with the first n natural numbers.

Example 4.2 *The set $\{a, b, c\}$ is finite because we can define $f(a) = 1$, $f(b) = 2$, $f(c) = 3$, which is a bijection to $\{1, 2, 3\}$.*

4.3.2 Infinite sets and bijections to $\mathbb{N}$

A set is *infinite* if it is not finite. Some infinite sets are still "countable" because they can be put in one-to-one correspondence (bijection) with the natural numbers $\mathbb{N}$. In such a case the elements of the set can be enumerated as *first*, *second*. *third*, and so on.

Definition. A set A is *countably infinite* or *denumerable* if there exists a bijection $f : A \to \mathbb{N}$.

Example 4.3

1. *The natural numbers $\mathbb{N}$ are countably infinite via the trivial bijection $f(n) = n$.*
2. *The set of even naturals $E = \{0, 2, 4, 6, \ldots\}$ is countably infinite. Define $f : \mathbb{N} \to E$ by $f(n) = 2n$. This is a bijection.*
3. *The set of odd naturals $O = \{1, 3, 5, 7, \ldots\}$ is also countably infinite. Define $g : \mathbb{N} \to O$ by $g(n) = 2n + 1$. This is a bijection.*

4.3.3 Integers and Rationals are countable

The integers $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ can be enumerated in a sequence:

$$0, -1, 1, -2, 2, -3, 3, -4, 4, \ldots$$

Define a bijection $h : \mathbb{N} \to \mathbb{Z}$ by

$$h(n) = \begin{cases} \frac{n}{2} & \text{if } n \text{ is even,} \\ -\frac{n+1}{2} & \text{if } n \text{ is odd.} \end{cases}$$

This shows that $\mathbb{Z}$ is countable.

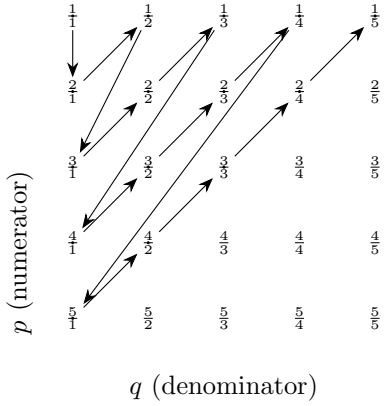
The rationals $\mathbb{Q} = \{\frac{p}{q} : p, q \in \mathbb{Z}, q \neq 0\}$ are also countable, though the proof is less obvious.

Step 1. First, consider only the positive rationals $\mathbb{Q}^+$. We can arrange them in a grid with numerator along one axis and denominator along the other:

	1	2	3	...
1	$\frac{1}{1}$	$\frac{1}{2}$	$\frac{1}{3}$	...
2	$\frac{2}{1}$	$\frac{2}{2}$	$\frac{2}{3}$	...
3	$\frac{3}{1}$	$\frac{3}{2}$	$\frac{3}{3}$	...
⋮	⋮	⋮	⋮	⋱

Note, however, that this does not give us an enumeration.

Step 2. Use Cantor's method: start at $\frac{1}{1}$, then $\frac{2}{1}, \frac{1}{2}$, then $\frac{3}{1}, \frac{2}{2}, \frac{1}{3}$, and so on, zig-zagging across the grid. This produces a sequence that eventually lists every positive rational.



Exercise 4.1 *1. Convince yourself that above ordering gives a bijection from* $\mathbb{N}$ *to ordered pairs* $(p, q), p > 0, q > 0$.

*2. Can you work out an explicit formula for the bijective function? (***This can be challenging***)*

Step 3. To avoid repetitions, we can restrict to fractions in lowest terms (e.g. $\frac{2}{2}$ is skipped since it equals $\frac{1}{1}$).

Step 4. To cover negative rationals as well, interleave them with positives:

$$0, \tfrac{1}{1}, -\tfrac{1}{1}, \tfrac{2}{1}, -\tfrac{2}{1}, \tfrac{1}{2}, -\tfrac{1}{2}, \ldots$$

This construction defines an explicit enumeration of $\mathbb{Q}$, so $\mathbb{Q}$ is countably infinite.

4.4 Equivalence Relations, Classes, and Partitions

In mathematics, we often want to group objects together when they share some common property. For example, in geometry all shapes that have the same size and shape are considered "congruent," and in number theory two integers that leave the same remainder when divided by n are considered "equivalent." These situations are captured formally by the concept of an *equivalence relation*.

Equivalence relations are important because they let us partition a large and possibly complicated set into smaller, simpler pieces (called *equivalence classes*). Each equivalence class collects all elements that are considered "the same" under the relation. Many areas of mathematics, and even computer science (e.g. hashing, classification, state-space reduction), rely on such partitions.

Definition. A relation R on a set A is an *equivalence relation* if for all $a, b, c \in A$:

- (Reflexive) $(a, a) \in R$,
- (Symmetric) $(a, b) \in R \Rightarrow (b, a) \in R$,
- (Transitive) $(a, b) \in R$ and $(b, c) \in R \Rightarrow (a, c) \in R$.

We will also write $(a, b) \in R$ as $a \; R \; b$ or as $a \sim b$.

Definition. Set of elements that are equivalent form an equivalent class. Given $a \in A$, the *equivalence class* of a under relation R is

$$[a] = \{x \in A : (a, x) \in R\}.$$

Here are some example of equivalent relations

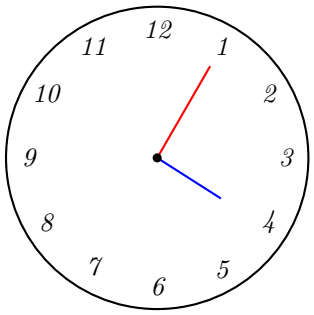
Example 4.4 *1.* ***Same birthday.*** *On the set of all students, define $x \sim y$ if x and y have the same birthday. Each equivalence class is a group of students born on the same day.*

2. ***Congruent triangles.*** *On the set of all triangles in the plane, define $T_1 \sim T_2$ if T_1 and T_2 are congruent (same shape and size). This relation is reflexive (every triangle is congruent to itself), symmetric (if T_1 is congruent to T_2, then T_2 is congruent to T_1), and transitive (if T_1 is congruent to T_2 and T_2 to T_3, then T_1 is congruent to T_3).*

3. ***Sets with same cardinality.*** *Clearly, $A \sim B$ if there exists a bijection between A and B. Since bijective functions have inverses that are bijections, and composition of two bijective functions is a bijection, we have that sets with same cardinality for an equivalent class.*

4. ***Congruence of integers (mod n).*** *Define $a \sim b$ if n divides $a - b$. We write this as $a \sim b \pmod{n}$ or $a \equiv b \pmod{n}$. This is reflexive ($a - a = 0$ is divisible by n), symmetric (if $a - b$ divisible by n, so is $b - a$), and transitive (if $a - b$ and $b - c$ divisible by n, so is $a - c$). The equivalence classes are the sets of integers with the same remainder when divided by n.*

For example, on a 12-hour clock, $16 \sim 4 \pmod{12}$; and, for the minutes hand, $65 \sim 5 \pmod{60}$;



Exercise 4.2 *Which of the following relations are not equivalent relations and why?*

1. *a R b if $a = b$.*
2. *a R b if $a \leq b$.*
3. *a R b if $gcd(a, b) = 1$.*

4.4.1 Equivalence classes and partitions

Theorem 4.1 *An equivalence relation defined on set A partitions A into disjoint equivalence classes: every element of A belongs to exactly one equivalence class, and the classes together cover all of A.*

Proof: We have to argue for two things – first, no element belongs to two or more equivalent classes; and second, the union of all the equivalence classes covers the whole set.

Suppose x belongs to two distinct equivalence classes. Then there exits $a, b \in A$ such that $x \sim a$ and $x \sim b$ but $a \not\sim b$. But this is not possible because $\sim$ is symmetric and transitive.

And, by reflexivity, each x is at least equivalent to itself. So, no element is left out. □

Example 4.5 *The relation Congruence of integers (mod 3) split $\mathbb{Z}$ into three classes:*

$$[0] = \{\ldots, -6, -3, 0, 3, 6, \ldots\}, \quad [1] = \{\ldots, -5, -2, 1, 4, 7, \ldots\}, \quad [2] = \{\ldots, -4, -1, 2, 5, 8, \ldots\}.$$

These three classes form a partition of $\mathbb{Z}$.

4.5 Modular Arithmetic, Magic Squares, and One-Time Pads

4.5.1 Modular arithmetic

Consider, for example, the set $\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$ consists of all integers modulo 7. This means that we can perform addition and multiplication, and then reduce the result to its remainder upon division by 7.

Addition. Examples:

$$3 + 5 \equiv 1 \pmod 7, \quad 6 + 4 \equiv 3 \pmod 7.$$

Every element has an *additive inverse*, i.e. a number x such that $a + x \equiv 0 \pmod 7$. We denote this as $x \equiv a^{-1} \pmod 7$

$$0^{-1} \equiv 0 \pmod 7, \quad 1^{-1} \equiv 6 \pmod 7, \quad 2^{-1} \equiv 5, \pmod 7, \quad 3^{-1} \equiv 4 \pmod 7, .$$

Multiplication. Examples:

$$3 \times 5 \equiv 1 \pmod 7, \quad 4 \times 6 \equiv 3 \pmod 7.$$

For multiplication, every nonzero element has a *multiplicative inverse*, i.e. $a \cdot x \equiv 1 \pmod 7$.

$$1^{-1} \equiv 1 \pmod 7, \quad 2^{-1} \equiv 4 \pmod 7, \quad 3^{-1} \equiv 5 \pmod 7, , \quad 6^{-1} \equiv 6 \pmod 7, .$$

Notice that 0 has no multiplicative inverse.

Thus $(\mathbb{Z}_7, +)$ is a finite arithmetic structure under addition, and $(\mathbb{Z}_7^{\times}, \cdot)$ with $\{1, 2, 3, 4, 5, 6\}$ is a finite arithmetic structure under multiplication. This has many interesting applications.

4.5.2 Magic Squares

A *magic square* is an arrangement of numbers in a square grid such that the sums of each row, column, and both diagonals are the same. Modular arithmetic allows us to construct magic squares in structures like $\mathbb{Z}_n$, where the "magic sum" is computed modulo n. This is a playful illustration of modular addition applied to combinatorial design.

A simple way to build a magic square *modulo* n is to take any ordinary magic square and reduce each entry modulo n. The LuoShu 3×3 square, which has a rich history in occult and numerology is given as

4	9	2
3	5	7
8	1	6

(each line sums to 15)

It reduces modulo 7 to

4	2	2
3	5	0
1	1	6

$\in \mathbb{Z}_7^{3\times 3}$.

Since $15 \equiv 1 \pmod 7$, every row, column, and diagonal now sums to $\overline{1} \in \mathbb{Z}_7$. For instance,

$$4 + 2 + 2 \equiv 8 \equiv 1 \pmod 7, \qquad 3 + 5 + 0 \equiv 8 \equiv 1 \pmod 7, \qquad 1 + 5 + 2 \equiv 8 \equiv 1 \pmod 7.$$

The diagonals also satisfy $4 + 5 + 6 \equiv 15 \equiv 1 \pmod 7$ and $2 + 5 + 1 \equiv 8 \equiv 1 \pmod 7$.

General recipe (odd moduli). Let S be any 3×3 magic square over the integers with magic sum M. For any modulus $n \geq 2$, the entry-wise reduction $\overline{S} \in (\mathbb{Z}_n)^{3\times 3}$ is a magic square in $\mathbb{Z}_n$ with magic sum $\overline{M} \in \mathbb{Z}_n$, because modular addition preserves equality of sums. More generally, for any $\alpha, \beta \in \mathbb{Z}_n$, the entry-wise transform $\alpha S + \beta$ is again a magic square modulo n with magic sum $\alpha M + 3\beta \pmod n$.

4.5.3 Perfect Secrecy and One-Time Pads

The idea of modular arithmetic is also central in cryptography. In the *one-time pad*, a message (plaintext) is converted into numbers (say letters $A = 0, \ldots, Z = 25$). A random secret key of the same length is chosen, and encryption is done by addition modulo 26:

$$\text{ciphertext}_i \equiv \text{plaintext}_i + \text{key}_i \pmod{26}.$$

Decryption uses subtraction modulo 26.

Claude Shannon showed that if the key is truly random, used only once, and kept secret, then the ciphertext reveals no information about the plaintext: this is called *perfect secrecy*. Thus, a simple application of modular addition gives us a theoretically unbreakable cryptosystem.

Summary

Modular arithmetic provides the framework to work with remainders in a structured way. It underlies recreational mathematics like magic squares, as well as fundamental cryptographic protocols such as the one-time pad.

Problems

1. Let $A = \{1, 2, 3\}$ and $B = \{a, b, c\}$. (a) List all elements of the Cartesian product $A \times B$. (b) Define two different binary relations $R_1, R_2 \subseteq A \times B$. (c) Which of them, if any, are functions?

2. Show that the relation $\leq$ on $\mathbb{N}$ is a subset of $\mathbb{N} \times \mathbb{N}$. Explicitly write out the first ten elements of this relation.

3. Give an example of a relation from $\{1, 2, 3\}$ to $\{a, b\}$ that is not a function. Explain why it fails to satisfy the definition of a function.

4. Consider the function $f : \mathbb{N} \to \mathbb{N}$ defined by $f(n) = 2n$. Argue that f is injective but not surjective.

5. Consider the function $g : \mathbb{N} \to \mathbb{N}$ defined by

$$g(0) = 0, \qquad g(n) = n - 1 \text{ for } n > 0.$$

   Show that g is surjective but not injective.

6. Let $h : \mathbb{Z} \to \mathbb{Z}$ be defined by $h(x) = x + 5$. Prove that h is bijective and describe its inverse function.

7. Define the function $p : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ by $p(a, b) = a \times b$. Argue formally that p is a well-defined function.

8. Construct an example of a function $f : A \to B$ where $A = \{1, 2, 3, 4\}$, $B = \{a, b\}$ that is neither injective nor surjective. Explain why.

9. Prove that if a function $f : A \to B$ is bijective, then there exists a unique inverse function $f^{-1} : B \to A$ such that $f^{-1}(f(a)) = a$ for all $a \in A$.

10. Consider the function $f : \mathbb{N} \to \mathbb{N}$ defined recursively as

$$f(0) = 0, \quad f(n + 1) = f(n) + (2n + 1).$$

    Show that $f(n) = n^2$. What kind of function is this (injective, surjective, bijective)?

11. Give an everyday example (outside mathematics) of: (a) an injective mapping, (b) a surjective mapping, and (c) a bijective mapping.

12. Challenge: Prove or disprove — the composition of two injective functions is injective, and the composition of two surjective functions is surjective. What about bijective functions?

13. On the set $\{1, 2, 3, 4, 5, 6\}$, define a relation R by $a \sim b$ if $a - b$ is divisible by 2.

    (a) Show that R is an equivalence relation.

    (b) List the equivalence classes.

14. Consider the relation "x has the same number of letters as y" on the set of English words. Prove or disprove that it is an equivalence relation. What do the equivalence classes look like?

15. Work out the addition and multiplication tables of $\mathbb{Z}_5$. Identify all additive and multiplicative inverses.

16. In $\mathbb{Z}_7$, solve the linear congruence $3x \equiv 2 \pmod 7$.

17. Find all solutions to $x^2 \equiv 1 \pmod{15}$.

18. Construct a bijection between the set of even numbers and $\mathbb{N}$. Then, using a diagram, show how integers $\mathbb{Z}$ can be listed in sequence, proving that they are countable.

19. Show that the set of rational numbers $\mathbb{Q}$ is countable by describing an explicit enumeration strategy.

20. Verify that the LuoShu square

4	9	2
3	5	7
8	1	6

    is a magic square. What is its magic sum?

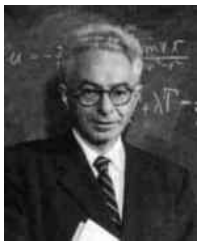
21. Construct a 3×3 magic square modulo 7 using the method of reducing an ordinary magic square. What is the magic sum in $\mathbb{Z}_7$?

22. In a one-time pad over the alphabet $\{A = 0, B = 1, \ldots, Z = 25\}$, encrypt the message `MATH` with the key `CODE`. Show the numerical steps modulo 26 and give the ciphertext.

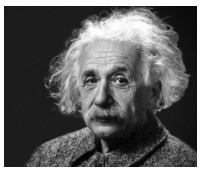
23. Explain why re-using the same key in a one-time pad scheme can destroy perfect secrecy. Give a simple example with short strings.

Chapter 5

We need precision in thought and action to win arguments



"Mathematics as an expression of the human mind reflects the active will, the contemplative reason, and the desire for aesthetic perfection. Its basic elements are logic and intuition, analysis and construction, generality and individuality." – Richard Courant



"Pure mathematics is, in its way, the poetry of logical ideas." – Albert Einstein

Logic is the language of reasoning. Whenever we argue in mathematics, make a claim in science, or even decide something in everyday life, we implicitly use the rules of logic. Boolean logic, named after the mathematician George Boole, provides a precise mathematical framework to study truth and falsity.

In this chapter, we will introduce the basic building blocks of logic, learn how to represent them with truth tables, explore important rules like De Morgan's laws, understand how to work with more than two statements at a time, and examine subtle but important ideas such as vacuous truth. Equally importantly, we will also study some of the standard methods of writing mathematical proofs. These include direct proofs, proofs by counter-examples, proofs by contrapositive, proofs by contradiction, and proofs by induction. Learning these strategies will allow you to apply logical reasoning to establish mathematical results with clarity and rigour.

The aim of this chapter is to provide a foundation that is both rigorous and intuitive, preparing you for deeper study in mathematics and computer science, where careful reasoning and proof techniques are essential.

5.1 Propositions, Basic Boolean Logic and Truth Tables

In mathematics and computer science, many problems boil down to deciding whether something is *true* or *false*. A statement that can be judged true or false is called a *proposition*. For example:

- "2 is an even number" is true.
- "5 is less than 3" is false.
- "$x + 2 = 7$" is a proposition, but its truth depends on the value of x.

Boolean logic studies how we can combine such statements using logical operations like **AND**, **OR**, **NOT**, and **IMPLIES**. These operations are the foundation of digital circuits, programming languages, and formal mathematical proofs.

5.1.1 The basic operations

We denote truth values as T (true) and F (false).

Conjunction (AND). The statement $p \wedge q$ is true only if both p and q are true. *Example:* "I will go for a walk *and* it is sunny." This is only true if both parts are true.

Disjunction (OR). The statement $p \vee q$ is true if at least one of p or q is true. *Example:* "I will have tea *or* coffee." In everyday language, "or" can sometimes mean "one but not both," but in logic the inclusive sense is used: tea, coffee, or both makes the statement true.

Negation (NOT). The statement $\neg p$ has the opposite truth value of p. *Example:* If $p =$"It is raining," then $\neg p =$"It is not raining."

Implication (IF...THEN). The statement $p \Rightarrow q$ means "p implies q" or "If p then q." It is false only when p is true but q is false. In all other cases it is true. Think of it as a promise: "If you pass the QRMT course, then you will get a laddoo."[1] The only way the promise fails is if you pass but still do not get a laddoo. If (unfortunately) you do not pass, the promise is not broken if you still get a laddoo, so the implication is considered true. Passing QRMT is sufficient to get a laddoo, so we say that "p is *sufficient* for q". Passing the course and yet not getting a laddoo will make the promise false, so we say that "q is *necessary* for p".

Biconditional (IF AND ONLY IF). The statement $p \Leftrightarrow q$ is true when both $p \Rightarrow q$ and $q \Rightarrow p$, i.e., p and q have the same truth value, either both true or both false. *Example:* "A number is even if and only if it is divisible by 2."

Exclusive OR (XOR). The statement $p \oplus q$ is true when exactly one of p or q is true, but not both. *Example:* "You can win first prize *or* second prize." You cannot win both at the same time.

5.1.2 Truth tables

Truth tables let us list all possible truth values of statements and see how the logical operations work.

p	q	$p \wedge q$	$p \vee q$	$p \Rightarrow q$	$p \Leftrightarrow q$	$p \oplus q$
T	T	T	T	T	T	F
T	F	F	T	F	F	T
F	T	F	T	T	F	T
F	F	F	F	T	T	F

[1] Note, however, that I am making no such promise. It is just a supposition for making a point.

De Morgan's Laws

These laws explain how negation interacts with AND and OR:

$$\neg(p \wedge q) \equiv (\neg p) \vee (\neg q), \qquad \neg(p \vee q) \equiv (\neg p) \wedge (\neg q).$$

Example 5.1 *Let $p =$ "bring a wizard to the QRMT class" $q =$ "bring a dragon to the QRMT class" Then $\neg(p \vee q)$ means "do not bring a wizard or a dragon to the QRMT class," which is the same as "do not bring a wizard to the QRMT class* and *do not bring a dragon to the QRMT class".*

Example 5.2 *In contrast, $\neg(p \wedge q)$ means "It is not the case that you bring both a wizard and a dragon to the QRMT class," which is logically equivalent to "either you do not bring a wizard to the QRMT class* or *you do not bring a dragon to the QRMT class (or both)."*

Exercise 5.1

1. *Convince yourself that the truth functions given in the table above, and the De Morgan's laws, are reasonable.*
2. *In particular, it is the truth table of $p \Rightarrow q$ that is most commonly used in logical deductions. Try to think of a few example English sentences that corroborate the truth table; especially the third row.*
3. *Argue, using both truth tables and language-based examples, that $p \Rightarrow q$ is equivalent to $\neg q \Rightarrow \neg p$. This is called* contrapositive.
4. *Argue, using both truth tables and language-based examples, that $p \Rightarrow q$ is equivalent to $\neg p \vee q$.*
5. *Argue, using both truth tables and language-based examples, that $p \Leftrightarrow q$ is equivalent to $\neg(p \oplus q)$.*
6. *Sometimes, in everyday life, we observe q, and hypothesize p and $p \Rightarrow q$, i.e., we hypothesize that p may have caused q. This is called* abductive reasoning. *Argue that such abductive reasoning will violate our rules of deduction in mathematics. However, abductive reasoning is essential for the sciences and the social sciences. Can you reason why, perhaps through some examples? The hypotheses generated through abductive reasoning of course need to be validated, even in the sciences and the social sciences.*

The truth tables that underlie propositional logic are not empirical discoveries but conventions inherited from linguistic and philosophical traditions. They represent a form of "agreed upon truth," codifying how we collectively interpret connectives such as "and," "or," "not," and "implies." In this sense they are *axiomatic* in nature: we do not prove that $\neg(p \vee q)$ should behave like $(\neg p \wedge \neg q)$, we simply accept the tabular assignments as the foundation upon which deductions are made. Logical proofs then proceed within this framework of shared agreement. If any one of us refuses to accept these conventions — if, for example, someone insists that "and," "or," or "implies" should work differently — then there is no common ground to move forward, and the very possibility of building logical arguments collapses.

5.1.3 Three-variable examples of truth tables

When we have three propositions p, q, r, there are $2^3 = 8$ possible combinations of truth values. Truth tables become longer, but the method is the same.

Example 1. Consider $p \wedge (q \vee r)$. We first compute $q \vee r$, then combine with p using AND.

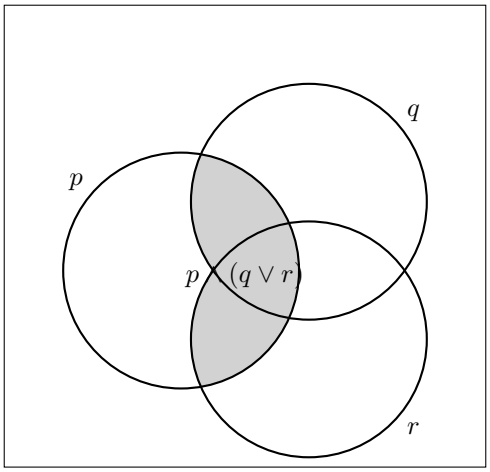
p	q	r	$q \vee r$	$p \wedge (q \vee r)$
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	F
F	T	T	T	F
F	T	F	T	F
F	F	T	T	F
F	F	F	F	F

Example 2. Consider $(p \Rightarrow q) \wedge (q \Rightarrow r)$. This corresponds to chaining two implications: "If p then q, and if q then r."

p	q	r	$p \Rightarrow q$	$q \Rightarrow r$	$(p \Rightarrow q) \wedge (q \Rightarrow r)$
T	T	T	T	T	T
T	T	F	T	F	F
T	F	T	F	T	F
T	F	F	F	T	F
F	T	T	T	T	T
F	T	F	T	F	F
F	F	T	T	T	T
F	F	F	T	T	T

Venn diagram illustration

The formula $p \wedge (q \vee r)$ can also be illustrated with sets. Think of p, q, r as sets of outcomes where each proposition is true. The shaded area corresponds to outcomes where p is true *and* at least one of q or r is true.



5.2 Vacuous Truth

The third line of the truth table for $p \Rightarrow q$, which suggests that if p is *false* and q is *true*, then $p \Rightarrow q$ is *true*, requires special attention. This forms the basis for many subtle logical arguments. A statement of the form "If p then q" is called an implication. As we saw, an implication is only false if p is true and q is false. Therefore, if p is false, the implication is always true. This phenomenon is called *vacuous truth*.

Example 5.3 *1. "If* 5 *is even, then* 5 *is prime." Since* 5 *is not even, the whole statement is true, regardless of the second part.*

2. *"All unicorns have wings." This can be written as "For every x, if x is a unicorn then x has wings." Since there are no unicorns, the condition never applies, and so the statement is vacuously true.*

3. *"If I am the king of France, then $2 + 2 = 5$." Because the condition "I am the king of France" is false, the implication is true.*

Vacuous truth is not a trick — it is a necessary feature of logic. It ensures that statements of the form "All elements of an empty set have property P" are automatically true. For example: "All prime numbers greater than 1000 and less than 1001 are even." There are no such numbers, so the statement is vacuously true.

We will use this quite a bit in the latter chapters.

5.3 Mathematical proofs

A proof is a method for establishing the truth of a statement. We use different methods in different spheres of life:

Rigour	Truth type	Field	Truth teller
0	Word of God	Religion	God/Priests
1	Authoritative truth	Business/School	Boss/Teacher
2	Legal truth	Judiciary	Law/Judge/Lawmakers
3	Philosophical truth	Philosophy	Plausible argument
4	Scientific truth	Physical sciences	Experiments/Observations
5	Statistical truth	Statistics	Data sampling
6	Mathematical truth	Mathematics	Logical deduction

Mathematical reasoning applies the highest standards of rigour for what may be considered as a *proof*, but only for very tightly defined domains. In mathematics we do not accept statements as true merely because they seem plausible or reasonable, or because many examples seem to support them. A *proof* is a logically complete argument that establishes truth from agreed assumptions, which may be definitions, axioms, or previously proved results. This chapter presents some core proof techniques, each with its own "feel" and natural use cases:

1. **Proof by Explicit Construction** — to prove that an element with some qualifying property exists, it is sufficient to construct an example.

2. **Proof by Counter-Example** — to disprove a universal claim, find one instance where it fails.

3. **Direct Proof** — derive the conclusion straight from the hypothesis using definitions and algebra.

4. **Proof by Contrapositive** — prove the logically equivalent statement $\neg Q \Rightarrow \neg P$ instead of $P \Rightarrow Q$.

5. **Proof by Contradiction** — assume the negation of what you want, reach an impossibility, and conclude the original claim.

6. **Proof by the Pigeonhole Principle** — to prove using the principle that if more objects are placed into fewer containers, then at least one container must hold more than one object.

7. **Proof by exhaustion of cases** — prove that statement hold for all of an exhaustive set of cases.

8. **Proof by Induction** — prove a base case and a step that carries truth from n to n+1.

9. **Proof of equivalence** – prove $P \iff Q$.

Along the way, we emphasize *how* to think when choosing a technique.

5.3.1 Proof by Explicit Construction

A *proof by explicit construction* demonstrates the truth of a statement of the form "there exists x such that $P(x)$" by actually exhibiting such an x and verifying that it satisfies $P(x)$. This is perhaps the most concrete kind of existence proof: instead of reasoning abstractly, we build or display the required object [2].

In our earlier discussion of *ruler and compass* constructions in Section 3 we showed, step by step, how to construct a length $a+1$ from a given length a, or how to realize the greatest common divisor of two numbers geometrically. Each of those is an instance of explicit construction: the proof of existence of a geometric object lies in the instructions themselves.

Here are some other examples of proof by explicit construction.

Theorem 5.1 *There exist integers x, y such that $14x + 21y = 7$.*

Proof: We explicitly construct one such solution: take $x = -1$, $y = 1$. Then

$$14(-1) + 21(1) = -14 + 21 = 7.$$

Thus such integers exist. In fact, by varying x and y we can generate an infinite family of solutions, but the single explicit example suffices to establish existence. □

Theorem 5.2 *There exist integers a, b, c such that $a^2 + b^2 = c^2$.*

Proof: Exhibit $(a, b, c) = (3, 4, 5)$. Indeed,

$$3^2 + 4^2 = 9 + 16 = 25 = 5^2.$$

Hence such integers exist. This triple is explicitly constructed and is known as the smallest nontrivial Pythagorean triple. □

Theorem 5.3 *For any rationals $r < s$, there exists a rational q with $r < q < s$.*

Proof: Explicitly construct $q = \frac{r+s}{2}$. Because $r < s$, we clearly have $r < q < s$. Moreover, since r and s are rational, their average q is also rational. Thus such a q exists. □

Theorem 5.4 *There exist integers x, y such that*

$$35x + 22y = 1.$$

Proof: To construct a solution, we compute the greatest common divisor of 35 and 22.

$$\begin{aligned} 35 &= 1 \cdot 22 + 13, \\ 22 &= 1 \cdot 13 + 9, \\ 13 &= 1 \cdot 9 + 4, \\ 9 &= 2 \cdot 4 + 1. \end{aligned}$$

Now back-substitute to express 1 as a combination of 35 and 22:

$$1 = 9 - 2 \cdot 4.$$

But $4 = 13 - 1 \cdot 9$, so

$$1 = 9 - 2(13 - 9) = 3 \cdot 9 - 2 \cdot 13.$$

[2] In ancient Mesopotamia (ca 2000 BCE), Babylonian mathematicians used algorithmic procedures to solve linear and quadratic equations for specific examples. The ancient Greeks – most notably Euclid (ca 300 BCE) – introduced geometric construction as a core part of geometric proofs. The kuṭṭaka (pulverizer) algorithm of Āryabhaṭa (5th c) (5th c), Jayadeva (c. 10th–11th c.), and later Bháskara II (12th c.) is a direct construction of integer solutions to linear Diophantine equations. Before the 19th century most proofs were constructive in nature.

Now substitute $9 = 22 - 13$:

$$1 = 3(22 - 13) - 2 \cdot 13 = 3 \cdot 22 - 5 \cdot 13.$$

Next, substitute $13 = 35 - 22$:

$$1 = 3 \cdot 22 - 5(35 - 22) = -5 \cdot 35 + 8 \cdot 22.$$

Thus, one explicit solution is

$$x = -5, \quad y = 8.$$

□

Remark. Equations such as above are called *linear Diophantine equations*. A solution may not always exist. The above method of finding a solution if it exists is called the *Extended Euclidean algorithm*, which not only proves that solutions exist when $\gcd(35, 22) = 1$, but by working through the steps we explicitly *construct* the solution [3].

Theorem 5.5 *The integer* 5 *has a multiplicative inverse modulo* 17*.*

Proof: We need an integer x such that

$$5x \equiv 1 \pmod{17}.$$

This is equivalent to solving the linear Diophantine equation

$$5x + 17y = 1$$

for integers x, y.

Apply the Euclidean algorithm:

$$17 = 3 \cdot 5 + 2,$$
$$5 = 2 \cdot 2 + 1.$$

Now back-substitute:

$$1 = 5 - 2 \cdot 2.$$

But $2 = 17 - 3 \cdot 5$, so

$$1 = 5 - 2(17 - 3 \cdot 5) = 7 \cdot 5 - 2 \cdot 17.$$

Thus

$$1 = 7 \cdot 5 + (-2) \cdot 17,$$

which shows that $x = 7$, $y = -2$ is an explicit solution.

Therefore

$$5 \cdot 7 \equiv 1 \pmod{17},$$

so 7 is the multiplicative inverse of 5 modulo 17. □

Remark. This construction not only proves that 5 has an inverse modulo 17, but also produces the explicit inverse 7. In fact, the extended Euclidean algorithm always gives such a construction whenever $\gcd(a, m) = 1$.

[3] The origins of this algorithm can be traced back to the **kuṭṭaka** (pulverizer) algorithm of Āryabhaṭa (5th c), Jayadeva (c. 10th–11th c.) and later Bháskara II (12th c.) refined and applied the kuṭṭaka extensively. Bháskara in the Līlāvatī and Bījagaṇita gives worked examples of solving linear Diophantine equations with what is recognizably the extended Euclid method.

When to prove by explicit construction. Proof by explicit construction is particularly valuable when the question is "does there exist?" and the object in question is concrete enough to build or write down. It contrasts with nonconstructive methods (such as contradiction or pigeonhole arguments, which we will study later) where existence is established without showing a specific example. Both are mathematically valid, but constructive proofs have the added advantage of providing insight into the nature of the object itself.

5.3.2 Proof by Counter-Example

A universal statement $\forall x\, P(x)$ is false if there exists a single x with $\neg P(x)$. Producing such an x *disproves* the claim completely [4]. For example, consider the exchange of Figure 5.1 on social media

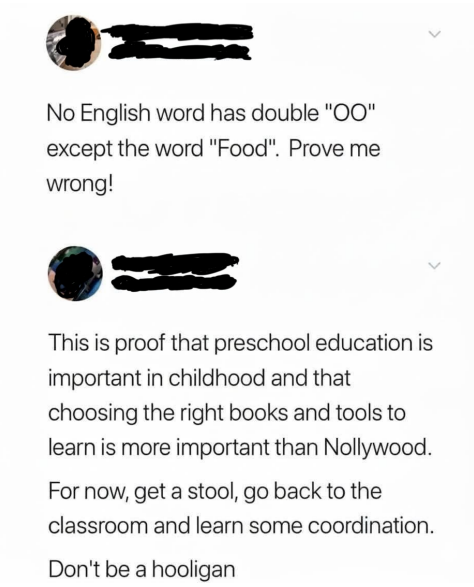


Figure 5.1: A counter-example to a universal claim

Exercise 5.2 *Which basic logical reasoning method has been used here to disprove the universal claim?*

Example 5.4 *Let us consider a few more examples.*

1. **Claim***: "The product of any two prime numbers is prime."*
   Counter-example. $2 \cdot 3 = 6$ *is not prime.*

2. **Claim***: "For all integers n, the number $n^2 + n + 41$ is prime."*
   Counter-example. *At $n = 41$, $41^2 + 41 + 41 = 41 \cdot 43$ is composite.*

.

What counter-examples teach. They refine sloppy universal claims. When a statement fails, analyzing *why* the witness breaks it often suggests a corrected statement. The expression $n^2 + n + 41$ produces prime numbers for all integers n with $0 \leq n \leq 39$, but fails afterwards.

[4] Philosophically and logically, the concept of a counterexample existed in ancient Greek dialectics and argumentation, notably in the Socratic method where contradictory examples were used to challenge general claims or definitions. In medieval and early modern mathematics, counterexamples became more systematically used to falsify conjectures. Famous historical examples include Euler's counterexample disproving the conjecture that all Fermat numbers are prime.

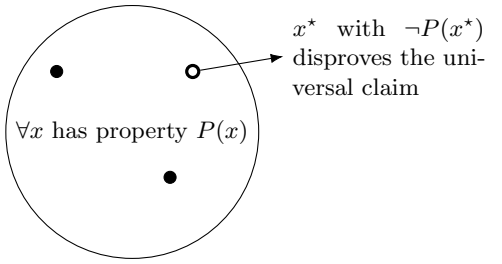


Figure 5.2: One counterexample $x^\star$ inside the "universe" circle shows the claim $\forall x\, P(x)$ is false.

5.3.3 Direct Proof

A direct proof of $P \Rightarrow Q$ proceeds as a straight line: Assume P $\rightarrow$ unpack definitions $\rightarrow$ algebra/logic $\rightarrow Q$.

Consider the following examples.

Definition. An integer n is *even* if $n = 2k$ for some integer k; it is *odd* if $n = 2k + 1$ for some integer k.

Theorem 5.6 *If a and b are even integers, then $a + b$ is even.*

Proof: Let $a = 2k$ and $b = 2m$ for integers k, m. Then

$$a + b = 2k + 2m = 2(k + m),$$

which is a multiple of 2, hence even. □

Theorem 5.7 *If x is odd, then x^2 is odd.*

Proof: Let $x = 2k + 1$. Then $x^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd. □

Theorem 5.8 $1 + 2 + 3 + \cdots + n = n(n + 1)/2$

Proof: Let $S = 1 + 2 + 3 + \cdots + n$.
Hence, $S = n + (n - 1) + (n - 2) + \cdots + 1$, in the reverse order. This implies that

$$2S = \underbrace{(n + 1) + (n + 1) + (n + 1) + \cdots + (n + 1)}_{n \text{ times}}$$

Thus, $S = n(n + 1)/2$. □

Theorem 5.9 *Every odd integer is equal to the difference between the squares of two integers.*

Proof: Let $a = 2k+1$ be an arbitrary odd integer. Then $a = 2k+1 = k^2+2k+1-k^2 = (k+1)^2-k^2$.
□

Theorem 5.10 *An integer n is divisible by 3 $\iff$ the sum of its digits is divisible by 3.*

Proof: Write n in decimal expansion as

$$n = d_0 + 10d_1 + 10^2 d_2 + \cdots + 10^k d_k,$$

where $d_0, d_1, \ldots, d_k$ are the digits of n.

Since $10 \equiv 1 \pmod 3$, it follows that

$$10^i \equiv 1 \pmod 3 \quad \text{for every } i \geq 0.$$

Therefore

$$n \equiv d_0 + d_1 + \cdots + d_k \pmod 3.$$

□

Theorem 5.11 *Let n be a positive integer with decimal form $n = 10q + u$, where u is the units digit and q is the number formed by the remaining digits. Then*

$$n \text{ is divisible by } 7 \iff q - 2u \text{ is divisible by } 7.$$

Proof: Working modulo 7, note first that $10 \equiv 3 \pmod 7$, hence

$$n = 10q + u \equiv 3q + u \pmod 7 \equiv 3(q - 2u) + 7u \pmod 7 \equiv 3(q - 2u) \pmod 7$$

Since 3 is invertible modulo 7 (indeed $3^{-1} \equiv 5 \pmod 7$), we have

$$3q + u \equiv 0 \pmod 7 \iff q - 2u \equiv 0 \pmod 7.$$

Combining with $n \equiv 3q + u \pmod 7$ yields

$$n \equiv 0 \pmod 7 \iff q - 2u \equiv 0 \pmod 7.$$

□

Example. For $n = 483$ we have $q = 48$, $u = 3$, so $q - 2u = 48 - 6 = 42$, a multiple of 7. Hence 483 is divisible by 7.

Exercise 5.3 *Argue that for bigger numbers, the test can be applied repeatedly by computing $q - 2u$ on the result.*

When to prefer direct proofs. They shine when definitions already carry the structure you need, or when simple algebra lifts P to Q. If you find yourself repeatedly "expanding the definitions" as your first move, you are in direct-proof territory.

5.3.4 Proof by Contrapositive

In Exercise 5.1 we argued that implications $P \Rightarrow Q$ and $\neg Q \Rightarrow \neg P$ are logically equivalent. Sometimes $\neg Q \Rightarrow \neg P$ is cleaner because "failing Q" has a rigid structure (e.g., divisibility, parity), while Q itself would require awkward casework.

Theorem 5.12 *For $n \in \mathbb{N}$, if $n \neq 0$ and $n \neq 1$, then $n^2 \neq n$.*

Proof: Let $P = (n \neq 0 \text{ and } n \neq 1)$ and $Q = (n^2 \neq n)$. We construct a direct proof of $\neg Q \Rightarrow \neg P$. Suppose $n^2 = n$. Then

$$n^2 - n = 0 \quad \Longrightarrow \quad n(n-1) = 0,$$

so $n = 0$ or $n = 1$. Hence the contrapositive $\neg Q \Rightarrow \neg P$ holds, and thus the original implication. □

Theorem 5.13 *If n^2 is odd, then n is odd.*

Proof: Assume that n is even, i.e., $n = 2k$ for some k. Then, $n^2 = 4k^2 = 2(2k^2)$ is also even. Hence the contrapositive $\neg Q \Rightarrow \neg P$ holds, and thus the original implication. □

Theorem 5.14 *If n^2 is divisible by 3, then n is divisible by 3.*

Proof: Assume that n is not divisible by 3. Then $n \equiv 1$ or $2 \pmod 3$. In both cases $n^2 \equiv 1 \pmod 3$, so n^2 is not divisible by 3. Hence the contrapositive $\neg Q \Rightarrow \neg P$ holds, and thus the original implication. □

Theorem 5.15 *If n does not divide ab, then n does not divide a and n does not divide b.*

Proof: Let (n divides a) or (n divides b).

If n divides a, then $a = nd$ for some $d > 0$. Thus $ab = ndb = n(db)$, which implies that n divides ab.

If n divides b, argue similarly.

□

Theorem 5.16 *Let $n \in \mathbb{Z}$. If $n^2 - 6n + 5$ is even, then n is odd.*

Proof: Let n be even, i.e., $n = 2k$ for some k. Then, $n^2 - 6n + 5 = (2k)^2 + 6(2k) + 4 + 1 = 2(2k^2 + 6k + 2) + 1$ which is odd. Hence the contrapositive is proved. □

Choosing contrapositive. Look for conclusions phrased as "is divisible by ·, is even, is nonnegative, is a subset of ·"—their negations often have crisp arithmetical or set-theoretic descriptions that are easy to exploit.

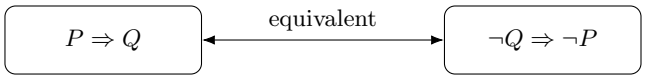


Figure 5.3: Proving the contrapositive proves the original implication.

5.3.5 Proof by Contradiction

To prove $P \Rightarrow Q$ by contradiction, assume both P and $\neg Q$. If these assumptions force an impossibility (a statement that cannot be true), then $\neg Q$ must be false, hence Q true [5]. This style of reasoning is also called reductio ad absurdum.

Consider the following examples.

Theorem 5.17 *If n is odd, then n^2 is odd.*

Proof: Let P be the statement "n is odd," and Q be the statement "n^2 is odd." Assume $P \wedge \neg Q$: that is, n is odd but n^2 is even. If n is odd, then $n = 2k + 1$ for some integer k. Then

$$n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1,$$

which is odd. This contradicts the assumption that n^2 is even. Therefore, n^2 must be odd. □

Theorem 5.18 *If ab is odd, then both a and b are odd.*

Proof: Let P be the statement "ab is odd," and Q be the statement "both a and b are odd." Assume $P \wedge \neg Q$: that is, ab is odd but at least one of a or b is even. If a is even, say $a = 2k$, then $ab = 2kb$ is even, contradiction. Similarly, if b is even then ab is even, contradiction. Therefore, both a and b must be odd. □

Theorem 5.19 *If n^2 is divisible by 3, then n is divisible by 3.*

Proof: Let P be the statement "n^2 is divisible by 3," and Q be the statement "n is divisible by 3." Assume $P \wedge \neg Q$: that is, n^2 is divisible by 3 but n is not divisible by 3. If n is not divisible by 3, then $n \equiv 1 \pmod 3$ or $n \equiv 2 \pmod 3$. In both cases,

$$n^2 \equiv 1 \pmod 3,$$

so n^2 is not divisible by 3. This contradicts our assumption. Therefore, n must be divisible by 3. □

Often, the antecedent P need not be explicit in contradiction proofs. In what follows, we prove that $\sqrt{2}$ which we constructed using ruler and compass in Exercise 3.5 is not a rational number.

Theorem 5.20 *$\sqrt{2}$ is irrational.*

Proof: We know from Exercise 3.5 that $\sqrt{2}$ can be constructed, so it exists. Assume $\sqrt{2} = \frac{p}{q}$ in lowest terms with $p, q \in \mathbb{Z}_{>0}, gcd(p, q) = 1$. Then $p^2 = 2q^2$, so p is even; write $p = 2r$. Substituting, $4r^2 = 2q^2$ gives $q^2 = 2r^2$, hence q is even. Thus p, q share a factor 2, contradicting lowest terms. □

In this example the antecedent P is the silent statement "the number $\sqrt{2}$ exists", and Q is the statement that $\sqrt{2}$ is irrational. Note, that in this case P is not a hypothesis but an independently established truth.

Also consider the following examples:

[5] Euclid's Elements contains many early examples of proof by contradiction. For instance, in Book 1, Proposition 6, Euclid proves that if two angles of a triangle are equal, the sides opposite these angles are equal by assuming the contrary and deriving a contradiction. In the section we illustrate two other famous one's – *$\sqrt{2}$ is irrational* and *there are infinitely many primes*. In Indian traditions too, the Jaina mathematicians (c. 6th–9th c.) often used impossibility reasoning, e.g. showing that certain infinite processes yield contradictions, to motivate definitions of infinity and infinitesimals. Bháskara's arguments about the impossibility of certain rational approximations can also be read as of the reductio-style.

Theorem 5.21 *Every integer $n > 1$ has at least one prime factor.*

Proof: Let $n > 1$. If n is prime, then it is its own prime factor.

Otherwise, suppose n is composite. Let d be the smallest divisor of n greater than 1. By definition, d divides n. We claim that d must be prime.

Suppose, for contradiction, that d is composite. Then $d = ab$ with $1 < a < d$ and $1 < b < d$. But then a divides d, and since d divides n, we also have a divides n. This contradicts the minimality of d, because a is a smaller divisor of n greater than 1.

Therefore d must be prime, and hence n has a prime factor in all cases. □

Theorem 5.22 *Let p be a prime. If p divides n, then p does not divide $n + 1$.*

Proof: Suppose p divides n and p divides $n + 1$. Then, for some integers a and b, $n = pa$ and $n + 1 = pb$.

We then have that $(n + 1) - n = 1 = p(b - a)$, or p divides 1, which is impossible. □

Theorem 5.23 *There exist infinitely many prime numbers.*

Proof:

Assume, to the contrary, that there are only finitely many primes $p_1, p_2, \ldots, p_n$. Consider the number

$$N = p_1 p_2 \cdots p_n + 1.$$

Clearly $N > 1$ and hence must have a prime divisor. But no p_i divides N, since dividing N by any p_i leaves a remainder 1. This contradicts the assumption that $p_1, \ldots, p_n$ were all the primes. Therefore, there must exist infinitely many primes [6] □

Theorem 5.24 *There exists an irrational number x such that x^2 is rational.*

Proof: Suppose, for contradiction, that no such number exists; that is, whenever x is irrational, x^2 is irrational as well. Consider $x = \sqrt{2}$. Then x is irrational, but

$$x^2 = (\sqrt{2})^2 = 2,$$

which is rational. This contradicts our assumption. Hence, there does exist an irrational number x such that x^2 is rational. □

Theorem 5.25 *If x and y are positive numbers with $x \neq y$, then*

$$\frac{x}{y} + \frac{y}{x} > 2.$$

Proof: Suppose, for contradiction, that $P = (x, y > 0,\ x \neq y)$, but not $(Q = \frac{x}{y} + \frac{y}{x} > 2)$. That is,

$$\frac{x}{y} + \frac{y}{x} \leq 2.$$

Since $x, y > 0$, we may multiply both sides by $xy > 0$ without reversing the inequality:

$$x^2 + y^2 \leq 2xy.$$

Rearranging,

$$(x - y)^2 \leq 0.$$

Since a square is always nonnegative, this forces $(x - y)^2 = 0$, i.e., $x = y$. This contradicts our assumption that $x \neq y$. Hence the assumption was false, and

$$\frac{x}{y} + \frac{y}{x} > 2.$$

□

Exercise 5.4 *What are the antecedents P and consequents Q in Theorems 5.23 and 5.24.*

[6] This proof was given by Euclid. This is also the first proof from the Proofs from the Book, which was written in the memory of the Hungarian mathematician Paul Erdős, who liked to talk about The Book, in which God maintains the perfect proofs for mathematical theorems.

When to use contradiction. Contradiction is powerful for showing that something is impossible – for example irrationality, no smallest positive rational, etc. – but it also works beautifully for proving existence: to show "there exists an object with property P," assume that no such object exists, and derive an inconsistency.

Contrapositive vs. contradiction

First, let us convince ourselves that the two are equivalent.

Exercise 5.5 *Argue that whatever can be proved by* contradiction *can also be proved by* contrapositive*, and vice versa.*

In general, the method of contrapositive is preferable whenever we can directly prove $\neg Q \Rightarrow \neg P$. In such situations starting with (P and $\neg Q$) instead of just $\neg Q$ and deriving $False$ would be a *roundabout* way of making the argument.

In contrast, consider Theorem 5.25 where we work with both P and $\neg Q$ to arrive at a contradiction. Working with just $\neg Q$ to prove $\neg P$ is possible, but the argument will then be not as clean.

Exercise 5.6 *Prove Theorem 5.25 by the method of contrapositive.*

5.3.6 Proof by the Pigeonhole Principle

The *Pigeonhole Principle* states that if $n+1$ or more objects are placed into n boxes, then some box must contain at least two objects. More generally, if N objects are placed into k boxes, then some box contains at least $\lceil N/k \rceil$ objects[7]. The pigeonhole principle itself can be proved by either contradiction or contrapositive [8].

Exercise 5.7 *Prove the pigeonhole principle using both contradiction and contrapositive.*

The pigeonhole principle however merits an independent consideration because this special form makes several proofs easier. This principle often provides quick existence proofs when explicit construction is difficult. It guarantees that a certain configuration must exist, even if we cannot point to the exact example.

Example 5.5 *In a group of 13 people, at least two must share a birth month. Here the "boxes" are the 12 months, and the "objects" are the 13 people. By the pigeonhole principle, some month contains at least two birthdays.*

Here are a few more examples.

Theorem 5.26 *Among any $n+1$ integers, there exist two with the same remainder when divided by n.*

Proof: The possible remainders upon division by n are $0, 1, 2, \ldots, n-1$, giving n "boxes." Placing $n+1$ integers into these boxes, two must land in the same box. Therefore two of the integers have the same remainder. □

Theorem 5.27 *Given n integers $a_1, a_2, \ldots, a_n$, there exists a non-empty subset whose sum is divisible by n.*

[7] $\lceil N/k \rceil$ is the smallest integer q such that $qk \geq N$.

[8] The first known formal appearance of the pigeonhole is often attributed to the German mathematician Peter Gustav Lejeune Dirichlet in 1834, who called it the "Schubfachprinzip" (drawer or box principle). The principle appears indirectly and informally much earlier, dating back at least to 1622 in a Latin work by the French Jesuit mathematician Jean Leurechon. But Indian combinatorial work (e.g. Pingala's prosody, c. 200 BCE) already involved reasoning about distributing syllables into patterns — essentially counting arrangements that could be seen pigeonhole-wise. Later, in combinatorial discussions in the Chandas-śāstra and in Bháskara's combinatorics, one finds implicit arguments about "if you have more patterns than slots, something repeats."

Proof: Consider the partial sums

$$s_k = a_1 + a_2 + \cdots + a_k, \quad k = 1, 2, \ldots, n.$$

There are n such sums. If any s_k is divisible by n, we are done. Otherwise, each s_k has a remainder in $\{1, 2, \ldots, n-1\}$. By the pigeonhole principle, two of the s_i, s_j (say $i < j$) have the same remainder. Then $s_j - s_i$ is divisible by n, and this difference is the sum of the subset $\{a_{i+1}, \ldots, a_j\}$. Hence such a subset always exists. □

Theorem 5.28 *In any group of n people, if friendship is always mutual, then at least two people have the same number of friends.*

Proof: Each person can have between 0 and $n-1$ friends. But it is impossible to have simultaneously one person with 0 friends and another with $n-1$ friends, since the "friend of all" would have to be friends with the "friend of none." Therefore, the possible friend counts are at most $n-1$ distinct numbers. With n people, two must share the same friend count. □

5.3.7 Proof by Exhaustion of cases

Sometimes a proposition cannot be proved in one uniform argument, but instead requires us to split the possible situations into a finite number of cases. A *proof by exhaustion of cases* works by checking each case separately and showing that the desired conclusion holds in all of them [9]. The structure is:

1. Partition the domain of the problem into finitely many exhaustive and mutually exclusive cases.
2. Prove the statement in each case individually using any of the proof techniques.
3. Conclude that the statement holds in general.

This method is indispensable when a property depends on a small number of discrete possibilities, such as parity (even/odd), sign (positive/negative/zero), or congruence classes modulo n.

Let us consider a few examples.

Theorem 5.29 *For any integer n, the number $n^2 + n$ is even.*

Proof: We consider two exhaustive cases:

- **Case 1:** n is even. Then $n = 2k$ for some integer k. So $n^2 + n = (2k)^2 + 2k = 4k^2 + 2k = 2(2k^2 + k)$, which is even.
- **Case 2:** n is odd. Then $n = 2k + 1$ for some integer k. So $n^2 + n = (2k+1)^2 + (2k+1) = 4k^2 + 6k + 2 = 2(2k^2 + 3k + 1)$, which is even.

In either case, $n^2 + n$ is even. Therefore the theorem holds. □

Theorem 5.30 *For any integers a and b, $|a| \cdot |b| = |ab|$.*

Proof: We proceed by cases on the signs of a and b:

- **Case 1:** $a \geq 0$, $b \geq 0$. Then $|a| = a$, $|b| = b$, so $|a||b| = ab = |ab|$.
- **Case 2:** $a \geq 0$, $b < 0$. Then $|a| = a$, $|b| = -b$, and $|a||b| = a(-b) = -ab$. Since $ab < 0$, $|ab| = -ab$, so equality holds.

[9] The earliest history of the method of exhaustion can be traced back to Antiphon of Athens (ca. 480-411 BCE), but the full logical method as a form of a rigorous proof was formalised by Eudoxus of Cnidus (ca 408-355 BCE). There are several examples in Euclid's Elements and the work of Archimedes. Āryabhaṭa (5th c.) and Bháskara II (12th c.) used case-based reasoning to establish divisibility properties and to reduce large problems to smaller congruence classes. In the Bījagaṇita, Bháskara explicitly gives different rules depending on whether numbers are odd or even – an early form of modular case-splitting. The most notable example of proof by case analysis is the modern computer aided proof of the four colour theorem.

- **Case 3:** $a < 0$, $b \geq 0$. Then $|a| = -a$, $|b| = b$, and $|a||b| = (-a)b = -ab$. Again $ab < 0$, so $|ab| = -ab$, equality holds.
- **Case 4:** $a < 0$, $b < 0$. Then $|a| = -a$, $|b| = -b$, so $|a||b| = (-a)(-b) = ab$. Here $ab > 0$, so $|ab| = ab$, equality holds.

In all possible cases, $|a||b| = |ab|$. □

Theorem 5.31 *There is no solution in integers to* $(x^2 - y^2)$ *mod* $4 = 2$.

Proof:

- **Case 1:** x is even and y is even $\Rightarrow x^2 = 4m, y^2 = 4n$ for some integers m and $n \Rightarrow (x^2 - y^2) = 4(m - n)$.
- **Case 2:** x is even and y is odd $\Rightarrow x^2 = 4m, y^2 = 4n + 1$ for some integers m and $n \Rightarrow (x^2 - y^2) = 4(m - n) - 1$.
- **Case 3:** x is odd and y is even $\Rightarrow x^2 = 4m + 1, y^2 = 4n$ for some integers m and $n \Rightarrow (x^2 - y^2) = 4(m - n) + 1$.
- **Case 4:** x is odd and y is odd $\Rightarrow x^2 = 4m + 1, y^2 = 4n + 1$ for some integers m and $n \Rightarrow (x^2 - y^2) = 4(m - n)$.

In all these four cases $(x^2 - y^2) \bmod 4 \neq 2$. □

Theorem 5.32 *An irrational raised to an irrational power may be rational.*

Proof: We already know that $\sqrt{2}$ is irrational. Let $a = \sqrt{2}^{\sqrt{2}}$. Two cases arise.

- **Case 1:** a is rational. Then, the proposition is clearly true.
- **Case 2:** a is irrational. In that case consider $a^{\sqrt{2}} = (\sqrt{2}^{\sqrt{2}})^{\sqrt{2}} = \sqrt{2}^{\sqrt{2}\sqrt{2}} = \sqrt{2}^2 = 2$ which is a rational.

□

When to use case analysis. Proof by exhaustion is most effective when the number of cases is small and natural, such as two parities, three sign possibilities, or finitely many congruence classes modulo n. However, it becomes impractical if the number of cases grows large. As a general strategy, one should strive to minimize the cases by exploiting symmetry, structure, or general arguments.

5.3.8 Proof by Induction

Proof by induction is a powerful technique for proving statements that are asserted to be true for all integers $n \geq n_0$. The method rests on two steps: establishing a base case and showing that if the statement holds for an arbitrary integer k, then it must also hold for $k + 1$.

The Induction Principle

Suppose $P(n)$ is a statement depending on an integer n. To prove that $P(n)$ is true for all integers $n \geq n_0$, we proceed as follows:

1. **Base case:** Verify that $P(n_0)$ is true.
2. **Induction step:** Assume $P(k)$ is true for some arbitrary integer $k \geq n_0$ (this assumption is called the *induction hypothesis*), and then show that $P(k + 1)$ must also be true.

If both steps succeed, then by the principle of mathematical induction, $P(n)$ is true for all integers $n \geq n_0$ [10]. Think of a staircase: show you can step onto the first stair (base case) and that from any stair you can step to the next (inductive step). Then you can reach all stairs.

Let us consider a few examples of proof by induction below.

Theorem 5.33 *For all $n \geq 1$,*

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Proof:
Base case: For $n = 1$, the left-hand side is 1, and the right-hand side is $\frac{1 \cdot 2}{2} = 1$. So the statement holds.
Induction hypothesis: Assume the formula holds for $n = k$, i.e.

$$1 + 2 + \cdots + k = \frac{k(k+1)}{2}.$$

Induction step: Now consider $n = k + 1$:

$$1 + 2 + \cdots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

This is exactly the formula with $n = k + 1$. Hence the result follows. □

Theorem 5.34 *For all $n \geq 1$,*

$$1 + 3 + 5 + \cdots + (2n-1) = n^2.$$

Proof:
Base case: For $n = 1$, the left-hand side is 1 and the right-hand side is $1^2 = 1$.
Induction hypothesis: Assume true for $n = k$, i.e.

$$1 + 3 + \cdots + (2k-1) = k^2.$$

Induction step: Then for $n = k + 1$,

$$1 + 3 + \cdots + (2k-1) + (2(k+1)-1) = k^2 + (2k+1) = (k+1)^2.$$

Thus the claim holds for $n = k + 1$. □

Theorem 5.35 *For all $n \geq 1$, $10^n - 1$ is divisible by 9.*

Proof:
Base case: For $n = 1$, $10^1 - 1 = 9$, which is divisible by 9.
Induction hypothesis: Assume $10^k - 1$ is divisible by 9, i.e., $10^k - 1 = 9m$ for some m.
Induction step: Then

$$10^{k+1} - 1 = 10 \cdot 10^k - 1 = 10(9m+1) - 1 = 90m + 9 = 9(10m+1).$$

Thus $10^{k+1} - 1$ is divisible by 9. □

[10] The idea of proof by induction is ancient, though the formal principle was only stated in the nineteenth century. In the Western tradition, inductive reasoning appears already in Euclid's *Elements* (Book IX, Proposition 8) and in his descent-style proof of prime factorisation (Book VII, Propositions 30–32). Later, Islamic mathematicians such as al-Karaji (10th–11th c.) used inductive arguments for binomial expansions, and in Europe, Maurolico (16th c.) and Pascal (17th c.) applied the method in combinatorics and series. By the time of Euler and Gauss, induction was common in number theory, and Peano (1889) finally codified it as an axiom of the natural numbers. In the Indian tradition, recursive and inductive reasoning also played a central role: Pingala (c. 200 BCE) described prosodic patterns equivalent to Pascal's triangle, and Bhāskara II (12th c.) used stepwise arguments in the *Līlāvatī* and *Bījagaṇita* to establish general formulas. Thus, both traditions employed inductive reasoning long before its modern formal axiomatization.

Theorem 5.36 *For all integers $n \geq 4$, $2^n > n^2$.*

Proof:
Base case: For $n = 4$, $2^4 = 16$ and $4^2 = 16$, so the inequality holds with equality. For $n = 5$, $2^5 = 32 > 25$, so the inequality holds strictly.
Induction hypothesis: Assume $2^k > k^2$ for some $k \geq 5$.
Induction step: Then

$$2^{k+1} = 2 \cdot 2^k > 2 \cdot k^2.$$

Since $k \geq 5$, we have $2k^2 \geq (k+1)^2$. Thus $2^{k+1} > (k+1)^2$. □

Theorem 5.37 *Suppose we have stamps of two different denominations, 3 paise and 5 paise. We want to show that it is possible to make up exactly any postage of 8 paise or more using stamps of these two denominations. Thus we want to show that every positive integer $n \geq 8$ is expressible as $n = 3i + 5j$ where $i, j \geq 0$.*

Proof:
Base case: For $n = 8$, we have $n = 3 + 5$, i.e. $i = j = 1$.
Induction hypothesis: $n = 3i + 5j$ for an $n \geq 8$, $i, j \geq 0$.
Induction step: Consider $n+1$. If $j = 0$ then clearly $i \geq 3$ and we may write $n+1$ as $3(i-3)+5(j+2)$. Otherwise $n + 1 = 3(i + 2) + 5(j - 1)$. □

Strong induction. Strong induction is a variant where we assume the statement holds for all integers up to k and use this to prove it for $k + 1$. Suppose $P(n)$ is a statement depending on an integer n. To prove that $P(n)$ is true for all integers $n \geq n_0$, we proceed as follows:

1. **Base case:** Verify that $P(n_0)$ is true.
2. **Induction step:** Assume $P(m)$ is true for all m, $n_0 \leq m \leq k$ for some arbitrary integer k (this assumption is called the strong version of the *induction hypothesis*), and then show that $P(k + 1)$ must also be true.

Exercise 5.8 *Show that whatever can be proved with strong induction can be proved using the original induction principle, and vice versa.*

Consider the following example.

Theorem 5.38 *Let $F_0 = 0$, $F_1 = 1$, $F_2 = 1, \ldots$ be the Fibonacci sequence where for all $n \geq 2$, $F_n = F_{n-1} + F_{n-2}$. Let $\phi = (1 + \sqrt{5})/2$. We now show that $F_n \leq \phi^{n-1}$ for all positive n.*

Proof:
Base case: For $n = 1$, we have $F_1 = \phi^0 = 1$.
Induction hypothesis: $F_n \leq \phi^{n-1}$ for all m, $1 \leq m \leq n$.
Induction step:

$$\begin{array}{rcll} F_{n+1} & = & F_n + F_{n-1} & \\ & \leq & \phi^{n-1} + \phi^{n-2} & \text{(by the induction hypothesis)} \\ & = & \phi^{n-2}(\phi + 1) & \\ & = & \phi^n & \text{(since } \phi^2 = \phi + 1\text{)} \end{array}$$

□

Theorem 5.39 (Fundamental Theorem of Arithmetic) *Every integer $n > 1$ can be written as a product of primes, and this representation is unique up to reordering of the primes.*

Proof: *Existence:* We proceed by induction on n. For $n = 2$, the result holds since 2 is prime. Assume every integer m with $2 \leq m < n$ has a prime factorisation. If n is prime, we are done. Otherwise, $n = ab$ with $1 < a, b < n$. By the induction hypothesis, both a and b factor into primes. Thus n factors into primes.

Uniqueness: Suppose

$$n = p_1 p_2 \cdots p_r = q_1 q_2 \cdots q_s,$$

with all p_i, q_j prime. Since $p_1 \mid q_1 q_2 \cdots q_s$, Euclid's Lemma implies $p_1 \mid q_j$ for some j. As both are prime, $p_1 = q_j$. Cancelling p_1 from both sides and repeating the argument gives uniqueness by induction on the number of prime factors. □

Exercise 5.9 *Complete the uniqueness part of the proof of the unique prime factorization theorem more formally using the principle of mathematical induction.*

Theorem 5.40 (Tiling a Deficient Chessboard with Trominoes) *A* tromino *is an L-shaped tile made from three unit squares, as shown in Figure 5.4. Rotations and reflections of the tile are also allowed.*

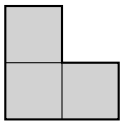


Figure 5.4: An L-shaped tromino consisting of three unit squares.

Suppose one square is removed from a $2^n \times 2^n$ chessboard, where $n \geq 1$. Show that the remaining board can always be tiled completely using L-shaped trominoes, irrespective of which square is removed.

Proof: Let $P(n)$ denote the statement

$P(n)$: Every $2^n \times 2^n$ board with one square removed can be tiled with L-shaped trominoes.

We shall prove $P(n)$ by mathematical induction.

Base case. For $n = 1$, the board has size

$$2^1 \times 2^1 = 2 \times 2.$$

It therefore consists of four unit squares. If any one square is removed, the remaining three squares form exactly one L-shaped tromino. Hence $P(1)$ is true.

Induction hypothesis. Suppose that $P(n)$ is true for some $n \geq 1$. Thus we assume that *any* $2^n \times 2^n$ board with one square removed can be tiled completely using L-shaped trominoes.

Inductive step. We must show that $P(n+1)$ is true. Consider a

$$2^{n+1} \times 2^{n+1}$$

board from which one arbitrary square has been removed.

Divide the board into four equal quadrants. Each quadrant is a $2^n \times 2^n$ board. The missing square must lie in exactly one of these four quadrants.

Without loss of generality, suppose that it lies in the top-left quadrant, as illustrated in Figure 5.5.

Now comes the crucial observation. The top-left quadrant already has one square missing. Place a single L-shaped tromino at the centre of the board so that it occupies one central square in *each of the other three quadrants.*

After placing this central tromino:

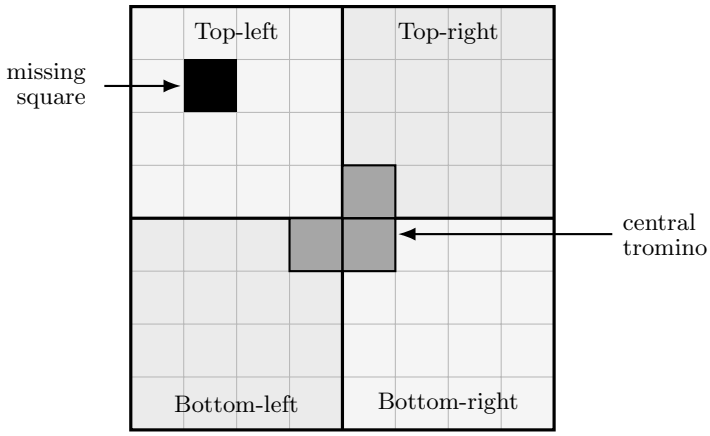


Figure 5.5: The $2^{n+1} \times 2^{n+1}$ board is divided into four $2^n \times 2^n$ quadrants. The original missing square is shown in black in the top-left quadrant. The shaded central L-shaped tromino occupies one square in each of the other three quadrants.

1. the top-left quadrant has exactly one square missing—the square that was originally removed;
2. the top-right quadrant has exactly one square unavailable—the square occupied by the central tromino;
3. the bottom-left quadrant similarly has exactly one square unavailable; and
4. the bottom-right quadrant also has exactly one square unavailable.

Thus we have transformed the original problem into four smaller problems of precisely the form covered by the induction hypothesis. Each quadrant is a $2^n \times 2^n$ board with one square missing.

By the induction hypothesis, each of the four quadrants can therefore be tiled completely with L-shaped trominoes. Together with the one tromino that we have already placed at the centre, these tilings cover the entire $2^{n+1} \times 2^{n+1}$ board except for the originally removed square.

Hence

$$P(n) \Longrightarrow P(n+1).$$

Since $P(1)$ is true, the Principle of Mathematical Induction gives

Every $2^n \times 2^n$ board with one square removed can be tiled completely with L-shaped trominoes.

□

Remarks.

- Induction proofs are especially useful for formulas involving sums, products, inequalities, or divisibility.
- The induction hypothesis is a temporary assumption used to prove the next case; it is not assumed globally.
- Strong induction is a variant where we assume the statement holds for all integers up to k and use this to prove it for $k+1$.

5.3.9 Proofs of Equivalence

Often in mathematics we want to prove a statement of the form

$$P \iff Q,$$

which reads "P if and only if Q" (abbreviated "P iff Q"). This means both $P \Rightarrow Q$ and $Q \Rightarrow P$ hold. To prove such an equivalence, we usually split the work into two parts:

1. Prove the *forward implication* $P \Rightarrow Q$.
2. Prove the *reverse implication* $Q \Rightarrow P$.

Equivalences require a special discussion because sometimes the two directions may use very different arguments. Consider the following examples.

Theorem 5.41 *An integer n is even $\iff$ n^2 is even.*

Proof: ($\Rightarrow$) Suppose n is even, say $n = 2k$. Then $n^2 = (2k)^2 = 4k^2 = 2(2k^2)$ is even.
($\Leftarrow$) Conversely, suppose n^2 is even. If n were odd, say $n = 2k + 1$, then $n^2 = (2k + 1)^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$, which is odd — a contradiction. Hence n must be even. □

Theorem 5.42 *The Well-Ordering Principle for $\mathbb{N}$ (Section 2.3.1) and the Principle of Mathematical Induction are equivalent.*

Proof:
($\Rightarrow$) Well-ordering implies mathematical induction
Assume the Well-Ordering Principle.
Suppose that $P(n)$ is a statement satisfying $P(0)$ and $P(n) \Longrightarrow P(n + 1)$ for every $n \in \mathbb{N}$.
We want to prove that $P(n)$ is true for every $n \in \mathbb{N}$.
Suppose, to the contrary, that this is not so. Then there is at least one natural number for which $P(n)$ is false. Consider the set

$$C = \{n \in \mathbb{N} : P(n) \text{ is false}\}.$$

Our assumption says that C is non-empty. By the Well-Ordering Principle, C therefore has a least element. Call it m.
Since $P(0)$ is true,

$$m \neq 0.$$

Hence $m > 0$, and therefore $m - 1 \in \mathbb{N}$.
Since m is the *least* member of C, $m - 1$ cannot belong to C. Consequently,

$$P(m - 1)$$

must be true.
But the inductive condition tells us that

$$P(m - 1) \Longrightarrow P(m).$$

Therefore $P(m)$ is true.
This contradicts the fact that $m \in C$, which says that $P(m)$ is false.
Hence C must be empty. There are no counterexamples, and therefore

$$P(n) \text{ is true for every } n \in \mathbb{N}.$$

Thus the Well-Ordering Principle implies the Principle of Mathematical Induction.
Notice the idea behind this argument. If induction were to fail, there would have to be a *first* place where it fails. But if the statement is true immediately before that first failure, the inductive step says that it must also be true at the supposed first failure. This is impossible.

(⇐)Mathematical induction implies well-ordering

Now assume the Principle of Mathematical Induction.

We want to prove the Well-Ordering Principle. Suppose, to the contrary, that it is false. Then there exists some non-empty set

$$S \subseteq \mathbb{N}$$

that has no least element.

We shall use induction to show that no natural number can belong to S. This will contradict the assumption that S is non-empty.

Define the statement

$$P(n): \qquad 0, 1, \ldots, n \notin S.$$

In other words, $P(n)$ says that none of the natural numbers from 0 through n belongs to S.

Base case. We claim that

$$0 \notin S.$$

For if $0 \in S$, then 0 would automatically be the least element of S, since no natural number is smaller than 0. This contradicts our assumption that S has no least element.

Therefore $P(0)$ is true.

Inductive step. Suppose $P(n)$ is true. Thus

$$0, 1, \ldots, n \notin S.$$

We must show that $P(n+1)$ is true.

Suppose instead that

$$n + 1 \in S.$$

Since none of

$$0, 1, \ldots, n$$

belongs to S, there can be no element of S smaller than $n + 1$. Therefore $n + 1$ would be the least element of S.

But we assumed that S has no least element. Hence

$$n + 1 \notin S.$$

Thus

$$P(n) \Longrightarrow P(n + 1).$$

By the Principle of Mathematical Induction, $P(n)$ is true for every $n \in \mathbb{N}$. Hence

$$n \notin S \qquad \text{for every } n \in \mathbb{N}.$$

Therefore

$$S = \emptyset.$$

But this contradicts our assumption that S is non-empty. Consequently, every non-empty subset of $\mathbb{N}$ must have a least element.

Thus mathematical induction implies the Well-Ordering Principle. □

Remark. The two principles describe the same fundamental structure of the natural numbers from different viewpoints.

The Well-Ordering Principle looks *backwards*: if something happens somewhere among the natural numbers, there must be a *first* place where it happens.

Mathematical induction looks *forwards*: start at 0, and if a property can always be carried from n to $n+1$, then it propagates through all the natural numbers.

This also explains why the method of the *smallest counterexample* is so closely related to induction. To prove that $P(n)$ holds for all n, suppose that counterexamples exist. Well-ordering gives us a smallest counterexample. We then use what is true for smaller numbers to show that this supposed smallest counterexample cannot actually be one.

Thus induction, well-ordering, and proofs using a smallest counterexample are not unrelated tricks. They are different manifestations of the same underlying structure of the natural numbers.

5.3.10 Equivalence of Several Statements: Circular Implication

We have seen that to prove two statements P and Q equivalent, we prove both directions

$$P \Longrightarrow Q \qquad \text{and} \qquad Q \Longrightarrow P.$$

What should we do if we want to prove that three statements P, Q, and R are all equivalent?

One possibility would be to prove every implication separately:

$$\begin{array}{lll} P \Rightarrow Q, & P \Rightarrow R, & Q \Rightarrow P, \\ Q \Rightarrow R, & R \Rightarrow P, & R \Rightarrow Q. \end{array}$$

But this is unnecessary. It is enough to prove the three implications

$$P \Longrightarrow Q, \qquad Q \Longrightarrow R, \qquad R \Longrightarrow P.$$

These form a *circular implication*:

$$\boxed{P \Longrightarrow Q \Longrightarrow R \Longrightarrow P.}$$

Why is this sufficient? For example, $P \Rightarrow R$ follows by combining

$$P \Rightarrow Q \Rightarrow R,$$

while $Q \Rightarrow P$ follows from

$$Q \Rightarrow R \Rightarrow P.$$

Similarly,

$$R \Rightarrow P \Rightarrow Q$$

gives $R \Rightarrow Q$. Thus every statement implies every other statement, and hence

$$P \Longleftrightarrow Q \Longleftrightarrow R.$$

This method is particularly useful when different links in the circle are naturally proved in different ways.

Example 1: Three characterizations involving even integers

Let $n \in \mathbb{Z}$. Consider the following three statements:

$$\begin{aligned} P: &\quad n \text{ is even}, \\ Q: &\quad n^2 \text{ is even}, \\ R: &\quad n^2 + 2n \text{ is even}. \end{aligned}$$

We shall prove that

$$P \Longleftrightarrow Q \Longleftrightarrow R$$

by establishing the circular implication

$$P \Longrightarrow Q \Longrightarrow R \Longrightarrow P.$$

$P \Rightarrow Q$. Suppose n is even. Then

$$n = 2k$$

for some $k \in \mathbb{Z}$. Therefore

$$n^2 = (2k)^2 = 4k^2 = 2(2k^2),$$

so n^2 is even. Hence

$$P \Longrightarrow Q.$$

$Q \Rightarrow R$. Suppose n^2 is even. Since $2n$ is also even,

$$n^2 + 2n$$

is the sum of two even integers and is therefore even. Hence

$$Q \Longrightarrow R.$$

$R \Rightarrow P$. For this direction it is convenient to prove the contrapositive. Suppose n is odd. Then $n + 2$ is also odd. Since

$$n^2 + 2n = n(n + 2),$$

and the product of two odd integers is odd, $n^2 + 2n$ is odd.

Thus

$$n \text{ odd} \Longrightarrow n^2 + 2n \text{ odd}.$$

Taking the contrapositive,

$$n^2 + 2n \text{ even} \Longrightarrow n \text{ even}.$$

Hence

$$R \Longrightarrow P.$$

We have therefore completed the circle:

$$\boxed{n \text{ even} \Longrightarrow n^2 \text{ even} \Longrightarrow n^2 + 2n \text{ even} \Longrightarrow n \text{ even}.}$$

Consequently,

$$\boxed{n \text{ is even} \iff n^2 \text{ is even} \iff n^2 + 2n \text{ is even}.}$$

Notice that the three parts of the proof need not use the same proof technique. The first implication follows directly from the definition of an even integer, the second uses the fact that the sum of two even integers is even, and the third is most naturally proved by contrapositive. Circular implication tells us how these separate arguments can be assembled into a single proof of equivalence.

Example 2: Divisibility by 6

Here is a slightly more substantial example. Let $n \in \mathbb{Z}$ and consider

$$\begin{aligned} P: &\quad 6 \mid n, \\ Q: &\quad 2 \mid n \text{ and } 3 \mid n, \\ R: &\quad 6 \mid n^2. \end{aligned}$$

We shall again prove

$$P \Longleftrightarrow Q \Longleftrightarrow R$$

by establishing

$$P \Longrightarrow Q \Longrightarrow R \Longrightarrow P.$$

$P \Rightarrow Q$. Suppose $6 \mid n$. Then

$$n = 6k$$

for some $k \in \mathbb{Z}$. But

$$n = 2(3k) \qquad \text{and} \qquad n = 3(2k).$$

Therefore

$$2 \mid n \qquad \text{and} \qquad 3 \mid n.$$

Hence

$$P \Longrightarrow Q.$$

$Q \Rightarrow R$. Suppose

$$2 \mid n \qquad \text{and} \qquad 3 \mid n.$$

Since 2 and 3 are relatively prime, this implies

$$6 \mid n.$$

Thus $n = 6k$ for some $k \in \mathbb{Z}$, and consequently

$$n^2 = 36k^2 = 6(6k^2).$$

Hence

$$6 \mid n^2,$$

and therefore

$$Q \Longrightarrow R.$$

$R \Rightarrow P$. Suppose

$$6 \mid n^2.$$

Then

$$2 \mid n^2 \qquad \text{and} \qquad 3 \mid n^2.$$

We use the fact that if p is prime and

$$p \mid a^2,$$

then

$$p \mid a.$$

Since both 2 and 3 are prime, it follows that

$$2 \mid n \qquad \text{and} \qquad 3 \mid n.$$

Since 2 and 3 are relatively prime,

$$6 \mid n.$$

Therefore

$$R \Longrightarrow P.$$

We have again completed the circle:

$$\boxed{6 \mid n \Longrightarrow (2 \mid n \text{ and } 3 \mid n) \Longrightarrow 6 \mid n^2 \Longrightarrow 6 \mid n.}$$

Thus

$$\boxed{6 \mid n \quad \Longleftrightarrow \quad (2 \mid n \text{ and } 3 \mid n) \quad \Longleftrightarrow \quad 6 \mid n^2.}$$

This example requires two elementary facts about divisibility:

$$\gcd(2,3) = 1 \quad \Longrightarrow \quad (2 \mid n \text{ and } 3 \mid n) \Longrightarrow 6 \mid n,$$

and, for a prime p,

$$p \mid n^2 \Longrightarrow p \mid n.$$

The latter is a consequence of Euclid's Lemma.

Circular implication in general

The same idea extends beyond three statements. Suppose we want to show that

$$P_1, P_2, \ldots, P_k$$

are all equivalent. It is sufficient to establish the cycle

$$\boxed{P_1 \Longrightarrow P_2 \Longrightarrow P_3 \Longrightarrow \cdots \Longrightarrow P_k \Longrightarrow P_1.}$$

Starting from any one of these statements, we can follow the arrows around the circle to reach any of the others. Hence every statement implies every other statement.

This can substantially reduce the amount of work required. If we tried to prove every directed implication separately, there would be

$$k(k-1)$$

implications to prove. A circular proof requires only

$$k$$

implications.

The important point, however, is not merely that circular implication saves work. It allows us to choose the *most natural direction* between successive statements. One implication may have a simple direct proof, another may follow from an already established theorem, and another may be easiest to prove by contrapositive. The resulting circle then establishes the equivalence of all the statements.

5.3.11 Conclusion

There is no single "best" technique; experienced problem solvers try viewpoints. If definitions seem aligned, push a direct proof. If "not Q" feels structured, flip to a contrapositive. If a statement ranges over integers, try induction. If you suspect a claim is too strong, hunt a counter-example. And whenever "assuming the opposite" quickly generates an impossibility, contradiction is often the sharpest tool.

Problems

1. **Truth Tables**
   (a) Construct the truth table for $(p \wedge q) \vee (\neg r)$.
   (b) Verify by a truth table that $(p \Rightarrow r) \wedge (q \Rightarrow r)$ is equivalent to $(p \vee q) \Rightarrow r$.
   (c) Determine whether $(p \Rightarrow q) \Rightarrow r$ and $p \Rightarrow (q \Rightarrow r)$ are equivalent.
   (d) Show by truth table that $p \vee (q \wedge r)$ is not equivalent to $(p \vee q) \wedge (p \vee r)$.

2. **Three-variable Logic**
   (a) Construct the truth table for $(p \oplus q) \oplus r$ and check whether it is associative.
   (b) Verify that $\neg(p \wedge q \wedge r)$ is equivalent to $\neg p \vee \neg q \vee \neg r$.
   (c) Show that $(p \Rightarrow q) \wedge (q \Rightarrow r)$ does not imply $(p \Rightarrow r)$ by giving a truth table.
   (d) Identify all assignments of p, q, r for which $(p \vee q) \Rightarrow (q \vee r)$ is false.

3. **Vacuous Truth**
   (a) State a universally quantified claim about an empty set and explain why it is true.
   (b) Explain why the statement "If n is an integer with $n^2 = -1$, then n is prime" is vacuously true.

(c) Let $A = \varnothing$. Prove that "For all $x \in A$, $x^2 = 0$" is vacuously true.

(d) Formulate a vacuous truth involving divisibility (e.g. "All integers divisible by both 2 and 3 and equal to 5 are even"), and justify why it is vacuously true.

4. **Proof by Explicit Construction**

(a) Find integers x, y such that $17x + 29y = 1$.

(b) Exhibit an explicit Pythagorean triple different from $(3, 4, 5)$ and $(5, 12, 13)$.

(c) Construct a rational number between $\frac{7}{9}$ and $\frac{8}{9}$.

(d) Show by construction that there exists an integer solution to $12x + 18y = 6$.

5. **Proof by Counter-Example**

(a) Disprove: "For all integers n, $n^2 - n + 41$ is prime."

(b) Disprove: "For all integers n, $n^3 + 2$ is prime."

(c) Disprove: "Every integer greater than 2 is the sum of two primes." (Give a counterexample.)

(d) Find a counterexample to: "If a divides bc, then a divides b."

6. **Direct Proof**

(a) Prove that the product of two even integers is even.

(b) Prove that if a is divisible by 12 and b is divisible by 6, then $a + b$ is divisible by 6.

(c) Show directly that if n is a multiple of 4, then n^2 is a multiple of 16.

(d) Prove directly that if $a \equiv b \pmod{m}$ and $c \equiv d \pmod{m}$, then $a + c \equiv b + d \pmod{m}$.

7. **Proof by Contradiction**

(a) Prove by contradiction that $\sqrt{7}$ is irrational.

(b) Show by contradiction that there is no integer solution to $2x = 2y + 1$.

(c) Prove by contradiction that there are infinitely many primes congruent to 1 modulo 4.

(d) Prove by contradiction that $\sqrt[3]{2}$ is irrational.

8. **Proof by Contrapositive**

(a) Prove: If n^2 is divisible by 9, then n is divisible by 3.

(b) Prove: If ab is even, then at least one of a or b is even.

(c) Prove: If n^2 is divisible by 8, then n is divisible by 2.

(d) Prove: If x^2 is divisible by 12, then x is divisible by 6.

9. **Proof by the Pigeonhole Principle**

(a) Show that among any 8 integers, two leave the same remainder modulo 7.

(b) Prove that in a group of 32 people, at least two share the same day of the month for their birthday.

(c) Show that in any set of 6 integers, at least three must have the same parity.

(d) Prove that in any group of 367 people, at least two have the same exact birthday (ignoring leap years).

10. **Proof by Exhaustion of Cases**

(a) Prove that the cube of any integer is congruent to $-1, 0$, or 1 modulo 7.

(b) Show that if n is an integer, then $n^2 \equiv 0, 1, 4 \pmod{5}$.

(c) By checking cases, show that for any integer n, $n^2 - n$ is even.

(d) Prove by exhaustion of parities that the product of two consecutive integers is even.

11. **Proofs of Equivalence**

(a) Prove that n is divisible by 2 if and only if n^2 is divisible by 4.

(b) Prove that $a \equiv b \pmod{n}$ if and only if a and b leave the same remainder when divided by n.

(c) Prove that an integer n is divisible by 11 if and only if the alternating sum of its digits is divisible by 11.

(d) Prove that a and b are both even if and only if $a + b$ is even and $a - b$ is even.

12. **Proof by Induction**

(a) Find the fallacy in the following proof by **PMI**.
Theorem Given any collection of n blonde girls. If at least one of the girls has blue eyes, then all n of them have blue eyes.
Proof: The statement is obviously true for $n = 1$. The step from k to $k + 1$ can be illustrated by going from $n = 3$ to $n = 4$. Assume, therefore, that the statement is true for $n = 3$ and let G_1, G_2, G_3, G_4 be four blonde girls, at least one of which, say G_1, has blue eyes. Taking G_1, G_2, and G_3 together and using the fact that the statement is true when $n = 3$, we find that G_2 and G_3 also have blue eyes. Repeating the process with G_1, G_2 and G_4, we find that G_4 has blue eyes. Thus all four have blue eyes. A similar argument allows us to make the step from k to $k + 1$ in general. □
Corollary. All blonde girls have blue eyes.
Proof: Since there exists at least one blonde girl with blue eyes, we can apply the foregoing result to the collection consisting of all blonde girls. □
Note: This example is from G. Pólya, who suggests that the reader may want to test the validity of the statement by experiment.

(b) Suban announces to the QRMT class:
"There will be a surprise test next week. You will not know in advance on which day it will be held."
One student in the class reasons as follows, by induction on $n = 5 - k$:

- The test cannot be on Friday (the last working day of the week, $k = 5, n = 0$), because if it hasn't happened before then, the class would know it must be on Friday — and so it would not be a surprise.
- Similarly, the test cannot be on Thursday, because if it hasn't happened before then, and Friday has already been ruled out, the class would know it must be on Thursday — and so it would not be a surprise.
- Continuing this reasoning backwards, the test cannot be on Wednesday, Tuesday, or Monday either.
- Therefore, the teacher cannot give a surprise test at all!

Yet, when Suban gives the test on Wednesday, the students are indeed surprised. Identify the flaw in the student's reasoning.

(c) Prove by induction that for all integers $n \geq 1$,

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

(d) Show by induction that $7^n - 1$ is divisible by 6 for all integers $n \geq 1$.

(e) Prove that for all $n \geq 1$, $n^3 - n$ is divisible by 6.

(f) Prove that for all integers $n \geq 4$,

$$n! > 2^n.$$

(g) Show that for all integers $n \geq 1$,

$$3^n \geq n^3.$$

(h) Prove using strong induction that every integer $n > 1$ can be written as a product of prime numbers.

(i) A tiling problem: Prove by strong induction that any $2^n \times 2^n$ chessboard with one square removed can be covered completely by L-shaped trominoes (three-square pieces).

Chapter 6

Induction gives us algorithms



"programming is as hard or as easy as proving"

Edsger W. Dijkstra
The next fifty years (excerpt)

Photo: © 2002 Hamilton Richards;
E. W. Dijkstra Archive, UT Austin.



"The Analytical Engine weaves algebraic patterns just as the Jacquard loom weaves flowers and leaves."

Ada Lovelace, 1843
As reproduced by the Computer History Museum

Portrait, c. 1838: Science Museum;
via the Computer History Museum.

In the chapter on proofs, induction allowed us to establish infinitely many claims using a base case and an inductive step. A closely related idea lets us *compute*: give the answer for a base case, and describe how to obtain a new answer from answers to smaller problems. We have already done this when we defined addition using the successor function. We now make that idea into a model of computation.

6.1 Why do we need a model of computation?

An algorithm is a finite, unambiguous description of a procedure for obtaining an answer from an input. For an algorithm that solves a problem on a stated domain, the procedure must terminate and give the right answer on every input in that domain. A description such as "find the answer" is not enough: we must say what operations are available and how to combine them.

A model of computation supplies these rules. It separates the mathematical procedure from the machine, programming language, or representation used to carry it out. It also makes three different questions precise: what answer is required, whether a procedure computes that answer, and how much work the procedure takes. Two procedures can compute the same function with very different amounts of work.

Our starting point is a function $f : \mathbb{N} \to \mathbb{N}$, where, as earlier, $\mathbb{N} = \{0, 1, 2, \ldots\}$. An input is a natural number and a terminating computation produces a natural number. We freely use functions of several arguments, such as add $: \mathbb{N}^2 \to \mathbb{N}$. Using several arguments makes the notation easier to read. Pairs can be encoded as single natural numbers; for example,

$$\langle a, b \rangle = \frac{(a+b)(a+b+1)}{2} + b.$$

This lists pairs by successive diagonals of $\mathbb{N}^2$. Its encoding and decoding are effective finite procedures. Repeating the construction encodes tuples. Finite strings, lists, and other finite data can likewise be represented by natural numbers. This explains why numerical functions are a useful starting point even when the eventual application concerns words or pictures.[1]

6.2 Functions defined by induction

Recall the successor function $S(n) = n + 1$. We may start with zero, successor, and functions that select an argument, and build new functions by *composition* and *recursion*. Composition means using the output of one function as the input of another: $h(n) = g(f(n))$. A basic recursion scheme is

$$f(\mathbf{x}, m) = \begin{cases} g(\mathbf{x}), & \text{if } m = 0, \\ h(\mathbf{x}, n, f(\mathbf{x}, n)), & \text{otherwise.} \end{cases}$$

Here $\mathbf{x}$ denotes a tuple of additional arguments, and g and h are functions already constructed. In the otherwise branch, n denotes the unique predecessor of m, so $m = S(n)$; this branch is used only when $m > 0$. The recursive call decreases the last argument by one. These rules determine a unique value on every input, provided the computations of g and h always finish on the inputs used. This scheme is called *primitive recursion*.

6.2.1 Addition and multiplication

Write $\operatorname{pred}(b)$ for the predecessor of a positive b, so $\operatorname{pred}(S(n)) = n$. It is used only in the nonzero branch. To compute $a + b$, begin with $a + 0 = a$. If we can compute $a + n$, applying successor gives $a + S(n)$. This proof pattern gives the algorithm

$$\operatorname{add}(a, b) = \begin{cases} a, & \text{if } b = 0, \\ S(\operatorname{add}(a, \operatorname{pred}(b))), & \text{otherwise.} \end{cases}$$

Correctness, by induction on b. *Basis:* $\operatorname{add}(a, 0) = a = a + 0$ for every a. *Induction hypothesis:* for an arbitrary integer $n \geq 0$ and every $a \in \mathbb{N}$, $\operatorname{add}(a, n) = a + n$. *Inductive step:*

$$\operatorname{add}(a, S(n)) = S(\operatorname{add}(a, n)) = S(a + n) = a + S(n).$$

The call decreases b to its predecessor, so it also terminates.

For multiplication, the base product is zero. If we can compute an, one more addition of a gives $aS(n)$. Thus

$$\operatorname{mul}(a, b) = \begin{cases} 0, & \text{if } b = 0, \\ \operatorname{add}(\operatorname{mul}(a, \operatorname{pred}(b)), a), & \text{otherwise.} \end{cases}$$

Correctness, by induction on b. *Basis:* $\operatorname{mul}(a, 0) = 0 = a \cdot 0$. *Induction hypothesis:* for an arbitrary integer $n \geq 0$, $\operatorname{mul}(a, n) = an$ for every $a \in \mathbb{N}$. *Inductive step:* using the already established correctness of addition,

$$\operatorname{mul}(a, S(n)) = \operatorname{add}(\operatorname{mul}(a, n), a) = an + a = aS(n).$$

Again, the second argument decreases to zero. For example, $\operatorname{mul}(4, 3)$ computes $4 + 4 + 4$ through three additions to an initial zero.

A notational simplification. From here onwards, we write $n + 1$ instead of $S(n)$, and $n - 1$ for the predecessor when $n > 0$. We also use ordinary addition and multiplication symbols for the functions just constructed. These symbols abbreviate the computations we have just defined.

[1] Ada Lovelace met Charles Babbage in 1833, when she was seventeen, and saw him demonstrate a working section of his calculating engine. In her 1843 notes she went beyond arithmetic to envisage operations on other kinds of symbols, even the composition of music. See the Computer History Museum's account.

6.2.2 Conditions, specifications, and algorithms

Our specification language also includes *if–then–else*. For example,

$$K(x) = \begin{cases} u(x), & \text{if } C(x), \\ v(x), & \text{otherwise.} \end{cases}$$

First evaluate the condition $C(x)$. If it is true, compute $u(x)$; otherwise compute $v(x)$. Only the selected branch is evaluated. With more than two cases, test the conditions in order and use the first that holds; "otherwise" covers the remaining inputs. For this to be an algorithm, the test itself must have a definite computational procedure. The curly braces alone do not make a mathematical specification executable.

For example, consider the following specification:

$$\operatorname{sqrt}(n) = \begin{cases} m, & \text{if some } m \in \mathbb{N} \text{ satisfies } m^2 = n, \\ 0, & \text{otherwise.} \end{cases}$$

The natural-number root, when it exists, is unique. This is a valid specification: return the exact root for a perfect square and zero otherwise. It also returns zero for $n = 0$; zero is therefore not a distinct failure code. But the first condition neither says how to find m nor how to establish that none exists. This description states the required answer without yet providing a method of computing it.

A procedure can start with a candidate $m = 0$ and test successive candidates:

$$R(n, m) = \begin{cases} m, & \text{if } m^2 = n, \\ 0, & \text{if } m^2 > n, \\ R(n, m+1), & \text{otherwise.} \end{cases}$$

Evaluate $R(n, 0)$. Each test is computable using multiplication and comparison. Candidates smaller than m have already been checked; once $m^2 > n$, no larger candidate can succeed. The search stops no later than $m = n + 1$. Thus it returns exactly the value specified above. For example, $R(9, 0) = R(9, 1) = R(9, 2) = R(9, 3) = 3$, whereas the search for 8 stops at $m = 3$ and returns zero. The distinction is between a condition describing an answer and instructions that actually find it.

We read our algorithmic equations as evaluation rules from left to right. An equation such as $f(n) = f(n)$, by itself, gives no procedure for finding a value. Neither a familiar function name nor mathematically valid notation supplies the missing computational steps.

An algorithm can be read from a proof. The examples suggest a method of construction. Rather than guess a sequence of steps, start with what is to be computed. Find a base case that can be solved directly. Then assume that a smaller instance can be solved, and show how its answer gives the answer to the current instance. The algorithm is a *computational rendering of this inductive proof*: the base case becomes a stopping rule, the induction hypothesis becomes a recursive call, and the inductive step becomes the computation using its result. Finding a useful induction may require ingenuity, but the proof guides the construction. [2]

A recursive call need not decrease an argument by exactly one. It may decrease some other natural-number measure. The Well-Ordering Principle rules out an infinite descent; induction on that measure proves correctness. In the examples below, we make both the measure and the proof explicit.

6.2.3 Quotient and remainder

For $b > 0$, the division theorem gives $a = bq + r$, with $0 \le r < b$. We use *induction on the quotient* q to construct the algorithms. When $q = 0$, $a < b$, so the quotient is zero and the remainder is a.

[2] Dijkstra once illustrated the limits of exhaustive testing with his university's multiplication hardware: at the quoted speed, trying every possible pair of inputs would take roughly 30,000 years. His point was that understanding why a procedure works can cover cases that we will never have time to test individually. See EWD303, "On the reliability of programs".

If $a = b(q+1)+r$, subtracting b leaves $bq+r$: solve this smaller-quotient problem, add one to its quotient, and keep its remainder. This gives

$$\operatorname{div}(a,b) = \begin{cases} 0, & \text{if } a < b, \\ \operatorname{div}(a-b,b)+1, & \text{otherwise,} \end{cases}$$

$$\operatorname{mod}(a,b) = \begin{cases} a, & \text{if } a < b, \\ \operatorname{mod}(a-b,b), & \text{otherwise.} \end{cases}$$

Subtraction and comparison are the natural-number operations introduced earlier. Here $a-b$ is used only when $a \geq b$.

Correctness, by induction on q. The precise claim is: for every $q \geq 0$, every $b > 0$, and every $0 \leq r < b$,

$$\operatorname{div}(bq+r,b) = q, \qquad \operatorname{mod}(bq+r,b) = r.$$

Basis: when $q = 0$, the first argument is $r < b$, so the two base branches return 0 and r. *Induction hypothesis:* for an arbitrary integer $q \geq 0$, both claims hold for all $b > 0$ and $0 \leq r < b$. *Inductive step:* for $a = b(q+1)+r$, we have $a \geq b$ and $a - b = bq + r$. Consequently,

$$\begin{aligned} \operatorname{div}(a,b) &= \operatorname{div}(bq+r,b)+1 = q+1, \\ \operatorname{mod}(a,b) &= \operatorname{mod}(bq+r,b) = r. \end{aligned}$$

Each call reduces the quotient by one, so after q calls the base case is reached. The algorithm need not *know* q in advance: q is the measure used in its proof, and $a < b$ detects the stopping case. For example,

$$\begin{aligned} \operatorname{div}(17,5) &= \operatorname{div}(12,5)+1 = (\operatorname{div}(7,5)+1)+1 = 3, \\ \operatorname{mod}(17,5) &= \operatorname{mod}(12,5) = \operatorname{mod}(7,5) = \operatorname{mod}(2,5) = 2. \end{aligned}$$

Division by zero is excluded from the domain.

6.2.4 Powers and factorials

For $x, n \in \mathbb{N}$, the identities $x^0 = 1$ and $x^n = x^{n-1}x$ suggest

$$\operatorname{pow}(x,n) = \begin{cases} 1, & \text{if } n = 0, \\ \operatorname{pow}(x,n-1)\cdot x, & \text{otherwise.} \end{cases}$$

Correctness, by induction on n. *Basis:* $\operatorname{pow}(x,0) = 1 = x^0$. *Induction hypothesis:* for an arbitrary integer $k \geq 0$, $\operatorname{pow}(x,k) = x^k$ for every $x \in \mathbb{N}$. *Inductive step:* $\operatorname{pow}(x,k+1) = \operatorname{pow}(x,k)x = x^k x = x^{k+1}$. We adopt $0^0 = 1$ in this discrete definition, consistent with the empty product. The exponent decreases to zero.

Similarly, $0! = 1$ and $n! = n(n-1)!$ give

$$\operatorname{fact}(n) = \begin{cases} 1, & \text{if } n = 0, \\ n \cdot \operatorname{fact}(n-1), & \text{otherwise.} \end{cases}$$

Correctness, by induction on n. *Basis:* $\operatorname{fact}(0) = 1 = 0!$. *Induction hypothesis:* for an arbitrary integer $k \geq 0$, $\operatorname{fact}(k) = k!$. *Inductive step:*

$$\operatorname{fact}(k+1) = (k+1)\operatorname{fact}(k) = (k+1)k! = (k+1)!.$$

The argument decreases to zero, proving termination. In each example, the same induction both suggests the algorithm and establishes its correctness.

6.2.5 Unfolding recursion: work that waits

Let us follow the factorial algorithm exactly, at the level where multiplication is available. Each recursive call leaves a multiplication waiting for its answer:

$$\begin{aligned}
\text{fact}(5) &= 5 \cdot \text{fact}(4) \\
&= 5 \cdot (4 \cdot \text{fact}(3)) \\
&= 5 \cdot (4 \cdot (3 \cdot \text{fact}(2))) \\
&= 5 \cdot (4 \cdot (3 \cdot (2 \cdot \text{fact}(1)))) \\
&= 5 \cdot (4 \cdot (3 \cdot (2 \cdot (1 \cdot \text{fact}(0))))) \\
&= 5 \cdot (4 \cdot (3 \cdot (2 \cdot (1 \cdot 1)))) \\
&= 5 \cdot (4 \cdot (3 \cdot (2 \cdot 1))) \\
&= 5 \cdot (4 \cdot (3 \cdot 2)) \\
&= 5 \cdot (4 \cdot 6) \\
&= 5 \cdot 24 \\
&= 120.
\end{aligned}$$

The expression first grows as the calls descend to zero. No multiplication can yet be completed under these evaluation rules: the inner answer is not available. After the base value 1 is obtained, the expression shrinks as the pending multiplications are performed. These are *deferred computations*. A direct evaluator remembers the unfinished calls in a *stack*.

Contrast this with computing the successive products $1, 5, 20, 60, 120, 120$ as we count down from 5 to 0. Here every multiplication can be completed immediately, and its result carried forward. In Section 6.4 we will derive the tail-recursive algorithm for this second process and trace the same input. Both function definitions are recursive; only the first process accumulates deferred multiplications.

6.2.6 Summing the values of a function

Suppose a procedure for computing $f(i) \in \mathbb{N}$ is already available and finishes for every argument used. We want to compute $\sum_{i=a}^{b} f(i)$ for $a, b \in \mathbb{N}$, taking an empty sum ($a > b$) as zero. The empty interval is the base case. For a nonempty interval, the first term plus the sum of the remaining terms gives the required answer:

$$\text{Sum}(f, a, b) = \begin{cases} 0, & \text{if } a > b, \\ f(a) + \text{Sum}(f, a+1, b), & \text{otherwise.} \end{cases}$$

Note that we have defined Sum as a "higher-order" function that takes an arbitrary function $f : \mathbb{N} \to \mathbb{N}$ as input. Choosing another f gives another summation procedure. For $f(i) = i^2$,

$$\text{Sum}(f, 2, 4) = 4 + (9 + (16 + 0)) = 29.$$

The additions are deferred until the recursive answers are available.

Correctness, by induction on the number ℓ of terms. Here $\ell = 0$ if $a > b$, and $\ell = b - a + 1$ otherwise. *Basis:* an empty interval returns 0, the empty sum. *Induction hypothesis:* for an arbitrary integer $k \geq 0$, the procedure returns the correct sum for *every* interval of k terms. *Inductive step:* an interval $[a, b]$ of $k + 1$ terms has k terms remaining in $[a + 1, b]$. Therefore

$$\text{Sum}(f, a, b) = f(a) + \text{Sum}(f, a+1, b) = f(a) + \sum_{i=a+1}^{b} f(i) = \sum_{i=a}^{b} f(i).$$

Each call removes one term, so the computation terminates. Notice that the first argument increases: it is the *number of remaining terms* that decreases. Taking $f(i) = i$, i^2, or 1 gives sums of integers, sums of squares, or a count of terms using the same proof and algorithm.

6.2.7 Perfect numbers

A positive integer is called a *perfect number* if the sum of its proper divisors add up to the number itself. a is a proper divisor of b if a is a divisor of b and $a \neq b$. The smallest examples of perfect numbers are 6 $(1 + 2 + 3 = 6)$ and 28 $(1 + 2 + 4 + 7 + 14 = 28)$ [3]. The next few perfect numbers are 496, 8128 and 33550336. Euclid devotes a chapter to perfect numbers in his *Elements*. There he proves that any number of the form $2^{p-1}(2^p - 1)$ is perfect, provided the odd factor $(2^p - 1)$, is prime. A few values of p for these perfect numbers are $p = 2, 3, 5, 7, 13, 17, 19, 61, 107, 127, 257$.

We define a function perfect : $\mathbb{N} \to \{true, false\}$ for determining whether a number is perfect or not in the following way.

$$\text{perfect}(n) = ((n \neq 0) \text{ and } (n = \text{addfactors}(n))$$

where the function addfactors : $\mathbb{N} \to \mathbb{N}$ computes the sum of the proper factors of n. We can define addfactors as

$$\text{addfactors}(n) = \text{Sum}(f, 1, n \; div \; 2)$$

where Sum is as defined above, and $f : \mathbb{N} \to \mathbb{N}$ is defined as

$$f(i) = \begin{cases} i & \text{if } n \; mod \; i = 0 \\ 0 & \text{otherwise} \end{cases}$$

Note that the n used in the definition of $f(i)$ is the same as in the function perfect.

Exercise 6.1 Establish the correctness of the algorithm for checking whether a number is perfect and estimate the number of steps required.

6.3 How much work? Complexity and Big O

Correctness does not tell us whether a computation is practical. *Time complexity* counts work as the input grows; *space complexity* measures the storage needed. We begin by counting the main operations: additions in summation, multiplications in factorial and powers, and remainder computations in Euclid's algorithm.[4][5]

6.3.1 Recurrences count the work

A *time-complexity recurrence* expresses the work on an input in terms of the work of its recursive calls, plus the work done outside those calls. It has a base case, just like the algorithm. This is an important tool for estimating steps: identify the calls, count the extra work, then unfold the recurrence or bound its solution by induction. Count every recursive call; two calls to the same smaller problem are still two computations unless the result is explicitly computed once and reused.

For factorial, let $T(n)$ count multiplications. Reading the definition gives

$$T(n) = \begin{cases} 0, & \text{if } n = 0, \\ 1 + T(n-1), & \text{otherwise.} \end{cases}$$

[3] The smallest perfect numbers 6 and 28 were known to the Hindus as well as the Hebrews. Some commentators of the bible regard 6 and 28 as the basic numbers of the Supreme Architect. They point to the 6 days of creation and the 28 days of the lunar cycle. Others go so far as to explain the imperfection of the second creation by the fact that eight souls, not six, were rescued in Noah's ark. Said St. Augustine: "Six is a number perfect in itself, and not because God created all things in six days; rather the converse is true; God created all things in six days because this number is perfect, and it would have been perfect even if the work of six days did not exist."

[4] These counts use whole-number operations as units of work. They need not equal counts of successor operations or individual bit operations. For example, a remainder computed by repeated subtraction may require many elementary steps. A more detailed analysis would expand the operations being counted.

[5] A number's value and the length of its representation are different measures. In binary, $n \geq 1$ has $L = \lfloor \log_2 n \rfloor + 1$ bits. A procedure using n steps can therefore take order 2^L steps in the input length. Large-number arithmetic also costs more than small-number arithmetic. We set these questions of bit complexity aside in this first discussion.

Unfolding gives $T(n) = 1 + T(n-1) = 2 + T(n-2) = \cdots = n + T(0) = n$. There are also $n+1$ factorial calls, and at most n pending multiplications. This measures the work of this algorithm, not a lower bound for every possible factorial algorithm.

For summation, counting one addition per term gives

$$A(\ell) = \begin{cases} 0, & \text{if } \ell = 0, \\ A(\ell-1) + 1, & \text{otherwise,} \end{cases} \qquad \text{so } A(\ell) = \ell.$$

There are also ℓ evaluations of f.[6]

For successor-based addition, exactly b successor operations are performed, so the work grows linearly with b. Multiplication makes b additions, each applying successor a times, for a total of ab successor operations. These simple counts already distinguish linear growth from growth in the product of two inputs.[7]

Definition 6.1 (Big O) *For nonnegative functions T and g, we write $T(n) = O(g(n))$ if there are constants $C > 0$ and n_0 such that*

$$T(n) \le Cg(n) \qquad \text{for all } n \ge n_0.$$

Thus $3n + 7 = O(n)$: for $n \ge 1$, $3n + 7 \le 10n$. Constant factors and lower-order terms do not affect this upper bound. Big O need not be tight: $3n + 7$ is also $O(n^2)$, although the linear bound tells us more. Typical orders of growth are $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(2^n)$. Big O itself does not mean "worst case"; we first choose whether T measures worst-case, average, or some particular execution cost.

6.3.2 Seeing the difference in growth rates

Figure 6.1 plots representative functions for the five orders of growth. The curves are $\log_2 n$, n, $n \log_2 n$, n^2, and 2^n themselves: Big O denotes a class of upper bounds, not one particular curve. Constant factors are taken as one for comparison.

Doubling n adds one to $\log_2 n$, doubles n, changes $n \log_2 n$ by a factor $2(1 + 1/\log_2 n)$, and quadruples n^2. For 2^n, doubling n *squares the amount of work*. Even increasing n by just one doubles it. These statements concern the displayed functions, not arbitrary algorithms known only to satisfy the corresponding Big O upper bounds.

How bad is exponential growth? Suppose an algorithm actually performs 2^n steps, and imagine a machine performing a billion (10^9) of those steps each second. Then:

n	Steps 2^n (approximately)	Time (approximately)
10	1.02×10^3	1.02 microseconds
30	1.07×10^9	1.07 seconds
50	1.13×10^{15}	13 days
100	1.27×10^{30}	4.02×10^{13} years

At $n = 100$, the other four representative counts are only about 6.64, 100, 664, and 10,000. A machine a thousand times faster buys the exponential algorithm only $\log_2 1000 \approx 10$ additional units of input in the same time budget. Better hardware changes a constant factor; a better algorithm may change the growth rate. Exponential algorithms can still be useful for small inputs, but scale very poorly. An $O(2^n)$ upper bound alone does not establish exponential behaviour; the example assumes an actual 2^n step count.

[6] If evaluating $f(i)$ costs $C_f(i)$ steps, the full count includes $\sum_{i=a}^{b} C_f(i)$ as well as additions and bookkeeping. Our addition count isolates the work of the summation procedure itself.

[7] Including tests and call bookkeeping adds work proportional to the number of calls. Under a constant cost for each such step, addition is bounded by a constant times $b + 1$, and multiplication by a constant times $ab + b + 1$. The extra terms account for zero inputs too.

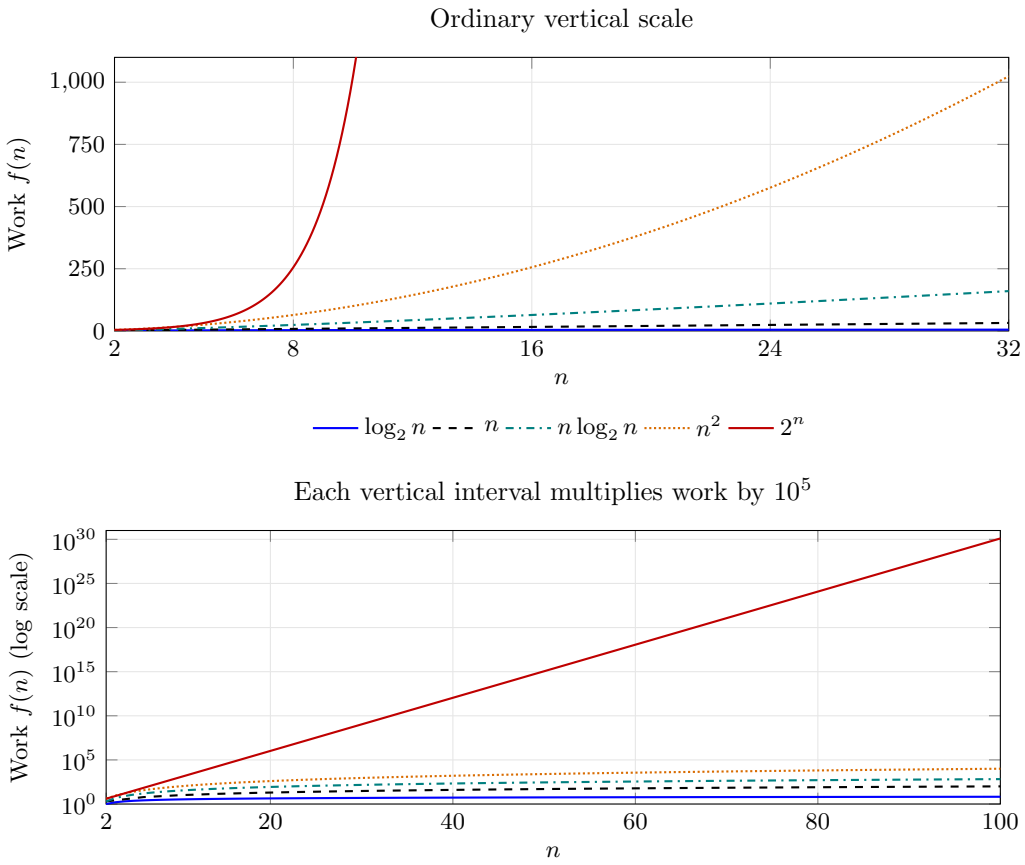


Figure 6.1: Five growth rates on two scales; the same styles identify curves in both panels. In the upper panel, 2^n leaves the displayed range just after $n = 10$. The logarithmic scale below keeps it visible up to $n = 100$; its straight appearance there represents exponential growth.

6.3.3 A faster way to compute powers

The power algorithm removes one factor at a time. Can we make a bigger reduction? If we know x^k, one squaring gives x^{2k}; one further multiplication by x gives x^{2k+1}. For a faster method[8] define the helper

$$\mathrm{sqr}(y) = y \cdot y.$$

Then use the parity of the exponent to choose the appropriate rule:

$$E(x,n) = \begin{cases} 1, & \text{if } n = 0, \\ \mathrm{sqr}(E(x, \mathrm{div}(n,2))), & \text{if } n \text{ is even}, \\ x \cdot \mathrm{sqr}(E(x, \mathrm{div}(n,2))), & \text{otherwise.} \end{cases}$$

Only one branch is used. In that branch the argument of sqr is evaluated once; the helper then multiplies the resulting number by itself. Thus there is just one recursive call at each level.

Correctness, by strong induction on the exponent n. *Basis:* $E(x,0) = 1 = x^0$ for every $x \in \mathbb{N}$. *Induction hypothesis:* for an arbitrary integer $n \geq 1$, assume $E(x,j) = x^j$ for every integer $0 \leq j < n$ and every $x \in \mathbb{N}$. *Inductive step:* put $k = \mathrm{div}(n,2)$, so $0 \leq k < n$. If $n = 2k$, the algorithm returns $(x^k)^2 = x^n$ by the hypothesis. If $n = 2k+1$, it returns $x(x^k)^2 = x^n$. In both cases the recursive exponent is smaller, so the computation also terminates.

Deriving the logarithmic bound. Let $M(n)$ be the number of multiplications made by this fast-power algorithm. Its exact recurrence is

$$M(n) = \begin{cases} 0, & \text{if } n = 0, \\ M(\lfloor n/2 \rfloor) + 1, & \text{if } n > 0 \text{ is even}, \\ M(\lfloor n/2 \rfloor) + 2, & \text{otherwise.} \end{cases}$$

There is one recursive call; an even exponent adds one multiplication, and an odd exponent adds two. For $n \geq 1$, after $L = \lfloor \log_2 n \rfloor + 1$ halvings the exponent is zero. Hence

$$L \leq M(n) \leq 2L, \qquad M(n) = O(\log n).$$

Equivalently, unfold the upper-bound recurrence $M(n) \leq M(\lfloor n/2 \rfloor) + 2$ for L levels. For $n = 16$, the exponents are $16, 8, 4, 2, 1, 0$, and the count is $1+1+1+1+2 = 6$, compared with 16 multiplications for ordinary power.

The helper sqr matters. Replacing it by two separate calls $E(x, \mathrm{div}(n,2)) \cdot E(x, \mathrm{div}(n,2))$ would repeat the smaller computation. For even n, the new multiplication count would satisfy $D(n) = 2D(n/2) + 1$. Unfolding this recurrence for $n = 2^k$ gives $D(2^k) = 3 \cdot 2^k - 1$, since $D(1) = 2$: the count is now linear in n. Halving the problem helps only if we avoid doing the same work twice. [9]

6.3.4 Euclid's algorithm: correctness and efficiency

For $a, b \in \mathbb{N}$ not both zero, the greatest common divisor is the largest positive integer dividing both. Euclid's algorithm gives a particularly clear example of a proof guiding an efficient computation.[10] Its key is the following claim.

[8] The idea of successive squaring has ancient roots. Piṅgala's *Chandaḥsūtra*, a Sanskrit treatise on prosody usually dated to around the third or second century BCE, describes a method of squaring and doubling to compute 2^n, the number of patterns of long and short syllables in a sequence of length n. This uses the essential idea behind fast exponentiation. See D. E. Knuth, *The Art of Computer Programming*, Vol. 2, *Seminumerical Algorithms*, Section 4.6.3.

[9] The logarithmic bound counts multiplications. It treats the halving and parity tests as auxiliary operations; implementing division by repeated subtraction would add further work. The sizes of the multiplied numbers matter in a bit-operation analysis.

[10] In Book VII, Proposition 2 of the *Elements*, Euclid describes the reduction by continually subtracting the smaller number from the larger. Computing a remainder packages several such subtractions into one step. The notation has changed, but the common-divisor argument is the same.

Claim. If $a = bq + r$, $b > 0$, and $0 \le r < b$, then $\gcd(a, b) = \gcd(b, r)$. *Proof:* let $d = \gcd(a, b)$. Since d divides both a and b, it divides $r = a - bq$, so it is a common divisor of b, r. Conversely, any positive common divisor c of b, r divides $bq + r = a$, so it is a common divisor of a, b and hence $c \le d$. Thus d is the greatest common divisor of b, r. This includes $r = 0$.

The claim reduces the second argument from b to $r < b$. Assuming we can solve all such smaller cases, we obtain the algorithm

$$G(a, b) = \begin{cases} a, & \text{if } b = 0, \\ G(b, \operatorname{mod}(a, b)), & \text{otherwise.} \end{cases}$$

Correctness, by strong induction on b. We prove $G(a, b) = \gcd(a, b)$ for every $b \ge 0$ and *every* $a > 0$. *Basis:* $G(a, 0) = a = \gcd(a, 0)$. *Induction hypothesis:* for an arbitrary integer $k \ge 0$, assume that for all integers $0 \le j \le k$ and all $u > 0$, $G(u, j) = \gcd(u, j)$. *Inductive step:* let $b = k + 1$ and $r = \operatorname{mod}(a, b)$. Since $0 \le r < b$ and $b > 0$,

$$G(a, b) = G(b, r) = \gcd(b, r) = \gcd(a, b).$$

The middle equality uses the induction hypothesis, and the last uses the claim. Quantifying over *every* first argument is essential: the recursive call changes it from a to b. The decreasing second argument also proves termination.

For completeness, if $a = 0 < b$, the algorithm returns $G(b, 0) = b$, as required. We may set $G(0, 0) = 0$ by convention to obtain a numerical function defined on every pair; this does not assert that zero has a greatest positive divisor. For example,

$$G(48, 18) = G(18, 12) = G(12, 6) = G(6, 0) = 6.$$

The following chapter on primes and gcd develops the number-theoretic consequences further.

Why logarithmically many steps? Let $C(a, b)$ count remainder computations. Directly from the algorithm,

$$C(a, b) = \begin{cases} 0, & \text{if } b = 0, \\ 1 + C(b, \operatorname{mod}(a, b)), & \text{otherwise.} \end{cases}$$

Merely observing that the second argument decreases proves termination, but gives only a crude linear bound. We need to show that it shrinks quickly. Put $r = \operatorname{mod}(a, b)$, so $0 \le r < b$. If $r = 0$, we stop after one step. If $0 < r \le b/2$, the new second argument is already at most half of b. Otherwise $b/2 < r < b$, so the next remainder is

$$\operatorname{mod}(b, r) = b - r < b/2,$$

because b contains exactly one copy of r. Thus within at most two remainder steps, either the computation stops or its second argument at least halves.

After at most $2\lceil \log_2 b \rceil$ steps, a starting $b \ge 1$ has therefore either reached zero or become at most 1; one further step then finishes. Hence

$$C(a, b) \le 2\lceil \log_2 b \rceil + 1 \quad (b \ge 1).$$

This shows that the algorithm requires $O(\log b)$ steps. It requires no assumption that $a \ge b$.[11]

6.4 Iteration through tail recursion

Return to factorial. To avoid deferred multiplications, let p record the product already computed and k count the factors still to be processed. We want $p = n!$ when $k = 0$. A useful relation to maintain throughout is

$$0 \le k \le n, \qquad p \cdot k! = n!.$$

[11] The bound counts remainder computations. It does not count the subtractions or bit operations used to implement each remainder.

Initially this holds with $(k, p) = (n, 1)$. If $k > 0$, replacing (k, p) by $(k - 1, pk)$ preserves the relation, since $(pk)(k - 1)! = pk!$. This reasoning *derives* the algorithm

$$F(k, p) = \begin{cases} p, & \text{if } k = 0, \\ F(k - 1, p \cdot k), & \text{otherwise.} \end{cases}$$

To compute $n!$, evaluate $F(n, 1)$. Its evaluation for $n = 5$ is

$$\begin{aligned} F(5, 1) &= F(4, 5) \\ &= F(3, 20) \\ &= F(2, 60) \\ &= F(1, 120) \\ &= F(0, 120) \\ &= 120. \end{aligned}$$

At each transition the multiplication is carried out *before* the next call. Compare the two factorial computations:

Deferred computation	**Direct accumulation**
$5 \cdot \text{fact}(4)$ waits for the inner answer.	$F(5, 1) \longrightarrow F(4, 5)$ carries a completed product.
Pending multiplications accumulate, then are performed on return.	Completed products are carried forward in p.
The evaluator remembers unfinished work.	The current state (k, p) contains all the information needed.

In F, nothing remains to be done with the result of the recursive call. Such a call is in *tail position*; a function whose recursive calls all have this form is *tail recursive*. A recursively written function can therefore generate an *iterative process*: repeatedly replace a current state by a next state, without accumulating unfinished calls. In general,

$$I(s) = \begin{cases} \text{answer}(s), & \text{if stop}(s), \\ I(\text{next}(s)), & \text{otherwise.} \end{cases}$$

The state s may be a tuple of natural numbers. Tail recursion alone does not guarantee termination: we must still show that a stopping state is reached.

6.4.1 Invariants are induction over state changes

An *invariant* is a statement about the current state, possibly also referring to the fixed original input, that holds initially and is preserved by every transition. The values of the state variables change; the truth of the statement does not. To establish correctness using an invariant, check:

1. *Initialisation:* the initial state satisfies it.
2. *Preservation:* if it holds before a nonterminal step, it holds after that step.
3. *Conclusion:* the invariant together with the stopping condition implies that the returned answer is the required one.

This is induction on the number t of transitions. Initialisation proves the claim for $t = 0$. For an arbitrary integer $t \geq 0$, assume the invariant holds after t transitions. Preservation proves it after the next transition, if one is made. A separate termination argument shows that a stopping state is eventually reached.

For $F(n, 1)$, the invariant we designed was $0 \leq k \leq n$ and $pk! = n!$. *Initialisation:* $1 \cdot n! = n!$. *Preservation:* $(pk)(k - 1)! = pk! = n!$ and $0 \leq k - 1 \leq n$ when $k > 0$. *Conclusion:* at $k = 0$,

$p \cdot 0! = p = n!$. *Termination:* k decreases by one and cannot decrease below zero. This shows how an invariant can guide the design, not merely check an algorithm after it has been written.

The same result can be proved by induction on k, the number of factors remaining. The claim is: *for every* $k, p \in \mathbb{N}$, $F(k,p) = p \cdot k!$. *Basis:* $F(0,p) = p = p \cdot 0!$. *Induction hypothesis:* for an arbitrary integer $k \geq 0$, $F(k,u) = u \cdot k!$ for *every* accumulator $u \in \mathbb{N}$. *Inductive step:*

$$F(k+1,p) = F(k,p(k+1)) = p(k+1)k! = p(k+1)!.$$

Taking $p = 1$ gives the desired factorial. The hypothesis must cover every accumulator, since its value changes in the recursive call.

6.4.2 Further examples of iterative processes

For addition, begin with $(a,b) = (a_0,b_0)$ and preserve $a + b = a_0 + b_0$:

$$A(a,b) = \begin{cases} a, & \text{if } b = 0, \\ A(a+1,b-1), & \text{otherwise.} \end{cases}$$

The initial equality is immediate. Each step preserves it because $(a+1)+(b-1) = a+b$. At termination $b = 0$, so the returned a equals $a_0 + b_0$; termination follows from decreasing b. For example, $A(4,3) = A(5,2) = A(6,1) = A(7,0) = 7$.

Multiplication and powers use an accumulator in the same way:

$$M(a,k,p) = \begin{cases} p, & \text{if } k = 0, \\ M(a,k-1,p+a), & \text{otherwise,} \end{cases}$$

$$P(x,k,p) = \begin{cases} p, & \text{if } k = 0, \\ P(x,k-1,p \cdot x), & \text{otherwise.} \end{cases}$$

For ab, start with $M(a,b,0)$ and maintain $p + ak = ab$. The initial equality is $0 + ab = ab$, a step preserves it since $(p+a) + a(k-1) = p + ak$, and $k = 0$ implies $p = ab$. For x^n, start with $P(x,n,1)$ and maintain $px^k = x^n$. It holds initially, is preserved by $(px)x^{k-1} = px^k$ for $k > 0$, and implies $p = x^n$ at $k = 0$. Both counters decrease to zero. Include $0 \leq k \leq b$ or $0 \leq k \leq n$, respectively, in the invariants.

Euclid's G is already tail recursive. For an initial pair (a_0,b_0) not both zero, its invariant is

$$a,b \in \mathbb{N}, \quad a+b > 0, \quad \gcd(a,b) = \gcd(a_0,b_0).$$

It holds initially. The common-divisor claim proves preservation under $(a,b) \mapsto (b, \operatorname{mod}(a,b))$, and $b > 0$ ensures the new pair is not $(0,0)$. When $b = 0$, the invariant says the returned a is the original gcd. The second argument decreases, proving termination. Thus the inductive proof and the invariant express the same mathematical reason for correctness in two forms.

6.4.3 Summation as iteration

For the same fixed function f as in Section 6.2.6, use c as the next index and s as the sum already accumulated:

$$J(f,c,b,s) = \begin{cases} s, & \text{if } c > b, \\ J(f,c+1,b,s+f(c)), & \text{otherwise.} \end{cases}$$

To compute $\sum_{i=a}^{b} f(i)$, start with $J(f,a,b,0)$. For $f(i) = i^2$ the computation is

$$J(f,2,4,0) = J(f,3,4,4) = J(f,4,4,13) = J(f,5,4,29) = 29.$$

Every addition is completed before the next call, in contrast with the nested additions in $\operatorname{Sum}(f,2,4)$.

If $a > b$, the initial call immediately returns the empty sum. Otherwise an invariant for the initial interval $[a, b]$ is

$$a \le c \le b + 1, \qquad s = \sum_{i=a}^{c-1} f(i).$$

Equivalently, the accumulated sum plus the remaining sum equals the target:

$$s + \sum_{i=c}^{b} f(i) = \sum_{i=a}^{b} f(i).$$

Initialisation: $c = a$ and $s = 0$, the sum over an empty prefix. *Preservation:* when $c \le b$, adding $f(c)$ extends the prefix by one term, and replacing c by $c + 1$ identifies the next unprocessed term. *Conclusion:* stopping gives $c = b + 1$, so $s = \sum_{i=a}^{b} f(i)$. *Termination:* $b - c + 1$, the number of remaining terms, decreases to zero. Again, the invariant tells us what to initialise, how to update the state, and when to return the answer.

Both versions evaluate f and add once per term: their addition count is $\ell = b - a + 1$ for $a \le b$. Tail recursion removes the chain of deferred additions without changing that count.[12]

What does tail recursion save? The direct factorial computation leaves n multiplications waiting for answers. The accumulator version still performs n multiplications, but carries only the current pair (k, p). It replaces a growing record of unfinished work by a fixed number of state variables.[13] This is a change in how the work is organised, not a reduction in the number of multiplications.

6.5 How far does this model reach?

6.5.1 Why these rules are a reasonable model

Imagine carrying out a calculation with pencil and paper, following explicit instructions without inventing new steps. At any moment there is a finite record: the input, intermediate numbers, and the instruction to perform next. One step reads part of this record, performs a specified elementary operation, and produces a new record. It may choose what to do next by a test. A long computation repeats such steps until its stopping condition is met.

Our model expresses these ingredients directly. Numbers encode finite data; composition connects one calculation to the next; if–then–else makes a choice using a computable test; and recursion describes repetition. As we saw with iteration, a tuple can hold the entire current state. Including the current instruction in that state also permits different operations at different stages. The amount of data and the number of steps need not have a fixed universal bound: each completed computation uses only finitely much. These observations explain the model's plausibility without reference to any particular machine or programming language.

There is one limitation to address. Primitive recursion repeats a construction a number of times controlled by a supplied natural number. It guarantees termination when its component computations terminate, but there are effective procedures that cannot be expressed using only those rules. To describe computation in general, we must also allow continuing until a computable test succeeds, without requiring a previously supplied bound on how many trials to make.

6.5.2 Bounded search and unbounded search

A *bounded search*, also called *bounded minimisation*, tests $y = 0, 1, \ldots, B$ and returns the first successful candidate, or reports that none in that range works. The bound B may depend on the input; it need not be a constant. If the test always finishes, the search always finishes. For square

[12] The work and storage needed inside f itself are additional in both versions.

[13] An evaluator must reuse tail calls as state transitions to obtain this storage saving; one that retains every call frame may still build a stack. A fixed number of variables is not a fixed number of bits: the accumulator eventually holds $n!$. Storage and work inside arithmetic helpers must also be counted in a more detailed analysis.

roots, such a bound is available: no natural-number root of n can exceed n when $n \geq 1$, and $n = 0$ is immediate. Our earlier procedure uses the sharper stopping test $m^2 > n$.

But failure up to B means only "no solution yet", unless we can prove that no later candidate could succeed. Imagine running a given procedure for successively larger limits: first 0 steps, then 1, then 2, and so on, each time checking whether it has finished. Every bounded trial is a finite task. Failure to finish within B steps does not show that the procedure will never finish. Requiring a suitable bound to be computed first would impose an extra task on every procedure. The ability to take the next step does not require such a bound to be known in advance.

Bounded search can already be expressed by primitive recursion, provided its bound and tests can: count through the permitted candidates and remember the first success. Adding this kind of search therefore gives no new computational power. Some terminating procedures lie beyond these rules, though proving this is beyond our scope. Allowing a more powerful procedure to compute the bound would simply move the difficulty into that procedure.

In *unbounded search*, we keep trying the next candidate until the test succeeds. For a previously constructed numerical procedure g, write

$$H(\mathbf{x}, y) = \begin{cases} y, & \text{if } g(\mathbf{x}, y) = 0, \\ H(\mathbf{x}, y+1), & \text{otherwise.} \end{cases}$$

Start with $H(\mathbf{x}, 0)$. Each test must finish before proceeding to the next candidate. If no test succeeds, or a required test never finishes, this computation produces no answer. "Unbounded" means no advance cutoff, not that a successful search performs infinitely many steps. If the first successful candidate is k and all preceding tests finish, only $k + 1$ candidates are tested.

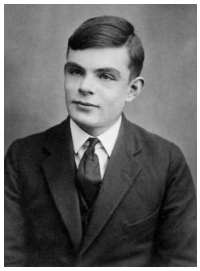
6.5.3 What completeness means

Informally, a *total function* assigns an answer to every input in its stated domain. A *partial function* may leave some inputs without an answer. In our computational setting, this latter possibility describes a procedure that runs forever on some inputs. All the arithmetic algorithms above finish on their stated domains; permitting unbounded search means that our language can also describe computations that fail to finish. For example, searching only for $y^2 = n$, with no overshoot test, never finishes on a nonsquare. The bounded square-root algorithm shows that this failure belongs to that particular search procedure, not to the problem.

Starting with the elementary numerical functions and closing under composition, primitive recursion, and unbounded search gives the standard *partial recursive* model. Computable conditionals can be expressed within it. Its rules themselves are mechanical, and the state-and-step view above explains why it is reasonable to regard them as sufficient for any effective calculation on finite data. This is an argument for the model's adequacy, not a proof that every imaginable mathematical function has an algorithm.

The same idea of effective calculation can be approached in different ways. Alan Turing described a machine that follows elementary rules for reading, writing, and moving among symbols. Alonzo Church approached computation through functions and rules for applying them. Their starting points look quite different, yet both models compute exactly the same numerical functions as the recursive model we have described. This equivalence is a mathematical theorem.

The agreement of these independently developed models gives us confidence that they capture the informal idea of calculation by explicit, mechanical rules. The assertion that they encompass *every effective mechanical procedure* is the *Church–Turing thesis*. Unlike the equivalence of the formal models, this assertion is not a mathematical theorem: it connects a precise mathematical definition with an informal idea. In Church's quotation, "recursive" includes the possibility of unbounded search; primitive recursion alone would not suffice. We need no knowledge of the other models to use ours. Its scope concerns what can be computed, not whether every procedure we write terminates or gives the intended answer.



"A function is said to be 'effectively calculable' if its values can be found by some purely mechanical process." – Alan Turing, *Systems of Logic Based on Ordinals* (1939), p. 166.



"a function of positive integers shall be called effectively calculable if it is recursive" – Alonzo Church, *An Unsolvable Problem of Elementary Number Theory* (1936), p. 346, footnote 3 (excerpt).

Problems

1. Unfold add$(2, 4)$ and fact(5). Mark the operations that are deferred. Trace the corresponding tail-recursive computations and identify their states.
2. State the induction hypothesis for division and remainder using the quotient q. Explain why using q in the proof does not require knowing it before running the algorithm.
3. Trace quotient and remainder for $(23, 4)$. Define a tail-recursive function that computes both together, using a remainder candidate and a quotient accumulator. State and prove its invariant.
4. Trace $G(105, 252)$. Explain why an initial assumption $a \geq b$ is unnecessary, and why the second argument is a suitable termination measure.
5. Prove that $5n^2 + 3n + 8 = O(n^2)$. Explain why an $O(n^2)$ upper bound alone does not establish that an algorithm needs quadratic time.
6. Compare the number of multiplications for 3^{16} using pow and E, following the definitions exactly. Explain what happens if sqr is replaced by two independently evaluated recursive calls.
7. The powering algorithm is based on successive multiplications. Similarly, one can devise an integer multiplication algorithm based on repeated additions. Design an iterative algorithm for integer multiplication in terms of adding, doubling and halving. The algorithm should work in $O(\lg n)$ time.[14]
8. Explain how the invariant for $F(k, p)$ determines its next state. Describe which value must be used in the multiplication if the two components of its state are updated one at a time.
9. Under the hypothetical rate of 10^9 steps per second, estimate the time for 2^{60} steps. How much does a thousandfold speed-up increase the input size possible in a fixed time? Compare with an algorithm taking n^2 steps.

[14] This algorithm, which is sometimes known as the "Russian peasant method" of multiplication, is very old. Examples of its use are found in the Rhind Papyrus, one of the two oldest mathematical documents in existence, written about 1700 B.C. (and copied from an even older document) by an Egyptian scribe named A'h-mose.

10. For $f(i) = 2i + 1$, unfold $\mathrm{Sum}_f(1, 3)$ and trace $J(f, 1, 3, 0)$. Check the invariant at each state, and state the addition-count recurrence. What changes if one evaluation of $f(i)$ costs i steps?

11. Prove the upper bound for fast power by induction using its recurrence. Explain why counting one recursive call instead of two is decisive.

12. In Euclid's algorithm, show that two consecutive positive remainders cannot both exceed half the earlier second argument. Use this to derive a logarithmic bound without assuming anything about Fibonacci numbers.

13. Explain why "no successful candidate at most B" does not usually mean "no successful candidate exists". What extra fact makes a bounded search sufficient in the square-root example?

14. Explain why the unbounded search for a square root may fail to halt. Modify it to return a special failure code when no natural-number square root exists, and explain why the modified search always terminates.

15. Define recursive and tail-recursive algorithms for $\prod_{i=a}^{b} f(i)$, with an empty product equal to 1. Prove the recursive version correct by induction on the number of factors. Give an invariant for the iterative version.

16. Let $B(0) = 1$ and $B(n) = B(n-1) + B(n-1)$ for $n > 0$. Show by induction on n that $B(n) = 2^n$. If the two recursive calls are evaluated separately, derive the recurrence for the number of additions and solve it. Rewrite the algorithm to use only n additions.

17. The Fibonacci numbers are $F_0 = 0$, $F_1 = 1$, and $F_{n+2} = F_{n+1} + F_n$. Design an iterative algorithm that starts with $(i, u, v) = (0, 0, 1)$ and maintains $u = F_i$, $v = F_{i+1}$. State its stopping condition, prove correctness, and count its additions.

18. Evaluate $G(34, 21)$ and $G(55, 34)$. Compare the successive remainders with the Fibonacci numbers. Explain why each reduction is small compared with the case in which the smaller input divides the larger exactly.

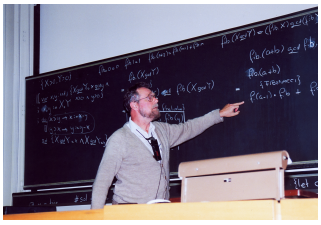
19. Construct a tail-recursive fast-power algorithm using a state (b, k, p) with invariant $pb^k = x^n$. Start with $(b, k, p) = (x, n, 1)$. What state change works when k is even? What works when it is odd? Prove that the algorithm terminates and uses logarithmically many multiplications. State what happens when $n = 0$.

20. To count how many values among $f(a), \ldots, f(b)$ are even, define a recursive algorithm using an if–then–else test. Prove it correct by induction on the number of terms. Then derive an iterative version and its invariant.

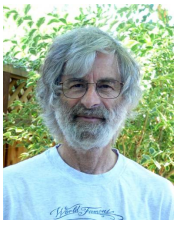
21. Suppose an algorithm has a correct invariant but its state never changes. Must it return the right answer? Give an example and explain precisely what an invariant proves and what a termination argument adds.

Chapter 7

Changing state, preserving truth: the Imperative model of computation



"one designs the correctness proof and derives the program from that proof."
—Edsger W. Dijkstra, EWD 1073 (excerpt).
ETH Zurich, 1994. Photo: Andreas F. Borchert, CC BY-SA 4.0.



"Invariants play an important role in understanding algorithms and programs."
—Leslie Lamport, *Computation and State Machines*, Section 5.1.1.
2013 Turing Award. Photo courtesy Leslie Lamport.

In Chapter 6, an iterative computation passed its current state to the next call. For factorial, the state (k, p) was replaced by $(k-1, pk)$ until $k = 0$. The invariant $p \cdot k! = n!$ explained why the answer survived each change. We now describe the same process by storing k and p and giving instructions that change their contents.

This is the *imperative model*: a computation is a sequence of commands that transform a stored state. We will keep asking two questions. What changes when a command is performed? What remains true despite that change? The first describes the computation; the second helps us design it and prove that it works.

Our procedures use mathematical notation and ordinary instructions, in the style of the ruler-and-compass constructions. They do not depend on a particular programming language. The available actions will be stated in each setting: storing a number, exchanging two entries, placing a tile, or moving a disk. A more detailed implementation would explain how to carry out these actions on a particular machine.

7.1 Variables become places to store values

In the equation $y = x + 1$, the letters denote mathematical values. In a functional call $F(4, 1)$, the parameters receive those values for that call; a later call $F(3, 4)$ has a new set of parameter values. Neither changes the meaning of the earlier call.

An imperative variable has a different role. Think of it as the name of a storage location, or a labelled box. The name stays the same while the value in the box may change. A *state* is a snapshot of the contents of the boxes. To describe the complete progress of a computation we also need to know which instruction comes next.

We write $x \leftarrow e$ for *assignment*: evaluate the expression e using the current contents of the variables, and then store the result in x, replacing its previous contents. Other locations retain their contents. Thus $x \leftarrow x + 1$ is a perfectly sensible command. If x contains 4, it will contain 5 afterwards. The equation $x = x + 1$, by contrast, is false for every integer x. We reserve $=$ for equality.

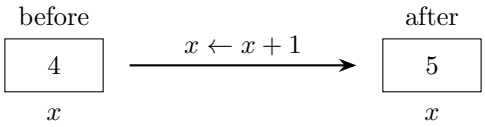


Figure 7.1: The location retains its name; assignment replaces its contents.

Start with $x = 4$ and perform $y \leftarrow x$ followed by $x \leftarrow x + 1$. The final state is $x = 5$, $y = 4$. The first command copied a value; it did not make y follow all later changes to x. In our model, differently named scalar variables denote separate locations. Every variable must be assigned a value before an instruction reads it.[1]

In the procedure displays, comments beginning with `# PRE`, `# INV`, and `# POST` state the required initial condition, loop invariant, and promised final condition. A `# BOUND` comment states a decreasing bound function for termination, defined below. A `# ASSERT` comment records an intermediate fact. They are mathematical statements, not instructions. A straight-line procedure has intermediate assertions rather than a loop invariant.

7.1.1 Order matters: exchanging two values

Suppose x and y initially contain A and B. Here A and B are fixed mathematical names for the original values, not locations that the procedure changes. We want to finish with $x = B$ and $y = A$. The commands $x \leftarrow y$ followed by $y \leftarrow x$ do not achieve this: the first destroys the old value of x, and both boxes finish with B. We must save the value before overwriting it.

Algorithm 8 Exchange the contents of two specified locations.

1: **procedure** EXCHANGE(x, y)
   `# PRE:` $x = A$, $y = B$; A, B are fixed original values.
2: Store the current contents of x in a temporary location t.
   `# ASSERT:` $t = A$, $x = A$, $y = B$.
3: $x \leftarrow y$
   `# ASSERT:` $t = A$, $x = B$, $y = B$.
4: $y \leftarrow t$
   `# POST:` $x = B$, $y = A$.

	x	y	t
initially	A	B	unused
after the first command	A	B	A
after the second command	B	B	A
after the third command	B	A	A

A command reads the state left by the preceding command. Translating a simultaneous state transition into separate assignments therefore requires care: save any old value that will still be needed.

[1] Real languages also offer ways for several names to refer to the same storage. We do not need that additional complication here. Our mathematical integers have no fixed upper limit; a machine implementation must provide suitable storage.

7.2 Commands, decisions, and repetition

We use three ways to organise commands.

1. *Sequence*: perform commands in the stated order.
2. *Selection*: **if** a condition is true, perform one block of commands; **otherwise**, perform the other. Only the chosen block is performed.
3. *Repetition*: **while** a condition is true, perform a block of commands, then test the condition again.

The condition of a while loop is checked *before* each repetition, so the block may be performed zero times. Indentation identifies the commands belonging to a block. A call to a named procedure performs its commands and then resumes the calling procedure.

For example, to store the larger of a and b in m, use

```
# PRE: a = A, b = B are comparable values.
if a ≥ b then
    m ← a
else
    m ← b
# POST: m = max(A, B), a = A, b = B.
```

The test asks a question about the current state; it does not change either value. An assignment changes the state. Keeping these roles separate is essential.

7.2.1 What a procedure promises



"The most important property of a program is whether it accomplishes the intentions of its user."
—C. A. R. Hoare, *An Axiomatic Basis for Computer Programming* (1969), p. 579.

Photograph by Rama, Wikimedia Commons, CC BY-SA 2.0 France.

Before designing a procedure, state its *precondition*: what must be true when it begins. State its *postcondition*: what it must make true when it finishes. For Exchange these are, respectively,

$$x = A \text{ and } y = B, \qquad x = B \text{ and } y = A.$$

An *assertion* is a mathematical statement about a state at a specified point in a procedure. It describes what we claim, rather than instructing the machine to make the claim true. Our proof must justify it.

Correctness has two parts: the procedure must finish, and its final state must satisfy the postcondition whenever its initial state satisfies the precondition. A rule that would give the right answer *if* it finished is not enough to solve the problem on every permitted input.

Our assertions give precise mathematical meaning to those intentions. In modern notation, a *Hoare triple*

$$\{P\}\ C\ \{Q\}$$

connects a precondition P, a command C, and a postcondition Q. It states that if P holds before C begins, and C terminates, then Q holds afterwards. For example,

$$\{x = a\} \quad x \leftarrow x + 1 \quad \{x = a + 1\},$$

where a denotes a fixed mathematical value.

Hoare developed rules for reasoning with such assertions in his paper *An Axiomatic Basis for Computer Programming*.[2]

The rule for a loop expresses precisely our invariant argument:

$$\frac{\{I \wedge B\}\ C\ \{I\}}{\{I\}\ \textbf{while}\ B\ \textbf{do}\ C\ \{I \wedge \neg B\}}.$$

If the body preserves I whenever the condition B holds, then I still holds when the loop exits, and B is false. Initialisation must establish I; the exit assertion $I \wedge \neg B$ must imply our desired postcondition.

The triple alone does not guarantee termination. That is the separate role of our decreasing bound function.

7.3 A loop invariant is an induction hypothesis

The tail-recursive factorial construction in Section 6.4, with its invariant proved in Section 6.4.1, gives the transition $(k, p) \mapsto (k - 1, pk)$. We now store the two components and update them with assignments. The same invariant guides these updates.

Start with the desired loop postcondition: $k = 0$ and $p = n!$. To generalise it, allow unfinished work, represented by $k!$, so that $p \cdot k! = n!$. At $k = 0$ this reduces to the required answer. At each test of the loop condition, we therefore claim

$$I: \qquad 0 \le k \le n \quad \text{and} \quad p \cdot k! = n!.$$

This is a *loop invariant*. Its truth is preserved from one test of the condition to the next, even though the values in it change. It need not hold between every pair of commands inside the loop.

To obtain $p = n!$ at the end, let $k!$ represent the work still to do. The invariant suggests starting with $k = n$ and $p = 1$, and stopping at $k = 0$. When $k > 0$, decreasing k by one must be balanced by multiplying p by the old k, because $(pk)(k - 1)! = pk!$. Thus the multiplication must precede the decrement. We obtain the following commands from that reasoning.

Algorithm 9 Factorial by successive changes of state.

```
1: procedure FACTORIAL(n)
   # PRE: n ∈ ℕ is fixed.
2:     k ← n
3:     p ← 1
   # INV: k, p ∈ ℕ, 0 ≤ k ≤ n, p · k! = n!.
   # BOUND: k.
4:     while k > 0 do
5:         p ← p · k
6:         k ← k − 1
   # POST: k = 0 and p = n!; n is unchanged.
7:     return p
```

[2] C. A. R. Hoare, "An Axiomatic Basis for Computer Programming," *Communications of the ACM*, 12(10), October 1969, pp. 576–580, 583. https://doi.org/10.1145/363235.363259. We use the now customary notation $\{P\}\,C\,\{Q\}$; Hoare originally wrote $P\{C\}Q$.

Initialisation. Before the first test, $k = n$ and $p = 1$, so I holds.

Preservation. Suppose I holds at a test for which $k > 0$. Call the values at that point K and P. The two commands produce $p = PK$ and $k = K - 1$. Since $K \geq 1$,

$$p \cdot k! = (PK)(K-1)! = PK! = n!.$$

The bounds on k persist too. This proves preservation for one execution of the *whole* loop body.

Exit. When the condition becomes false, $k \leq 0$. Together with the invariant this gives $k = 0$, and hence $p = n!$.

Termination. A *bound function* assigns a nonnegative integer to each state at a loop test. Whenever the body executes, its value must strictly decrease from one test to the next. It therefore decreases monotonically along the computation, and cannot decrease forever. If its initial value is B, there can be at most B repetitions, provided each execution of the body itself finishes. The bound need not decrease by exactly one, and the loop may stop before it reaches zero.

For factorial the bound function is k: it starts at n and decreases by one on every repetition. A bound function is also called a *loop variant*. An invariant tells us what remains true; a bound function establishes progress towards termination. Merely never increasing is insufficient: a constant value would permit endless repetition. Nor is strict decrease among nonnegative real numbers enough: the sequence $1, 1/2, 1/4, \ldots$ never ends. Here we use nonnegative integers.

Where was induction used? We induct on the number j of completed repetitions. The invariant holds for $j = 0$. For every integer $j \geq 0$, if it holds after j repetitions and another repetition is performed, preservation shows that it holds after $j + 1$. Thus it holds at every reached test, including the final one. This is the induction principle applied to a computation's history.

For $n = 4$, the states (k, p) at successive tests are

$$(4, 1),\ (3, 4),\ (2, 12),\ (1, 24),\ (0, 24).$$

They are the same states as in the tail-recursive version. The model of description has changed; the invariant and the computation have not.

7.3.1 From the postcondition to the invariant and the loop

We first stated the factorial postcondition, then generalised it to an invariant before writing commands. The invariant supplied the initial state, the update, and the stopping condition. Reversing the two assignments would multiply by the new k and break that reasoning. This order of development applies to the remaining examples too. Begin with the final assertion, and generalise it to describe partial progress: replace a completed range by a growing prefix, allow an unfinished remainder, or retain a smaller equivalent problem. Keep the conditions that protect data and preserve the meaning of the answer. The invariant must be weak enough to hold initially and strong enough to yield the postcondition at exit. This is a design method, not a mechanical recipe for finding an invariant. For a loop with invariant I and condition C, prove:

1. The initialisation establishes I.
2. If I and C hold, the body re-establishes I.
3. I together with the failure of C implies the required answer.
4. A nonnegative integer bound function strictly decreases on every execution of the body, and each execution finishes.

The first three establish the answer on exit; the fourth establishes that exit is reached. A loop that repeatedly leaves k unchanged can preserve $k \geq 0$ forever. Preservation alone does not imply progress.

7.4 Familiar computations, now with stored state

7.4.1 Summation: the processed part and the remaining part

Recall the recursive sum in Section 6.2.6 and its iterative form in Section 6.4.3. We now implement that same accumulation by assignments.

Let $f : \mathbb{N} \to \mathbb{N}$ be a function whose values can always be computed, and let $a, b \in \mathbb{N}$. The loop postcondition is $k > b$ and $s = \sum_{i=a}^{b} f(i)$, taking an empty sum to be zero. Generalise the completed upper endpoint b to $k - 1$: only the terms before the moving index k have been added. The inputs and the rule for f remain fixed throughout the procedure. Choose a location s for the sum so far and a location k for the next index to be processed. The invariant, including the empty-interval case, is

$$a \le k \le \max(a, b + 1), \qquad s = \sum_{i=a}^{k-1} f(i).$$

The empty processed part suggests $k = a$, $s = 0$. To extend it by one term, add $f(k)$ before advancing k.

The same invariant can be written as

$$s + \sum_{i=k}^{b} f(i) = \sum_{i=a}^{b} f(i).$$

Work already done and work still to do together make the whole task. This is a useful pattern for discovering invariants.

Algorithm 10 Accumulate the sum of successive function values.

1: **procedure** SUM(f, a, b)
   `# PRE:` $a, b \in \mathbb{N}$; $f : \mathbb{N} \to \mathbb{N}$ is computable; inputs fixed.
2: $s \leftarrow 0$
3: $k \leftarrow a$
   `# INV:` $s \in \mathbb{N}$, $a \le k \le \max(a, b + 1)$,
   `#` $s = \sum_{t=a}^{k-1} f(t)$; f, a, b unchanged.
   `# BOUND:` $\max(0, b - k + 1)$.
4: **while** $k \le b$ **do**
5: $s \leftarrow s + f(k)$
6: $k \leftarrow k + 1$
   `# POST:` $k > b$, $s = \sum_{t=a}^{b} f(t)$ (zero for an empty interval).
7: **return** s

Initially the sum in the invariant is empty. If the old index is $K \le b$, then the updated values satisfy

$$s_{\text{new}} = \sum_{i=a}^{K-1} f(i) + f(K) = \sum_{i=a}^{K} f(i), \qquad k_{\text{new}} = K + 1.$$

Thus the invariant is preserved. Induction on the number of completed repetitions, beginning at zero and with the step valid for every integer $j \ge 0$ for which the loop continues, proves it at every test. At exit, if $a \le b$, then $k = b + 1$, so the sum is the desired one. The bound function $\max(0, b - k + 1)$ decreases by one whenever the body executes. Each evaluation of f finishes by the precondition. If $a > b$, the loop is skipped and returns zero, as required.

The procedure evaluates f exactly $b - a + 1$ times when $a \le b$. Its total time must also include the cost of those evaluations.

7.4.2 Quotient and remainder together

Section 6.2.3 constructed quotient and remainder by repeated subtraction. Here we compute both together, retaining the same relationship that justified the inductive step. For fixed $a \geq 0$ and $b > 0$, the postcondition is $a = bq + r$, $q \geq 0$, and $0 \leq r < b$. Generalise it by dropping the final restriction $r < b$: the remainder may still contain whole copies of b. Maintain

$$a = bq + r, \qquad q \geq 0, \quad r \geq 0.$$

Here q counts the copies of b already removed, while r is what remains. The invariant suggests $q = 0, r = a$ initially. If $r \geq b$, subtracting b from r must be balanced by adding one to q. Stopping when $r < b$ then gives both the quotient and the required remainder bound. These choices follow from the relationship we have decided to preserve.

Algorithm 11 Each repetition removes one more copy of the divisor.

```
1: procedure DIVIDE(a, b)
   # PRE: a, b ∈ ℕ and b > 0.
2:     q ← 0
3:     r ← a
   # INV: q, r ∈ ℕ, a = bq + r; a, b fixed.
   # BOUND: r.
4:     while r ≥ b do
5:         r ← r − b
6:         q ← q + 1
   # POST: a = bq + r, q ≥ 0, 0 ≤ r < b; inputs unchanged.
7:     return (q, r)
```

Initialisation is immediate. If $a = bQ + R$ and $R \geq b$, then $a = b(Q + 1) + (R - b)$, so the full body preserves the invariant and nonnegativity. This is induction on the number of completed subtractions, which is the quotient counter q: the base is $q = 0$, and the step applies to every integer $q \geq 0$ for which another subtraction is made. At exit $0 \leq r < b$, giving precisely the quotient and remainder conditions. The bound function is the nonnegative remainder r; it strictly decreases on every repetition. For $(a, b) = (17, 5)$, the states (q, r) at tests are $(0, 17), (1, 12), (2, 7), (3, 2)$.

7.4.3 Euclid: preserving the answer while changing the problem

Section 6.3.4 proved the common-divisor property, and Section 6.4.2 expressed it as an invariant. The functional transition $(u, v) \mapsto (v, \mathrm{mod}(u, v))$ preserves the gcd. To realise it with assignments, compute the remainder before overwriting u.

For fixed inputs $a, b \in \mathbb{N}$ not both zero, the loop postcondition is $v = 0$ and $u = \gcd(a, b)$. Since $\gcd(u, 0) = u$ for $u > 0$, rewrite the answer as $\gcd(u, v) = \gcd(a, b)$ at $v = 0$. Generalise by allowing v to be nonzero, while keeping the gcd fixed. This gives the invariant

$$u, v \in \mathbb{N}, \qquad u + v > 0, \qquad \gcd(u, v) = \gcd(a, b).$$

Initialising $(u, v) = (a, b)$ establishes it. The common-divisor property suggests the update above, while the stopping condition $v = 0$ makes the invariant yield $u = \gcd(a, b)$. Temporary storage lets us implement that update without losing an old value that is still needed.

Algorithm 12 Preserve the gcd, using temporary storage for the remainder.

```
1: procedure EUCLID(a, b)
   # PRE: a, b ∈ ℕ, not both zero.
2:     u ← a
3:     v ← b
   # INV: u, v ∈ ℕ, u + v > 0, gcd(u, v) = gcd(a, b); a, b fixed.
   # BOUND: v.
4:     while v > 0 do
5:         r ← mod(u, v)
6:         u ← v
7:         v ← r
   # POST: v = 0, u = gcd(a, b); inputs unchanged.
8:     return u
```

The chosen invariant holds initially. The common-divisor argument from Section 6.3.4 proves preservation for the complete body. Induction on the number of completed repetitions proves it at every test, with base zero and the preservation step for every reached iteration. On exit $v = 0$, so $u = \gcd(a, b)$. The new v is smaller than the old positive v. Thus v is a bound function, proving termination. The logarithmic bound on the number of remainder steps from Section 6.3.4 is unchanged by this translation.

7.5 Lists and a non-numerical invariant

Suppose a list occupies indexed locations $L[1], \ldots, L[m]$. An entry may contain a letter, a name, or another finite object. Reading $L[i]$ retrieves that entry; assigning to $L[i]$ replaces it. We can reverse the list by exchanging its outermost entries and moving inwards.

Let L_0 be the fixed original list of length m. Write $L[a..b]$ for an indexed segment, empty if $a > b$, and write $\mathrm{rev}(S)$ for a sequence S read backwards. These describe mathematical sequences; no extra copies need be stored. Relax the final requirement $L = \mathrm{rev}(L_0)$ by marking exactly which segments are finished. At each loop test the invariant is

$$\begin{gathered} |L| = m, \quad 1 \le \ell \le m+1, \quad 0 \le r \le m, \\ \ell \le r+1, \qquad \ell + r = m+1, \\ L[1..\ell-1] = \mathrm{rev}(L_0[r+1..m]), \\ L[\ell..r] = L_0[\ell..r], \\ L[r+1..m] = \mathrm{rev}(L_0[1..\ell-1]). \end{gathered}$$

Thus the current left end is the reversal of the corresponding original right end; the current right end is the reversal of the original left end; and the entire middle is still unchanged. The two finished ends are not asserted to be reversals of *each other*. Their relation is to the original list, which need not be a palindrome.

$$\underbrace{L[1..\ell-1]}_{\text{finished left end}} \qquad \underbrace{L[\ell..r]}_{\text{unchanged middle}} \qquad \underbrace{L[r+1..m]}_{\text{finished right end}} .$$

Initialising $\ell = 1, r = m$ makes both finished ends empty. While $\ell < r$, the equality $\ell + r = m + 1$ identifies two original entries that belong in each other's final positions. Exchanging them and moving the boundaries inwards extends both finished ends. When $\ell \ge r$, the middle is empty or is the single central entry, whose original position is already its final position. The invariant then implies that the whole list is reversed. This gives the procedure below.

Algorithm 13 Reverse a list by modifying its entries.

```
1: procedure REVERSE(L, m)
   # PRE: L = L_0, |L_0| = m ≥ 0; m fixed.
2:     ℓ ← 1
3:     r ← m
   # INV: |L| = m, 1 ≤ ℓ ≤ m + 1, 0 ≤ r ≤ m,
   # ℓ ≤ r + 1, ℓ + r = m + 1.
   # L[1..ℓ − 1] = rev(L_0[r + 1..m]).
   # L[ℓ..r] = L_0[ℓ..r].
   # L[r + 1..m] = rev(L_0[1..ℓ − 1]).
   # BOUND: max(0, r − ℓ + 1).
4:     while ℓ < r do
5:         Exchange the contents of locations L[ℓ] and L[r].
6:         ℓ ← ℓ + 1
7:         r ← r − 1
   # POST: ℓ ≥ r, |L| = m, L = rev(L_0).
```

Initially the outside part is empty. If $\ell < r$, the two untouched end entries belong in each other's positions because $r = m + 1 - \ell$. Exchanging them and moving the boundaries preserves the claim. Induction on the number of completed exchanges, with base zero and the step for each integer $j \geq 0$ at which another exchange is made, proves the invariant.

At exit there are either no unprocessed entries or just the middle entry, which is already in its correct position. Thus the list is reversed. The bound function $\max(0, r - \ell + 1)$ decreases by two whenever the loop runs, so it terminates after $\lfloor m/2 \rfloor$ exchanges. For example,

$$[\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}] \longrightarrow [\mathrm{E}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{A}] \longrightarrow [\mathrm{E}, \mathrm{D}, \mathrm{C}, \mathrm{B}, \mathrm{A}].$$

The output is a changed arrangement, and the invariant describes which part of that arrangement is finished.

7.6 Searching for an entry in a list

Random access means that $L[i]$ gives direct access to the entry at index i; it does not mean that the index is chosen by chance. We count an entry access, assignment, or comparison as one operation.[3]

We shall work on a segment $L[l..r]$ of a list of length m, where $1 \leq l \leq r + 1 \leq m + 1$. It is empty when $l = r+1$. The parameters l, r, m, x remain fixed. All index variables are integers. Assertions over empty segments are true. In every procedure below the input assertions, loop invariants, and output assertions are written as comments beginning with `#`. They describe the states to be established, not extra commands to execute.

7.6.1 Sequential search

Specify the answer. Given an established segment and a target x, return its first matching index, or -1 if there is no match. The list must remain unchanged. Use a Boolean variable $found$ and an index i. The loop postcondition is:

$$(found \text{ and } \ (l \leq i \leq r \text{ and } L[i] = x)) \text{ or } (\neg found \text{ and } (L[t] \neq x \quad \text{for every } l \leq t < r + 1)).$$

Either $found$ is true and $L[i]$ is the first match, or $found$ is false and $i = r + 1$, which indicates that every entry of the segment differs from x. Generalise the completed unsuccessful scan by allowing i to mark an unfinished position.

[3] This is a model of indexed storage. Following links in a chain or comparing long strings can have additional costs.

Relax the final assertion to obtain the invariant. Instead of claiming that the entire segment has been searched, describe the part before i. At every loop test require the following invariant to hold

$$l \le i \le r+1,$$
$$(found \text{ and } \ (l \le i \le r \text{ and } L[i] = x)) \text{ or } (\neg found \text{ and } (L[t] \neq x \quad \text{for every } l \le t < i))$$

The prefix before i has been ruled out. If $found$ is true, the current entry is a valid first match. If it is false, the current entry may still need to be checked: false does not assert absence from the whole segment. This distinction keeps the invariant true both before and after success.

Initialise, then preserve. Set $found$ to false and $i = l$. The ruled-out prefix is empty. Continue while no match is recorded and $i \le r$. If $L[i] = x$, set $found$ to true and retain i as the answer. Otherwise increase i: the failed comparison allows this entry to join the ruled-out prefix. At exit either a first match is recorded, or $i = r + 1$ and the entire segment has been ruled out. The commands follow from these assertions.

Algorithm 14 Search by extending a prefix known not to contain the target.

```
1:  procedure SequentialSearch(L, l, r, x)
    # PRE: |L| = m, 1 ≤ l ≤ r + 1 ≤ m + 1; x is established.
2:      found ← false; i ← l
    # INV: (found and (l ≤ i ≤ r and L[i] = x)) or (¬found and (L[t] ≠ x for every l ≤ t < i)),
    # l ≤ i ≤ r + 1
    # BOUND: r − i + 1 if not found, otherwise 0.
3:      while ¬found and i ≤ r do
4:          if L[i] = x then
5:              found ← true
6:          else
7:              i ← i + 1
    # POST: (found and (l ≤ i ≤ r and L[i] = x)) or (¬found and (L[t] ≠ x for every l ≤ t <
    r + 1))
8:      if found then
9:          return i
10:     else
11:         return −1
```

Induct on the number of completed iterations. Initialisation proves the base case zero; the two cases just described preserve every clause for any integer $t \ge 0$ at which another iteration is made. On success the loop stops; on failure $r - i + 1$ decreases. Equivalently, use the bound function $r - i + 1$ when $found$ is false and zero otherwise. Each executed iteration strictly decreases it. There are at most $r - l + 1$ comparisons for a nonempty segment, so the worst-case work is linear in its length.

7.6.2 Binary search

Now consider that the given segment is sorted in nondecreasing order. Return any matching index, or -1 if the target is absent, without changing the list. We can then construct a more efficient search algorithm.

This is also a search problem, and the loop postcondition is unchanged:

$$(found \text{ and } \ (l \le i \le r \text{ and } L[i] = x)) \text{ or } (\neg found \text{ and } (L[t] \neq x \quad \text{for every } l \le t \le r)).$$

Since the list is sorted in the increasing order, a middle entry smaller than x excludes itself and every entry to its left; a larger entry excludes itself and every entry to its right. We therefore relax the final assertion by keeping a candidate interval $[left, right]$ inside the original segment.

The invariant. Let $mid = \mathrm{div}(left + right, 2)$. Then, we can define the invariant condition by relaxing the postcondition as:

$$(found \text{ and } \ (l \le mid \le r \text{ and } L[mid] = x))$$
$$\text{or}$$
$$(\neg found \text{ and } ((L[t] \ne x \quad \text{for every } l \le t < left) \text{ and } (L[t] \ne x \quad \text{for every } right < t \le r).$$

This statement concerns all occurrences, including duplicates. A recorded match lies within the remaining interval; an empty interval certifies absence.

Initialise and derive the loop. Take $found =$ false, $left = l$, $right = r$. No entries have been excluded and the invariant holds. While there is no recorded match and the interval is nonempty, inspect its middle. Equality sets $found$ to true. The two strict inequality cases move the appropriate boundary past the middle entry. Sortedness is exactly what makes these exclusions preserve the invariant.

Algorithm 15 Bracket every possible match between two moving boundaries.

```
1:  procedure BinarySearch(L, l, r, x)
    # PRE: |L| = m, 1 ≤ l ≤ r + 1 ≤ m + 1;
    # L[l..r] is sorted; entries and x are established
2:      found ← false; left ← l; right ← r
    # INV: (found and (l ≤ mid ≤ r and L[mid] = x))
    #  OR
    # (¬found and ((L[t] ≠ x  for every l ≤ t < left) and (L[t] ≠ x  for every right < t ≤ r)
    # BOUND: right − left + 1 if not found, otherwise 0.
3:      while ¬found and left ≤ right do
4:          mid ← div(left + right, 2)
5:          if L[mid] = x then
6:              found ← true
7:          else if L[mid] < x then
8:              left ← mid + 1
9:          else
10:             right ← mid − 1
    # POST: (found and (l ≤ mid ≤ r and L[mid] = x)) or (¬found and (L[t] ≠ x  for every l ≤
    t ≤ r))
11:     if found then
12:         return mid
13:     else
14:         return −1
```

A bound function is $right - left + 1$ when $found$ is false and zero when it is true. It is nonnegative at each test and strictly decreases on every iteration, including the successful one. An unsuccessful iteration at least halves the interval length; a successful one ends the search. For a segment of length n, the inspection count obeys $B(0) = 0$ and $B(n) \le 1 + B(\lfloor n/2 \rfloor)$ for $n > 0$. Thus the total work is $O(\log n)$, as with the halving recurrence in Section 6.3.3. For $L = [2, 5, 8, 12, 16, 18, 23, 31]$, $l = 1, r = 8$, searching for 18 inspects indices $4, 6$ and succeeds. Searching for 17 inspects $4, 6, 5$ and finishes with $left = 6, right = 5$, certifying absence.

7.7 Sorting by selecting the smallest remaining entry

Suppose we want to sort a segment $L[l..r]$ in nondecreasing order. Write L_0 for the original list, and assume $1 \le l \le r + 1 \le |L| + 1$. The segment is empty when $l = r + 1$. The entries can be compared, and the indices l and r remain fixed.

The postcondition has three parts:

$$L[l..r] \text{ is sorted in nondecreasing order,}$$
$$L[l..r] \text{ is a rearrangement of } L_0[l..r],$$
$$\text{every entry outside } [l, r] \text{ is unchanged.}$$

A rearrangement preserves every entry, including its multiplicity. Throughout the procedure, the length of the list is unchanged.

We develop the procedure in two stages. First we describe the outer loop, leaving its inner task specified by a postcondition. Then we derive a procedure that accomplishes that task.

7.7.1 The outer loop: extend a sorted partition

Generalise the final assertion by introducing a boundary i. The segment $L[l..i-1]$ is the sorted part, while $L[i..r]$ is the remaining part. Calling the latter "unsorted" means that we have not yet established its order.

A sorted prefix alone is not enough. To extend it by choosing the smallest remaining entry, we also require that every entry already in the prefix is at most every remaining entry. Our outer invariant is

$$\begin{aligned} I_{outer} \quad = \quad & l \le i \le r+1, L[l..i-1] \text{ is sorted}, L[a] \le L[b] \qquad \text{whenever } l \le a < i \le b \le r, \\ & L[l..r] \text{ rearranges } L_0[l..r], \text{ entries outside } [l, r] \text{ and the list length are unchanged.} \end{aligned}$$

Assertions about all entries of an empty segment hold automatically. Thus $i = l$ establishes the invariant: the sorted part is empty.

Suppose $i \le r$. What must be accomplished before advancing the boundary? The required assertion, which we call Q, is

$$\begin{aligned} Q \quad = \quad & L[l..i] \text{ is sorted}, L[a] \le L[b] \qquad \text{whenever } l \le a \le i < b \le r, \\ & L[l..r] \text{ rearranges } L_0[l..r], \text{ entries outside } [l, r] \text{ and the list length are unchanged.} \end{aligned}$$

Here i has not yet changed. Once Q holds, the assignment $i \leftarrow i + 1$ restores the outer invariant.

To establish Q, select a minimum entry of $L[i..r]$ and exchange it with $L[i]$. The old sorted prefix remains unchanged. Its entries are at most the selected minimum, and that minimum is at most every entry still to its right. An exchange also preserves multiplicities. We postpone the implementation of this task.

Algorithm 16 The outer loop, with the inner task specified (Statement 4) but not yet implemented.

```
1: procedure SelectionSort(L, l, r)
   # PRE: L = L_0, 1 ≤ l ≤ r + 1 ≤ |L| + 1; entries are comparable.
2:     i ← l
   # INV: I_outer
   # BOUND: r − i + 1.
3:     while i ≤ r do
4:         Establish Q by selecting a minimum of L[i..r] and exchanging it with L[i].
             # INNER POST: Q
5:         i ← i + 1
   # POST: L[l..r] is sorted and rearranges L_0[l..r];
   # outside entries and the list length are unchanged.
```

The instruction to establish Q is a task still to be developed. Assuming it finishes with its stated postcondition, the following increment preserves the outer invariant.

The bound function $r - i + 1$ is a nonnegative integer at every loop test and decreases by one on each completed iteration. To finish the termination argument, we must show that the inner task terminates.

7.7.2 The inner task: find a minimum and exchange it

The inner task must establish Q. As argued above, it is enough to find an index p satisfying

$$R = i \le p \le r, \qquad L[p] \le L[t] \quad \text{for every } i \le t \le r,$$

without changing the list during the search, and then exchange $L[p]$ with $L[i]$.

This is the postcondition of the minimum-search loop. Generalise its completed search range $[i, r]$ to $[i, j-1]$, where j is the next position to inspect. The inner invariant becomes

$$I_{inner} = i + 1 \le j \le r + 1, \qquad i \le p < j, \qquad L[p] \le L[t] \quad (i \le t < j).$$

The list remains unchanged during this loop, as do i, l, r. Consequently, the outer invariant continues to hold during the search.

Initially, set $p = i$ and $j = i+1$. The inspected segment contains just one entry, which is necessarily a minimum of that segment. If $j \le r$, compare $L[j]$ with the recorded minimum. When $L[j] < L[p]$, replace p by j; otherwise retain p. Advancing j then extends the inspected segment by one entry and restores the invariant.

Algorithm 17 The independent refinement of the inner task.

```
1: procedure SelectAndPlaceMinimum(L, i, r)
   # PRE: the outer invariant holds and i ≤ r.
2:     p ← i
3:     j ← i + 1
   # INV: I_inner
   # BOUND: r − j + 1.
4:     while j ≤ r do
5:         if L[j] < L[p] then
6:             p ← j
7:         j ← j + 1
   # LOOP POST: R,
   # The list is unchanged, so the outer invariant still holds.
8:     Exchange the contents of L[p] and L[i].
   # POST: Q: L[l..i] is sorted;
   # every entry of L[l..i] is at most every entry of L[i + 1..r]; L[l..r] rearranges L_0[l..r]; outside
   entries and length unchanged.
   #
```

bound function $r - j + 1$ is nonnegative at each test and strictly decreases on every iteration. At exit $j = r + 1$, so p identifies a minimum of the whole remaining segment. The exchange establishes Q, completing both the inner task and the justification of the outer loop. If $p = i$, the exchange leaves the list unchanged.

Replacing the specified task in the outer outline by SelectAndPlaceMinimum gives the complete selection-sort procedure.

Algorithm 18 The Selection Sort Algorithm.

```
1:  procedure SelectionSort(L, l, r)
    # PRE: L = L_0, 1 ≤ l ≤ r + 1 ≤ |L| + 1; entries are comparable.
2:      i ← l
    # INV: I_outer
    # BOUND: r − i + 1.
3:      while i ≤ r do
4:          p ← i
5:          j ← i + 1
              # INV: I_inner
              # BOUND: r − j + 1.
6:          while j ≤ r do
7:              if L[j] < L[p] then
8:                  p ← j
9:              j ← j + 1
              # LOOP POST: R,
              # The list is unchanged, so the outer invariant still holds.
10:         Exchange the contents of L[p] and L[i].
              # INNER POST: Q: L[l..i] is sorted;
              # every entry of L[l..i] is at most every entry of L[i + 1..r]; L[l..r] rearranges L_0[l..r];
    outside entries and length unchanged.
11:         i ← i + 1
    # POST: L[l..r] is sorted and rearranges L_0[l..r];
    # outside entries and the list length are unchanged.
```

7.7.3 Counting the work

For a segment of length $n = r - l + 1$, let $T(n)$ be the number of comparisons required. Then, the recurrence for $T(n)$ is governed by

$$\begin{aligned} T(1) &= 0 \\ T(n) &= T(n-1) + (n-1) \end{aligned}$$

Unfolding the recurrence we obtain

$$T(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

entry comparisons. There are at most n exchanges, including exchanges of an entry with itself. Thus the running time is $O(n^2)$ under constant-cost comparisons, accesses, and exchanges. The comparison count is the same even when the original segment is already sorted.

7.8 Procedures can be recursive too

Imperative computation does not require us to abandon recursion. A procedure can call itself on a smaller task. Each call has its own input parameters and temporary locations; when that call finishes, execution resumes at the next instruction in its caller. Unfinished calls require storage, just as deferred computations did in the previous chapter.

A procedure such as Factorial returns a value. A procedure such as Exchange or Reverse changes specified storage. The following procedures change a board or a puzzle configuration. Their specifications must say both what they accomplish and what they leave undisturbed. This allows the result of one call to be used safely by the next.

7.8.1 Trominoes: carrying out an inductive proof

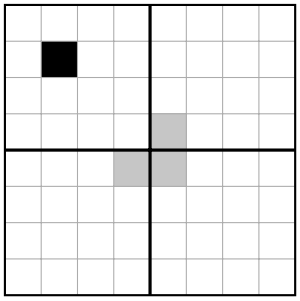
Recall the proof of Theorem 5.40 that a $2^n \times 2^n$ board with one square missing can be tiled by L-shaped trominoes. Its inductive step was already a plan of action: divide the board into quadrants, place one central tromino, and solve four smaller problems.

For a recursive procedure, broaden the description of a missing square. A square may be *reserved*: either it is the original missing square or it is already covered by a tile placed at an earlier stage. A call must fill all other squares of its region, and must not touch the reserved square or anything outside that region. All other squares in the region are initially empty. This is the precondition that the recursive calls actually need.

The quadrant loop's postcondition is that all four quadrants are tiled, with their reserved squares and everything outside the region unchanged, and no overlaps. Generalise "all four" to "the first j": after j calls, where $0 \le j \le 4$, the first j quadrants in clockwise order are completely covered except for their reserved squares; the remaining quadrants have only their reserved squares unavailable and are otherwise empty. All reserved squares, and everything outside the region, are unchanged. No tiles overlap. The central tile has already been placed. The case $j = 0$ describes the state immediately after that placement. Completing one ready quadrant establishes the case $j + 1$ without disturbing the others; $j = 4$ gives the required covering. The inductive specification for a recursive call is exactly what is needed to preserve this invariant.

Algorithm 19 Recursive algorithm for tiling a deficient $2^n \times 2^n$ board. Click here to download a Python code that illustrates the solution,

```
1: procedure TILE(B, n, x)
Require: B is a 2^n × 2^n region, n ≥ 0, with one reserved square x; its other squares are empty.
2:     if n = 1 then
3:         Place one L-shaped tromino on the three squares of B other than x
4:         return
5:     Divide B into four equal quadrants B_1, B_2, B_3, B_4
6:     Let B_i be the quadrant containing the missing square x
7:     Place an L-shaped tromino at the centre so that it occupies one central square in each
       quadrant B_j, j ≠ i
       j ← 1
8:     while j ≤ 4 do
9:         if j = i then
10:            x_j ← x
11:        else
12:            x_j ← the central square of B_j covered by the newly placed tromino
13:        TILE(B_j, n − 1, x_j)
14:        j ← j + 1
```



Black: original reserved square

Grey: central tile; one reserved square in each other quadrant

Figure 7.2: One placement creates four smaller instances of the same task.

Correctness follows Theorem 5.40.

If $T(n)$ counts tiles placed, then

$$T(0) = 0, \qquad T(n) = 4T(n-1) + 1 \quad (n \geq 1), \qquad T(n) = \frac{4^n - 1}{3}.$$

The last equality follows by unfolding the recurrence or by counting squares. With constant work per call apart from its subcalls, the running time is proportional to the board's area 4^n. It is exponential in n, but linear in the number of squares to be covered. Here the size parameter matters: the produced tiling itself contains that many pieces of information.

7.8.2 The Tower of Hanoi: a computation that produces actions

Three pegs hold n disks of different sizes. Initially all disks are on one peg, with smaller disks above larger ones. Move the tower to another peg, moving only one top disk at a time and never placing a larger disk on a smaller one. The third peg is available as a spare.

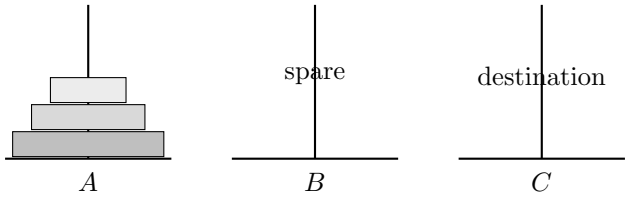


Figure 7.3: Three disks initially on A. Move the whole tower to C.

To uncover the largest disk, first move all smaller disks to the spare peg. Then move the largest disk to the destination. Finally move the smaller tower onto it. The proof suggests the procedure directly.

The promised result is that the specified tower moves from A to C in legal order, using only legal moves, and all larger disks remain where they were. Allowing larger disks underneath the moving tower is important: that is the situation faced by the recursive calls.

Throughout these actions the disks must remain distributed among the three pegs without loss or duplication, each peg must have smaller disks above larger ones, and all disks outside the specified tower must remain in place. These are the properties to preserve. Since this procedure is recursive rather than a loop, its main correctness argument will use the inductive specification of each call.

Algorithm 20 Move n disks from the first named peg to the last, using the middle peg as spare. Click here to download a python code that illustrates the solution.

1: **procedure** HANOI(n, A, B, C)
`# PRE:` $n \in \mathbb{N}$. The n disks form a top tower on A; the pegs are distinct. Any other disks on the three pegs are larger than these n disks.
`# INV (safety):` no disks lost or duplicated; each peg remains ordered;
`#` all disks outside this tower remain in place.
2: **if** $n > 0$ **then**
3: Call HANOI$(n-1, A, C, B)$.
`# ASSERT:` the smaller tower is on B; the largest disk is exposed on A.
4: Move the top disk from A to C.
5: Call HANOI$(n-1, B, A, C)$.
`# POST:` the specified tower is on C in order, all moves were legal,
`#` and every other disk is unchanged.

Correctness by induction on n. For $n = 0$, doing nothing meets the specification. Suppose for some integer $k \geq 0$ that every call moving k disks satisfies the specification. In a call moving $k+1$ disks, the first recursive call legally moves the k smaller disks from A to B, leaving the largest

disk exposed. The top of C is empty or larger than that disk, so the single move from A to C is legal. The second recursive call legally moves the smaller tower from B to C, on top of the disk just moved. Both calls preserve all larger disks. This proves the step. The call parameter decreases to zero, proving termination.

For two disks, the actions are $A \to B$, $A \to C$, $B \to C$. For three disks, they are

$$A \to C,\ A \to B,\ C \to B,\ A \to C,\ B \to A,\ B \to C,\ A \to C.$$

Each arrow means "move the top disk", not "move the entire tower". The output is an ordered sequence of actions. Changing their order can make a move illegal, even if the same peg names occur.

If $H(n)$ counts moves, then

$$H(0) = 0, \qquad H(n) = 2H(n-1) + 1 \quad (n \geq 1).$$

Adding one on both sides and unfolding gives $H(n) + 1 = 2^n$, so $H(n) = 2^n - 1$. This exponential growth is unavoidable for three pegs. Before the first move of the largest disk, all smaller disks must have been transferred to a single other peg. After the last move of the largest disk onto its destination, all smaller disks again form a tower on another peg and must be transferred onto it. These are two separate transfers of $n - 1$ disks, with at least one move of the largest disk between them. Thus, if $M(n)$ is the minimum number of moves, $M(n) \geq 2M(n-1) + 1$. Induction on n, with base $M(0) = 0$ and step $M(k+1) \geq 2(2^k - 1) + 1 = 2^{k+1} - 1$ for every integer $k \geq 0$, gives the lower bound. Our procedure attains it.

At one move per second, 64 disks require about 585 billion years. A short recursive description can specify a very long computation. [4]

7.9 What the two models share

For a tail-recursive procedure, the arguments describing the current state become stored variables; the stopping test becomes a loop condition; and the recursive transition becomes the loop body. Its invariant carries over. Separate assignments must implement the intended transition correctly, which may require temporary storage.

For general recursion, unfinished calls also form part of the state. Hanoi must remember to move the largest disk and make the second call after its first call finishes. A stack can record these pending tasks. We can in principle replace recursive calls by a loop processing such a stack, but this does not remove the pending work or its storage requirements. Recursion and iteration remain different ways of organising computation within an imperative description.

The functional viewpoint emphasises how an answer is built from answers to smaller problems. The imperative viewpoint emphasises how instructions change a stored state. Neither confines us to numerical answers: a tiling, a reversed list, or a sequence of moves can be represented as finite data. The numerical model of Chapter 6 can encode such data; here we describe it in the form most natural for the task.

The thread connecting all these examples is the proof. An induction tells us how to divide a recursive task. An invariant tells us how to change a state without losing the relationship that leads to the answer. In both cases, reasoning is part of constructing the procedure.

Problems

For each loop you design, first state its postcondition, generalise it to an invariant, and then derive the commands. Give a nonnegative integer bound function and show that it strictly decreases on every iteration.

[4] Édouard Lucas published the puzzle in 1883 under the name N. Claus de Siam, an anagram of Lucas d'Amiens. Its accompanying temple story is part of the puzzle's presentation, not evidence of an ancient algorithm. See the MacTutor historical account.

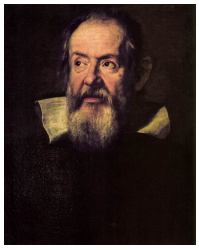
1. Start with $x = 3, y = 7$. Trace $x \leftarrow x + y$ followed by $y \leftarrow x - y$. Add one assignment that exchanges the original values. Explain why a temporary location is often easier to reason about.
2. Translate $(u, v) \mapsto (v, u + v)$ into sequential assignments. Give a failing translation and explain which old value it loses.
3. Trace Factorial for $n = 0$ and $n = 5$. At which point inside the body can its invariant fail? Show that it is restored before the next test.
4. Design an increasing-counter factorial loop with invariant $p = k!$ and $0 \leq k \leq n$. Give initialisation, preservation, exit, and termination arguments. Compare its update order with Factorial.
5. Translate repeated-addition multiplication into a loop. Use the invariant $p + ak = ab$ from Section 6.4.2, with fixed inputs $a, b \geq 0$. Count additions.
6. Construct an imperative power procedure from $px^k = X^N$, where X, N are fixed inputs. Compare Sections 6.3.3 and 6.4.2. Use even and odd cases to halve k on each repetition. Specify assignment order and prove correctness.
7. Trace Sum for $f(i) = i^2$, $a = 2$, $b = 5$. Explain why advancing k before adding $f(k)$ changes the terms summed. Handle $a > b$.
8. Trace Divide for $(23, 4)$. Prove that its repetition count is the quotient. Explain why $b > 0$ is necessary for termination.
9. Replace Euclid's body by $u \leftarrow v$ followed by $v \leftarrow \mathrm{mod}(u, v)$. Find a counterexample and explain the failure in terms of storage.
10. Give a loop and a true invariant for which the loop never ends. Explain which part of a correctness proof is missing.
11. Trace Reverse on lists of lengths $0, 1, 2$, and 6. Prove that no entry outside the list is accessed. Count the temporary locations needed.
12. Design a procedure that finds a largest entry of a nonempty list. Use an invariant describing the largest entry in the processed prefix. State what happens when the largest value occurs more than once.
13. Draw every tile placed by Tile on a 4×4 board with one corner missing. Repeat with an interior square missing. Label the order of placements, following the stated quadrant order.
14. Why must Tile permit a reserved square that is already occupied? Explain why "one square is physically missing" is not a suitable specification for every recursive call.
15. Prove $T(n) = (4^n - 1)/3$ from its recurrence by induction on n, explicitly quantifying the induction hypothesis. Relate this to area.
16. Trace Hanoi for three disks, recording the peg state after each move. Check legality as well as the final destination.
17. Prove the lower bound for Hanoi by induction on the number of disks. Which step relies on having exactly three pegs and fails with a fourth?
18. Describe a stack of pending Hanoi tasks that a loop could process. In what order must tasks be placed on a last-in, first-out stack to reproduce the recursive procedure? Distinguish the task stack from the pegs.
19. For Exchange, Sum, Tile, and Hanoi, state the input, output or changed state, and permitted actions. Explain how the non-numerical data could, in principle, be encoded by natural numbers.
20. Trace SequentialSearch on a list containing several copies of its target. Prove that it returns the first occurrence. Give its best and worst cases.

21. Trace BinarySearch for an absent target smaller than every entry, larger than every entry, and between two consecutive entries. Include the empty list and the one-entry list.
22. Modify BinarySearch to return the first occurrence of its target. State a new invariant that records a possible answer as the interval shrinks.
23. Find an unsorted list on which BinarySearch incorrectly reports absence. Identify precisely which preservation argument fails.
24. In Stage 1 of insertion sort, why is a sorted prefix alone an incomplete invariant? Explain the roles of multiplicities and unchanged entries.
25. Starting from the assertion Q, justify every clause of the inner invariant. Give an example where the entire stored list is temporarily not a rearrangement of its original contents.
26. Give a nonnegative real-valued function that strictly decreases in an infinite loop. Explain why our definition requires integer values.
27. Give bound functions for both searches, including successful iterations. Why does the number of uninspected entries alone fail to strictly decrease on a successful iteration?
28. Trace every assignment while inserting 1 into the sorted prefix $[2, 4, 5]$. Mark the logical hole and the saved value after each shift.
29. Why does replacing $L[j-1] > x$ by $L[j-1] \geq x$ in InsertionSort still sort the values but change the order in which equal entries can finish? Use labelled cards such as 2_a and 2_b to illustrate.
30. Prove that insertion sort makes exactly $m(m-1)/2$ shifts on m distinct entries presented in decreasing order. State the induction parameter and quantify the induction hypothesis.
31. Suppose an unchanging list of length m will be searched q times. Compare q linear searches with insertion sort followed by q binary searches, using the bounds in this chapter. Explain why these bounds alone do not determine an exact practical crossover point.

Chapter 8

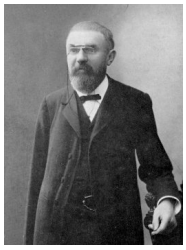
Measuring the world with geometry

Geometry as a way of reasoning



"It is written in the language of mathematics, and its characters are triangles, circles, and other geometric figures"
—Galileo Galilei, *The Assayer* (1623), excerpt; translated by Stillman Drake.

Portrait by Justus Sustermans, 1636. Public domain.



"it is by logic one demonstrates, by intuition one invents."
—Henri Poincaré, *Science and Method*, Book II, Chapter II; excerpt, translated by G. B. Halsted.

Frontispiece of *Last Thoughts*, 1913. Public-domain image (Wikimedia Commons).

How can we measure a building without climbing it? Locate ourselves from distances to landmarks? Find a shortest route that has to touch a boundary? Choose a shape that uses a fixed amount of material efficiently? These questions ask us to do more than remember a formula. We must decide what to represent, what to measure, and why the resulting construction works.

In the preceding chapters, a proof guided the construction of an algorithm. Geometry lets us follow the same approach with diagrams. A figure suggests relationships; a proof explains which of them must hold. We will use similar triangles, lengths, areas, and volumes that are already familiar, and turn them into tools for solving practical problems.

8.1 From a place to a geometric model

A building has windows, ledges, and an irregular roof. For a shadow measurement, we may represent it by a vertical segment of unknown length. We represent level ground by a straight line and sunlight by parallel rays. These choices form a *model*: we retain the features relevant to the question and set others aside.

A conclusion can be correct within a model and still give a poor answer about the place being measured. Perhaps the ground slopes, the building leans, or we measure the shadow of the wrong part of the roof. We therefore keep two questions separate: does the reasoning follow from the assumptions, and are those assumptions suitable here?

Throughout this chapter we use plane Euclidean geometry for local maps, and straight-line distance unless another notion of distance is specified. Diagrams explain the construction; their drawn sizes are not measurements.

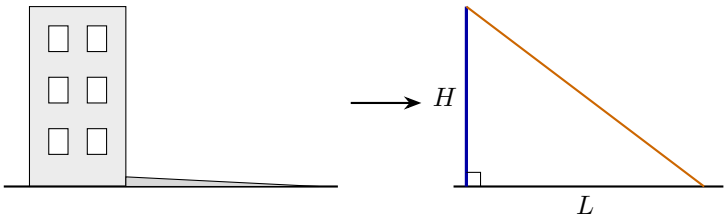


Figure 8.1: A model makes selected relationships explicit. The right angle is an assumption to check, not a consequence of the drawing.

8.2 Measuring what we cannot reach

8.2.1 A building and a stick

The problem. Determine the height H of a vertical building from a stick of known height $h > 0$ and simultaneous shadow lengths $s > 0$ (stick) and $L > 0$ (building). Assume level ground, vertical objects, and parallel sunlight.

Place a vertical stick of known height h on the same level ground, and measure its shadow s and the building's shadow L at the same time. Assume the tips of both shadows can be identified and the Sun is above the horizon, so $s > 0$.

The two right triangles have the same angle with the sunlight, whose rays we treat as parallel. They are similar, and hence

$$\frac{H}{L} = \frac{h}{s}, \qquad H = \frac{hL}{s}.$$

The inaccessible length has been replaced by three accessible ones.

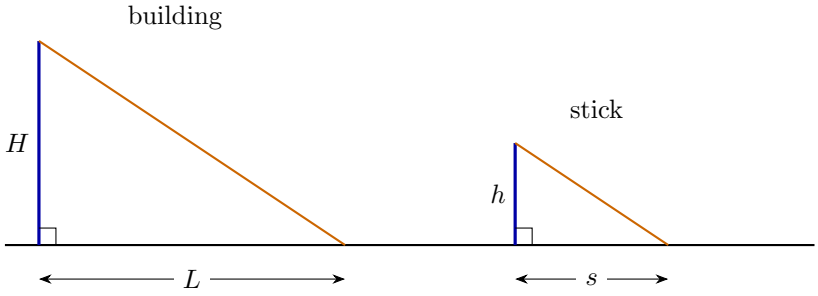


Figure 8.2: Equal sunlight angles and vertical objects give similar triangles.

A measurement procedure. First check verticality and the ground. Measure h, then measure s and L as nearly simultaneously as possible. Calculate hL/s and record the units and the conditions under which the measurement was made. For $h = 1.5$ metres, $s = 2$ metres, and $L = 18$ metres, the estimate is $H = 13.5$ metres. The numerical answer alone does not tell us how accurate it is.

8.2.2 How much should we trust the answer?

Suppose $h = 1.5$ metres is treated as exact, but the shadow measurements lie in the intervals

$$17.8 \le L \le 18.2, \qquad 1.9 \le s \le 2.1.$$

The problem. Under the same shadow model, find lower and upper bounds for H that contain every value consistent with these measurement intervals. All lengths in the displayed intervals are in metres.

Increasing L increases H, while increasing the positive denominator s decreases it. Consequently,

$$\frac{1.5(17.8)}{2.1} \le H \le \frac{1.5(18.2)}{1.9}, \qquad 12.71\ldots \le H \le 14.36\ldots.$$

A conservative rounded report is $12.7 \le H \le 14.4$ metres. This interval accounts for the stated measurement uncertainty within the model; it does not account for a leaning stick or sloping ground.

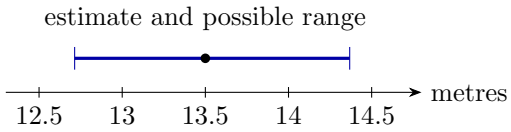


Figure 8.3: An interval often communicates a measurement more honestly than extra decimal places.

If a shadow is very short, a small absolute error can be a large fraction of its length. A longer shadow may help, but a low Sun can make its tip harder to identify. A useful procedure must consider both the geometry and the quality of the observations.

8.2.3 Can a photograph replace a shadow?

A shadow turned an inaccessible height into a ratio of accessible lengths. Could a smartphone photograph do the same job? A photograph also records rays of light, but its geometry is different. Sunlight was modelled by parallel rays; a camera is modelled by rays passing through a single point, the *camera centre*. Similar triangles will again do the work.

A dark room, a small hole, and a picture. The point-camera model has a physical counterpart: a dark box with a small hole in one side and a screen opposite it. Light from the top of an object passes through the hole and reaches the lower part of the screen; light from the bottom reaches the upper part. The image is inverted. A darkened room can do the same thing, which explains the name *camera obscura*, Latin for "dark room". A pinhole camera uses this principle without a lens. The image can be viewed on a screen; recording it permanently requires a light-sensitive surface.

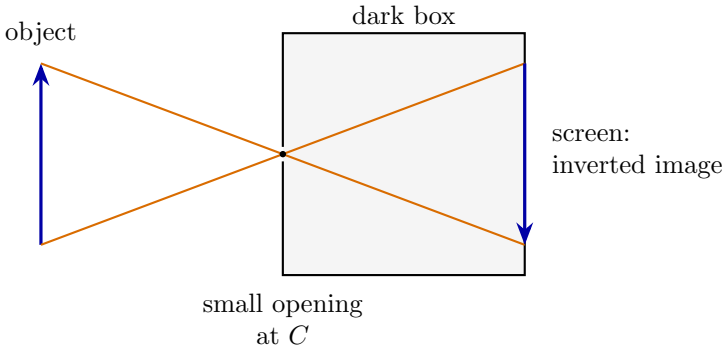


Figure 8.4: The camera obscura principle. Rays cross at the small opening; the screen is behind it, on the opposite side from the object. The arrows represent the object and its inverted image, not the direction of light. The ideal pinhole cannot physically exist. Modern cameras are approximations of the ideal model.

Pinhole-image effects were known in antiquity. In the eleventh century, Ibn al-Haytham (Alhazen, c. 965–c. 1040) investigated light and vision through mathematical reasoning and experiments,

including dark-room arrangements. His work helped establish how straight-travelling light produces images and how vision depends on light entering the eye. [1]



Ibn al-Haytham, as imagined in the frontispiece of Hevelius's *Selenographia* (1647). A later representation, not a contemporary likeness. Polona / Wikimedia Commons; public domain.



Leonardo da Vinci (1452–1519). Portrait attributed to his pupil Francesco Melzi, Royal Collection. Wikimedia Commons; public domain.

Figure 8.5: Optics and pictorial perspective study a common geometry: how points in space correspond to points in an image.

Leonardo da Vinci studied light, vision, and perspective, and used linear perspective to organise painted space. In *The Last Supper*, the receding architectural lines help create the impression that a flat wall opens into a room. In a perspective construction, one imagines straight lines from the scene to a fixed eye and marks where they cross a picture plane: the same geometric arrangement as our point-camera model.[2]

Later camera obscuras often used lenses to admit more light and mirrors to redirect the image, and became aids to drawing. A smartphone likewise uses lenses and a sensor, rather than a literal pinhole. Nevertheless, central projection remains a useful geometric model when lens distortion is negligible or corrected. The camera makes an image from a scene; our measurement problem uses known features of the scene to reason back from the image.

The problem. A single photograph shows two vertical objects standing on the same flat, horizontal ground. One has known height $h_0 > 0$; the other has unknown height $h > 0$. Their tops and feet are visible. Four marked ground points are also visible, and their positions on a ground plane are known. Determine h without measuring the camera's height. For this construction, assume that the camera is higher than both objects, that neither stands directly below it, and that their feet and the point below the camera are not collinear. We will explain why these assumptions are useful.

What does an image point tell us? A pixel identifies a viewing line through the camera centre, not a unique point in space. Every point on that line projects to the same image position. However, if we know that a point lies on the ground plane, the intersection of its viewing line with that plane locates it.

For now, suppose a function H performs this intersection calculation:

$$H(x, y) = \text{the ground coordinates corresponding to image position } (x, y).$$

We will construct H from the four marked points later. It maps positions in the photograph to positions on a plan of the ground. It does not recover the height of an arbitrary object by itself.

[1] For Ibn al-Haytham's experimental approach, see UNESCO, "Ibn al-Haytham's scientific method", and "Who was Ibn al-Haytham?". The Swiss Camera Museum discusses the camera obscura's ancient antecedents and later use as a drawing instrument.

[2] See the Museo del Cenacolo Vinciano's discussion of perspective in *The Last Supper*. The use of linear perspective is distinct from evidence of tracing a particular painting with an optical device.

Let C be the camera centre and O its vertical projection onto the ground. Write $c = CO$ for the camera height. Let T and F be the top and foot of the unknown object, and let t, f be their image positions. We avoid the image inversion problem by considering an equivalent virtual image plane at the same distance in front of the camera, Convince yourself that the similar triangle relations hold. Then

$$H(f) = F, \qquad H(t) = R.$$

Here R is where the line from C through T, continued beyond T, meets the ground. *Mapping the image of the top does not give the foot.* It gives a different point on the same viewing line. The assumption $c > h$ ensures that this line meets the ground beyond the object. However, convince yourself that this is not strictly required.

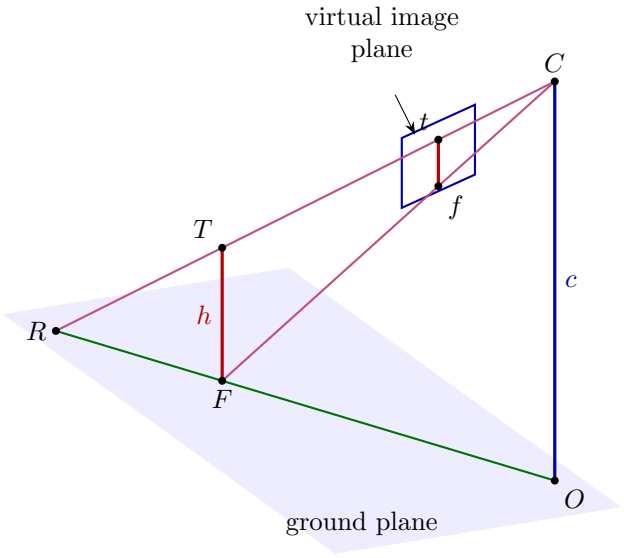


Figure 8.6: A spatial view of the construction, redrawn from the supplied metrology sketch. The viewing lines through image points t, f meet the ground at R, F. The camera centre C is vertically above O. The image plane shown in front of C is a convenient *virtual* plane, not the physical sensor behind the opening.

For drawing purposes we may place an imaginary image plane in front of C, where it cuts the viewing lines before they reach the scene. This gives an upright image and simplifies the diagram. Moving to this virtual plane does not change the viewing lines or their ground intersections. The physical pinhole image in Figure 8.4 is on the other side of C and is inverted. Figure 8.7 isolates the vertical cross-section that we need for the proof.

The triangles TFR and COR share the angle at R and each has a right angle. Consequently,

$$\frac{h}{c} = \frac{FR}{OR}, \qquad h = c\,\frac{FR}{OR}.$$

The lengths on the right are distances on the ground plan, after applying H. Ratios of uncorrected pixel distances generally do not give the same answer.

Finding the point below the camera. For the reference object, write $F_0 = H(f_0)$ and $R_0 = H(t_0)$, using its foot and top image positions. The camera and each vertical object lie in a vertical plane. That plane meets the ground in a line through O. Thus O, F, R are collinear, and so are O, F_0, R_0. Draw the two full lines:

$$O = \text{line}(F, R) \cap \text{line}(F_0, R_0).$$

This explains the noncollinearity assumption in the problem: the two lines must be distinct to determine their intersection. O need not lie inside the photographed patch of ground.

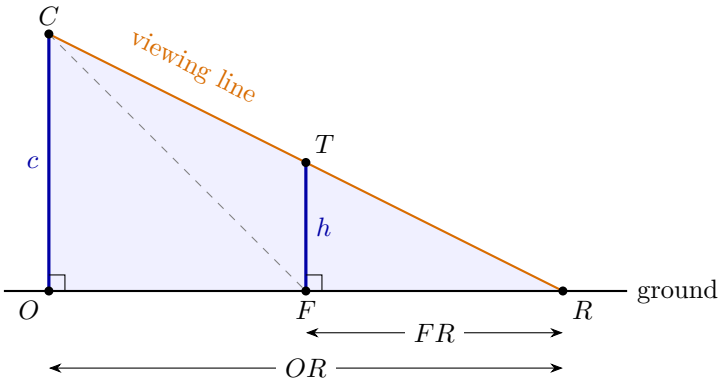


Figure 8.7: A vertical cross-section through the camera and the object. The image of the foot maps to F; the image of the top maps to R. The right triangles TFR and COR are similar. The image plane is omitted so that the geometry of the measurement is visible.

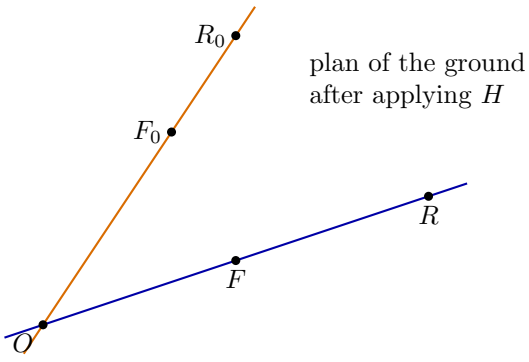


Figure 8.8: Two vertical objects locate the point O below the camera. Join each mapped foot to its mapped top and extend the lines to meet. These are ground-plane lines, not the vertical objects themselves.

Eliminating the camera height. The reference object gives another pair of similar triangles, so

$$\frac{h_0}{c} = \frac{F_0R_0}{OR_0}.$$

Dividing the equation for the unknown object by this equation eliminates c:

$$\boxed{h = h_0 \, \frac{FR/OR}{F_0R_0/OR_0}.}$$

For example, if $FR/OR = 0.6$, $F_0R_0/OR_0 = 0.4$, and $h_0 = 1$ metre, then $h = 1.5$ metres. Multiplying every ground-plan length by the same scale factor changes neither ratio. Thus an absolute ground-distance scale is unnecessary once a reference height is supplied. Note, that if H is known then no measurements on ground are necessary.

Four points straighten the ground plane. How do we obtain H? Imagine marking a rectangle on the ground and photographing it at an angle. Its image usually looks like a quadrilateral: parallel sides may appear to converge. Suppose we know which image corner corresponds to each corner of the rectangle. We seek a mapping that sends those four image positions back to their positions on the ground plan. The required mapping is a *homography*, or plane perspective transformation. It preserves straight lines, but generally not lengths, angles, or parallelism.

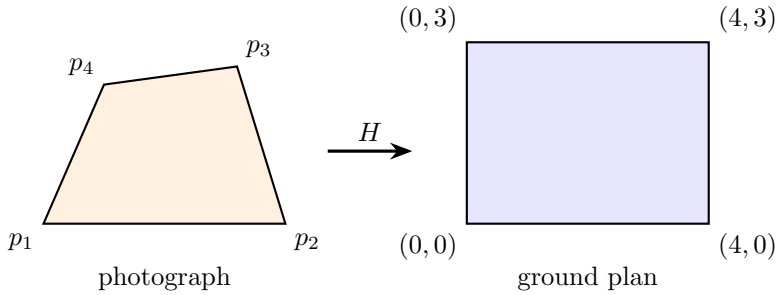


Figure 8.9: Four correspondences calibrate the mapping: $p_1 \mapsto (0,0)$, $p_2 \mapsto (4,0)$, $p_3 \mapsto (4,3)$, $p_4 \mapsto (0,3)$. The ground coordinates must describe the marked points; they cannot be assigned arbitrarily to four unrelated clicks.

Four pairs of corresponding points determine a homography, provided the points are distinct and no three are collinear in either plane. For our rectangle, measure its side lengths or their ratio and use coordinates such as $(0,0), (4,0), (4,3), (0,3)$. Click the four image corners in the matching order. A supplied computational procedure finds the mapping. It can then be evaluated at the top and foot image positions, including positions outside the calibration quadrilateral where the mapping is finite. The four points calibrate the ground mapping; the vertical objects supply the additional information needed to measure height.[3]

An optional algebraic glimpse. In suitable image coordinates, the mapping can be written

$$H(x,y) = \left(\frac{a_1x + a_2y + a_3}{c_1x + c_2y + 1}, \quad \frac{b_1x + b_2y + b_3}{c_1x + c_2y + 1} \right).$$

There are eight coefficients. A correspondence $(x_i, y_i) \mapsto (X_i, Y_i)$ gives two equations; for example,

$$a_1x_i + a_2y_i + a_3 = X_i(c_1x_i + c_2y_i + 1),$$

[3]The familiar perspective correction in a document-scanning application uses the same idea: map the photographed page corners to a rectangle. See OpenCV's introduction to homography. For the broader subject, see *Foundations of Computer Vision*, "Single View Metrology."

and similarly for Y_i. Since all four coordinates in a correspondence are known, these are linear equations in the unknown coefficients. Four pairs give eight equations. Under the stated conditions they determine the mapping. We can use this computation without developing its full algebra here. A zero denominator represents a viewing line parallel to the ground, for which no finite ground intersection exists.

From the proof to a procedure. First calibrate H using the four ground correspondences. Click the top and foot of each object, and map those four positions with H. Intersect the two mapped top–foot lines to obtain O. Calculate the two distance ratios and multiply their quotient by the known height h_0. Every step has a purpose supplied by the geometric argument.

What can go wrong? All clicks and calibration points must refer to the same camera view. The model assumes a point camera with negligible or corrected lens distortion, a flat ground plane, and truly vertical objects. If the two ground lines nearly coincide, their intersection is poorly determined. If a top is near camera height, its viewing line is nearly parallel to the ground and its mapped position can be very far away: small clicking errors may then produce large height errors. More calibration points can help check a fitted mapping, but do not correct a wrong geometric model. These are the same questions about assumptions and uncertainty that arose when we measured shadows.

8.2.4 Across a river, without crossing it

The problem. Determine the perpendicular distance AT to a visible but inaccessible landmark T, using only constructions and length measurements on the accessible side of a straight baseline through A. Assume level terrain and unobstructed sight lines. Specify which accessible lengths suffice.

Choose a landmark T on the opposite bank and a point A on our bank so that AT is perpendicular to an accessible baseline. We want the length AT; it need not be the river's width everywhere. Choose B along the baseline, and continue in the same direction to C with $AB = BC$. From C walk along a perpendicular, away from the river, until a point D is reached such that D, B, T lie on one straight line. All walking and tape measurements take place on our side.

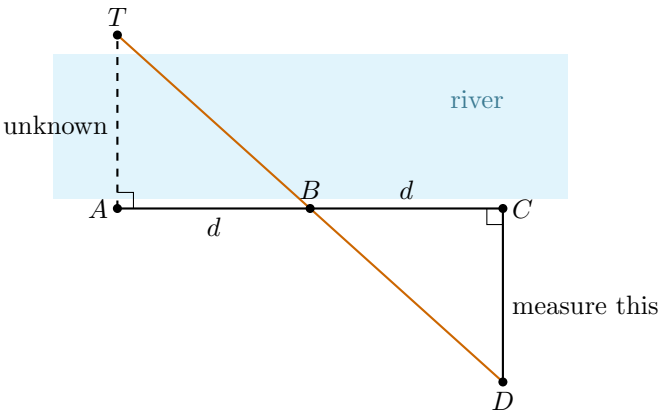


Figure 8.10: The right triangles TAB and DCB are similar. Equal baseline segments make their corresponding heights equal.

The angles at B are vertically opposite, and the angles at A and C are right angles. Similarity therefore gives

$$\frac{AT}{CD} = \frac{AB}{BC} = 1.$$

Thus measuring CD determines AT. If equal baseline segments are inconvenient, the same proof gives $AT = CD\,(AB/BC)$. The construction assumes enough clear ground behind C and a visible

line of sight to T.[4]

8.3 Measuring the Earth: shadows, angles, and a long journey

Our shadow calculation treated the ground as flat and vertical sticks as parallel. Over a small region this is a useful approximation. Over a long north–south journey, however, the direction of the local vertical changes. On a spherical Earth, vertical sticks point along radii from the centre. That change of direction allows us to measure the size of the Earth without travelling all the way around it.

In the third century BCE, Eratosthenes used observations associated with Alexandria and Syene, near present-day Aswan in Egypt, to estimate Earth's circumference. Before following the argument, we need a way to describe positions on a sphere.[5]



Figure 8.11: *Eratosthenes Teaching in Alexandria*, Bernardo Strozzi, 1635. Montreal Museum of Fine Arts. This much later painting is an imaginative representation, not a contemporary likeness. Image source: Wikimedia Commons; public domain.

The problem. Assume Earth is a sphere and that two observation sites lie on the same meridian. Given their surface separation $d > 0$ and the angle $0 < \theta < 180°$ between their local verticals, determine Earth's great-circle circumference C. Explain how solar observations can supply θ and what ground measurement is still required.

8.3.1 Latitude and longitude: angles that locate a place

Imagine Earth as a sphere with centre O, turning about the line joining the North and South Poles. The plane through O perpendicular to this axis cuts the sphere in the *equator*. Its latitude is 0°.

The *latitude* of a point P is the angle its radius OP makes with the equatorial plane. We specify north or south: the North Pole has latitude 90° N, the South Pole 90° S. Points of equal latitude form circles called *parallels*. Apart from the equator, these circles are smaller than a great circle, which is a circle whose centre is O.

[4] A useful geometric construction converts an inaccessible measurement into an accessible one. Its proof also explains which field conditions are essential: here, the two perpendiculars, the baseline ratio, and the alignment of T, B, D.

[5] Eratosthenes' own account is lost. The familiar Alexandria–Syene construction comes through later writers, including Cleomedes. We present its idealised geometry rather than a reconstruction of every historical observation. See the Mathematical Association of America's historical discussion.

A *meridian* is a semicircle running from the North Pole to the South Pole. Choose one as the *prime meridian*. The *longitude* of a point is the angle from that reference meridian to its own, measured east or west about Earth's rotation axis. We use 0° to 180° E or W; 180° E and 180° W describe the same meridian. At a pole all meridians meet, so longitude does not distinguish a position there.

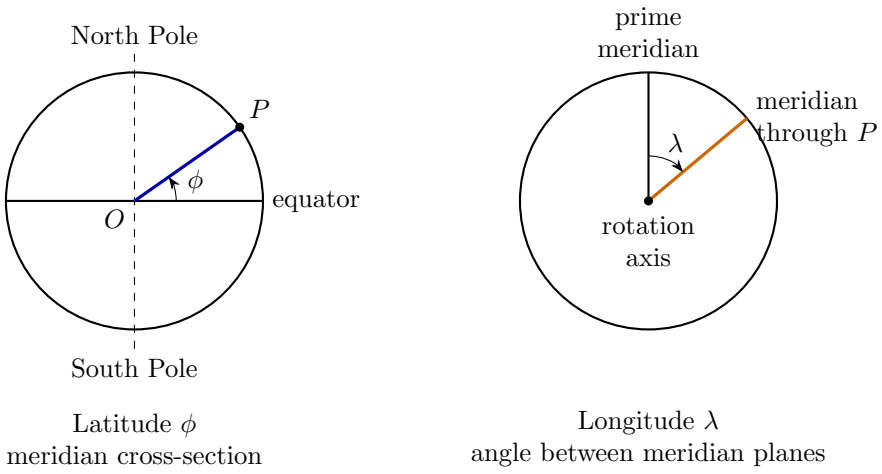


Figure 8.12: Two separate views define the two angles. Left: a slice through the poles and P. Right: a view along the rotation axis; each meridian plane is represented by a radial line. The chosen angular separation is illustrative.

For example, 30° N, 20° E specifies a point north of the equator on the meridian 20° east of the reference. These are angles, not distances in kilometres. A degree of longitude spans a smaller distance along a parallel near a pole than it does at the equator, because the parallel itself is smaller.[6]

If two places lie on the same meridian, their north–south surface separation is an arc of a great circle. The angle it subtends at the centre is the difference of their signed latitudes. Thus two points at 24° N and 31° N on the same meridian are separated by 7° of arc. If their longitudes differ as well, latitude difference alone does not give the angle between their radii. This is why the north–south alignment matters in the following argument.

8.3.2 From a shadow angle to a central angle

The idealised account begins at *local solar noon* on the northern summer solstice. Solar noon means the Sun's daily passage across the local meridian; it need not occur at 12 on a civil clock. At Syene the Sun was reported directly overhead: a vertical stick had no shadow. At Alexandria, farther north, a vertical stick did cast a shadow. The stick made an angle of about 7.2° with the incoming sunlight.

We make three geometric assumptions: Earth is spherical, sunlight reaches the two sites in parallel rays, and the sites lie on the same meridian. In this idealisation they share solar noon. The overhead ray at Syene lies along the local radius. Because the ray at Alexandria is parallel to it, the angle between that ray and Alexandria's vertical is equal to the angle between the two radii at Earth's centre.

The angle can be found from the right triangle formed by a vertical stick, its shadow, and a ray from the Sun. Measure the stick and the shadow, construct a similar triangle on paper, and read the angle between the stick and the ray with a protractor. We do not need trigonometry for this step. A report that the angle is one fiftieth of a full turn is equivalent to $360°/50 = 7.2°$.

[6]The modern reference for longitude is associated with Greenwich. Unlike the equator, this reference is a human choice, not a feature singled out by the rotation axis. Our latitude is defined for a sphere; precise surveying accounts for Earth's flattening. See NOAA's explanations of latitude and longitude.

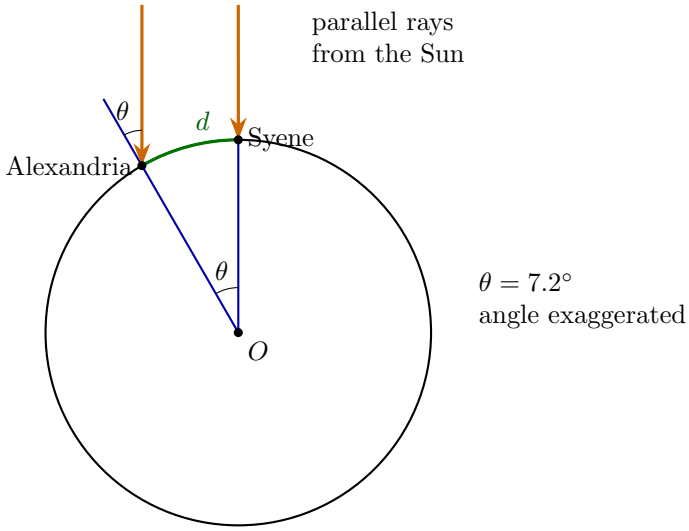


Figure 8.13: The shadow angle measures the turn between local verticals. The surface distance d is the arc between the sites, not the straight chord through Earth. The circle is a meridian cross-section, not a map with north at the top.

Let d be the north–south surface distance between the sites and C Earth's circumference along a great circle. Arcs of a circle are proportional to their central angles, so

$$\frac{d}{C} = \frac{\theta}{360^\circ}, \qquad C = \frac{360^\circ}{\theta}\, d.$$

For $\theta = 7.2^\circ$, this gives $C = 50d$. The traditional distance of 5000 stadia therefore gives 250,000 stadia for the circumference. For a modern classroom illustration, using $d \approx 800$ kilometres gives $C \approx 40{,}000$ kilometres. This rounded illustration is not an exact conversion of the ancient unit.[7]

8.3.3 Why did the measurement involve such a long journey?

The shadow gives an *angle*, not a length. A small globe and a large globe can have the same angle between two radii. To determine which scale we are dealing with, we need one measured distance on the ground. The geometric argument removes the need to travel around the whole Earth; it does not remove the need for a distance between separated sites.

Obtaining that distance in antiquity was a substantial surveying and travel task, whether done for this purpose or taken from existing geographical knowledge. A long trek could establish a baseline through measured stages or careful pacing. Simply counting the distance along a winding road or river would not suffice: the calculation needs the corresponding meridian arc, so detours must be allowed for. We should not turn this practical requirement into an unsupported claim that Eratosthenes personally walked from Alexandria to Syene.[8]

There is a second reason to use widely separated sites: the signal becomes easier to measure. On a sphere of circumference 40,000 kilometres,

$$\theta = 360^\circ \frac{d}{40{,}000}.$$

[7] The length of the stadion used in this calculation is disputed, and ancient sources also report 252,000 stadia for Eratosthenes' result. It is better to state the ancient result in its own units than attach a spurious exact percentage accuracy to it. See MacTutor's biography and the MAA discussion cited above.

[8] Accounts of how the distance was obtained differ; the surviving evidence does not securely establish a particular journey or a newly commissioned walking survey. For the historical distance problem, see "Distance from Alexandria to Syene," MAA. The long trek explains the practical work behind a ground baseline, not an independently established episode in Eratosthenes' life.

A baseline of 1 kilometre gives only 0.009°; 100 kilometres gives 0.9°; 800 kilometres gives 7.2°. A very short walk produces almost parallel local verticals and a tiny difference of shadow angles. A long baseline makes that difference more conspicuous.

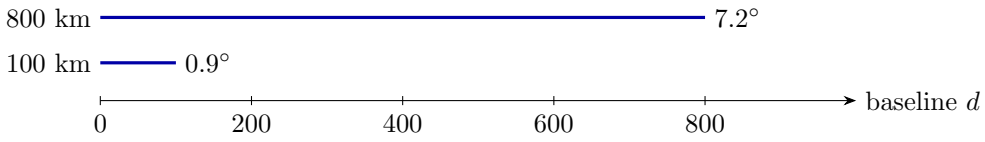


Figure 8.14: On the same sphere, a longer north–south baseline produces a larger angular difference. Distances are drawn to scale; the angles are given as labels.

For example, suppose the angle can be read only within 0.1° and ignore distance error for the moment. With $d = 800$ kilometres and a reading of 7.2°, the possible circumference lies between

$$\frac{360(800)}{7.3} \approx 39{,}452 \quad \text{and} \quad \frac{360(800)}{7.1} \approx 40{,}563 \text{ kilometres.}$$

For $d = 100$ kilometres and a reading of 0.9°, the same angular uncertainty gives a much wider interval: 36,000 to 45,000 kilometres. These are illustrative measurement bounds, not claims about the accuracy of ancient instruments. A longer baseline helps with a fixed angular error, but measuring that baseline introduces its own difficulties.

8.3.4 What the model tells us, and what it assumes

The cities are not exactly on the same meridian, and Syene was not exactly at the location where the solstice Sun would be overhead. Earth is also not a perfect sphere. These are limitations of the simplified account, not flaws in its geometric deduction. The achievement is the method: a terrestrial distance and an astronomical angle together determine a planetary length scale.

Latitude makes the method especially transparent: along one meridian, the difference in latitude is the measured central angle. Longitude tells us why an east–west separation cannot be ignored. Places at different longitudes generally reach solar noon at different times. Merely reading two shadows at the same clock time need not reproduce the ideal experiment.

We have moved from a flat local model to a spherical global one. Neither is useful at every scale. The earlier shadow calculation neglects the change in vertical over a small area; Eratosthenes' calculation deliberately makes that change large enough to measure.

8.4 Locating a point from distances

8.4.1 Coordinates turn a diagram into equations

Choose perpendicular axes and a common unit. A point $P = (x, y)$ records two signed displacements. By the Pythagorean theorem, the distance from $P = (x, y)$ to $A = (a, b)$ is

$$PA = \sqrt{(x - a)^2 + (y - b)^2}.$$

A negative coordinate is a direction, not a negative length. Coordinates let us describe the same geometric relationships algebraically.

8.4.2 One circle, two circles, three measurements

Knowing that a point is five units from A confines it to a circle; it does not tell us which point on that circle it is. A second distance usually leaves two possibilities. A third suitably placed landmark can distinguish them.

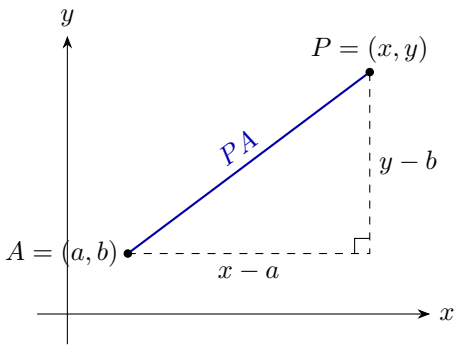


Figure 8.15: For the configuration drawn, the horizontal and vertical lengths are $x - a$ and $y - b$. In general use their absolute values; squaring removes the signs.

The problem. Given $A = (0, 0)$ and $B = (6, 0)$, find all points P with $PA = PB = 5$. Determine whether one additional exact distance from $C = (3, 6)$ distinguishes those points. All distances are Euclidean.

If $P = (x, y)$ and $PA = PB = 5$, then

$$x^2 + y^2 = 25, \qquad (x - 6)^2 + y^2 = 25.$$

Subtracting the equations gives $12x - 36 = 0$, so $x = 3$. Substitution gives $y^2 = 16$, leaving $P = (3, 4)$ or $P = (3, -4)$. A third landmark $C = (3, 6)$ distinguishes them: their distances from C are 2 and 10, respectively.

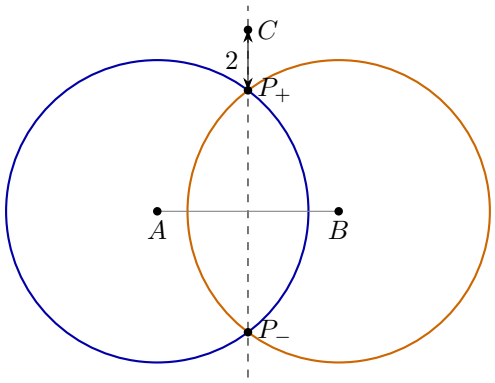


Figure 8.16: Two distances leave a reflection ambiguity. A landmark off the baseline can distinguish the two points.

Two circles can also be tangent, disjoint, or coincident. With measured rather than exact distances, three circles may have no common point. This can indicate measurement error rather than a failure of geometry. Each uncertain distance is better represented by a thin ring; plausible locations lie in the overlap of the rings.

8.4.3 Which clinic is nearest?

The problem. Given finitely many distinct clinic locations in the Euclidean plane, describe, for each clinic, every point at least as close to it as to any other clinic. Include ties on shared boundaries. Assume straight-line travel is possible and all clinics are equally suitable.

Suppose two clinics are at $A = (-2, 0)$ and $B = (2, 0)$. For $P = (x, y)$,

$$PA^2 - PB^2 = (x + 2)^2 + y^2 - \big((x - 2)^2 + y^2\big) = 8x.$$

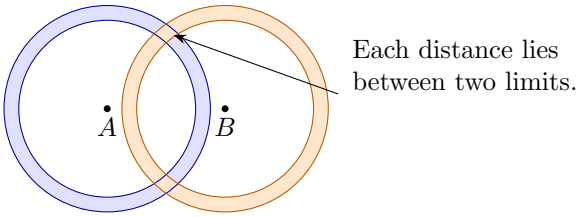


Figure 8.17: Uncertain measurements give regions of possible locations, often with more than one separate component.



Figure 8.18: Georgy Voronoi (1868–1908), whose name is attached to these nearest-site regions. Photograph, photographer unknown; Wikimedia Commons, marked public domain.

Thus P is nearer A when $x < 0$, nearer B when $x > 0$, and equally far from both when $x = 0$. The boundary is the perpendicular bisector of AB. Translating or rotating the picture does not change the result.

For several clinics, the region belonging to A consists of all points at least as near to A as to every other clinic. For each competitor, keep the half-plane containing A bounded by their perpendicular bisector. The intersection of those half-planes is A's *Voronoi region*.[9] Boundary points may belong to more than one region because of ties.

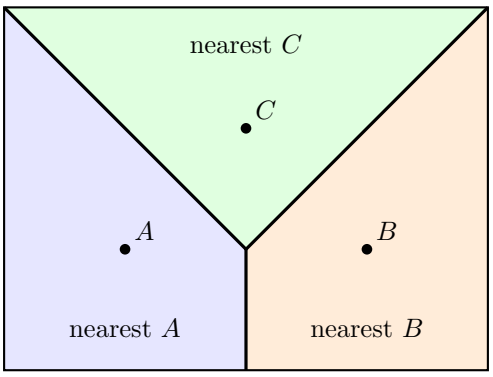


Figure 8.19: Three nearest-clinic regions, clipped to a rectangular map. Here $A = (-2, 0)$, $B = (2, 0)$, and $C = (0, 2)$; the three boundaries meet at $(0, 0)$.

A region is not necessarily bounded: a clinic near the edge of the set of sites can have a region extending indefinitely. The border of a printed map is not a nearest-distance boundary. Nor does "nearest" automatically mean "best served": travel time, capacity, and barriers may matter more.

[9] The regions are named after Georgy Voronoi (also written Voronoy, 1868–1908). For further examples and applications, see Wikipedia, "Voronoi diagram."

8.4.4 Changing the meaning of distance

The problem. Given two grid intersections $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ with integer coordinates, determine the shortest route length using unit horizontal or vertical street segments. Every segment is available and has unit cost.

On an ideal rectangular street grid, suppose we can move only horizontally and vertically and every street is available. The shortest route length is

$$d_{\text{grid}}(P, Q) = |x_P - x_Q| + |y_P - y_Q|.$$

Every route must cover at least those horizontal and vertical displacements; a route that never reverses direction attains their sum. This proves the formula. Between $(0, 0)$ and $(3, 2)$ it gives 5, whereas straight-line distance gives $\sqrt{13}$. The nearest-clinic boundaries can change when the distance model changes.

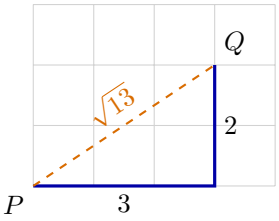


Figure 8.20: The straight route is shorter, but the street model does not permit it. A shortest admissible route has length 5.

8.5 Finding a shortest route

8.5.1 A route that must touch a straight boundary

The problem. Let A and B lie strictly on the same side of a straight boundary ℓ. We want a route from A to B that touches ℓ, perhaps to collect water along a straight stretch of riverbank. Assuming open terrain and equal cost per unit length, choose X on ℓ to minimise $AX + XB$.

Reflect B across ℓ to obtain B'. Every point X of ℓ satisfies $XB = XB'$, so

$$AX + XB = AX + XB' \geq AB'.$$

The inequality is the triangle inequality. Equality holds when X is where the straight segment AB' crosses ℓ. Because A and B' are on opposite sides, that intersection exists. The lower bound is attained, which proves that the constructed route is shortest.

From the proof to a procedure. Construct the perpendicular from B to ℓ and extend it by the same length on the other side to obtain B'. Draw AB' and mark its intersection X with ℓ. Follow AX and then XB. The proof supplies both the construction and the certificate that no other touching point is better.

For $A = (0, 3)$, $B = (6, 2)$, and ℓ the horizontal axis, $B' = (6, -2)$. The straight segment drops five units in six horizontal units, and reaches the axis after a drop of three, so $X = (18/5, 0)$. The shortest length is $AB' = \sqrt{61}$. If only a finite stretch of bank is accessible, first check that X belongs to it; the construction for an unrestricted line need not solve the constrained problem.

8.5.2 Unfolding a surface

A route drawn straight through a box is not a route along its surface. But unfolding adjacent flat faces preserves lengths within those faces. A shortest route crossing a specified shared edge can therefore be found by laying the faces flat and drawing a straight segment, provided that segment stays within the unfolded faces and crosses the intended edge.

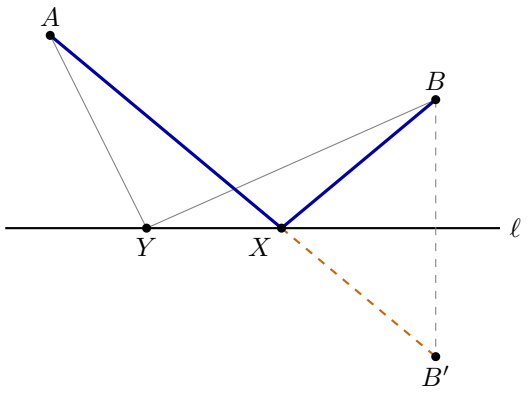


Figure 8.21: Reflect the destination. The best broken route becomes a straight segment; a different touching point Y gives a longer route.

The problem. For a rectangular box with edge lengths $a = 4$, $b = 3$, $c = 2$ and opposite corners P, Q, compare the three two-face routes obtained by unfolding rectangles of dimensions $(a + b) \times c$, $(a + c) \times b$, and $(b + c) \times a$. Check that each straight segment stays on those faces, and find the shortest of these candidates. Lengths use a common unit.

The three unfoldings give the candidate lengths

$$\sqrt{(a+b)^2 + c^2} = \sqrt{53}, \quad \sqrt{(a+c)^2 + b^2} = \sqrt{45}, \quad \sqrt{(b+c)^2 + a^2} = \sqrt{41}.$$

These are lengths of admissible surface routes, not of the space diagonal. The third is the shortest of these candidates. Comparing candidates is a useful step, but a claim of global optimality also needs a proof that no other face sequence does better; we do not assume that merely from a drawing.

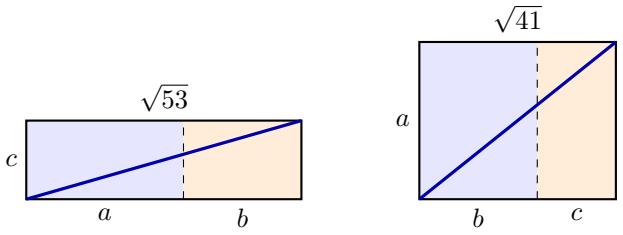


Figure 8.22: Two of the three candidate unfoldings. Dashed lines are folds, not barriers. The endpoints of each diagonal represent opposite corners of the box.

Try it with paper. Make a $4 \times 3 \times 2$ box, mark opposite corners, and transfer the shorter route from its unfolding back onto the box. It will bend in space at the fold although it was straight on the paper. The relevant geometry is that of the permitted surface.

8.6 Shape, size, and efficiency

8.6.1 The largest rectangular garden

The problem. Suppose a rectangular garden has a fixed perimeter $P > 0$, with all four sides fenced. If its side lengths are $x, y > 0$, then $x + y = P/2$. We want to maximise its area xy.

The identity

$$(x + y)^2 - (x - y)^2 = 4xy$$

gives

$$xy = \frac{P^2}{16} - \frac{(x-y)^2}{4} \le \frac{P^2}{16}.$$

Equality holds exactly when $x = y = P/4$. Thus the square has the greatest area *among rectangles with that perimeter*. The restriction to rectangles is part of the claim. We have both an upper bound and a shape that attains it, just as the reflection argument gave a lower bound and a route attaining it.

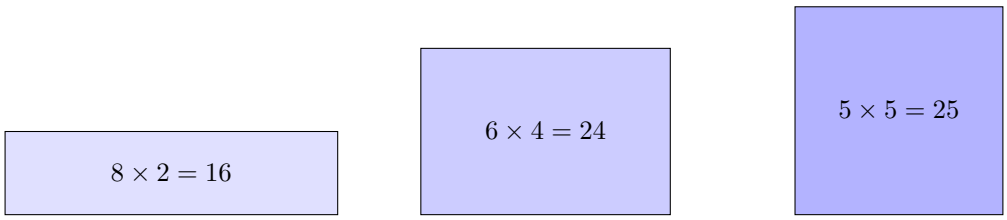


Figure 8.23: Each rectangle has perimeter 20. Equalising the side lengths increases area.

A changed problem. Now an existing straight wall can supply one entire side at no fencing cost. It is long enough for any proposed garden. With the same available fencing length P, determine the largest rectangular area and its dimensions.

The constraint changes to $2x + y = P$, where x measures distance away from the wall. Then

$$xy = x(P - 2x) = \frac{P^2}{8} - 2\left(x - \frac{P}{4}\right)^2.$$

The optimum now has $x = P/4$ and $y = P/2$: it is not a square. A correct answer belongs to a specified problem, not just to a familiar picture.

8.6.2 Enlarging a shape

The problem. A plane figure or solid is enlarged uniformly by a factor $k > 0$ in every linear dimension. Determine the factors by which its area or volume changes. For a solid, also determine the change in surface-area-to-volume ratio.

If every length of a shape is multiplied by $k > 0$, corresponding areas are multiplied by k^2 and volumes by k^3. For a rectangle this follows from $(kx)(ky) = k^2xy$; for a rectangular box it follows from $(kx)(ky)(kz) = k^3xyz$. Decomposing a polygon into triangles gives the same area rule, and the volume rule extends to similar solids.

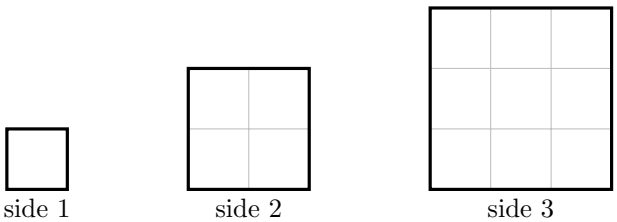


Figure 8.24: Scaling every length by k requires k^2 unit squares to cover the enlarged square when k is an integer. The algebraic rule also holds for noninteger $k > 0$.

Doubling a box's dimensions therefore requires four times as much surface material but gives eight times the capacity, if thickness is neglected. The surface-area-to-volume ratio is divided by two. These statements compare *similar* boxes; changing only one dimension is a different operation.

8.6.3 Cutting a cube into pieces

The problem. A cube of side $s > 0$ is cut by planes parallel to its faces into equal cubes, with $m \geq 1$ divisions along each edge. Neglect the thickness of cuts. Determine the number of pieces, their total volume and total surface area, and the newly exposed area when all pieces are separated.

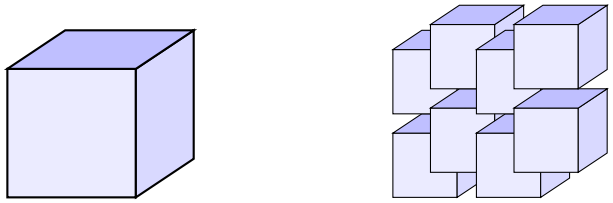
Take a cube of side s, and divide each edge into m equal parts, where $m \geq 1$ is an integer. There are m^3 small cubes, each of side s/m. Their combined volume is

$$m^3(s/m)^3 = s^3,$$

but the sum of all their surface areas is

$$m^3\, 6(s/m)^2 = 6ms^2.$$

That is m times the surface area of the original cube. The extra area comes from newly exposed cut faces, not from extra volume. If the pieces remain pressed together, those internal faces are not exposed to the surroundings.



one cube eight pieces

Figure 8.25: For $m = 2$, volume is unchanged but the total surface area of the separated pieces doubles. The gaps illustrate separation and are not part of the original cube.

This helps explain why crushing or cutting a material can make surface processes faster. Geometry identifies the changed area; a prediction of the actual rate also needs a model of the physical process.

8.6.4 Packaging: state what is being optimised

The problem. Among all closed boxes with a square base and prescribed volume $V > 0$, find dimensions that minimise total surface area, and prove the minimum. The dimensions may be any positive real numbers; ignore material thickness, seams, and waste in cutting.

Consider a closed box with square base of side x and height h, required to contain V cubic units. Ignoring thickness and seams,

$$x^2h = V, \qquad S = 2x^2 + 4xh = 2x^2 + \frac{4V}{x}.$$

For $V = 1000$, the boxes $(x, h) = (5, 40), (10, 10), (20, 2.5)$ have surface areas $850, 600, 1000$, respectively. A table suggests a good design, but three trials do not prove optimality.

Here a proof is possible without calculus. Put $s = \sqrt[3]{V}$ and $t = x/s > 0$. Then

$$\frac{S}{2s^2} - 3 = t^2 + \frac{2}{t} - 3 = \frac{(t-1)^2(t+2)}{t} \geq 0.$$

Thus $S \geq 6V^{2/3}$, with equality exactly at $t = 1$: the cube is best within this family of closed boxes. An open box has only one base, and extra conditions such as shelf height, openings, seams, or strength change the problem. The model tells us what our optimisation does and does not establish.

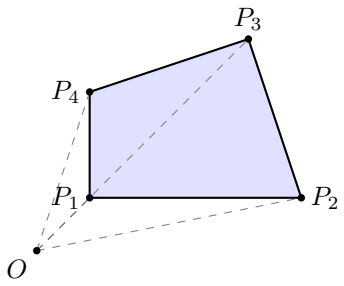
8.7 A bridge back to algorithms: area from vertices

The problem. Given the coordinates of the $n \geq 3$ vertices of a simple polygon in boundary order, compute its area. Give a procedure whose arithmetic-operation count grows linearly with n. The input boundary does not cross itself; clockwise and anticlockwise orders must both be accepted.

A survey may give the vertices of a field rather than its area. Suppose $P_i = (x_i, y_i)$, $i = 1, \ldots, n$, list the vertices of a simple polygon in order around its boundary. "Simple" means that its edges do not cross. Set $P_{n+1} = P_1$. Its area is

$$A = \frac{1}{2}\left|\sum_{i=1}^{n}(x_i y_{i+1} - x_{i+1}y_i)\right|.$$

This is the *shoelace formula*. The order of the vertices is essential; sorting them arbitrarily by one coordinate can describe a different boundary.



Signed triangle areas
cancel outside the polygon.

Figure 8.26: Summing oriented triangles from an origin gives the polygon area, even when that origin is outside.

Why does this work? The signed area of the triangle OP_iP_{i+1} is $\frac{1}{2}(x_i y_{i+1} - x_{i+1}y_i)$, positive for an anticlockwise turn and negative for a clockwise turn. The formula follows by subtracting right-triangle areas, or by taking base times perpendicular height. Triangulate the polygon, and orient each triangle anticlockwise. In the sum of their coordinate expressions, every internal edge occurs in both directions and cancels. Only the boundary terms remain. Reversing the boundary order changes the sign; the absolute value recovers ordinary area.

For $(0,0), (4,0), (4,3), (0,3)$ the sum is 24, giving area 12. For the quadrilateral in the figure the sum is 18, giving area 9. A self-crossing boundary needs separate interpretation: signed regions can cancel, so the formula need not give the total area one intended.

The desired loop postcondition is that an accumulator z contains all n terms. Generalise "all" to "the first k":

$$0 \leq k \leq n, \qquad z = \sum_{i=1}^{k}(x_i y_{i+1} - x_{i+1}y_i).$$

Begin with $k = 0, z = 0$. While $k < n$, add the term with index $k + 1$ to z, then increment k. The displayed assertion is preserved, and the bound function $n - k$ decreases by one. Return $|z|/2$. This connects a geometric proof directly to an iterative procedure requiring $O(n)$ arithmetic operations.

8.8 What a geometric answer should contain

We have repeatedly followed the same route: state the question, draw a model, identify assumptions, prove a relationship, and turn it into a calculation or construction. Finally, interpret the result in the original setting. A number without units, a diagram without justified relationships, or an optimum without a specified constraint is only part of an answer.

Geometry also gives us ways to check our work. Does the formula have the right units? Does it behave sensibly when a length doubles? Does an equality case attain the bound? Does the

constructed route stay in the permitted region? These questions connect visual understanding to mathematical proof.

Problems

Unless stated otherwise, coordinates refer to a Euclidean plane with a common unit on both axes. Draw labelled diagrams where helpful. For measurement problems, distinguish the conclusion within the stated model from possible errors in the model itself.

1. **Height from shadows.** A vertical stick of height 1.2 metres casts a shadow of length 0.8 metres, while a vertical tree casts a shadow of length 7 metres at the same instant. Assume level ground and parallel sunlight. Estimate the tree's height using similarity, and identify two physical departures from the assumptions that would invalidate the estimate.
2. **Bounds on the height.** In the preceding shadow model, take the stick height to be exactly 1.2 metres, its shadow length to lie in $[0.75, 0.85]$ metres, and the tree's shadow length in $[6.8, 7.2]$ metres. Find the smallest interval containing every tree height allowed by these bounds, with no further relationship assumed between the measurement errors. Give exact endpoints and an outward-rounded interval to two decimal places.
3. **An unequal-baseline river construction.** Points A, B, C lie in that order on a straight baseline, with $AB = 12$ and $BC = 8$ metres. The segments AT and CD are perpendicular to the baseline on opposite sides, T, B, D are collinear, and $CD = 10$ metres. Determine AT by similarity. Derive the general relation between AT, AB, BC, CD and explain what simplifies when $AB = BC$.
4. **Design a field measurement.** Choose one inaccessible height or distance on campus, and define precisely its endpoints. Propose a procedure using a tape, a vertical stick, and sight lines (you need not perform it). Supply a labelled diagram, a list of accessible measurements, the assumptions and formula used to infer the desired length, and one repeat measurement that would help detect an error. Explain what that repeat measurement checks.
5. **Locating a point.** Let $A = (0, 0)$ and $B = (8, 0)$. Find all points P satisfying $PA = PB = 5$. Give coordinates for a third landmark whose distance to P distinguishes the possible points, and for a different landmark whose distance does not. Compute the relevant distances to justify both choices.
6. **How many circle intersections?** Two circles have radii $r_1, r_2 > 0$ and distinct centres at distance $d > 0$. Give necessary and sufficient conditions, in terms of r_1, r_2, d, for zero, one, and two intersection points. Distinguish external tangency from internal tangency, and disjoint circles lying outside one another from one lying inside the other. Illustrate each case.
7. **Three Voronoi regions.** The only sites are $A = (0, 0)$, $B = (6, 0)$, and $C = (0, 6)$. For each site, derive inequalities defining the points at least as close to it as to both others. Draw all three regions in the whole plane, mark the parts of the bisectors where nearest-site ties occur, and identify the point tied among all three sites.
8. **Straight-line distance versus road distance.** Construct a connected road network containing a home H and two clinics A, B, with $HA < HB$ in Euclidean distance but with the shortest road route from H to A longer than that from H to B. Give coordinates for every junction, list all roads and their lengths, and calculate both shortest road distances. Use straight road segments with lengths given by their coordinates; travel is allowed only along the listed roads.
9. **Counting shortest grid routes.** In the unobstructed integer square grid, every horizontal or vertical edge has length one and may be traversed in either direction. Find the shortest length from $(0, 0)$ to $(3, 2)$ and count all routes attaining it. Explain why no shortest route can include a westward or southward step.
10. **A route touching a boundary.** Let $A = (0, 2)$ and $B = (8, 4)$. Among all broken straight routes $A \to X \to B$ with $X = (t, 0)$, $t \in \mathbb{R}$, find the minimising point X and the minimum

length. Prove optimality by reflection. Repeat with the additional constraint $0 \le t \le 1$, giving a proof that your constrained minimiser is best.

11. **Equal angles at the boundary.** Let A, B lie strictly on the same side of a line ℓ, and let $X \in \ell$ minimise $AX + XB$. Prove that the smaller angle between AX and ℓ equals the smaller angle between BX and ℓ. Include the case in which both segments are perpendicular to ℓ, and identify where reflection is used.

12. **Routes on a box.** A rectangular box has edge lengths $5, 4, 1$ units. Join opposite corners by the three two-face routes obtained from unfolded rectangles $(5+4)\times 1$, $(5+1)\times 4$, and $(4+1)\times 5$. Draw each unfolding with its fold and endpoints, find the three lengths, and identify the smallest. Check that each diagonal crosses the intended shared edge within that edge. Compare only these candidates; a proof about all possible face sequences is not required.

13. **Fencing a garden.** Exactly 40 metres of fencing is available for a rectangular garden with positive side lengths. Find the maximum area and the dimensions attaining it (a) when all four sides must be fenced and (b) when an existing straight wall supplies one side. In (b) assume the wall is long enough and the remaining three sides use all the fencing. Prove both maxima algebraically.

14. **A scale model.** A model building is geometrically similar to the actual building, with every model length equal to $1/50$ of the corresponding actual length. Find the ratios model:actual for corresponding areas and volumes. If A square centimetres of model surface requires painting, express the corresponding actual area in square centimetres and explain why multiplying A by 50 is incorrect.

15. **Cutting a cube.** A solid cube of side 12 cm is cut into cubes of side 3 cm by planes parallel to its faces. Neglect material lost in cutting. Find the number of pieces, their combined volume, and the sum of their surface areas. After every piece is separated, how much exposed area is additional to the original cube's surface area? Include units.

16. **A closed box.** A closed box has a square base of side $x > 0$ cm, height $h > 0$ cm, and volume 216 cubic centimetres. Neglect thickness and seams. Find the least possible total surface area, prove that no admissible box has a smaller area, and identify all dimensions attaining the minimum. Do not use calculus.

17. **An open box.** An open-top box has a square base of side $x > 0$ and prescribed volume $V > 0$. Its height is $h = V/x^2$ and its material area is $S = x^2 + 4V/x$. Prove without calculus that the unique minimum occurs at $x^3 = 2V$, give the minimum area in terms of V, and show that the optimal height is $x/2$. You may adapt the nonnegative factorisation used for the closed box.

18. **Area from coordinates.** A polygon has vertices $(0,0), (5,0), (5,2), (2,4), (0,2)$ in the listed boundary order, with the last vertex joined to the first. Calculate its area using the shoelace formula. Verify it independently by decomposing the polygon into a rectangle and a triangle. Give the area in square coordinate units.

19. **Translation and the shoelace sum.** Let $P_i = (x_i, y_i)$, $1 \le i \le n$, be a closed polygonal sequence with $P_{n+1} = P_1$. For fixed real numbers a, b, replace every vertex by $(x_i + a, y_i + b)$. Prove that the signed sum $\sum_{i=1}^{n}(x_i y_{i+1} - x_{i+1} y_i)$ is unchanged by expanding the translated sum and cancelling terms. Deduce area invariance for a simple polygon.

20. **A crossed boundary.** Join $(0,0), (2,2), (0,2), (2,0)$ in that order and close the boundary. Draw the resulting crossed figure. Compute both the signed shoelace sum and half its absolute value. Separately find the sum of the ordinary areas of its two triangular lobes. Explain why these results differ and which hypothesis of the polygon-area formula fails.

21. **Earth's circumference from latitude.** Model Earth as a sphere. Two sites on the same meridian have latitudes 18° N and 25.5° N, and their meridian arc separation is 825 km. Estimate the great-circle circumference. If the sites have different longitudes, explain why their latitude difference cannot simply be substituted for the angle between their radii in the same calculation.

22. **An angle does not fix a size.** Fix an angle $0 < \theta < 180°$. Draw two circles of different radii, each with two radii enclosing θ. Explain why the same shadow-angle observation could be consistent with either sphere. Identify one additional length measurement that would determine the circumference and give the formula.

23. **Uncertainty in Earth's circumference.** In a spherical model, two sites on one meridian have separation $d \in [595, 605]$ km and central angle $\theta \in [5.3°, 5.5°]$. Assume no further relation between these uncertainties. Find the smallest interval containing every value of $C = 360°d/\theta$ consistent with the bounds. State which endpoints determine each extreme and give an outward-rounded interval in whole kilometres.

24. **Choosing a baseline.** Assume a spherical Earth of circumference 40,000 km. Compare meridian baselines of 100 km and 800 km. Calculate their true central angles. For each, suppose the reported angle equals that true angle but has uncertainty $\pm 0.1°$; treat the distance as exact. Calculate the resulting circumference bounds and compare their widths. Explain why a longer baseline helps here, and why a winding road's length cannot replace the meridian arc length.

25. **Latitude is not longitude.** On a sphere, let P have coordinates 20° N, 30° E and Q have coordinates 20° N, 60° E. State their common coordinate and longitude difference. Using a diagram, explain why that longitude difference is an angle about the rotation axis, whereas $\angle POQ$ is measured at the sphere's centre O between the two radii; they are not equal here. Explain why their common parallel is not a great circle, while a full circle through a meridian is. No trigonometric calculation is required.

26. **Height from a photograph.** In the construction of Section 8.2.3, the mapped points are $F = (4, 0)$, $R = (10, 0)$, $F_0 = (0, 3)$, and $R_0 = (0, 5)$. The reference object has height $h_0 = 1$ metre. Locate O, calculate both ground-distance ratios, and determine the unknown height h. Infer the camera height as a check, and verify the assumption that it exceeds both object heights. The ground coordinates use one common, arbitrary unit.

27. **What the photograph does not tell us.** Assume the point-camera model and horizontal ground. Explain why $H(t)$, for the image t of an object's top, is generally not its foot. Draw a vertical cross-section for an object whose height equals the camera height and explain why $H(t)$ then has no finite value. Finally, explain why the two mapped top–foot lines cannot locate O uniquely when both objects and O lie on one ground line.

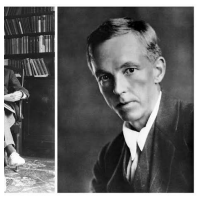
28. **Calibrating and checking the mapping.** A marked rectangle on the ground has side lengths in the ratio $4 : 3$. Its four photographed corners p_1, p_2, p_3, p_4 are given in cyclic order, with p_1p_2 corresponding to the longer side. State the four correspondences that map it to a ground rectangle with corners $(0, 0), (4, 0), (4, 3), (0, 3)$. Explain why the rectangle's absolute size is unnecessary for the height-ratio formula. If a fifth marked point is known to be the centre of the rectangle, describe how its image can check the computed H. Must that image be the arithmetic mean of the four image-corner coordinates? Explain using the distinction between perspective and ordinary averaging.

Chapter 9

Primes, GCD, and and the intrigue of Cryptography



"The primes are the jewels in the crown." – Carl Friedrich Gauss



"A mathematician, like a painter or a poet, is a maker of patterns. If his patterns are more permanent than theirs, it is because they are made with ideas. And the primes are the most eternal patterns of all." – G. H. Hardy



"God may not play dice with the universe, but something strange is going on with the prime numbers." – Paul Erdős

The theory of prime numbers and their properties forms the foundation of number theory. At first sight, the study of primes may seem like a purely mathematical pursuit. Yet today prime numbers play a crucial role in modern life, because they underpin cryptographic systems that secure communication over the internet.

In this chapter we introduce the basic ideas of primes and factorisation, methods for computing greatest common divisors, some crucial algorithms and theorems about primes, and finally explain how these notions are used in public-key cryptography, especially the RSA cryptosystem.

9.1 Primes

We have informally discussed primes several times in these notes. We give a formal definition here

Definition 9.1 *A* prime number *is an integer $p > 1$ whose only positive divisors are* 1 *and p itself.*

Numbers greater than 1 that are not prime are called *composite*. $2, 3, 5, 7, 11$ are some examples of primes, while $12 = 3 \times 4$ is composite. In what follows, we develop some properties of prime numbers.

9.1.1 There are infinitely many primes

In Theorem 5.23 we proved there are infinitely many primes. We repeat the theorem here for the sake of completeness.

Theorem 9.1 (Euclid) *There are infinitely many prime numbers.*

Proof: Suppose there are only finitely many: $p_1, \ldots, p_k$. Consider $N = p_1 \cdot p_2 \cdots p_k + 1$. No p_i divides N, since each leaves remainder 1. Thus N is prime or divisible by a new prime, contradicting the assumption[1]. □

9.1.2 Unique prime factorisation

The *Unique Prime Factorisation theorem* (Theorem 5.39) establishes that primes are the "atoms" of arithmetic: every integer factors into primes, and no further. We again repeat it here for completeness.

Theorem 9.2 (Fundamental Theorem of Arithmetic) *Every integer $n > 1$ can be written as a product of primes. This factorisation is unique up to the order of the factors.*

Proof: *Existence:* If n is prime, we are done. Otherwise, assume that the claim is true for all m such that $2 \leq m < n$. Let $n = ab$ with $a, b < n$. By the induction hypothesis, each of a and b has a prime factorisation, hence so does n.

Uniqueness: Suppose

$$n = p_1 p_2 \cdots p_k = q_1 q_2 \cdots q_m$$

with all p_i, q_j prime. Then p_1 divides the right side, hence some q_j. But since q_j is prime, $p_1 = q_j$. Cancelling, we continue inductively to conclude that the multisets $\{p_i\}$ and $\{q_j\}$ are identical. □

9.1.3 The Sieve of Eratosthenes

One of the oldest known algorithms for finding prime numbers is due to *Eratosthenes of Cyrene* (around 200 BCE). The method, called the Sieve of Eratosthenes, systematically eliminates composite numbers from a list, leaving only primes.

> Sift the Two's and Sift the Three's:
> The Sieve of Eratosthenes.
> When the multiples sublime,
> The numbers that remain are Prime.
>
> – Anonymous

[1] Assume you have a finite list of primes. Multiply them together and add 1. Either that number is prime — or you just found a new factor. Either way, you're wrong. Welcome to mathematics!

The Algorithm

To find all primes up to a given number N:

1. Write down the list of integers $2, 3, 4, \ldots, N$.
2. Start with the first number $p = 2$. It is prime.
3. Cross out all multiples of p greater than p itself in the list.
4. Find the next uncrossed number. This is the next prime. Set p to this number and repeat Step 3.
5. Continue until $p^2 > N$. All remaining uncrossed numbers are prime.

Example 9.1 *Primes up to 30*

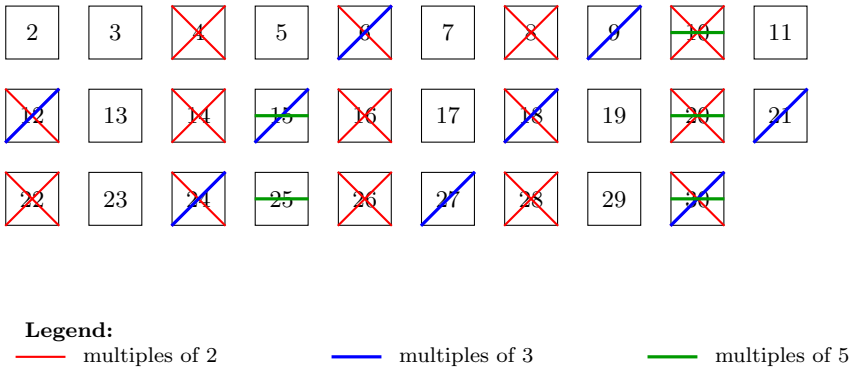


Figure 9.1: Sieve of Eratosthenes up to 30. Uncrossed boxes are primes.

Start with the numbers 2 *to* 30 *(see Figure 9.1).*

- 2 *is prime. Cross out* $4, 6, 8, \ldots, 30$.
- *Next uncrossed is* 3. *Cross out* $6, 9, 12, \ldots, 30$.
- *Next uncrossed is* 5. *Cross out* $10, 15, 20, 25, 30$.
- *Next uncrossed is* 7. *Since* $7^2 = 49 > 30$, *we stop.*

The remaining numbers are $2, 3, 5, 7, 11, 13, 17, 19, 23, 29$, *exactly the primes* ≤ 30.

Remarks

- The sieve is simple and efficient for generating primes up to a moderately large bound.
- It takes $O(N \log \log N)$ steps for large N, making it far faster than testing each number individually for divisors.
- This method, though ancient, is still used sometimes[2].

Exercise 9.1 *Find all primes between 2 and 100 using the Sieve of Eratosthenes.*

[2]Unfortunately, this algorithm is too inefficient in practice. The problem of primality testing – how to test whether a given integer is a prime – has been open at least since the times of Euclid and Eratosthenes. The first "efficient" solution for the problem was the AKS algorithm from IIT Kanpur in 2002. In practice, an even faster Miller-Rabin test is used, but it uses probabilistic coin tosses as a primitive, and can give a wrong answer with a very small probability. We will study some of these methods later in these otes

9.1.4 The Prime Number Theorem

The distribution of primes among the natural numbers is highly irregular: sometimes primes cluster close together, other times they are far apart. A natural question, posed already by Gauss and Legendre around the year 1800 when he was a teenager[3], is: *How many primes are there up to a large number n?*

Theorem 9.3 (Prime Number Theorem) *Let $\pi(n)$ denote the number of primes $\leq n$. Then*

$$\pi(n) \sim \frac{n}{\ln n}, \quad \text{as } n \to \infty,$$

meaning that

$$\lim_{n\to\infty} \frac{\pi(n)}{n/\ln n} = 1.$$

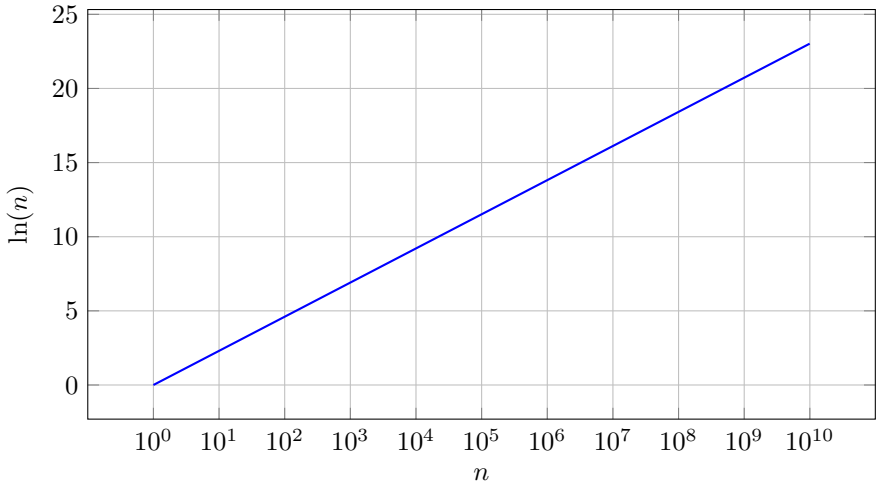


Figure 9.2: The natural logarithm $\ln(n)$ grows very slowly, even as n reaches 10^{10}.

In words: the density of primes around n is roughly $\frac{1}{\ln n}$. The natural logarithm function $\ln n$ is a rather slowly growing function. It can be intuitively understood as a way of answering the question: "How much time does it take to grow from 1 to n, assuming continuous growth at a fixed rate?". See Figure 9.2 for a plot. We will revisit the natural logarithm function later in these notes. So, primes are not very sparse at all[4]. For example, around $n = 10^6$, about one in every 14 numbers is prime.

The proof of the prime number theorem is unfortunately out of the scope of these notes. Interested readers will have to follow up QRMT with some courses on calculus, analysis and algebra for the proof to be accessible.

Exercise 9.2 *Use a calculator to stimate the number of primes between 10^6 and 10^7 using the prime number theorem.*

[3]Historians like to quip: "Even at 15, Gauss preferred to keep his prime secrets private."

[4]Carl Friedrich Gauss (1792–93), as a teenager, investigated prime tables and conjectured the approximation $\pi(n) \approx \mathrm{Li}(n)$, where $\mathrm{Li}(n) = \int_2^n \frac{dt}{\ln t}$ is the logarithmic integral. He recorded this insight in his diary, long before publishing it. Adrien-Marie Legendre independently proposed the formula $\pi(n) \approx \frac{n}{\ln n - 1.08366}$ in 1798. The theorem was finally proved independently in 1896 by Jacques Hadamard and Charles Jean de la Vallée Poussin, using complex analysis and properties of the Riemann zeta function. Their proof showed how deep connections between prime numbers and complex analysis truly are. Interested students will have to wait for a few later course in mathematics to study the proofs

9.1.5 Fermat's Little Theorem

Prime numbers have remarkable properties in modular arithmetic. One of the most beautiful and useful is a result discovered by Pierre de Fermat in 1640, long before the modern development of number theory. It shows that when we raise a number to a large power, the remainder upon division by a prime behaves in a very simple way.

Fermat wrote that if p is a prime and a is not divisible by p, then $a^{p-1} \equiv 1 \pmod{p}$. He gave no proof, and the first rigorous demonstrations came later from Leonhard Euler and others.

This theorem is central in number theory. It lies behind efficient algorithms for primality testing, and it forms one of the key mathematical tools used in public-key cryptography. Before proving it, let us state it precisely.

Theorem 9.4 (Fermat's Little Theorem) *If p is prime and $p \nmid a$, then*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Proof: Multiplication by a permutes the set $\{1, 2, \ldots, p-1\}$ modulo p (why?) Thus

$$a \cdot 2a \cdots (p-1)a \equiv 1 \cdot 2 \cdot \cdots (p-1) \pmod{p}.$$

This gives $a^{p-1}(p-1)! \equiv (p-1)! \pmod{p}$. Since $(p-1)!$ is not divisible by p, we may cancel to obtain $a^{p-1} \equiv 1 \pmod{p}$. □

Exercise 9.3 *Argue that multiplication by a permutes the set $\{1, 2, \ldots, p-1\}$ modulo p. That is, the operation is injective.*

The Fermat's little theorem can be used for primality testing as follows:

Fermat Primality Test Algorithm

1. Given an integer $n > 2$, choose a random integer a where $1 < a < n-1$ and $\gcd(a, n) = 1$.
2. Compute $a^{n-1} \bmod n$.
3. If
$$a^{n-1} \not\equiv 1 \pmod{n},$$
then n is definitely composite.
4. If
$$a^{n-1} \equiv 1 \pmod{n}$$
holds for several randomly chosen values of a, then n is probably prime.

Limitations

- The test is *probabilistic*, not deterministic.
- Certain composite numbers, known as *Fermat pseudoprimes*, can satisfy the congruence for some bases a, causing false positives.
- To reduce error probability, test multiple values of a independently.

Example 9.2 *For $n = 11$:*

$$2^{10} \equiv 1 \pmod{11}, \quad 3^{10} \equiv 1 \pmod{11},$$

so 11 is probably prime.

For a composite number $n = 15$:

$$2^{14} \not\equiv 1 \pmod{15},$$

so 15 is composite.



"The Euclidean algorithm is one of the oldest nontrivial algorithms that has survived to the present day. It can be considered the first efficient algorithm ever devised." – Donald Knuth

9.2 The Greatest Common Divisor

When working with integers, one of the most natural questions is: given two numbers, what is the largest number that divides them both? For example, for 18 and 24 the answer is 6, since 6 is the largest integer that divides both without remainder. This simple idea is called the *greatest common divisor*, or *gcd* for short.

The gcd plays a central role in number theory. It captures the notion of the "common part" of two numbers, and it is fundamental for simplifying fractions, solving equations in integers, and developing modular arithmetic. As we will see, the efficient algorithm to compute gcds, due to Euclid, is one of the oldest and most beautiful algorithms in all of mathematics.

Definition 9.2 *The* greatest common divisor $\gcd(a, b)$ *of two integers* a, b *is the largest integer that divides both.*

Example 9.3

$$\gcd(18, 24) = 6, \qquad \gcd(13, 27) = 1.$$

9.2.1 Euclid's Algorithm

Euclid, over 2000 years ago, discovered a remarkably efficient method, *Euclid_gcd*, for computing $\gcd(a, b)$, $a > 0, b \geq 0$:

$$\begin{array}{ll} \mathit{Euclid_gcd}(a, b) = a & \text{if } b = 0 \\ \mathit{Euclid_gcd}(a, b) = \mathit{Euclid_gcd}(b,\ a \bmod b) & \text{otherwise} \end{array}$$

Repeating this step until the remainder is 0 yields the gcd.

Example 9.4 *To compute* $\gcd(252, 105)$*:*

$$252 = 105 \cdot 2 + 42, \quad \mathit{Euclid_gcd}(252, 105) = \mathit{Euclid_gcd}(105, 42),$$

$$105 = 42 \cdot 2 + 21, \quad \mathit{Euclid_gcd}(105, 42) = \mathit{Eucid_gcd}(42, 21),$$

$$42 = 21 \cdot 2 + 0 \quad \Rightarrow \quad \mathit{Euclid_gcd}(21, 0) = 21.$$

Correctness of Euclid's GCD algorithm

To prove the correctness of Euclid's algorithm, we first require the following result which was proved by Euclid.

Claim: If $a = qb + r$, $0 < r < b$, then $gcd(a, b) = gcd(b, r)$

Proof: If $d = gcd(a, b)$ then $d \mid a$ (d divides a) and $d \mid b$ which, in turn, implies that $d \mid (a - qb)$, or $d \mid r$. Thus d is a common divisor of b and r. If c is any common divisor of b and r, then $c \mid (qb + r)$ which implies that $c \mid a$. Thus c is a common divisor of a and b. Since d is the largest divisor of both a and b, it follows that $c \leq d$. It now follows from definition that $d = gcd(b, r)$. □

We can then prove the correctness of *Euclid_gcd* using the principle of mathematical induction.

Theorem 9.5 *For all $b \geq 0$, for all $a > 0$, Euclid_gcd(a, b) computes* gcd(a, b)*, the largest common divisor of a and b.*

Proof:

Basis. $b = 0$. If $b = 0$ then for all $a > 0$, $Euclid_gcd(a, b) = a = gcd(a, b)$.

Induction hypothesis. For all $b \leq k$ such that $0 \leq b$, for all $a > 0$, $Euclid_gcd(a, b) = gcd(a, b)$.

Induction step. Consider $b = k + 1$, $a > 0$.

$$\begin{array}{lcll} Euclid_gcd(a, b) & = & Euclid_gcd(b, a \bmod b) & \\ & = & gcd(b, a \bmod b) & \text{by the inductive hypothesis} \\ & = & gcd(a, b) & \text{by the Claim above} \end{array}$$

□

Definition 9.3 *Two positive integers are* relatively prime *if their gcd is 1, i.e., they have no non-trivial factors in common.*

9.2.2 The Extended Euclidean Algorithm

Euclid's algorithm is remarkable because it quickly computes the gcd of two numbers. But sometimes we need more than just the gcd: we want to write the gcd as a sum of multiples of the two numbers themselves[5].

For example, for $a = 30$ and $b = 12$, not only is $\gcd(30, 12) = 6$, but also

$$6 = 30 \times (-1) + 12 \times 3.$$

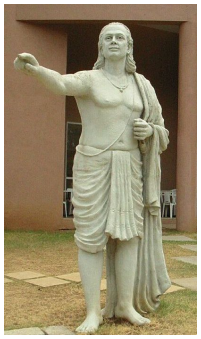
In other words, the gcd can be expressed as a whole-number combination of a and b. This means we can "build" the gcd by taking suitable multiples of 30 and 12. This fact is called Bézout's identity.

The *Extended Euclidean Algorithm* is just a way to find these multipliers at the same time as we compute the gcd.

How it works (idea). We start the same way as in Euclid's algorithm:

$$a = q_1 b + r_1, \quad b = q_2 r_1 + r_2, \quad \ldots$$

until we reach a remainder 0. While we do this, we also keep track of how each remainder is made from a and b. At the end, the last nonzero remainder is the gcd, and the numbers in front of a and b give us the multipliers we want.



Āryabhaṭa's statue in the Inter-University Centre for Astronomy and Astrophysics (IUCAA) at Pune

[5] Euclid tells us what the gcd is; the extended Euclid tells us how to make it.

Example 9.5 *Let's find the gcd of* 26 *and* 7*, and also write it as a combination of* 26 *and* 7*.*

$$26 = 3 \times 7 + 5, \quad 7 = 1 \times 5 + 2, \quad 5 = 2 \times 2 + 1, \quad 2 = 2 \times 1 + 0.$$

So $\gcd(26, 7) = 1$*.*
Now work backwards:

$$1 = 5 - 2 \times 2,$$

$$1 = 5 - 2 \times (7 - 1 \times 5) = 3 \times 5 - 2 \times 7,$$

$$1 = 3 \times (26 - 3 \times 7) - 2 \times 7 = 3 \times 26 - 11 \times 7.$$

So we have

$$1 = 3 \times 26 + (-11) \times 7.$$

This shows that -11 *is a multiplier for* 7 *that makes* $7 \times (-11) \equiv 1 \pmod{26}$*. In other words,* 7 *has an inverse modulo* 26*, and it is* 15 *(since* $-11 \equiv 15 \pmod{26}$*)* [6]*.*

The steps of the general algorithm can then be given as:

General Algorithm (Extended Euclid). Given two integers a and b:

1. If $b = 0$, then $\gcd(a, 0) = a$ and we return $(x, y) = (1, 0)$.
2. Otherwise, divide a by b:
$$a = qb + r, \quad 0 \leq r < b.$$
3. Inductively apply the algorithm to (b, r) to get
$$\gcd(b, r) = xb + yr.$$
4. Substitute $r = a - qb$ to express the gcd in terms of a and b:
$$\gcd(a, b) = ya + (x - qy)b.$$
5. Return $(x', y') = (y,\ x - qy)$.

At the end, we obtain integers x, y such that

$$ax + by = \gcd(a, b).$$

Exercise 9.4 *(Bháskara I, c. 600) "Tell me at once, O mathematician, that number which leaves unity as remainder when divided by any of the numbers from 2 to 6 but is exactly divisible by 7." [Ans : 301]*

Exercise 9.5 *(Mahāvīra, c. 850) "Five heaps of fruits added with two fruits were divided equally between nine travellers; six heaps added with four fruits were divided amongst eight; four heaps increased by one fruit were divided amongst seven. Tell the number of fruits in each heap." [(Least) Ans : 194]*

[6] We had discussed this earlier in Theorem 5.4. The origins of this algorithm can be traced back to Āryabhaṭa (5th c), Jayadeva (c. 10th–11th c.) and later Bháskara II (12th c.). Bháskara in the Līlāvatī and Bījagaṇita gives worked examples of solving linear Diophantine equations with what is recognizably the extended Euclid method.

Why this matters. The extended Euclidean algorithm is very useful:

- It gives the gcd of two numbers.
- It shows how to write the gcd using those two numbers.
- If the gcd is 1, it also finds a modular inverse, which is essential in cryptography.

But how many steps does the Euclidean algorithm take? Is it slow or fast? Amazingly, the answer lies in the famous Fibonacci sequence[7] defined as:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_k &= F_{k-1} + F_{k-2}, \text{ for } i \geq 2 \end{aligned}$$

The following theorem establishes the worst-case run rime of the Euclidean algorithm:

Theorem 9.6 *If $a > b \geq 1$ and the invocation Euclid_gcd(a, b) performs $k \geq 1$ repeated calls, then $a \geq F_{k+2}$ and $b \geq F_{k+1}$.*

Exercise 9.6 *Prove the above theorem using the principle of mathematical induction.*

The following result immediately follows:

Theorem 9.7 (Lamé's theorem) *For any integer $k \geq 1$, if $a > b \geq 1$ and $b < F_{k+1}$, then Euclid_gcd(a, b) makes fewer than k repeated calls.*

It is well known that $F_k \leq \phi^{k-1}$ for all $k > 0$, where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio.

Exercise 9.7

1. *Prove that $F_k \leq \phi^{k-1}$ for all $k > 0$ using the principle of mathematical induction.*
2. *Argue that if $b < F_{k+1} \leq \phi^k$, then the number of steps required will be approximately* $\log b$*, which is rather small. So, the algorithm is very efficient.*

9.3 Chinese Remainder Theorem

The Chinese Remainder Theorem (CRT) is one of the most beautiful and useful results in number theory. It says that if we know the remainder of a number when divided by several pairwise coprime moduli, then we can uniquely determine the number modulo the product of those moduli.

In other words, instead of working with a single large modulus, we can break a problem into several smaller ones, solve them separately, and then combine the answers back together. This is extremely powerful in computations, because working modulo small numbers is easier.

The theorem was discovered in ancient China, and first appeared in the work of Sun Tzu around the 3rd century AD, who posed problems like: *"Find a number which leaves remainder 2 when divided by 3, remainder 3 when divided by 5, and remainder 2 when divided by 7."* The answer is unique modulo $3 \cdot 5 \cdot 7 = 105$.

The CRT has many applications:

- Efficient computation in modular arithmetic (e.g. fast exponentiation).
- Cryptography, especially RSA, where it is used to speed up decryption.
- Error correction in coding theory.

Thus, the CRT is not only a striking piece of mathematics but also a practical tool.

[7] Though named after Fibonacci, also referred to as the *Leonardo of Pisa*, the sequence was known to ancient Indian mathematicians like Pingala, Bharata muni, Virahanka, Gopala and Hemachandra.

Example 9.6 *An example (Sun Tzu, 3rd century AD).*

"There are certain things whose number is unknown. If we count them by threes, we have two left over; by fives, we have three left over; and by sevens, two are left over. How many things are there?"

Suppose we want to find a number N such that

$$N \equiv 2 \pmod 3, \quad N \equiv 3 \pmod 5, \quad N \equiv 2 \pmod 7.$$

Step 1. The moduli $3, 5, 7$ are pairwise coprime, so the CRT applies. The product is $M = 3 \cdot 5 \cdot 7 = 105$.

Step 2. For each congruence, compute

$$M_1 = \frac{M}{3} = 35, \quad M_2 = \frac{M}{5} = 21, \quad M_3 = \frac{M}{7} = 15.$$

Step 3. Find numbers y_i such that

$$M_1 y_1 \equiv 1 \pmod 3, \quad M_2 y_2 \equiv 1 \pmod 5, \quad M_3 y_3 \equiv 1 \pmod 7.$$

- $35 \equiv 2 \pmod 3$*, and* $2 \cdot 2 \equiv 1 \pmod 3 \implies y_1 = 2$.
- $21 \equiv 1 \pmod 5$*, so* $y_2 = 1$ *works.*
- $15 \equiv 1 \pmod 7$*, so* $y_3 = 1$ *works.*

Step 4. Combine everything:

$$N \equiv a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3 \pmod M,$$

where $(a_1, a_2, a_3) = (2, 3, 2)$.

$$N \equiv 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 \pmod{105}.$$

$$N \equiv 140 + 63 + 30 = 233 \pmod{105}.$$

Step 5. Reduce: $233 \equiv 23 \pmod{105}$.

Therefore, the solution is

$$N \equiv 23 \pmod{105}.$$

Indeed, $23 \equiv 2 \pmod 3$, $23 \equiv 3 \pmod 5$*, and* $23 \equiv 2 \pmod 7$.

Theorem 9.8 (Chinese Remainder Theorem) *Let $n_1, \ldots, n_k$ be pairwise relatively prime (i.e. $\gcd(n_i, n_j) = 1$ for $i \neq j$), and $N = n_1 \cdots n_k$. Then for any integers $a_1, \ldots, a_k$ there is a unique solution $x \pmod N$ to*

$$x \equiv a_i \pmod{n_i}, \qquad i = 1, \ldots, k.$$

Proof: Let $N_i = N/n_i$. Since $\gcd(N_i, n_i) = 1$, there exists y_i with $N_i y_i \equiv 1 \pmod{n_i}$. Then $x = \sum a_i N_i y_i$ solves the congruences. Uniqueness follows: if x and x' are both solutions, then each n_i divides $x - x'$, hence so does N. □

This also has an useful special form:

Theorem 9.9 *Let $n_1, n_2, \ldots, n_k$ be pairwise relatively prime, and let $N = n_1 n_2 \cdots n_k$. Then for all integers x, a,*

$$x \equiv a \pmod{n_i} \text{ for each } i = 1, 2, \ldots, k \quad \iff \quad x \equiv a \pmod N.$$

Proof: ($\Rightarrow$) Assume $x \equiv a \pmod{n_i}$ for each i. Then $n_i \mid (x-a)$ for all i. Since the n_i are pairwise coprime, we claim that

$$n_1 n_2 \cdots n_k \mid (x-a).$$

Thus $x \equiv a \pmod{N}$.

($\Leftarrow$) Conversely, if $x \equiv a \pmod{N}$ then $n \mid (x-a)$. Since $n_i \mid N$, we have $n_i \mid (x-a)$ for each i, i.e. $x \equiv a \pmod{n_i}$ for all i.

This proves the equivalence. □

Exercise 9.8 *(Yih-hiang, c. 700) "Find the number of completed units of work, the same number of units to be performed by each of four sets of 2, 3, 6, 12 workmen, such that after certain whole days' work, there remains 1, 2, 5, 5 units not completed by the respective sets." [(Least) Ans : 17]*

9.4 Public-Key Cryptography

From ancient times to the digital present, cryptography[8] has carried with it an air of mystery and intrigue. In the past, emperors and generals relied on secret codes to protect messages from falling into enemy hands. The Caesar cipher, for instance, shifted letters by a fixed number of places in the alphabet, and even such a simple trick was often enough to change the course of a battle, either way.

But as history has shown, secrecy is never easy to preserve. Every cipher invites the challenge of decryption: a contest between the code maker and the code breaker. This cat-and-mouse game gave rise to ingenious methods on both sides — from the codebreakers at Bletchley Park during World War II, who cracked the German Enigma machine, to modern-day mathematicians working on factoring large numbers.

Public key cryptography is one of the most astonishing turns in this story. Until the 1970s, it was believed that secure communication required two parties to share a secret key in advance. The invention of public key cryptography overturned this assumption: it showed that two people could establish secure communication without ever having met before, using only openly published keys.

This breakthrough was so surprising that it seemed almost paradoxical at first. How can one lock a message with a key that is public, and yet keep it safe from anyone but the intended recipient? The resolution lies in deep number theory, and in particular in the surprising difficulty of problems such as factoring large numbers. It is here that the elegance of mathematics meets the intrigue of secrecy.

In a public key cryptosystem, each participant has

- a **public key** (made widely known),
- a **private key** (kept secret).

The public and the secret keys specify functions that can be applied to any message M which belongs to the set of permissible messages $\mathcal{D}$. The public and secret keys for a participant are a "matched pair" in that they specify functions that are inverses of each other. That is,

$$\begin{aligned} M &= S(P(M)), \\ M &= P(S(M)) \end{aligned}$$

for any $M \in \mathcal{D}$. The challenge in designing a workable public-key cryptosystem is in figuring out how to create a system in which we can reveal a transformation $P()$, without thereby revealing how to compute the inverse transformation $S()$. Public-key cryptography solves this.

9.4.1 How to encrypt?

Suppose Bob wishes to send a message M to Alice, encrypted so that it will look like unintelligible gibberish to an eavesdropper. The procedure for doing so in a public-key cryptosystem is:

[8] See here for an abridged history of cryptography.

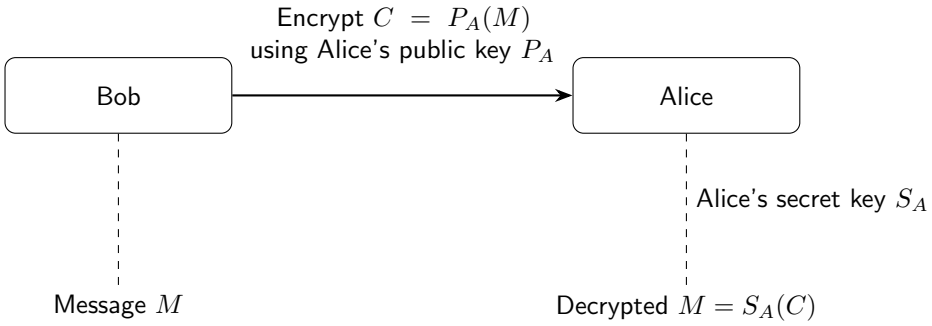


Figure 9.3: Encrypting a message with the receiver's public key.

- Bob obtains Alice's key P_A, from a public directory, or directly from Alice.
- Bob computes the *ciphertext* $C = P_A(M)$, corresponding to the message M and sends C to Alice.
- When Alice receives the ciphertext C, she applies her secret key S_A to retrieve the original message: $M = S_A(C)$.

Since S_A and P_A are inverse functions, Alice can recover M from C. Only Alice is able to compute M from C, as only Alice knows S_A.

9.4.2 Digital signature

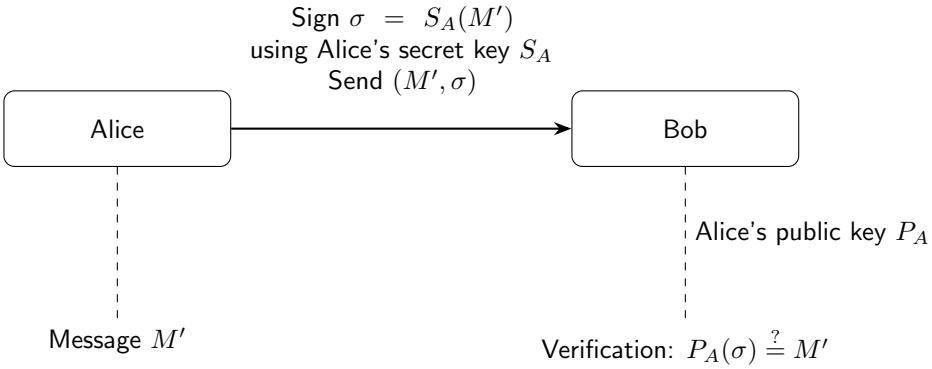


Figure 9.4: Digital signature: Alice signs the message M' with her secret key to produce $\sigma = S_A(M')$. Bob verifies by checking $P_A(\sigma) = M'$ using the public key.

A signature is a device by which the receiver of the signed message can be assured of the identity of the sender and the integrity of the message. Digital signatures are fairly easy to implement in a public-key cryptosystem. Suppose Alice wishes to send Bob a digitally signed response M'. The procedure is as follows:

- Alice computes her *digital signature* σ for the message M' using her secret key S_A and the relation $\sigma = S_A(M')$.
- Alice sends the message-signature pair (M', σ) to Bob.
- When Bob receives (M', σ), he can verify that it came from Alice by using Alice's public key P_A to verify the relation $M' = P_A(\sigma)$. (We presume that M' contains Alice's name, so that Bob knows whose public key to use.) If the equation holds, then Bob concludes that Alice had

sent it. If the equation does not hold, then he concludes that either M' or σ was corrupted by transmission errors, or (M', σ) is an attempted forgery. Note that appending a digital signature necessarily increase the size of the total message.

One can easily combine the two schemes to send signed and encrypted messages.

Exercise 9.9 *Combine the two schemes above to derive a protocol for sending signed and encrypted messages.*

9.5 The RSA Cryptosystem


Ronald Rivest


Adi Shamir


Leonard Adleman

The RSA cryptosystem was proposed by Ronald Rivest, Adi Shamir and Leonard Adleman in 1977. They got the Turing award for their result in 2002.

9.5.1 The RSA procedure

To create his public and secret keys, a person uses the following procedure:

- Select two large prime numbers, p and q, typically of 300 to 600 digits each. To select primes, choose random numbers of this size and test for primality using the Fermat's Little Theorem (Theorem 9.4). By the Prime Number Theorem (Theorem 9.3) we are guaranteed to succeed in just a few trials.
- Compute $n = pq$.
- Select a small odd integer e that is relatively prime to $m = (p-1)(q-1)$. Use Euclid's GCD to test whether a randomly chosen e is relatively prime to m.
- Compute d, the multiplicative inverse of e modulo m. Use the Extended Euclid Algorithm.
- Publish the pair $P = (e, n)$ as his RSA public key.
- Keep secret the pair $S = (d, n)$ as his RSA secret key.

For this scheme, the domain $\mathcal{D}$ is $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$. That is, the message M must be such that $M \in \mathbb{Z}_n$.The transformation of a message M associated with a public key $P = (e, n)$ is

$$P(M) = M^e \mod n$$

The transformation of the ciphertext C associated with a secret ket $S = (d, n)$ is

$$S(C) = C^d \mod n$$

These equations apply both to encryption and signatures.

9.5.2 Why does it work?

It is easy to see that P and S are inverses:

$$M^{ed} \equiv M \mod n$$

Proof: Note that $m = (p-1)(q-1)$. Since e and d are multiplicative inverses modulo m. We have that

$$ed = 1 + km = 1 + k(p-1)(q-1)$$

Thus if $M \not\equiv 0 \mod p$, we have that

$$\begin{aligned} M^{ed} &\equiv M(M^{p-1})^{k(q-1)} \mod p \\ &\equiv M(1)^{k(q-1)} \mod p \\ &\equiv M \mod p \end{aligned}$$

Note that $M^{p-1} \equiv 1 \mod p$ by Fermat's little theorem (Theorem 9.4).

If $M \equiv 0 \mod p$, we anyway have that

$$M^{ed} \equiv M \mod p$$

Repeating the same argument as above, we have that

$$M^{ed} \equiv M \mod q$$

Hence, by the Chinese Remainder Theorem (Therem 9.8) we have that

$$M^{ed} \equiv M \mod pq \equiv M \mod n$$

□

9.5.3 How secure is RSA?

Multiplying two large primes to form n is easy, but given only n it is extremely difficult to recover p and q with current knowledge. RSA security relies on this asymmetry: the public modulus n reveals nothing practical about the private key. At present, the only known way to obtain d from e and n is to factor n into its prime factors p and q, then compute m and proceed as above. But no one knows how to factor large integers efficiently, despite several centuries of effort. Using known methods, factoring $n = pq$ where p and q are 600-digit primes would require years on today's fastest supercomputers. Until someone comes up with an efficient way to factor, or discovers some other way to compute d from e and n, the system is reasonably secure for all practical purposes.

RSA remains one of the most commonly use public-key encryption system in the digital world, including in banking, stock markets and other financial trasactions.

Summary

- Primes are the building blocks of integers.
- Euclid's algorithm computes gcds efficiently.
- There are infinitely many primes, and every integer has a unique prime factorisation.
- Fermat's little theorem and the CRT are central in modular arithmetic.
- Public-key cryptography allows secure communication without prior key sharing.
- RSA uses primes and modular arithmetic, with security based on the difficulty of factoring.

Problems

1. **Prime numbers and factorisation**

   (a) Prove that there are infinitely many primes of the form $4k + 3$.

   (b) Factor 123456 into primes using successive gcd computations.

   (c) Use the sieve of Eratosthenes to list all primes up to 200. How efficient is this compared to trial division?

2. **Greatest common divisor**

   (a) Compute $\gcd(414, 662)$ using Euclid's algorithm. Show all intermediate steps.

   (b) Use the extended Euclidean algorithm to find integers x, y such that $414x + 662y = \gcd(414, 662)$.

3. **Fermat's little theorem**

   (a) Verify Fermat's little theorem for $a = 7, p = 13$.

   (b) Show by counterexample that Fermat's little theorem does not hold if p is composite.

   (c) Explain how Fermat's little theorem can be used as a primality test. What are its limitations?

4. **Chinese remainder theorem**

   (a) Solve the system of congruences:

   $$x \equiv 2 \pmod 3, \quad x \equiv 3 \pmod 5, \quad x \equiv 2 \pmod 7.$$

   What is the smallest positive solution?

   (b) A recent visitor to Ashoka University, Prof Amartya Kumar Dutta of ISI Kolkata gave the following problems:

      i. In a science camp of 210 students, creativity tests were planned in each of the subjects Mathematics, Physics and Chemistry. The students were to attempt the questions in groups of 5, 6 and 7 respectively. Unfortunately, a few students could not turn up due to unforeseen circumstances and it was found that dividing students into groups of 5, 6 and 7 would leave remainders 2, 1 and 5 respectively. Using kuṭṭaka, a professor of mathematics quickly realised how many students were present and suggested how the students could be grouped if uniformity in number was to be insisted. What was the total number of students present?

      ii. (Hardy-Wright) From Monday, 1 January 2018, six professors begin two-year courses of lectures on Monday, Tuesday, Wednesday, Thursday, Friday, and Saturday. They announce their intention of lecturing at intervals of two, three, four, one, six, and five days respectively. However, no lecture is to take place on a Sunday and a professor would omit his lecture if it falls on a Sunday. For instance, the dates of the lectures of the first professor were January 1,3,5,9,11,13, ... (omitting 7 January, a Sunday). What will be the date of the first Sunday on which all six professors find themselves compelled to omit a lecture?

   (c) Explain why the Chinese Remainder Theorem is crucial for speeding up RSA decryption.

5. **Cryptography and RSA**

   (a) Work out a small RSA example: choose primes $p = 17, q = 23$, construct public/private keys, and encrypt/decrypt the message $M = 42$.

   (b) Suppose Eve can factor the modulus n. Show why RSA becomes insecure.

(c) Consider the Caesar cipher. Implement it with shift $k = 3$, encrypt the message "HELLO", then decrypt it.

(d) Compare the Caesar cipher with RSA in terms of key space, security, and vulnerability to brute-force attack.

6. **Reflection and open-ended problems**

(a) Discuss why factoring large numbers is computationally difficult, and why this difficulty underpins public-key cryptography.

(b) Research the Miller–Rabin primality test. How does it improve upon Fermat's test?

(c) Find a real-world application where digital signatures are used. Explain the role of public and private keys in that context.

Chapter 10

Counting to Beat the Odds



"There is no computation without the art of orderly enumeration." – Paraphrased from Mahāvīra, *Gaṇitasārasaṇgraha* (9th century CE)



"There is no end to what can be counted; the more we count, the more we see patterns emerge." – Paraphrased from the notebooks of Srinivasa Ramanujan

10.1 Introduction

Counting is one of the oldest and most universal human activities. Long before the invention of symbols or written numbers, people used stones, knots, or scratches on bones to keep track of livestock, harvests, or trade. From these humble origins grew the entire edifice of arithmetic, combinatorics, and probability theory.

In modern computer science and mathematics, counting is not merely an act of enumeration but a way of understanding structure. How many possible passwords of a given length can be formed? In how many distinct ways can a set of vertices in a network be connected? In how many ways can certain objects be arranged? Each of these questions is, at heart, a counting problem.

The goal of this chapter is to develop systematic ways of counting the number of possible outcomes in a finite discrete system, and then to use these counts to reason about uncertainty and probability.

- We first learn the *basic counting principles*—the addition and multiplication rules—that form the backbone of all enumeration.
- We then study *permutations* and *combinations*, which tell us how many ways elements can be arranged or selected.
- We then study about the *binomial theorem* and *binomial coefficients*.

- Finally, we build upon these ideas to develop the foundations of *discrete probability*—reasoning about likelihood when outcomes are finite and equally likely.

10.2 The rules of sum and product

10.2.1 The rule of sum

If a task can be done in m ways, and another independent task can be done in n ways, and the two tasks cannot be done simultaneously, then there are $m + n$ ways of choosing one of the two tasks.

Example 10.1 *1. Suppose you may take either a bus or a train to reach a city. If there are 3 bus routes and 2 train routes, then there are* $3 + 2 = 5$ *ways to travel.*

2. Suppose the sections 1, 2 , and 3 of QRMT have 85, 90 and 88 students respectively. If we have to choose one class representative from the combined three sections, then there are $85 + 90 + 88 = 263$ *ways to choose the representative.*

10.2.2 The rule of product

If a task consists of two successive stages, the first of which can be done in m ways and the second in n ways, then the entire task can be done in $m \times n$ ways.

Example 10.2 *1. Suppose you must choose a shirt and a pair of trousers. If there are 4 shirts and 3 trousers, then there are* $4 \times 3 = 12$ *possible outfits.*

2. Suppose the sections 1, 2 , and 3 of QRMT have 85, 90 and 88 students respectively. If we have to choose one class representative from each of the three sections independently, then there are $85 \times 90 \times 88 = 673200$ *ways of choosing the three representatives.*

10.3 Permutations

Definition 10.1 *A* permutation *of a set of n objects is an ordered arrangement of all the objects. The number of permutations of n objects is $n!$, read as "n factorial":*

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1.$$

The first can be chosen from n objects, the second from the remaining $n - 1$ objects, the third from the remaining $n - 2$, and so on till 1.

Example 10.3 *How many ways can 5 students line up for a photograph?*
Solution: *There are* $5! = 120$ *possible line-ups.*

Next, consider the number of ways of placing three balls coloured red, blue, and white in 10 boxes numbered $1, 2, 3, \ldots, 10$, if each box can hold only one ball. The red ball can be placed in any of the 10 boxes, the blue ball can be place in any of the 9 remaining boxes, and the white ball can be placed in any of the remaining 8 boxes. Thus the total number of distinct ways to place these balls is $10 \times 9 \times 8 = 720$.

In general, if we have to place r distinctly coloured balls in n distinctly numbered boxes, then the number of distinct ways is

$$n(n-1)(n-2)\cdots(n-r+1) = \frac{n!}{(n-r)!}$$

Example 10.4 *1. In how many ways four examinations can be scheduled in a week so that no two examinations are scheduled on the same way? Considering the days as boxes and the examinations as distinct balls we obtain the result as* $7 \times 6 \times 5 \times 4 = 840$.

2. *How many 3-digit numbers can be formed using the digits* $\{1, 2, 3, 4, 5\}$ *without repetition? Arguing similarly we arrive at* $5 \times 4 \times 3 = 60$.

3. *How many four-digit decimal numbers can be formed without repeating the digits. This according to the above analysis is* $10 \times 9 \times 8 \times 7 = 5040$. *However,* $9 \times 8 \times 7 = 504$ *of them have 0 as the first digit. So, the number of distinct numbers that can be formed is* $5040 - 504 = 4536$. *We can also arrive at this result from the first principles by excluding 0 in the leading position. We then get the number of distinct numbers is* $9 \times 9 \times 8 \times 7 = 4536$.

An equivalent problem of placing distinct balls in boxes is that of arranging distinct objects. By permuting r of n distinct objects we mean to arrange r of these n objects in some order. For example, there are six ways to permute two out of three objects a, b, c. These are ab, ba, ac, ca, bc, cb. Arranging r on n objects amounts to filling r positions with r objects. We get the following definition:

Definition 10.2 Restricted permutations*: If only* r *elements out of* n *distinct elements are to be arranged, the number of permutations is*

$$P(n, r) = \frac{n!}{(n-r)!}.$$

Example 10.5 *How many ways can 3 books be arranged on a shelf from a collection of 10?*

$$P(10, 3) = \frac{10!}{7!} = 720.$$

10.4 Counting functions and power sets

Let us consider the problem of placing 3 distinctly coloured balls into 10 distinctly numbered boxes, but let a box hold as many balls as we wish. Since each of the red, blue, and the white ball can be placed in any of the 10 boxes the total number of ways is

$$10 \times 10 \times 10 = 1000$$

This is exactly the total number of functions that can be defined from the set of balls to the set of boxes.

In general, If there are r distinct balls (the domain has r elements) to be placed in n distinct boxes (the co-domain has n elements) and multiple balls can be mapped on to a box, then the total number of ways is n^r.

Example 10.6 *Let us revisit the problem of scheduling four examinations in a week, but without any restrictions on number of examinations per day. This is exactly the problem of determining the number of functions that can be defined from the courses to days of the week. The result is* $7^4 = 2401$.

Exercise 10.1 *Suppose that the popular tea company serve tea to their customers with 12 types of ingredients, each ingredient can be present or absent. How many different types of tea can they serve?*

Exercise 10.2 *An n-bit binary number is defined as* $b_0 + b_1 \times 2^1 + b_2 \times 2^2 + \ldots + b_{n-1} \times 2^{n-1}$*, where each* b_i *is 0 or 1. How many n-bit binary numbers are there? How many will have an even number of 1s?*

Exercise 10.3 *How many functions (truth tables) of n boolean input and 1 boolean output can we define?*

10.4.1 Power set

The Power set $\mathcal{P}(A)$ of a set A is defined as the set of all subsets of A. For example, the set of all subsets of A is $\mathcal{P}(A) = \{\phi, \{0\}, \{1\}, \{0, 1\}\}$. Suppose the size of A is r. What is the size of $\mathcal{P}(A)$?

Consider the problem of placing each of the r elements in two boxes – inside the subset or outside. Since the 2^r ways of placing the r elements, there are 2^r elements in $\mathcal{P}(A)$.

10.5 Combinations

Consider now the problem of placing three balls, all red, in 10 boxes numbered $1, 2, \ldots, 10$. What are the total number of ways the balls can be placed if each box can hold at most one ball?

If the balls were of three shades of red, and hence distinguishable, then the answer would be $P(10, 3)$. However, if they are indistinguishable then 3! of the ways of their placement actually becomes one, and the number of ways reduces to $P(10, 3)/3!$.

A problem equivalent to placing r indistinguishable objects in n numbered boxes is that of selection of r objects from n distinct objects. If we were to select r objects from n, we can imagine the n objects as boxes, and mark the selected ones.

Definition 10.3 *A* combination *is a selection of objects without regard to order. The number of combinations of r objects chosen from n is*

$$C(n, r) = \frac{n!}{r!(n-r)!}.$$

This is also sometimes denoted as $\binom{n}{r}$.

Example 10.7 *1. Suppose Dining@Ashoka wants to schedule* Tinda Kala Chana *at lunch three times a week. They can clearly do so in $\binom{7}{3} = \frac{7!}{3!4!} = 35$ ways.*

2. From 10 professors, in how many ways can a committee of 3 be chosen? The answer is $\binom{10}{3} = \frac{10!}{3!\,7!} = 120$.

Exercise 10.4 *Clearly $\binom{n}{r} = \binom{n}{n-r}$. Give a combinatorial argument for the above.*

Example 10.8 *Consider a $p \times q$ chess board. In how many ways a rook can move from the bottom-leftmost corner to the top-rightmost, moving only towards right and top? There are total of $p-1$ right moves and $q-1$ top moves to be made. Think of them as red and blue balls respectively. Then, the problem is equivalent to the number of ways of choosing $p-1$ red balls from $p+q-2$ total balls, which is $\binom{p+q-2}{p-1}$. This is also exactly equal to $\binom{p+q-2}{q-1}$.*

Exercise 10.5 *Suppose we have to place r balls in n numbered boxes, where q_1 them are of one colour, q_2 of them are in second colour, ..., and q_k of them are of the kth colour. Argue that the total number of ways to place the balls is*

$$\frac{P(n, r)}{q_1! q_2! \cdots q_k!}$$

Example 10.9 *Among 11 professors there are $C(11, 5) = 462$ ways of selecting a committee of 5 members. Also, there are $C(10, 4) = 210$ ways to select a committee of 5 members so that a particular professor, say professor Rajan, is always included, and there are $C(10, 5) = 252$ ways of selecting a committee of 5 members so that professor Rajan is always excluded.*

We now ask in how many ways we can select a committee of 5 members so that at least one of prof Rajan and prof Suban will be included. The number of selections including both is $C(9, 3) = 84$. The number of selections including Rajan but excluding Suban is $C(9, 4) = 126$, as is the number of selections including Suban but excluding Rajan. Thus, the total number of ways is

$$84 + 126 + 126 = 336$$

Alternatively, since the total number of selections excluding both Rajan and Suban is $C(9, 5)$, the total number of ways of selection is

$$C(11, 5) - C(9, 5) = 462 - 126 = 336$$

10.5.1 Combination with repetitions

Consider the problem of placing r identical balls in n numbered boxes, but allowing as many balls in a box as we wish.

As an equivalent problem, consider the problem of arranging $n+1$ 1s and r 0s with a 1 at the beginning and a 1 at the end of each arrangement. If we consider the 1s as inter-box partitions and 0s as balls, then every such arrangement is a way of placing the r balls in the n boxes with possible repetitions. For example, let $n = 5$, and $r = 4$. Then the sequence

$$1011001101$$

represents placing 1 ball in the first box, none in the second, 2 balls in the third box, none in the fourth, and 1 ball in the fifth box. The number of ways of arranging r 0s and $n+1$ 1s with 1s at both ends is

$$C(n+r-1, r) = \frac{(n+r-1)!}{r!(n-1)!}$$

Example 10.10 *1. The number of ways to select three days in a week with repetitions allowed is* $C(9,3) = 84$.

2. The number of ways to select seven out of three days with repetitions allowed is $C(9,7) = 36$.

3. When three dices are rolled, the number of different outcomes is $C(8,3) = 56$.

10.6 The principle of Inclusion and Exclusion

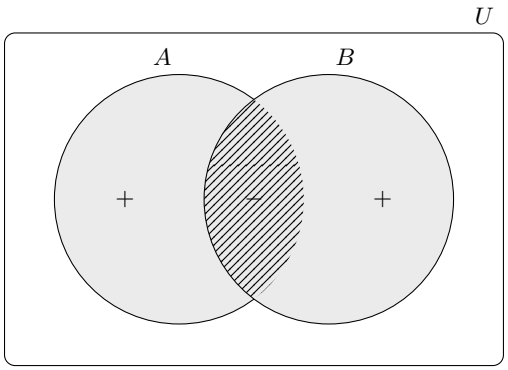


Figure 10.1: The principle of inclusion and exclusion (2 sets).

The problem of Example 10.9 can also be solved by the following method. Among the 462 ways of selecting the committee of 5, let A_1 and A_2 be the set of ways of selection that include Rajan and Suban respectively. Now, since

$$\begin{aligned} &| A_1 | = C(10,4) = 210 \\ &| A_2 | = C(10,4) = 210 \\ &| A_1 \cap A_2 | = C(9,3) = 84 \end{aligned}$$

We. can estimate $| A_1 \cup A_2 |$, the number ways to include at least one of Rajan and Suban, as

$$| A_1 \cup A_2 | = | A_1 | + | A_2 | - | A_1 \cap A_2 | = 210 + 210 - 84 = 336$$

The last subtraction is necessary because the number of ways where both Rajan and Suban are included is double counted in $| A_1 | + | A_2 |$. This method of counting is called the *principle of inclusion and exclusion.*

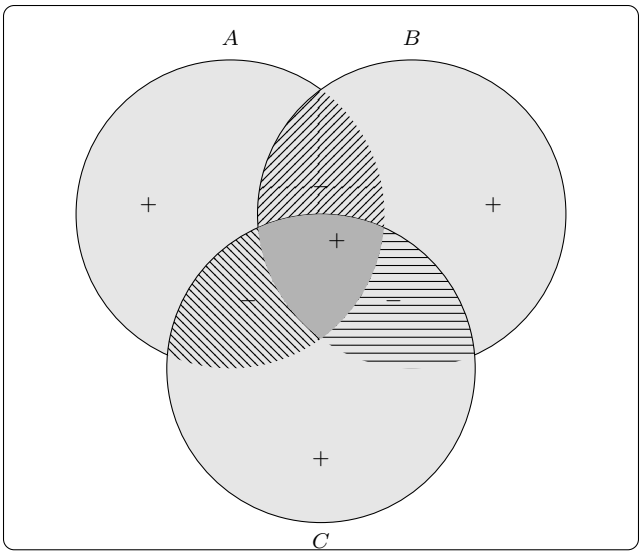


Figure 10.2: The principle of inclusion and exclusion (3 sets).

For any two finite sets A and B,

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

The subtraction corrects for the double counting of elements common to both sets (See Figure 10.1) .

Example 10.11 *Let A be the set of students taking Mathematics (40 students) and B be those taking Physics (30 students), with 10 students taking both. Then, the size of the set taking at least one of Mathematics and Physics is*

$$|A \cup B| = 40 + 30 - 10 = 60.$$

For three finite sets A, B, C,

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|.$$

Exercise 10.6 *Argue the correctness of the above formula (see Figure 10.2).*

We can generalize the above for n sets $A_1, A_2, \ldots, A_n$ as,

$$\left|\bigcup_{i=1}^{n} A_i\right| = \sum_i |A_i| - \sum_{i<j} |A_i \cap A_j| + \sum_{i<j<k} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n+1} |A_1 \cap A_2 \cap \cdots \cap A_n|.$$

Example 10.12 (Derangements) *How many permutations of $\{1, 2, 3\}$ have no fixed point (i.e., no element appears in its original position)? Let A_i denote the set of permutations that fix i.*

$$|A_1| = |A_2| = |A_3| = 2, \quad |A_i \cap A_j| = 1, \quad |A_1 \cap A_2 \cap A_3| = 1.$$

Thus,

$$|\overline{A_1 \cup A_2 \cup A_3}| = 3! - (3 \times 2) + (3 \times 1) - 1 = 6 - 6 + 3 - 1 = 2.$$

Hence, there are 2 derangements.

Example 10.13 *Suppose an Ashoka student wants to make up a schedule for a seven-day period during which she will study only one subject each day. She is taking four subjects - Environmental Studies (EVS), Great Books (GB), Economic Politics and Society (EPS) and Quantitative Reasoning*

and Mathematical Thinking (QRMT). Clearly, there are 4^7 *different schedules. We want to know the number of schedules that devote at least one day to each subject?*

Let A_1 *denote the set of schedules in which EVS is never studied; likewise* A_2, A_3, A_4 *for GB, EPS, and QRMT respectively. Then,*

$$A_1 \cup A_2 \cup A_3 \cup A_4$$

is the set of schedules in one or more of the subjects is not included. Since

$$\begin{array}{l} \mid A_1 \mid = \mid A_2 \mid = \mid A_3 \mid = \mid A_4 \mid = 3^7 \\ \mid A_1 \cap A_2 \mid = \mid A_1 \cap A_3 \mid = \mid A_1 \cap A_4 \mid = \mid A_2 \cap A_3 \mid = \mid A_2 \cap A_4 \mid = \mid A_3 \cap A_4 \mid = 2^7 \\ \mid A_1 \cap A_2 \cap A_3 \mid = \mid A_1 \cap A_2 \cap A_4 \mid = \mid A_1 \cap A_3 \cap A_4 \mid = \mid A_3 \cap A_3 \cap A_4 \mid = 1^7 \\ \mid A_1 \cap A_2 \cap A_3 \cap A_4 \mid = 0 \end{array}$$

We obtain

$$\mid A_1 \cup A_2 \cup A_3 \cup A_4 \mid = 4(3^7) - 6(2^7) + 4$$

Consequently, the number of schedules in which all subjects will be included is

$$4^7 - (4(3^7) - 6(2^7) + 4)$$

Exercise 10.7 *Convince yourself* $P(7,4) * 4^3$ *– there are* $P(7,4)$ *ways to schedule 4 subjects for 4 of the 7 days, and* 4^3 *ways to schedule three subjects for the remaining three days – is not the correct answer.*

10.7 Recurrences: Counting by Relations

Consider the following algorithms for sorting a list of integers in ascending order. Suppose we start with a sequence in random order such as

$$\mid 8\ 3\ 5\ 9\ 7$$

The symbol $\mid$ denotes that the elements to the right are unsorted. At the first step we can search through the list for the minimum element, and swap it with the element in the first position. We obtain a sorted part and an unsorted part

$$3 \mid 8\ 5\ 9\ 7$$

Then, keeping the sorted part fixed, we can search through the rest of the list for the minimum element and swap it with the first element in the unsorted part to obtain

$$3\ 5 \mid 8\ 9\ 7$$

Repeating the process we obtain in a sequence

$$3\ 5\ 7 \mid 9\ 8$$

and

$$\begin{array}{c} 3\ 5\ 7\ 8 \mid 9 \\ 3\ 5\ 7\ 8\ 9 \mid \end{array}$$

Our problem is to count the number of comparisons required to sort a random sequence of n elements.

We can do this using the principle of mathematical induction. Let $T(n)$ be the number of comparisons required to solve a sorting problem of size n. If $n = 1$ then the sequence is sorted by default. Otherwise, we need to search through n elements to find the minimum which requires

$n-1$ comparisons, and are left with an identical subproblem of size $n-1$ which forms the induction hypothesis $T(n-1)$. Hence,

$$T(n) = \begin{cases} 0 & \text{if } n = 1 \\ T(n-1) + (n-1) & \text{if } n > 1 \end{cases}$$

The above is called a *recurrence relation*. We can solve this by telescoping

$$\begin{aligned} T(n) &= T(n-1) + (n-1) \\ &= T(n-2) + (n-2) + (n-1) \\ &\vdots \quad \vdots \\ &= T(1) + 1 + 2 + \ldots + (n-1) \\ &= 1 + 2 + \ldots + (n-1) = n(n-1)/2 \end{aligned}$$

Thus we require $n(n-1)/2$ comparisons.

A *recurrence relation* defines a sequence by relating each term to earlier ones. Recurrences arise naturally in counting when a choice splits a problem into smaller subproblems. In fact, all counting problems can be expressed through recurrences. Consider the following examples.

Example 10.14 *Consider the counting problem of number of distinct ways of arranging n objects. There is only one way to arrange an empty set of objects, which forms the base case $P(0) = 1$. Suppose $P(n-1)$ is the number of ways of arranging $n-1$ objects for $n > 0$ (the induction hypothesis). Then, the nth object can be inserted in each of the $P(n-1)$ arrangements in n ways. Thus we have*

$$P(n) = \begin{cases} 1 & \textit{if } n = 0 \\ n * P(n-1) & \textit{if } n > 0 \end{cases}$$

which evaluates to $P(n) = n!$

Example 10.15 *Consider the rook problem of Example 10.8. In how many ways a rook can move from the bottom-leftmost corner to the top-rightmost in a $p \times q$ chessboard, moving only towards right and top? If $p = 1$ or $q = 1$ (a 1-dimensional chessboard), then there clearly is only one way giving us our base case. Otherwise, by the rule of sum, it is the sum of the total number of ways by taking the first step towards the right and the total number of ways by taking the first step towards the top. We obtain*

$$Rook(p,q) = \begin{cases} 1 & \textit{if } p = 1 \textit{ or } q = 1 \\ Rook(p-1,q) + Rook(p,q-1) & \textit{otherwise} \end{cases}$$

Note that $Rook(p-1,q)$ and $Rook(p,q-1)$ are identical subproblems of smaller sizes, and hence are valid induction hypotheses. We saw earlier that this must be equal to $\binom{p+q-2}{p-1} = \binom{p+q-2}{q-1}$.

10.7.1 Pascal's triangle

The fact that the above recurrence evaluates to a combination suggests that combination may itself be expressed as a recurrence. Consider the problem of selecting r identical balls from n boxes without repetition. Clearly, if $r = 0$ or $r = n$ we have exactly one way, giving our bases cases as $C(n,0) = C(n,n) = 1$. Now, by the rule of sum, the total number of ways of selecting is equal to the sum of the total number of ways of selecting by picking one from the nth box and the total number of ways by not picking at all from the nth box. This gives us

$$C(n,r) = \begin{cases} 1 & \text{if } r = 0 \text{ or } r = n \\ C(n-1,r-1) + C(n-1,r) & \text{otherwise} \end{cases}$$

The above recurrence leads us to the famous triangle of numbers called the Pascal's triangle[1].

[1]The triangular arrangement of binomial coefficients is commonly associated with Blaise Pascal, whose posthumous work (1665) presented it systematically. However, the structure was known much earlier. The Italian algebraist Niccolò Tartaglia published its first six rows in 1556. Even earlier, it appeared in the writings of the Chinese mathematician Yàng Huì and in the work of the Persian scholar Omar Khayyām. Consequently, the triangle is known by different names in different traditions: Yàng Huì triangle in China, Khayyām's triangle in Persia, and Tartaglia's triangle in Italy.

Starting with $n = 0$ the triangle is

$$\begin{array}{ccccccc} 1 & & & & & & \\ 1 & 1 & & & & & \\ 1 & 2 & 1 & & & & \\ 1 & 3 & 3 & 1 & & & \\ 1 & 4 & 6 & 4 & 1 & & \\ 1 & 5 & 10 & 10 & 5 & 1 & \\ 1 & 6 & 15 & 20 & 15 & 6 & 1 \end{array}$$

Note that each number $\binom{n}{r}$ can be obtained by adding the two to the top and left in the previous row. Instead of computing $n!$ and $r!$ which are rather inefficient, the triangle gives faster method of computing $\binom{n}{r}$.

Exercise 10.8 *A curious fact about the Pascal's triangle is that is we start from each of the 1s in the first column and add up the numbers along the anti-diagonals (in the direction ↗), then we obtain the Fibonacci numbers. For example*

$$\begin{array}{l} 1 \\ 1 \\ 1+1=2 \\ 1+2=3 \\ 1+3+1=5 \\ 1+4+3=8 \\ 1+5+6+1=13 \\ 1+6+10+4=21 \\ \vdots \end{array}$$

Can you figure out why?

Example 10.16 *A shop keeper in the Asawarpur village adjoining Ashoka university thought of the following problem: how many ways can one make a change of ₹1, given coins of denomination 50p, 25p, 10p, 5p and 1p? More generally, can one write a function to compute the number of ways to change any given amount of money? After giving some thought to the problem she noted that the total number of ways to make change for some amount is equal to the number of ways to make change for the amount without using any of the first kind of coin, plus the number of ways to make change assuming we do use a coin of the first kind. But the latter number is equal to the number of ways to make change for the amount that remains after using one coin of the first kind. With this solution in mind she approached the students of QRMT for a recurrence for the problem. Assume that there is a function $d : \mathbb{N} \to \mathbb{N}$ that given you the denomination of the nth type of coin (for example, $d(1) = 1$, $d(2) = 25$, $d(3) = 10$, $d(4) = 5$ and $d(5) = 50$)*[2].

Exercise 10.9 *Suppose the amount for which change has to be generate is a. Argue that a solution to the counting problem is given by the following recurrence.*

$$Coin(a, n) = \begin{cases} 0 & \text{if } a < 0 \text{ or } n = 0 \\ Coin(a, n-1) + Coin(a - d(n), n) & \text{otherwise} \end{cases}$$

10.8 The Binomial Theorem

When we expand an expression of the form $(x + y)^n$ for a positive integer n, we obtain a sum of products involving powers of x and y. For small n, we can compute these expansions by hand:

$$(x + y)^2 = x^2 + 2xy + y^2, \qquad (x + y)^3 = x^3 + 3x^2y + 3xy^2 + y^3.$$

[2]This is a special case of a general class of problems called the partition problem

A clear pattern emerges: each term consists of a power of x multiplied by a power of y, and the coefficients follow the rows of Pascal's triangle. The binomial theorem makes this pattern precise, expressing $(x+y)^n$ as a sum in which the coefficients are exactly the binomial numbers $\binom{n}{k}$.

Theorem 10.1 (Binomial Theorem) *For any integer $n \geq 0$ and for all commuting symbols or numbers x, y,*

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

Proof: (**A combinatorial proof**) Expand $(x+y)^n$ as a product of n identical factors:

$$(x+y)^n = (x+y)(x+y)\cdots(x+y).$$

A term $x^{n-k}y^k$ is formed by choosing y from exactly k of the n factors and choosing x from the remaining $n-k$ factors. The number of ways to choose which k factors contribute the y is $\binom{n}{k}$. Thus,

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

□

Proof: (**An inductive proof**) We prove the identity by induction on n.
Base case: $n = 0$. $(x+y)^0 = 1$ agrees with the right-hand side.
Inductive step: Assume that for some $n \geq 0$,

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k. \qquad (*)$$

We prove the identity for $n+1$.

$$(x+y)^{n+1} = (x+y)(x+y)^n = (x+y)\sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

Distribute:

$$(x+y)^{n+1} = \sum_{k=0}^{n} \binom{n}{k} x^{n+1-k} y^k + \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^{k+1}. \qquad (1)$$

We now *reindex* the second sum. Let $j = k+1$. Then when $k = 0$, $j = 1$, and when $k = n$, $j = n+1$. Also $k = j-1$. Therefore:

$$\sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^{k+1} = \sum_{j=1}^{n+1} \binom{n}{j-1} x^{n-(j-1)} y^j = \sum_{j=1}^{n+1} \binom{n}{j-1} x^{n+1-j} y^j. \qquad (2)$$

Now substitute (2) into (1) and combine the sums by matching powers:

$$(x+y)^{n+1} = \binom{n}{0} x^{n+1} + \sum_{j=1}^{n} \left(\binom{n}{j} + \binom{n}{j-1} \right) x^{n+1-j} y^j + \binom{n}{n} y^{n+1}. \qquad (3)$$

Finally, we apply **Pascal's identity**:

$$\binom{n}{j} + \binom{n}{j-1} = \binom{n+1}{j}.$$

Substituting this into (3) gives:

$$(x+y)^{n+1} = \sum_{j=0}^{n+1} \binom{n+1}{j} x^{n+1-j} y^j.$$

This is exactly the binomial expansion for $(x+y)^{n+1}$, completing the induction.

$$\boxed{(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k \text{ for all } n \geq 0.}$$

□

Corollaries

1. $(1+1)^n = \sum_{k=0}^{n} \binom{n}{k} = 2^n$. (Give a purely combinatorial argument for this identity.)
2. $(1-1)^n = \sum_{k=0}^{n} (-1)^k \binom{n}{k} = 0$ for $n \geq 1$.
3. The coefficient of $x^{n-k}y^k$ in $(x+y)^n$ is $\binom{n}{k}$.

Example 10.17 *Using the binomial theorem,*

$$(2x-3)^4 = \sum_{k=0}^{4} \binom{4}{k} (2x)^{4-k}(-3)^k = 16x^4 - 96x^3 + 216x^2 - 216x + 81.$$

10.9 Discrete Probability

Probability provides a mathematical language for reasoning about uncertainty. Everyday situations such as tossing a coin, rolling a die, shuffling cards, choosing passwords, or hashing data involve outcomes that cannot be predicted with certainty in advance. Yet, they are not completely unpredictable: the set of all possible outcomes is known, and many useful questions concern the *likelihood* of particular events. In this section we develop a framework for thinking systematically about such situations.

We begin by describing random experiments and their sample spaces, and how events are formed from outcomes. We then introduce probability measures and the basic axioms that govern them. Next, we discuss random variables, which allow us to assign numerical values to outcomes and reason about distributions, independence, and conditional probabilities. Bayes' theorem provides a powerful tool for updating beliefs in the light of new information. Finally, we conclude with the concepts of information and entropy, which quantify uncertainty and play a central role in modern data science, coding theory, and learning systems.

10.9.1 Experiments, Sample Spaces, and Events

A *random experiment* is a process whose outcome cannot be predicted with certainty in advance, even though all possible outcomes are known. The set of all possible outcomes is called the *sample space*, denoted S. An *event* is a subset $E \subseteq S$.

Example 10.18 *1. Tossing a fair coin:* $S = \{H, T\}$.

2. Rolling a fair die: $S = \{1, 2, 3, 4, 5, 6\}$.

3. Drawing a card from a standard deck of 52 cards.

Two events A and B are *mutually exclusive* if $A \cap B = \varnothing$. A collection of events $A_1, \ldots, A_k$ is *exhaustive* if $A_1 \cup A_2 \cup \cdots \cup A_k = S$.

10.9.2 Probability Measure and Axioms

A *probability measure* assigns to each event $E \subseteq S$ a number $P(E)$ satisfying:

$$P(E) \geq 0, \qquad P(S) = 1, \qquad .$$

When S is finite and all outcomes are equally likely,

$$P(E) = \frac{|E|}{|S|}.$$

Example 10.19 *1. P(even number when rolling a die) $= \frac{3}{6} = \frac{1}{2}$.*

2. *P(ace from a deck) $= \frac{4}{52} = \frac{1}{13}$.*

3. *Consider the problem of dealing a poker hand out of a deck of cards. The sample space consists of $C(52, 5)$ different hands that can be dealt. We assume that all of these are equally probable. What is the probability of getting four aces? We note that 48 possible outcomes from the sample space contain 4 aces (1 way to choose 4 aces and 48 ways to choose the remaining card). So, the probability is $48/C(52, 5) = 0.0000185$.*

4. *What is the probability that in the QRMT class of 88 students, no two have the same birthday (assuming no leap year)? Each of the 88 students can have their birthdays in 365 ways, so the sample space is 365^{88}. Out of these $P(365, 88)$ correspond to the distribution of birthdays so that no two are the same. So, the probability that no two have the same birthday is $P(365, 88)/365^{88} \approx 10^{-5}$, which is rather small. Hence, the probability is close to 1 (almost certain) that at least two people have the same birthday. For how many students is the probability nearly 0.5?*

5. *Suppose 8 Ashoka students are in a tug-of-war team. What is the probability that there are exactly two from each of the four years? Clearly, the sample space is of size 4^8. The total number of ways to select exactly two from each batch is $8!/(2!2!2!2!)$. So the probability is*

$$\frac{8!}{2!2!2!2!4^8} = 0.0385$$

6. *Consider the experiment of shooting a target until there is a hit where the probability of a hit or a miss in a single shot is 1/2 (not a very good shooter). The, the probability of k misses before a hit is $2^{-(k+1)}$. Let A denote the event that there is a hit in no more than 5 misses. Then $A = \{h, mh, mmh, mmmh, mmmmh, mmmmmh\}$. Thus*

$$P(A) = \sum_{k=0}^{5} 2^{-(k+1)} = 0.984$$

Let B denote the event that there is a hit after odd number of misses. Then[3]*,*

$$P(B) = \sum_{i=1}^{\infty} 2^{-2i} = 1/3$$

On the other hand, if C is the event that there is a hit after an even number of misses, then

$$P(C) = \sum_{i=1}^{\infty} 2^{-2i+1} = 2/3$$

[3]We will study the methods to evaluate such infinite summations in Section 12.1.

10.9.3 Random Variables, probability distribution, expectation and variance

A *random variable* is a function

$$X : S \longrightarrow \mathbb{R}.$$

It assigns a numerical value to each outcome of the experiment.

Example 10.20 *If $S = \{1, 2, 3, 4, 5, 6\}$, then $X(\omega) = \omega$ simply records the number rolled.*

The *distribution* of X is given by

$$P(X = x) = |\{\omega \in S : X(\omega) = x\}|/|S|.$$

The *expectation* or *expected value* of a discrete random variable X is the weighted average of its possible values, where each value is weighted by its probability. It captures the *long-run average* or the *mean* outcome if the experiment is repeated many times.

Let X take values $x_1, x_2, \ldots, x_n$ with probabilities $P(X = x_i)$, or more generally let X take countably many values. Then the expectation of X is defined as

$$\mathbb{E}[X] = \sum_x x\, P(X = x).$$

Interpretation. If we repeat the experiment many times and record the values of X, the average of the recorded values will be close to $\mathbb{E}[X]$. Thus, expectation represents the *typical* or *average* outcome in the long run.

Example 10.21 1. ***Fair Die.*** *The random variable X equals the number rolled:*

$$\mathbb{E}[X] = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = 3.5.$$

The expected value need not be a possible outcome.

2. ***Indicator Variables.*** *If I_A is the indicator of event A (i.e., $I_A = 1$ if A occurs and 0 otherwise), then*

$$\mathbb{E}[I_A] = P(A).$$

So expectations generalize probabilities.

Example 10.22 *Consider rolling two fair six-sided dice. Let X be the value on the first die and Y the value on the second die. Define the random variable*

$$S = X + Y,$$

the sum *of the two dice.*

Since each die has 6 faces and the dice are independent, the sample space consists of 36 *equally likely ordered pairs:*

$$S = \{(1, 1), (1, 2), \ldots, (6, 6)\}, \qquad P((i, j)) = \frac{1}{36}.$$

The possible values of the sum S range from 2 *to* 12*. To compute $P(S = s)$, we count how many ordered pairs (i, j) satisfy $i + j = s$.*

$$P(S = s) = \frac{|\ \{(i, j) :\ i + j = s\}\ |}{36}.$$

The number of such pairs increases from $s = 2$ up to $s = 7$, then decreases symmetrically:

$$P(S = s) = \begin{cases} \frac{s-1}{36}, & 2 \le s \le 7, \\ \frac{13-s}{36}, & 8 \le s \le 12. \end{cases}$$

Distribution Table.

s	2	3	4	5	6	7	8	9	10	11	12
number of pairs (i,j)	1	2	3	4	5	6	5	4	3	2	1
$P(S=s)$	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$

Explanation. *The value $S=7$ is the most likely because there are more pairs of dice that add to 7 than to any other sum:*

$$(1,6),(2,5),(3,4),(4,3),(5,2),(6,1).$$

There are 6 such pairs, giving $P(S=7)=6/36=1/6$.

The distribution is symmetric because (i,j) and (j,i) contribute to sums equally. Values near the center have the largest number of combinations, while values near the extremes have fewer.

Example 10.23 *Suppose we repeat an experiment consisting of independent identical trials, each of which results in a* success *with probability p and a* failure *with probability $1-p$, where $0<p\leq 1$.*

Let T be the random variable defined as:

$$T = \textit{the number of trials until the first success occurs.}$$

Thus T takes values in $\{1,2,3,\dots\}$.

Probability Distribution. *For the first success to occur on trial k, the first $k-1$ trials must be failures (each with probability $1-p$), and the k-th trial must be a success.*

$$P(T=k)=(1-p)^{k-1}\,p, \qquad k=1,2,3,\dots$$

This distribution is called the Geometric distribution *with parameter p.*

Verification. *The probabilities sum to 1*[4]*:*

$$\sum_{k=1}^{\infty}(1-p)^{k-1}p = p\sum_{k=0}^{\infty}(1-p)^{k} = p\cdot\frac{1}{1-(1-p)}=1.$$

Expected Value.

$$\mathbb{E}[T]=\sum_{k=1}^{\infty}k\,(1-p)^{k-1}p=\frac{1}{p}.$$

Thus, the average number of trials needed to get a success is $\dfrac{1}{p}$.

Example. *If the probability of success on each trial is $p=\frac{1}{4}$ (for example, getting a four when rolling a die), then*

$$\mathbb{E}[T]=\frac{1}{p}=4.$$

On average, one must roll the die 4 times to get the first four.

While the expectation of a random variable measures its average value, the *variance* measures how much the values of the random variable tend to spread out around the mean.

Let X be a discrete random variable with expectation $\mu=\mathbb{E}[X]$. The *variance* of X is defined as

$$\operatorname{Var}(X)=\mathbb{E}\big[(X-\mu)^2\big].$$

The quantity $(X-\mu)^2$ measures the squared deviation of X from its mean, and the expectation averages this deviation over all outcomes.

[4]We will study the methods to evaluate such infinite summations in Section 12.1.

The square root of the variance is called the *standard deviation*:

$$\sigma = \sqrt{\mathrm{Var}(X)}.$$

Often it is easier to compute variance using the identity

$$\mathrm{Var}(X) = E[X^2] - (E[X])^2.$$

This follows by expanding $(X - \mu)^2 = X^2 - 2\mu X + \mu^2$ and taking expectation.

10.9.4 The Bernoulli Distribution

Introduction

Many experiments in probability have only two possible outcomes — such as *success/failure*, *yes/no*, or *head/tail*. The simplest random variable that models such an experiment is called a *Bernoulli random variable*.

Definition

A random variable X is said to have a *Bernoulli distribution* with parameter p (where $0 \leq p \leq 1$) if

$$P(X = 1) = p \quad \text{and} \quad P(X = 0) = 1 - p.$$

We write this as

$$X \sim \mathrm{Bernoulli}(p).$$

Here $X = 1$ represents "success" and $X = 0$ represents "failure".

Probability Mass Function

The probability mass function (PMF) of X can be compactly expressed as

$$P(X = x) = p^x (1 - p)^{1-x}, \qquad x \in \{0, 1\}.$$

Expectation and Variance

The *expected value* (mean) and *variance* of a Bernoulli random variable are

$$E[X] = p, \qquad \mathrm{Var}(X) = p(1 - p).$$

Proof:

$$E[X] = (0)(1 - p) + (1)(p) = p,$$

and

$$E[X^2] = (0^2)(1 - p) + (1^2)(p) = p.$$

Therefore

$$\mathrm{Var}(X) = E[X^2] - (E[X])^2 = p - p^2 = p(1 - p).$$

Example

Consider a fair coin toss where success is "getting a head". Then $p = \frac{1}{2}$ and

$$P(X = 1) = P(X = 0) = \tfrac{1}{2}.$$

Hence $E[X] = \frac{1}{2}$ and $\mathrm{Var}(X) = \frac{1}{4}$.

Remarks

- The Bernoulli distribution is the simplest discrete distribution.
- The sum of n independent Bernoulli(p) random variables follows a *Binomial* distribution Binomial(n, p).
- Many random processes—such as coin tossing, system reliability, and binary classification—are modeled as Bernoulli trials.

10.9.5 Independence

Two events A and B are independent if

$$P(A \cap B) = P(A)P(B)$$

Two random variables X and Y are independent if for all x, y,

$$P(X = x, Y = y) = P(X = x)\, P(Y = y).$$

Example 10.24 *1. Consider two independent coin tosses. The sample space is* $S = \{hh, ht, th, tt\}$. *If* $P(h) = P(t) = 1/2$ *for each toss, then* $P(hh) = P(ht) = P(th) = P(tt) = 1/4$.

2. Let $X =$ *result of die 1,* $Y =$ *result of die 2. Then* X *and* Y *are independent:*

$$P(X = 4, Y = 2) = \frac{1}{6} \cdot \frac{1}{6}.$$

3. Roll two fair dice. Let

$$A = \{\textit{the first die shows a } 4\}, \qquad B = \{\textit{the second die shows an even number}\}.$$

Then

$$P(A) = \frac{1}{6}, \qquad P(B) = \frac{1}{2}.$$

Since the dice are independent,

$$P(A \cap B) = P(\textit{first die is } 4 \textit{ and second die is even}) = \frac{1}{6} \cdot \frac{1}{2} = \frac{1}{12}.$$

4. Select one card uniformly from a standard 52-card deck. Let

$$C = \{\textit{the card is red}\}, \qquad D = \{\textit{the card is a Heart}\}.$$

Then:

$$P(C) = \frac{26}{52} = \frac{1}{2}, \qquad P(D) = \frac{13}{52} = \frac{1}{4}.$$

However,

$$P(C \cap D) = P(\textit{the card is a red Heart}) = \frac{13}{52} = \frac{1}{4}.$$

But

$$P(C)\, P(D) = \frac{1}{2} \cdot \frac{1}{4} = \frac{1}{8}.$$

The events C *and* D *are* not *independent. Knowing that the card is a Heart* already implies *that it is red. So the occurrence of* D *changes the probability of* C.

10.9.6 Conditional Probability

In many situations, additional information is available about the outcome of an experiment. Such information can change the likelihood of events. The idea of *conditional probability* allows us to update probabilities when we learn that some event has already occurred.

If $P(B) > 0$, $P(A \mid B)$ is the probability of A given that B has occurred is given as

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}.$$

Example 10.25 *Let $A =$ "card is a King", $B =$ "card is a face card". Then*

$$P(A \mid B) = \frac{4/52}{12/52} = \frac{1}{3}.$$

Bayes' Theorem

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

More generally, if $\{A_1, \ldots, A_k\}$ partitions S,

$$P(A_i \mid B) = \frac{P(B \mid A_i)P(A_i)}{\sum_{j=1}^{k} P(B \mid A_j)P(A_j)}.$$

Example 10.26

1. *A disease has a probability $P(D) = 0.01$. A test has a probability $P(\textit{positive} \mid D) = 0.99$ and $P(\textit{positive} \mid \overline{D}) = 0.05$.*

$$P(D \mid \textit{positive}) = \frac{0.99 \cdot 0.01}{0.99 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.166.$$

2. *A coin was chosen at random and tossed. The probability that a fair coin was chosen and head shows is 1/3. The probability that a fair coin was chosen and tail shows is also 1/3. The probability that an unfair coin was chosen and head shows is 1/12. The probability that an unfair coin was chosen and head shows is 1/4.*

   Clearly, the probability that head shows is

$$\frac{1}{3} + \frac{1}{12} = \frac{5}{12}$$

   and the probability that an unfair coin was chosen is

$$\frac{1}{12} + \frac{1}{4} = \frac{1}{3}$$

   Thus, the conditional probability that an unfair coin was chosen given that head shows is

$$\frac{1/12}{5/12} = \frac{1}{5}$$

   and the conditional probability that head shows given an unfair coin was chosen is

$$\frac{1/12}{1/3} = \frac{1}{4}$$

3. *Three dice were rolled. Given that no two faces were the same, what is the probability that there was an ace? Let A denote the event that there was an ace, and B the event that no two faces were the same. Note that*

$$P(B) = \frac{P(6,3)}{6^3} \qquad P(A \cap B) = \frac{3P(5,2)}{6^3}$$

   Hence,

$$P(A \mid B) = \frac{3P(5,2)}{P(6,3)} = \frac{1}{2}$$

Problems

1. A department offers 6 foundation courses, 8 elective courses, and 3 project courses.
   (a) In how many ways can a student choose exactly one course of any type?
   (b) In how many ways can a student choose one foundation and one elective course?

2. A password consists of either:
   - any string of 4 lowercase letters, or
   - any string of 3 digits.

   How many possible passwords are there?

3. A student rolls a fair six-sided die until a 6 is obtained. How many different length-n outcome sequences are possible? (Hint: sequences end with a 6.)

4. How many permutations of the letters A, B, C, D, E begin with a consonant and end with a vowel?

5. In how many ways can 10 runners finish a race if ties are not allowed? How many ways if the top 3 places matter but the remaining order does not?

6. How many functions are there from an n-element set to a k-element set? How many of these functions are injective?

7. Show that a set of size n has 2^n subsets. (Hint: Use the binary string representation or apply the product rule.)

8. Prove that $\binom{n}{k} = \binom{n}{n-k}$ using
   (a) a bijection between k-subsets and $(n-k)$-subsets,
   (b) counting of bitstrings of length n with k ones.

9. A class of 40 students must form project teams of size 3. How many different teams are possible? How many different collections of *five disjoint* teams of size 3 can be formed?

10. In a group of 120 students,

    $$|A| = 50, \quad |B| = 60, \quad |C| = 40, \quad |A \cap B| = 20, \ |A \cap C| = 15, \ |B \cap C| = 18, \ |A \cap B \cap C| = 8.$$

    How many students take at least one of the three courses?

11. How many integers between 1 and 1000 are divisible by at least one of 2, 3, or 5?

12. Prove the general n-set inclusion–exclusion formula. (Hint: Count how many times each element is included/excluded.)

13. Let a_n be the number of binary strings of length n with no two consecutive 1s.
    (a) Derive a recurrence for a_n.
    (b) Compute a_1, a_2, a_3, a_4, a_5.
    (c) Show that the recurrence is the Fibonacci recurrence.

14. Let D_n be the number of derangements of n items. Using the argument from class, show that

    $$D_n = (n-1)(D_{n-1} + D_{n-2}).$$

15. Using the combinatorial argument "subsets that contain n and subsets that do not," prove Pascal's identity
$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

16. **Binomial theorem.** Expand $(x+y)^6$ and determine the coefficient of x^4y^2 and of x^2y^4. Verify using $\binom{6}{2} = \binom{6}{4}$.

17. Show that
$$\sum_{k=0}^{n} \binom{n}{k}^2 = \binom{2n}{n}.$$
(Hint: Interpret both sides combinatorially.)

18. Each student in a class of n students writes down the last digit of their phone number (an integer from 0 to 9, assumed equally likely).
    (a) What is the probability that all n students have *different* last digits? (Express the answer as a product.)
    (b) Evaluate this probability for $n = 5$ and for $n = 10$.
    (c) For what smallest n is this probability less than 0.5?

19. How many 5-card poker hands are possible? How many contain at most four aces?

20. How many subsets of 1,2,...,nhave an even number of elements? (Show your reasoning.)

21. Roll two fair dice.
    (a) Write down the sample space explicitly.
    (b) Compute the distribution of the sum.
    (c) Compute $P(\text{sum is even})$, $P(\text{sum is a prime})$, and $P(\text{sum} = 10)$.

22. Let X be the number of heads in 4 tosses of a fair coin. Compute $\mathbb{E}[X]$ (a) from the definition, (b) using linearity of expectation.

23. For the same random variable X, compute $\mathrm{Var}(X)$ using
$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2.$$

24. A fair die is rolled twice. Let X be the maximum of the two outcomes. Find the probability distribution of X and compute $\mathbb{E}[X]$.

25. Two fair dice are rolled. Let A be the event "sum is 7" and B be the event "first die is even." Are A and B independent? Justify carefully.

26. You pick a card from a well-shuffled deck. Let A be the event "the card is red," and B be "the card is a king." Are A and B independent?

27. In a class, 40% of students play basketball, 30% play football, and 15% play both. Compute $P(\text{basketball} \mid \text{football})$.

28. Three cards are drawn without replacement from a standard deck. Compute the probability that all three are face cards given that the first is a face card.

29. A spam-filtering system flags emails as "spam" or "not spam." Suppose:
$$P(\text{email is spam}) = 0.20,$$
$$P(\text{flagged as spam} \mid \text{spam}) = 0.95, \qquad P(\text{flagged as spam} \mid \text{not spam}) = 0.05.$$
If an incoming email is flagged as spam, compute the probability that it is actually spam.

30. Let $X \sim \text{Bernoulli}(p)$. Compute the distribution, expectation, and variance of

$$S_n = X_1 + X_2 + \cdots + X_n,$$

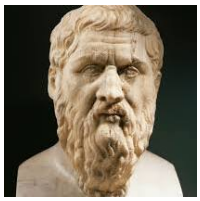
where the X_i are independent Bernoulli(p) random variables.

Chapter 11

Even infinity admits a hierarchy we must confront: The Real Numbers



"God created the integers; all else is the work of man" – Leopold Kronecker



"He is unworthy of the name of man who is ignorant of the fact that the diagonal of a square is incommensurable with its side" – Plato

The real numbers are the backbone of modern mathematics. They unify discrete and continuous quantities, allow us to measure lengths, areas, speeds, and forces, and they form the natural domain for calculus, physics, probability, and geometry. Yet the construction of the real numbers is not obvious, and their necessity arises from deep conceptual difficulties that puzzled mathematicians for millennia.

In this chapter we try to build a conceptual and computational foundation for understanding the real numbers, their structure, and the functions defined on them.

11.1 Why Do We Need Real Numbers?

As we saw earlier in these notes, mathematics begins with the natural numbers $\mathbb{N}$, which suffice for counting and simple arithmetic. But subtraction motivates the integers $\mathbb{Z}$, division motivates the rationals $\mathbb{Q}$, and geometry quickly reveals that even rational numbers are insufficient.

11.1.1 Incompleteness of the Rationals and the irrational persecution of Hippasus

In Theorem 5.20 we discussed a classical proof that shows that $\sqrt{2} \notin \mathbb{Q}$. This was shocking in antiquity and led to the development of more sophisticated number systems. Thus geometry forces us beyond the rationals.

> "He is unworthy of the name of man who is ignorant of the fact that the diagonal of a square is incommensurable with its side" – (Plato)

Hippasus is often credited with the discovery of irrational numbers. According to common folklore the Pythagoreans was rather upset with this discovery, and Hippasus is supposed to have drowned at sea, apparently as a punishment from the gods for divulging this troublesome fact.

There are more reason as to why rational are incomplete. Although $\mathbb{Q}$ is dense on the line, it contains "holes". A sequence of rational numbers may converge to a limit that is not rational.

Consider, for example,

$$y_n = \sum_{k=1}^{n} 10^{-k!}$$

Each y_n is rational, but the limit is the Liouville constant, which is not rational.

Theorem 11.1 *The Liouville constant*

$$L = \sum_{k=1}^{\infty} \frac{1}{10^{k!}} = \frac{1}{10} + \frac{1}{10^2} + \frac{1}{10^6} + \frac{1}{10^{24}} + \cdots$$

is irrational.

Proof: We will use the fact that a decimal number is rational if and only if its decimal expansion is *eventually periodic*; that is, after some point its digits repeat with a fixed period. We will prove this in a exercise problem at the end of this chapter.

Let us analyse the decimal expansion of L. The term $\frac{1}{10^{k!}}$ contributes a single '1' in the $k!$-th decimal place and zeros elsewhere. Thus L has a '1' in the decimal positions

$$1!,\ 2!,\ 3!,\ 4!,\ \ldots$$

and all other digits are 0. Therefore the distances between successive positions of '1' are

$$\begin{aligned} 2! - 1! &= 1 \\ 3! - 2! &= 4 \\ 4! - 3! &= 18 \\ 5! - 4! &= 96 \end{aligned}$$

and in general

$$(k+1)! - k! = k! \cdot k$$

These gaps grow without bound as $k \to \infty$.

Now suppose, for the sake of contradiction, that L is rational. Then its decimal expansion would be eventually periodic: there exist integers $N \geq 1$ and $p \geq 1$ such that all digits from position N onward repeat with period p. In particular, after position N, if a digit '1' occurs once, then the same digit '1' must occur again exactly p places later, and then again p places after that, and so on. Thus the distances between successive '1's beyond N must all be *exactly* p.

However, choose k large enough that $k! \cdot k > p$ and also $k! > N$. Then between the '1' at position $k!$ and the next '1' at position $(k+1)!$ there are

$$(k+1)! - k! = k! \cdot k$$

zeros — a gap strictly greater than p. This contradicts the assumption that the decimal expansion is eventually periodic with period p.

Hence no such period p can exist, and the decimal expansion of L is not eventually periodic. Therefore L is irrational. □

Hence, rhe rational numbers are not complete. Limits of rational sequences need not be rational. To properly define limits, infinite series, areas, lengths, and solutions of differential equations, we need a larger number system.

11.1.2 Zeno's paradox and infinite processes: Achilles and the Tortoise

Zeno of Elea (5th century BCE) presented several paradoxes to argue that motion is impossible. His most famous example is the paradox of *Achilles and the Tortoise*, which troubled mathematicians and philosophers for centuries. We now explain the paradox in detail and show how it naturally leads to the concept of limits and infinite series.

The Setup

Suppose Achilles, the fastest of the Greek heroes, runs ten times faster than a tortoise. The tortoise is given a head start of D metres. Common sense says Achilles will quickly overtake the tortoise. Zeno's reasoning, however, appeared to show otherwise.

Zeno's Argument

Zeno argued as follows:

1. Achilles first runs to the tortoise's starting point (distance D).
2. During that time, the tortoise moves ahead a little.
3. Achilles then runs to this new position.
4. Again the tortoise moves slightly ahead.
5. This continues forever, *ad infinitum*.

Thus Achilles must complete *infinitely many tasks*: first reach D, then $D(1+r)$, then $D(1+r+r^2)$, and so on, where r is the ratio of the tortoise's speed to Achilles' speed. Zeno concluded that Achilles can never overtake the tortoise.

Mathematical Formulation

Let v_A be Achilles' speed and v_T be the tortoise's speed, with $v_A > v_T$. Let the tortoise's head start be D.

The time Achilles takes to reach the tortoise's initial position is

$$t_1 = \frac{D}{v_A}.$$

During this time the tortoise moves

$$D_1 = v_T t_1 = D \cdot \frac{v_T}{v_A}.$$

Next, Achilles takes

$$t_2 = \frac{D_1}{v_A} = \frac{D}{v_A}\left(\frac{v_T}{v_A}\right)$$

to reach that point; during this period the tortoise moves

$$D_2 = D\left(\frac{v_T}{v_A}\right)^2.$$

Continuing, we obtain an infinite sequence of positive times

$$t_1,\ t_2,\ t_3,\ \ldots$$

with

$$t_n = \frac{D}{v_A}\left(\frac{v_T}{v_A}\right)^{n-1}.$$

Thus the total time required for Achilles to reach all points previously occupied by the tortoise is

$$T = t_1 + t_2 + t_3 + \cdots = \frac{D}{v_A}\left(1 + r + r^2 + r^3 + \cdots\right),$$

where

$$r = \frac{v_T}{v_A} \quad \text{and} \quad 0 < r < 1.$$

Resolution via convergent geometric series

We will show in Section 12.1 that the infinite geometric series with $0 < r < 1$ satisfies

$$1 + r + r^2 + r^3 + \cdots = \frac{1}{1-r}.$$

Hence

$$T = \frac{D}{v_A} \cdot \frac{1}{1-r} = \frac{D}{v_A - v_T}.$$

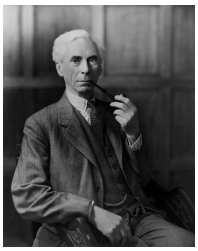
That is, the infinite sum actually converges to a finite quantity, a rational number if r is rational. This is the standard "relative speed" formula for the time taken for a faster runner to catch a slower one. The total time T is finite.

Achilles does indeed overtake the tortoise.

What Zeno misunderstood

Zeno implicitly assumed that a process involving infinitely many steps must take an infinite amount of time. Modern mathematics shows that an infinite series of positive quantities may converge to a finite sum. Here the quantities t_n become smaller so rapidly that their infinite sum remains finite.

This is only made rigorous in the *real numbers*, which allow us to define limits and convergent series. In this sense, Zeno's paradox hints at the need for a complete number system.



"The paradoxes of Zeno the Eleatic and the difficulties in the analysis of space, of time, and of motion, are all completely explained by means of the modern theory of continuity. This is because a non-contradictory theory has been found, according to which the continuum is composed of an infinity of distinct elements; and this formerly appeared impossible" – Bertrand Russell

Understanding the resolution of Zeno's paradox requires understanding limits, convergence, and the structure of the real numbers. The resolution uses *infinite series* and the idea of a *limit*. These require a complete number system — the real numbers. A follow up course on *Calculus* is strongly recommended.



"It is by the idea of limit alone that the calculus becomes a rigorous science." – A.–L. Cauchy, Cours d'Analyse (1821)



"The calculus was the first achievement of modern mathematics and it is difficult to overestimate its importance. I think it defines more unequivocally than anything else the inception of modern mathematics; and the system of mathematical analysis, which is its logical development, still constitutes the greatest technical advance in exact thinking." – John von Neumann

11.2 The Real Line as a Continuum

One way to view the real line is as the set of all (possibly infinite, non-repeating) decimals. Another is to view the real line as the completion of the rationals under limits of Cauchy sequences.

Definition 11.1 (Cauchy Sequence) *A sequence (x_n) of real or rational numbers is called a* Cauchy sequence *if for every $\varepsilon > 0$ there exists $N \in \mathbb{N}$ such that for all $m, n \geq N$,*

$$|x_m - x_n| < \varepsilon.$$

In other words, the terms of the sequence eventually become arbitrarily close to one another. It turns out that every Cauchy sequence converges in $\mathbb{R}$, but not necessarily in $\mathbb{Q}$, which shows that the rationals are not complete.

11.2.1 Cantor: The Reals are uncountable



"I see it, but I don't believe it!" – Georg Cantor

In 1874, Cantor used his celebrated *diagonal argument* to prove that no list can contain all real numbers in $(0, 1)$.

Theorem 11.2 (Cantor's Diagonal Argument) *The interval $(0, 1)$ is uncountable. In particular, there is no bijection between the natural numbers $\mathbb{N}$ and the real numbers in $(0, 1)$.*

Proof: Suppose, for the sake of contradiction, that $(0, 1)$ is countable. Then every real number in $(0, 1)$ has a decimal expansion of the form

$$0.a_1a_2a_3\ldots,$$

and our assumption implies that it is possible to list *all* such numbers in a single infinite sequence. Thus suppose we have an enumeration

$$x_1, x_2, x_3, \dots$$

of all real numbers in $(0, 1)$, where each x_n is written in decimal form as

$$\begin{aligned} x_1 &= 0.a_{11}a_{12}a_{13}a_{14}\dots \\ x_2 &= 0.a_{21}a_{22}a_{23}a_{24}\dots \\ &\vdots \\ x_n &= 0.a_{n1}a_{n2}a_{n3}a_{n4}\dots \\ &\vdots \end{aligned}$$

Here $a_{nk} \in \{0, 1, 2, \dots, 9\}$ is the k-th digit of the n-th number in the list.

Important convention. To avoid the ambiguity of decimal expansions such as $0.2499999\ldots = 0.25$, we assume that no number in the list ends in an infinite tail of 9's. Every real number in $(0, 1)$ has at least one representation without trailing 9's, so this assumption does not omit any real number.

Now we construct a new number y whose decimal digits differ from the diagonal digits of the list. Define $y = 0.b_1b_2b_3\dots$ by

$$b_k = \begin{cases} 5, & \text{if } a_{kk} \neq 5, \\ 4, & \text{if } a_{kk} = 5. \end{cases}$$

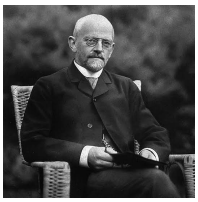
In other words, b_k is chosen so that it *differs* from a_{kk}. This is always possible because there are ten possible digits, and we deliberately avoid choosing the digit 9, so y will not end in an infinite string of 9's.

Consider any index $n \in \mathbb{N}$. We compare y with the n-th number x_n in the list. The n-th digit of x_n is a_{nn}, while the n-th digit of y is b_n, which was chosen to be *different* from a_{nn}. Therefore,

$$y \neq x_n.$$

Since this argument holds for every n, we conclude that the constructed number y is different from x_n for all $n \in \mathbb{N}$. But this contradicts our assumption that $(0, 1)$ was completely enumerated by the sequence $x_1, x_2, x_3, \dots$.

Thus our original assumption was false. No such complete enumeration exists, and therefore $(0, 1)$ is uncountable. □



"No one shall expel us from the paradise that Cantor has created." – David Hilbert

The result was revolutionary and laid the foundations of modern set theory. Quite amazingly, *diagonalization* as a proof technique has also turned out to be foundational in logic and computer science.

11.3 Algebraic and Transcendental Numbers

The irrationals split into two classes.

A real number x is *algebraic* if it is a root of (a solution of) an equation like

$$a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0 = 0$$

with rational coefficients $a_0, a_1, \ldots, a_n$. Expressions such as the left hand side of the above equation are called *polynomials* and the solutions are called its *roots*.

Some examples are

$$\sqrt{2}, \qquad \sqrt[3]{5}, \qquad \frac{1+\sqrt{5}}{2}$$

If a number is not algebraic, it is *transcendental*. Some famous examples are e and π which will soon discuss.

Liouville (1844) constructed numbers that are "too well approximated" by rationals:

$$L = \sum_{k=1}^{\infty} 10^{-k!}$$

Liouville proved that L is transcendental.

> "Transcendental numbers are not rare; they are, in fact, the rule." — *Kurt Mahler*

11.3.1 How fast do our money grow? Simple and compound interests

When we invest in a financial instrument our money earns *interest* and grows.

Simple interest assumes that interest is calculated only on the original principal, and that previously earned interest does not itself earn further interest.

Let

$$P = \text{principal (initial amount)}, \qquad r = \text{annual interest rate}, \qquad t = \text{time in years}.$$

Under simple interest, the total amount after t years is

$$A(t) = P(1 + rt).$$

This is a linear function of t. The amount increases at a constant rate Pr per year, so the graph of $A(t)$ against t is a straight line.

Example 11.1 *Suppose you invest*

$$P = ₹10{,}000$$

at an annual simple interest rate of $r = 0.08$ *(i.e. 8% per year). Then*

After 1 year:	$A(1) = 10000(1 + 0.08) = 10800$
After 3 years:	$A(3) = 10000(1 + 0.08 \times 3) = 10000(1 + 0.24) = 12400$
After 10 years:	$A(10) = 10000(1 + 0.08 \times 10) = 10000(1 + 0.80) = 18000$

In each year, the amount increases by the same amount:

$$Pr = 10000 \times 0.08 = ₹800$$

Under *compound interest*, interest is added periodically and in each period the previously earned interest itself earns further interest. If P is the principal, r is the annual interest rate, and interest is compounded once per year, then the amount after t years is

$$A_c(t) = P(1 + r)^t.$$

Example 11.2 *For the same values as in the simple–interest example,*

$$P = 10{,}000, \qquad r = 0.08,$$

we obtain

$$A_c(t) = 10000(1.08)^t$$

Thus,

After 1 year:	$A(1) = 10000(1.08) = 10800$
After 3 years:	$A(3) = 10000(1.08)^3 = 12597.12$
After 10 years:	$A(10) = 10000(1.08)^10 = 21589.25$

While simple interest grows linearly, compound interest grows exponentially. Over long durations the difference becomes significant, and compounded growth eventually "pulls away" from linear growth.

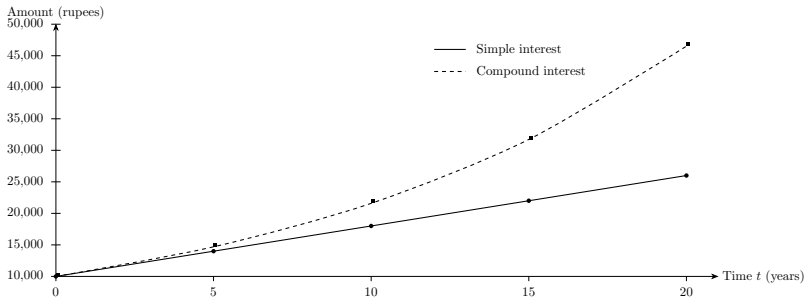


Figure 11.1: Simple vs annually compounded growth at 8% for $P = 10{,}000$ over 20 years. Vertical scale: $y = (A(t) - 10000)/5000$.

As n increases, this expression approaches the continuously compounded growth

$$A(t) = Pe^{rt},$$

revealing the natural appearance of the constant e. Thus the transition from simple to compound interest provides a natural pathway from linear to exponential growth.

11.3.2 From compound interest to the number e

Compound interest provides a natural setting in which the exponential number e appears. Suppose a principal amount P is invested at an annual interest rate r, but interest is now credited not once a year, but n times per year. In each compounding period the interest rate is r/n, so the amount after one year becomes

$$A_n(1) = P\left(1 + \frac{r}{n}\right)^n.$$

For example:

- with $n = 1$ (annual compounding), we obtain $P(1 + r)$,
- with $n = 4$ (quarterly compounding), we obtain $P\left(1 + \frac{r}{4}\right)^4$,
- with $n = 12$ (monthly compounding), we obtain $P\left(1 + \frac{r}{12}\right)^{12}$,
- with $n = 365$ (daily compounding), we obtain $P\left(1 + \frac{r}{365}\right)^{365}$.

As n increases, interest is credited more and more frequently, and the amount grows slightly larger. A natural mathematical question is:

$$\textit{What happens as } n \to \infty?$$

To study this limit, let us first focus on the core expression

$$\left(1+\frac{1}{n}\right)^n.$$

One can check numerically that

$$\left(1+\frac{1}{n}\right)^n = 2.0000,\ 2.2500,\ 2.3707,\ 2.3979,\ 2.7048,\ \ldots$$

for $n = 1, 2, 3, 4, 10, \ldots$, and that the sequence increases and approaches a limiting value. This limit is denoted by the symbol e[1].

$$e = \lim_{n\to\infty}\left(1+\frac{1}{n}\right)^n \approx 2.71828$$

The limit can also be written as an infinite sum. The full Binomial expansion becomes

$$\left(1+\frac{1}{n}\right)^n = 1+1+\frac{1}{2}\left(1-\frac{1}{n}\right)+\frac{1}{6}\left(1-\frac{1}{n}\right)\left(1-\frac{2}{n}\right)+\cdots.$$

As $n\to\infty$, $\frac{1}{n}\to 0$, Hence,

$$e = 1+1+\frac{1}{2!}+\frac{1}{3!}+\frac{1}{4!}+\cdots$$

Thus, continuous compounding at rate r leads to the amount

$$A(1) = Pe^r$$

In general, after t years the continuously compounded amount becomes[2]

$$A(t) = Pe^{rt}.$$

This is the fundamental exponential-growth formula. The appearance of e is therefore not mysterious: it arises naturally as the limiting value of compound interest when interest is credited infinitely often.

> *"The number e is the unique real number for which continuous growth at rate 1 yields a unit increase in one time unit."*

This interpretation makes e one of the most important constants in mathematics, appearing in finance, probability theory, calculus, differential equations, and many dynamical processes in the natural and social sciences.

11.3.3 The Number π from Geometry

All circles are similar: any circle can be obtained from any other by a uniform scaling. Under scaling by a factor $k > 0$, every length is multiplied by k. Hence circumference and diameter scale by the same factor, so their ratio is unchanged.

Definition 11.2 *For every circle,*

$$\pi = \frac{C}{D} = \frac{C}{2r}.$$

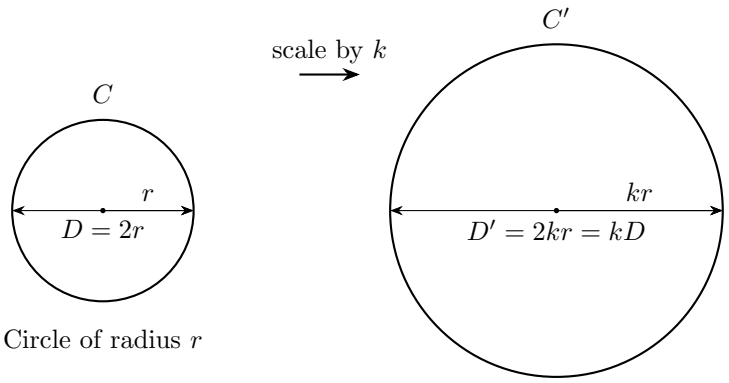
For a disk of radius r, the area is given as

$$A(r) = \pi r^2.$$

Thus $\pi = A(1)$: the area of the unit disk.

[1] Estimating its value will require some advanced techniques which an interested reader may look up.

[2] Jacob Bernoulli discovered this limit while studying compound interest in 1683. Leonhard Euler later linked it to calculus and defined it analytically.



$$C' = kC, \quad D' = kD \quad \Rightarrow \quad \frac{C'}{D'} = \frac{C}{D}.$$

Figure 11.2: Scaling a circle by factor k multiplies both circumference and diameter by k, so the ratio C/D is the same for all circles.

Approximations of π

Archimedes approximated π using inscribed and circumscribed n-gons. Using a 96-gon, he showed

$$3\frac{10}{71} < \pi < 3\frac{1}{7}.$$

The Leibniz formula for π obtained using a geometric series approximation and integration is given as

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$$

This series converges slowly but gives an exact expression. This series is also sometimes called the Madhava-Leibniz series as it was first discovered by Madhava of Sangamagrama[3]. It can be shown that π is a transcendental number.

Problems

1. (**Incompleteness of the rationals**) Give an example of a Cauchy sequence of rational numbers that does *not* converge to any rational number. Explain why this demonstrates a "gap" in $\mathbb{Q}$.

2. (**Achilles and the Tortoise**) In the Achilles–Tortoise setup where Achilles runs at twice the speed of the tortoise, write the explicit geometric series representing Achilles' motion. Compute its sum and argue why the paradox dissolves once infinite sums are allowed.

3. (**Density**) Prove that between any two irrational numbers there exists a rational number. (Hint: scale and shift.)

4. (**Decimal expansions**) Give an example of a non-terminating repeating decimal and a non-repeating decimal. Classify each as rational or irrational and justify your answer.

5. (**Cantor's diagonal argument**) Explain why the diagonal method does not contradict the fact that decimal representations have non-uniqueness (e.g. $0.4999\ldots = 0.5$). Why does the argument still go through?

[3]See https://en.wikipedia.org/wiki/Approximations_of_π for a history of various approximations of π.

6. (**Countability vs uncountability**) Show that the set of all infinite binary sequences is uncountable by giving a diagonal proof analogous to Cantor's proof for $(0, 1)$.

7. (**Algebraic vs transcendental**) Prove that the number

$$L = \sum_{k=1}^{\infty} 10^{-k!}$$

is irrational. (Hint: compare L with rational numbers truncated after $n!$ decimal places.)

8. (**From compound interest to** e) Compute $\left(1 + \frac{1}{n}\right)^n$ for $n = 1, 2, 5, 10, 100, 1000$. Does the sequence appear to converge? How does this motivate the definition of e?

9. (e **as a limit**) Show that the sequence

$$a_n = \left(1 + \frac{1}{n}\right)^{n+1}$$

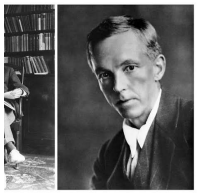
is bounded above by e. (Hint: use monotonicity and the binomial expansion.)

10. (**Archimedes' method**) Using a circumscribed n-gon around a unit circle, derive an upper bound on π. Compute the bound explicitly for $n = 6$ and $n = 12$.

11. (**Transcendentals exist "in abundance"**) Show that the set of algebraic numbers is countable. Conclude that "almost all" real numbers are transcendental.

Chapter 12

The Real Toolkit: Sequences, Functions, and Equations



"A mathematician, like a painter or a poet, is a maker of patterns" – G. H. Hardy

The real numbers are more than a continuum filling the gaps left by the rationals—they form a versatile algebraic toolkit for describing change, predicting patterns, and solving practical problems. In this chapter we develop the basic algebra of the reals as it appears in everyday mathematical modelling: from arithmetic and geometric progressions; from polynomials and their remarkable power of unique interpolation to the curve-fitting techniques that underlie modern data analysis; and from real-valued functions to systems of linear equations that allow us to solve for unknowns in structured ways. Together, these ideas illustrate how the algebra of the real numbers becomes a unifying language for patterns, growth, and relationships in both mathematics and the world around us.

12.1 Arithmetic and geometric progressions

Arithmetic and geometric progressions are two of the simplest infinite families of sequences, and they sit at the gateway between discrete sums and the idea of limits. They also provide the first nontrivial examples of closed-form formulas for sums.

12.1.1 Arithmetic progressions (AP)

Definition 12.1 *A sequence (a_n) is an* arithmetic progression *if the difference between consecutive terms is constant. That is, there exists a real number d such that*

$$a_{n+1} - a_n = d \quad \textit{for all } n \geq 1.$$

The number d is called the common difference.

Hence an AP has the form

$$a,\ a+d,\ a+2d,\ \ldots,\ a+(n-1)d,\ \ldots$$

and the n-th term is

$$a_n = a + (n-1)d.$$

Sum of the first n terms: a double-counting derivation

Let

$$S_n = a + (a+d) + (a+2d) + \cdots + (a+(n-1)d)$$

be the sum of the first n terms.

Write the same sum in reverse order:

$$S_n = (a+(n-1)d) + (a+(n-2)d) + \cdots + (a+d) + a.$$

Now add these two expressions term-by-term. Each pair adds to the same value[1]:

$$\big(a+(k-1)d\big) + \big(a+(n-k)d\big) = 2a + (n-1)d.$$

There are n such pairs, so

$$2S_n = n\big(2a + (n-1)d\big).$$

Therefore,

$$\boxed{S_n = \frac{n}{2}\big(2a + (n-1)d\big) = \frac{n}{2}\,(a_1 + a_n).}$$

Example 12.1 (Sum of the first n natural numbers) *Here $a = 1$ and $d = 1$, so*

$$S_n = \frac{n}{2}(2 + (n-1)) = \frac{n(n+1)}{2}.$$

Example 12.2 (Sum of the first n odd numbers) *The odd numbers form an AP:*

$$1, 3, 5, \ldots, (2n-1),$$

with $a = 1$, $d = 2$. Thus

$$S_n = \frac{n}{2}\big(2 \cdot 1 + (n-1) \cdot 2\big) = \frac{n}{2}(2n) = n^2.$$

So the sum of the first n odd numbers is n^2.

12.1.2 Geometric progressions (GP)

Definition 12.2 *A sequence (g_n) is a* geometric progression *if the ratio between consecutive terms is constant. That is, there exists a real number r such that*

$$\frac{g_{n+1}}{g_n} = r \quad \textit{for all } n \geq 1,$$

provided $g_n \neq 0$. The number r is called the common ratio.

A GP has the form

$$a,\ ar,\ ar^2,\ \ldots,\ ar^{n-1},\ \ldots$$

and the n-th term is

$$g_n = ar^{n-1}.$$

[1] This "pairing" trick is often attributed to the young Gauss, who is said to have summed $1 + 2 + \cdots + 100$ instantly by noticing that $1 + 100 = 2 + 99 = \cdots = 101$.

Sum of the first n terms

Let

$$S_n = a + ar + ar^2 + \cdots + ar^{n-1}.$$

Multiply both sides by r:

$$rS_n = ar + ar^2 + \cdots + ar^n.$$

Subtract:

$$S_n - rS_n = a - ar^n.$$

Hence

$$(1-r)S_n = a(1-r^n),$$

and therefore, if $r \neq 1$,

$$\boxed{S_n = a\,\frac{1-r^n}{1-r}.}$$

When $r = 1$, the GP is constant and $S_n = na$.

Infinite geometric series

If $|r| < 1$, then $r^n \to 0$ as $n \to \infty$, so

$$S_\infty = \lim_{n\to\infty} S_n = \lim_{n\to\infty} a\,\frac{1-r^n}{1-r} = \frac{a}{1-r}.$$

Thus

$$\boxed{a + ar + ar^2 + \cdots = \frac{a}{1-r} \quad (|r| < 1).}$$

This is one of the earliest places where limits naturally enter: an infinite sum makes sense only because the partial sums converge.

Example 12.3 (Zeno's geometric series)

$$\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \cdots$$

is a GP with $a = \frac{1}{2}$ *and* $r = \frac{1}{2}$*, so*

$$S_\infty = \frac{1/2}{1 - 1/2} = 1.$$

This is exactly the mathematical resolution of Zeno's paradox.

Example 12.4 (A repeating decimal as a GP)

$$0.\overline{3} = 0.3 + 0.03 + 0.003 + \cdots$$

is a GP with $a = 0.3$ *and* $r = 0.1$*. Hence*

$$0.\overline{3} = \frac{0.3}{1 - 0.1} = \frac{0.3}{0.9} = \frac{1}{3}.$$

Remarks

- AP sums grow *quadratically* in n when $d \neq 0$, reflecting linear increments.
- GP sums grow *exponentially* in n when $|r| > 1$, reflecting multiplicative change.
- The infinite GP formula is the first rigorous example of a convergent infinite series, and is a key bridge to calculus.

12.2 Polynomials

Polynomials are among the most fundamental objects in algebra. They form a rich class of functions that are easy to compute with, easy to approximate, and powerful enough to model a wide variety of phenomena. In this section we introduce the basic theory of polynomials, discuss the unique interpolation theorem, and show how polynomials naturally arise in curve fitting and other applications.

12.2.1 Definition and degree

A *polynomial* over the real numbers is a function of the form

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n,$$

where $a_0, a_1, \ldots, a_n \in \mathbb{R}$ and n is a nonnegative integer. The number n is the *degree* of the polynomial (provided $a_n \neq 0$).

- A polynomial of degree 0 is a constant function.
- A polynomial of degree 1 is a linear function. It gives the equation of a straight line.
- A polynomial of degree 2 is a quadratic function.
- A polynomial of degree 3 is a cubic function, and so on.

The set of all polynomials with real coefficients is denoted by $\mathbb{R}[x]$. Polynomials are defined everywhere on $\mathbb{R}$. They also satisfy the useful property that if

$$P(x) = 0 \quad \text{for infinitely many distinct } x \in \mathbb{R},$$

then P must be the zero polynomial. This is called the *identity theorem for polynomials.*

12.2.2 Basic algebraic properties

Polynomials can be added and multiplied term-by-term, following the same rules as with ordinary algebraic expressions. Here we illustrate both operations with simple examples.

$$\deg(P+Q) \leq \max(\deg P, \deg Q), \qquad \deg(PQ) = \deg P + \deg Q,$$

unless cancellation occurs in the leading terms.

Polynomial addition

Let

$$P(x) = 3x^3 - 2x + 5, \qquad Q(x) = 4x^2 + x - 1.$$

To compute $P(x) + Q(x)$, we combine like terms:

$$\begin{aligned} P(x) + Q(x) &= (3x^3) + (4x^2) + (-2x + x) + (5 - 1) \\ &= 3x^3 + 4x^2 - x + 4. \end{aligned}$$

Thus

$$\boxed{P(x) + Q(x) = 3x^3 + 4x^2 - x + 4.}$$

Polynomial multiplication

Let

$$A(x) = 2x^2 - 3x + 1, \qquad B(x) = x - 4.$$

Multiplying out (using distributivity):

$$\begin{aligned} A(x)B(x) &= (2x^2)(x) + (2x^2)(-4) + (-3x)(x) + (-3x)(-4) + (1)(x) + (1)(-4) \\ &= 2x^3 - 8x^2 - 3x^2 + 12x + x - 4. \end{aligned}$$

Now combine like terms:

$$A(x)B(x) = 2x^3 - 11x^2 + 13x - 4.$$

So

$$\boxed{A(x)B(x) = 2x^3 - 11x^2 + 13x - 4.}$$

12.2.3 The decimal system as a polynomial representation

Every finite decimal expansion

$$d_k d_{k-1} \cdots d_1 d_0$$

represents a number that can be interpreted as a polynomial in 10:

$$d_k 10^k + d_{k-1} 10^{k-1} + \cdots + d_1 10 + d_0.$$

In this sense, the decimal numeral system is simply a *polynomial in the base* 10.

Example 12.5 *1. A whole number:*

$$374 = 3 \cdot 10^2 + 7 \cdot 10^1 + 4 \cdot 10^0.$$

2. A larger number:

$$5{,}182 = 5 \cdot 10^3 + 1 \cdot 10^2 + 8 \cdot 10 + 2.$$

3. A decimal (fractional) number: For digits after the decimal point, we use negative powers of 10:

$$46.327 = 4 \cdot 10^1 + 6 \cdot 10^0 + 3 \cdot 10^{-1} + 2 \cdot 10^{-2} + 7 \cdot 10^{-3}.$$

Thus every decimal numeral is nothing more than a polynomial expression in the base 10 with coefficients drawn from $\{0, 1, \ldots, 9\}$. This viewpoint is fundamental in computer science (base 2), engineering (base 16), and number theory (general base-b expansions).

Exercise 12.1 *Explain the junior school algorithm for integer multiplication in terms of polynomial multiplications. Explain how is this repeated addition?*

12.2.4 The unique interpolation theorem

One of the most remarkable facts about polynomials is that they can be used to fit any finite collection of data points with a unique polynomial of sufficiently small degree.

Theorem 12.1 (Unique Interpolating Polynomial) *Let $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$ be $n+1$ data points with*

$$x_i \neq x_j \quad \text{whenever } i \neq j.$$

Then there exists a polynomial $P(x)$ of degree at most n such that

$$P(x_i) = y_i \quad \text{for all } i = 0, 1, \ldots, n,$$

and this polynomial is unique.

Proof: We prove existence and uniqueness.

Existence (Lagrange construction)

For each $k \in \{0, 1, \dots, n\}$ define the *Lagrange basis polynomial*

$$L_k(x) = \prod_{\substack{0 \le j \le n \\ j \ne k}} \frac{x - x_j}{x_k - x_j}.$$

This is well-defined because $x_k - x_j \neq 0$ for $j \neq k$.

Claim 1. For all $i, k \in \{0, \dots, n\}$,

$$L_k(x_i) = \delta_{ik} = \begin{cases} 1, & i = k, \\ 0, & i \neq k. \end{cases}$$

Proof of Claim 1. If $i = k$, then every factor in the product equals

$$\frac{x_k - x_j}{x_k - x_j} = 1,$$

so $L_k(x_k) = 1$. If $i \neq k$, then the product defining L_k contains the factor with $j = i$, namely

$$\frac{x_i - x_i}{x_k - x_i} = 0,$$

so $L_k(x_i) = 0$. This proves the claim. □

Now define

$$P(x) = \sum_{k=0}^{n} y_k \, L_k(x).$$

Each L_k is a polynomial of degree n (a product of n linear factors), so P is a polynomial of degree at most n.

Evaluate P at each interpolation point x_i:

$$P(x_i) = \sum_{k=0}^{n} y_k \, L_k(x_i) = \sum_{k=0}^{n} y_k \, \delta_{ik} = y_i.$$

Hence P interpolates all the given data. Existence is proved.

Uniqueness

We need one standard fact.

Lemma. A nonzero polynomial of degree at most n can have at most n distinct real roots.
Proof of Lemma. We proceed by induction on n. For $n = 0$, a nonzero constant polynomial has no roots. Assume the statement true for degree at most $n - 1$. Let Q be a nonzero polynomial of degree at most n with a root a. Then $(x - a)$ divides $Q(x)$, so

$$Q(x) = (x - a)R(x)$$

for some polynomial R of degree at most $n - 1$. Any root of Q different from a must be a root of R, and by the induction hypothesis R has at most $n - 1$ distinct roots. Thus Q has at most n distinct roots in total. □

Now suppose P and Q are two polynomials of degree at most n both interpolating the data:

$$P(x_i) = Q(x_i) = y_i \quad \text{for all } i = 0, \dots, n.$$

Consider their difference

$$R(x) = P(x) - Q(x).$$

Then $\deg R \leq n$, and

$$R(x_i) = P(x_i) - Q(x_i) = 0 \quad \text{for all } i = 0, \ldots, n.$$

So R has at least $n+1$ distinct roots. By the lemma, the only polynomial of degree at most n that can have $n+1$ roots is the zero polynomial. Hence $R \equiv 0$, i.e.

$$P(x) = Q(x) \quad \text{for all } x.$$

Therefore the interpolating polynomial is unique. □

A simple example

Given

$$(0, 1), \qquad (1, 2), \qquad (2, 5),$$

there is a unique quadratic polynomial through these points. Using the Lagrange formula,

$$P(x) = 1 \cdot \frac{(x-1)(x-2)}{(0-1)(0-2)} + 2 \cdot \frac{(x-0)(x-2)}{(1-0)(1-2)} + 5 \cdot \frac{(x-0)(x-1)}{(2-0)(2-1)}.$$

Simplifying yields

$$P(x) = x^2 + 1.$$

12.2.5 Polynomials and curve fitting

Interpolation provides an exact polynomial passing through given data, but we often only have noisy measurements or more data points than we want the degree of the polynomial to match. In practice, measured data points rarely lie exactly on a straight line, a quadratic curve, or any simple mathematical model. We therefore need a way to choose a "best" curve that captures the overall trend of the data, without requiring a perfect fit to every point. The most widely used approach is the *method of least squares*, which selects the curve whose predicted values come, on average, as close as possible to the observations.

The idea is simple to state: among all candidate lines or curves, choose the one that *minimises the total squared error*

$$\sum_{i=1}^{n} (y_i - P(x_i))^2,$$

where $P(x)$ is the model we want to fit. Squaring the errors penalises large deviations more strongly and leads to a unique, stable solution. Although the derivation is not required here, the resulting formulas give clean, explicit expressions for the best-fitting straight line or quadratic polynomial.

Historically, the method was developed by Carl Friedrich Gauss in the early 19th century in his work on determining planetary orbits from noisy astronomical measurements. Gauss famously used least squares to recover the orbit of the asteroid *Ceres* in 1801, demonstrating that the best-fitting curve—in the least-squares sense—provides the most reliable prediction from imperfect data. Today the same principle forms the basis of regression, statistical modelling, and modern machine learning.

We will not describe his methods in these notes and leave it for a future course. In what follows we give some examples to illustrate the use of least squares approximation.

Example 12.6 (Linear Fit Example (Economics): Wage vs. Experience) *A common empirical observation in labour economics is that, over an early career window, wages rise approximately linearly with experience. Suppose we measure monthly wage W (in thousand rupees) for workers with different years of experience x:*

Experience x (years)	*0*	*2*	*4*	*6*	*8*	*10*
Wage W (₹'000/month)	*30*	*38*	*46*	*55*	*63*	*70*

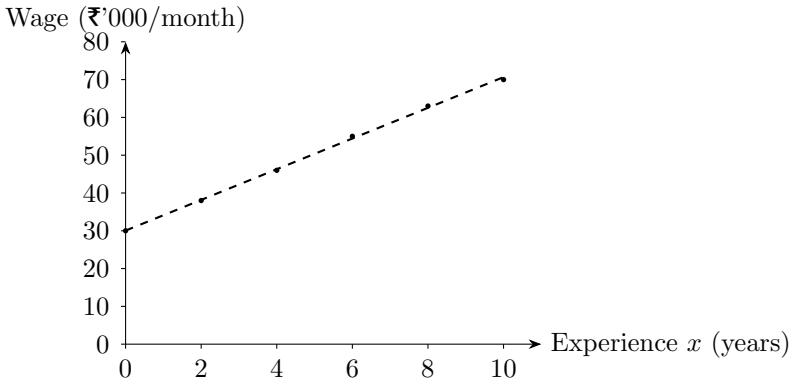


Figure 12.1: Wage vs. experience.

We model this by a straight line

$$W(x) \approx mx + c.$$

A least–squares calculation gives (See Figure 12.1)

$$m \approx 4.06, \qquad c \approx 30.05,$$

so the fitted wage equation is

$$\boxed{W(x) \approx 4.06x + 30.05.}$$

Interpretation. *The slope means that each additional year of experience raises expected wage by about ₹4,060 per month in this range. The intercept estimates a starting wage around ₹30,000 per month.*

Example 12.7 (Quadratic Fit Example (Economics): Profit vs. Output) *In many production settings, profits rise with output at first, but beyond some point diminishing returns and rising costs reduce profits. This often produces a concave (downward-opening) quadratic relationship.*

Suppose a firm varies daily output q (in hundreds of units) and observes profit Π (in thousand rupees):

Output q (hundreds)	*0*	*1*	*2*	*3*	*4*	*5*	*6*
Profit Π (₹'000)	*0*	*8*	*14*	*18*	*20*	*19*	*15*

We fit a quadratic model

$$\Pi(q) \approx aq^2 + bq + c, \qquad a < 0.$$

Least squares gives (See Figure 12.2)

$$a \approx -1.18, \qquad b \approx 9.68, \qquad c \approx -0.29,$$

so

$$\boxed{\Pi(q) \approx -1.18q^2 + 9.68q - 0.29.}$$

Interpretation. *The negative coefficient a reflects diminishing returns. The profit-maximising output is near the vertex:*

$$q^* = -\frac{b}{2a} \approx \frac{9.68}{2.36} \approx 4.1,$$

i.e., roughly 4.1 hundred units per day.

Polynomials offer some of the simplest yet most expressive "laws of variation" in mathematics, and they occur in many different contexts.

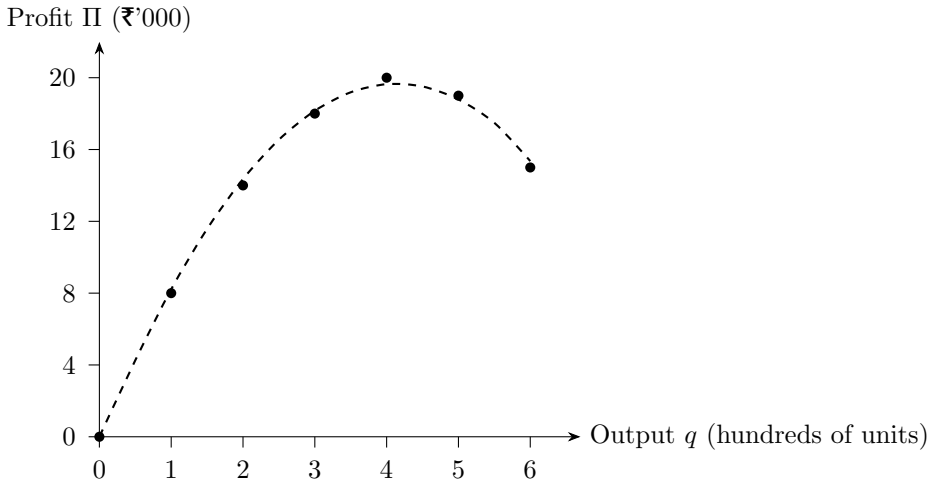


Figure 12.2: Profit vs. output showing a concave quadratic least-squares fit.

12.3 Systems of linear equations

A system of linear equations is a collection of m equations in n unknowns $x_1, x_2, \ldots, x_n$ of the form

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{in}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ \vdots \qquad\qquad\qquad\qquad\quad &\;\;\vdots \;\; \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned}$$

where the a_{ij} and b_i are known real numbers and the x_j are the unknowns. In general, if the equations are independent (none can be obtained as a combination of the others) and $m = n$ (as many equations as unknowns), then there is a unique solution for the x_is.

Linear systems are among the oldest and most fundamental objects in algebra: they form the basis of modern numerical computation, optimization, statistics, and scientific modelling. Despite their simple appearance, linear systems encode a remarkable variety of real-world relationships.

12.3.1 Why linear equations matter

Even when the underlying model is complicated or nonlinear, it is often approximated by a linear system. This process, called *linearisation*, relies on the fact that over sufficiently small ranges many functions behave almost linearly. Because linear equations can be solved efficiently, stably, and exactly by well-understood methods, they serve as the computational workhorse of applied mathematics.

Linear systems also arise naturally whenever several quantities influence one another additively. In economics, physics, and engineering, additive effects are frequently easier to measure or estimate, so linear models become the default starting point for analysis. In higher dimensions, linear systems form the algebraic language of vectors, matrices, and multivariable functions which an interested reader may study in a later course.

Example 12.8 (A simple example puzzle) *In a lab there are two kinds of metal blocks, type A and type B. Let x be the weight (in kg) of one A-block and y the weight of one B-block.*

- *Three A-blocks and two B-blocks weigh* 18 *kg.*
- *Two A-blocks and five B-blocks weigh* 24 *kg.*

Question: *Find x and y.*

Solution. *The statements translate directly into the linear system*

$$3x + 2y = 18,$$
$$2x + 5y = 24.$$

Multiply the first equation by 5 *and the second by* 2*:*

$$15x + 10y = 90,$$
$$4x + 10y = 48.$$

Subtracting gives

$$11x = 42 \quad \Rightarrow \quad x = \frac{42}{11}.$$

Substitute into $3x + 2y = 18$*:*

$$3 \cdot \frac{42}{11} + 2y = 18 \quad \Rightarrow \quad \frac{126}{11} + 2y = \frac{198}{11} \quad \Rightarrow \quad 2y = \frac{72}{11} \quad \Rightarrow \quad y = \frac{36}{11}.$$

$$\boxed{x = \frac{42}{11} \approx 3.82 \ kg, \qquad y = \frac{36}{11} \approx 3.27 \ kg.}$$

This illustrates a key point: linear systems arise whenever several unknown quantities contribute additively to measured outcomes. The same algebraic tool solves problems in mixtures, balances, circuits, flows, and data fitting.

12.3.2 Some real-world examples of linear systems

1. Economics: input–output models. In Leontief's input–output theory, each industry requires fixed proportions of output from other industries. If x_j is the total output of industry j, linear balance equations

$$x_i = a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n + d_i$$

relate production to inter-industry consumption and final demand d_i. This leads directly to a system of linear equations solved for $x_1, \ldots, x_n$.

2. Electronics: nodal analysis in circuits. Using Kirchhoff's current laws, the voltages $V_1, \ldots, V_n$ at electrical nodes satisfy

$$\sum_j G_{ij}(V_i - V_j) = I_i,$$

where G_{ij} are conductances and I_i are source currents. The result is a linear system $GV = I$, solved billions of times per second inside circuit simulators.

3. Data analysis: linear regression. A straight-line fit $y = mx + c$ to data (x_i, y_i) leads to the normal equations

$$\begin{pmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{pmatrix} \begin{pmatrix} m \\ c \end{pmatrix} = \begin{pmatrix} \sum x_i y_i \\ \sum y_i \end{pmatrix},$$

a 2×2 linear system whose solution yields the least-squares fit.

4. Physics and engineering: equilibrium conditions. Static equilibrium of forces in structures—bridges, frames, trusses—is governed by equations of the form

$$\sum F_x = 0, \qquad \sum F_y = 0,$$

which, when decomposed by components, give linear relations between unknown tensions and forces.

Summary

Systems of linear equations are central because:

- they model many real-world relationships directly,
- they provide the local approximation to nonlinear systems,
- they admit efficient and reliable solution methods,
- and they unify ideas across economics, science, engineering, and data analysis.

They form the algebraic backbone of the "real toolkit" of functions, approximations, and computational methods.

12.4 Some fascinating real-valued functions

Real-valued functions are the language in which we describe patterns, growth, decay, saturation, symmetry, oscillation, and randomness. In this section we collect a few of the most important and beautiful functions that appear repeatedly across mathematics, the sciences, economics, and data analysis. For each function we list salient properties and a couple of canonical uses.

12.4.1 The quadratic function

$$f(x) = ax^2 + bx + c, \qquad a \neq 0.$$

Salient properties.

- Graph is a parabola opening upward if $a > 0$ and downward if $a < 0$.
- Vertex at
$$x^* = -\frac{b}{2a}, \qquad f(x^*) = c - \frac{b^2}{4a}.$$
- Symmetric about $x = x^*$.

Uses.

- Uniform acceleration: $s(t) = \frac{1}{2}gt^2 + vt + s_0$.
- Concave approximations in economics (diminishing returns).

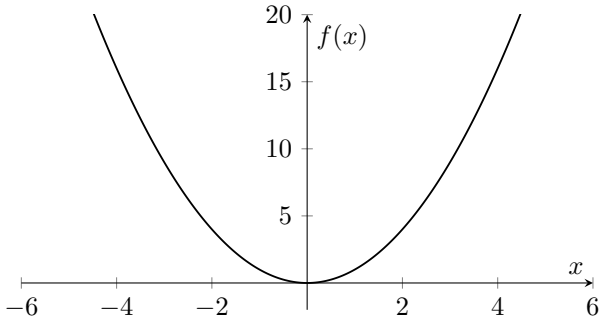


Figure 12.3: A quadratic: $f(x) = x^2$.

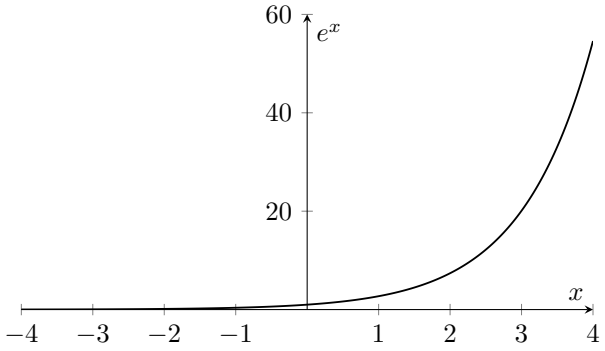


Figure 12.4: Exponential growth: $f(x) = e^x$.

12.4.2 The exponential function

$$f(x) = e^x.$$

Properties: always positive, strictly increasing, derivative equals itself.
Uses: population growth, compound interest, differential equations.

12.4.3 The negative exponential

$$f(x) = e^{-x}.$$

Properties: positive, strictly decreasing, approaches 0 as $x \to \infty$.
Uses: radioactive decay, discounting, cooling.

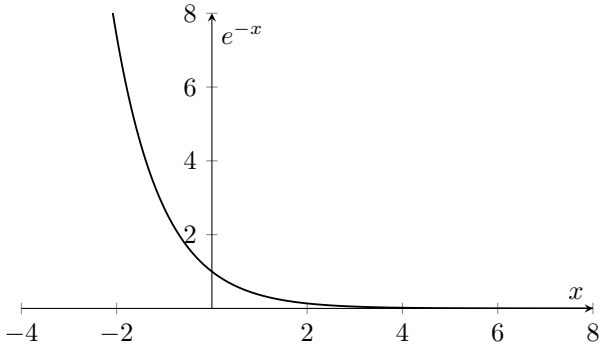


Figure 12.5: Exponential decay.

12.4.4 The logarithm

$$f(x) = \ln x, \qquad x > 0.$$

Properties: strictly increasing, slow growth, inverse of e^x.
Uses: multiplicative-to-additive conversion, data compression, statistics.

12.4.5 The sinusoid

$$f(x) = \sin x.$$

Properties: periodic, oscillatory, bounded, smooth.
Uses: waves, AC electricity, tides, Fourier series.

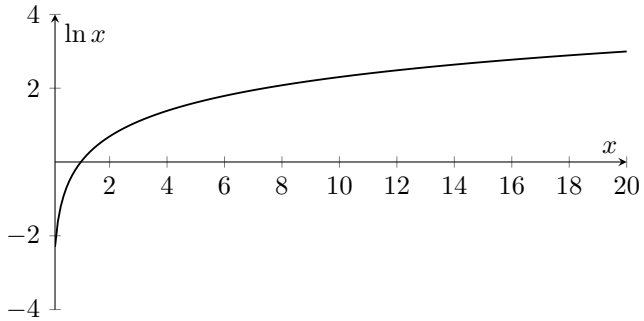


Figure 12.6: Logarithmic growth: $f(x) = \ln x$.

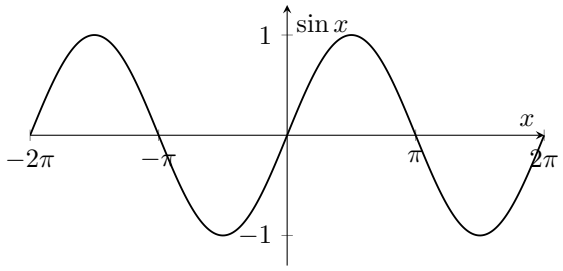


Figure 12.7: A sinusoidal oscillation: $f(x) = \sin x$.

12.4.6 The Gaussian (normal curve)

$$f(x) = \frac{1}{\sqrt{2\pi}}\, e^{-x^2/2}.$$

Properties: symmetric, bell-shaped, rapidly decaying tails.
Uses: measurement error, statistics, noise modelling.

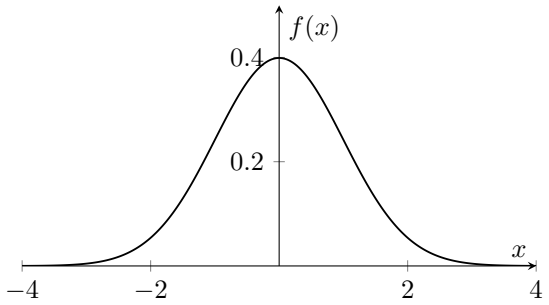


Figure 12.8: Standard Gaussian density.

12.4.7 The logistic function

$$f(x) = \frac{1}{1 + e^{-x}}.$$

Properties: S-shaped, bounded, symmetric about $x = 0$.
Uses: constrained growth, technology adoption, logistic regression.

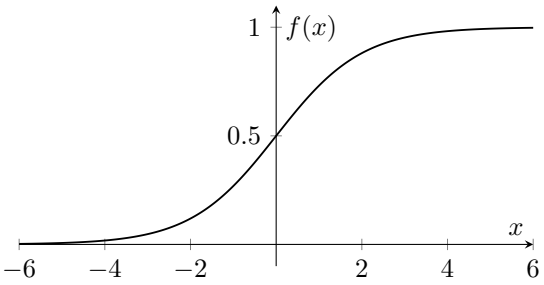


Figure 12.9: Logistic (saturation) curve.

A closing remark

These functions recur because each captures a deep "shape" that nature and society repeatedly exhibit: quadratic curvature, exponential growth and decay, logarithmic compression, sinusoidal oscillation, Gaussian randomness, and logistic saturation. Learning their properties is like acquiring a small but powerful vocabulary for the real world.

Summary

The real numbers complete the rationals, resolve ancient paradoxes, and support the modern theory of limits, continuity, and calculus. They underpin geometry, physics, finance, and modelling, and form the domain for the most important functions in science. Understanding real numbers is therefore a gateway into nearly every field of mathematics and its applications.

Problems

1. (**Arithmetic sequences**) An arithmetic progression has first term a and common difference d. Show that the average of the first n terms equals the average of the first and nth terms. Use this fact to re-derive the sum formula $S_n = \frac{n}{2}(2a + (n-1)d)$.

2. (**Geometric growth**) A bacteria culture triples every hour. If it starts with B_0 bacteria at time $t = 0$, express B_t as a geometric progression. How long does it take to exceed $50\,B_0$?

3. (**Polynomial addition and multiplication**) Let

$$p(x) = 3x^3 - 2x + 7, \qquad q(x) = x^2 - 4.$$

   Compute $p(x) + q(x)$ and $p(x) \cdot q(x)$ and state their degrees. Explain why degrees add under multiplication.

4. (**Decimal as a polynomial**) Interpret the decimal number 40756 as a polynomial in 10 evaluated at 10. Rewrite it using powers of 2 instead of 10 by expanding it in base 2.

5. (**Interpolation**) Given three points $(0, 1)$, $(1, 3)$, $(2, 6)$:

   (a) Construct the Lagrange basis polynomials $L_0(x)$, $L_1(x)$, $L_2(x)$.

   (b) Write down the interpolating polynomial explicitly.

   (c) Evaluate it at $x = 1/2$.

6. (**Overfitting with polynomials**) Explain why there always exists a polynomial of degree $n - 1$ passing through n given data points, but why such a polynomial might be a poor model for prediction.

7. (**Linear regression intuition**) Consider the data points $(0,3)$, $(1,2)$, $(2,2)$, $(3,5)$. Sketch a line that "best fits" the data by eye. Explain qualitatively why the least-squares line minimises the sum of squared vertical errors.

8. (**Systems of equations**) Solve the system:

$$\begin{cases} 3x + y = 7, \\ 2x - y = 1. \end{cases}$$

   Then interpret the solution as the intersection of two lines.

9. (**Linearity in modelling**) A factory produces two products A and B. Each unit of A requires 2 hours of machine time and 3 units of raw material. Each unit of B requires 1 hour of machine time and 4 units of raw material. If the factory uses 23 hours of machine time and 41 units of raw material in total, set up the linear system and solve for the production quantities.

10. (**Uniqueness of solutions**) Give an example of a 2×2 linear system with:

    - exactly one solution,
    - infinitely many solutions,
    - no solution.

    Explain geometrically why these cases correspond to intersecting lines, coincident lines, and parallel lines.

11. (**Function families**) For each of the functions below—quadratic, exponential, logarithmic, sinusoidal, logistic—provide:

    (a) a real-world phenomenon it models,

    (b) whether it is increasing, decreasing, periodic, or saturating,

    (c) the effect of scaling the input (e.g. replacing x by kx).

12. (**Gaussian decay**) Show that the Gaussian function

$$g(x) = e^{-x^2}$$

    decays faster than any exponential e^{-cx} as $x \to \infty$. (Hint: compare exponents.)